

Affinity SDK – Complete Scripting Tutorial

Affinity SDK — Complete Scripting Tutorial

Copyright © 2026

All rights reserved.

Fonts: IBM Plex Serif, IBM Plex Sans, IBM Plex Mono

Source Code: <https://github.com/peterfrieese/eightbyten>

PDF generated from source.

This PDF is not affiliated with, endorsed by, or sponsored by Affinity by Canva.

For everyone learning to automate Affinity by Canva.

Contents

Chapter 1: Affinity SDK — Complete Learning Tutorial	1	PART I. PHASE 1 — FOUNDATIONS	7
1.1 What this tutorial is	1	Chapter 2: Phase 1 — Foundations	9
1.2 Official SDK documentation	1	2.1 Lessons	10
1.3 What is the Affinity SDK	1	2.2 What to expect	10
1.4 Who this tutorial is for	2	Chapter 3: Lesson 01 — Environment Setup & Execution Model	11
1.5 What you will be able to build	2	3.1 What this lesson covers	11
1.6 How the tutorial is structured	2	3.2 The execution model	11
1.7 Phase overview	3	3.3 The module system	12
1.8 How to use this tutorial	4	3.4 console.log() — your only output	12
1.9 The three rules everything else builds on	4	3.5 File system access	12
1.10 Running a script	5	3.6 Application API	12
1.11 SDK module reference	5	3.7 Key gotchas	14
1.12 Supporting files	6	3.8 Exercises	14
		3.9 SDK modules used in this lesson	14
		3.10 See also	14
		3.11 Examples	14
		Chapter 4: Lesson 02 — Application & Document Model	17
		4.1 What this lesson covers	17
		4.2 The document hierarchy — JS API vs node tree	17
		4.3 Getting the current document	18
		4.4 All open documents	19
		4.5 Spreads vs pages	19
		4.6 The selection / currentSpread ordering rule	19
		4.7 Document state flags	19
		4.8 Creating a new document	20
		4.9 Async document operations	20
		4.10 Document properties	20
		4.11 Document units and page properties	24
		4.12 Key gotchas	24
		4.13 Exercises	25
		4.14 SDK modules used in this lesson	25
		4.15 See also	25
		4.16 Examples	25
		Chapter 5: Lesson 03 — Node Model & Tree Traversal	27
		5.1 What this lesson covers	27
		5.2 The node tree	27
		5.3 Traversing layers	28
		5.4 Multi-spread traversal	28
		5.5 `/collection` — lazy functional traversal	28
		5.6 Node type guards	29
		5.7 Working with the selection	30
		5.8 Node interfaces reference	30
		5.9 NodeChildType	30
		5.10 Node navigation shortcuts	31
		5.11 ContainerNode — named layer groups	31
		5.12 Node convenience methods	31

5.13 Per-node tag metadata	32
5.14 Exploring nodes with `/inspect`	33
5.15 Key gotchas	34
5.16 Exercises	34
5.17 SDK modules used in this lesson	35
5.18 Further examples	35
5.19 See also	35
5.20 Examples	35

PART II. PHASE 2 — CORE API 39

Chapter 6: Phase 2 — Core API 41

6.1 The core loop	41
6.2 Lessons	42
6.3 What to expect	42

Chapter 7: Lesson 04 — The Command Pattern 43

7.1 What this lesson covers	43
7.2 The golden rule	43
7.3 Basic command execution	44
7.4 CompoundCommandBuilder — one undo step for many changes	44
7.5 AddChildNodesCommandBuilder — creating new nodes	44
7.6 Reading command results	45
7.7 Preview mode	45
7.8 Async command execution	45
7.9 Programmatic selection	46
7.10 Undo and redo	46
7.11 Common DocumentCommand factories ..	47
7.12 Key gotchas	47
7.13 Exercises	47
7.14 SDK modules used in this lesson	47
7.15 Further examples	48
7.16 See also	48
7.17 Examples	48

Chapter 8: Lesson 05 — Fills & Strokes 51

8.1 What this lesson covers	51
8.2 Colours	51
8.3 FillDescriptor	52
8.4 Gradient fills	52
8.5 Applying fills to selected nodes	53
8.6 Three fill channels	54
8.7 Line styles	54
8.8 Line caps and joins	55
8.9 Dash patterns	56
8.10 LineStyleMask — partial updates	56
8.11 Stroke alignment	56
8.12 Opacity and blend mode	57
8.13 Hatch fills	57
8.14 Bitmap fills	58
8.15 Mesh fills	58
8.16 Diffusion fills	59
8.17 Doc convenience fill methods	59
8.18 VectorNodeDefinition — multi-fill and multi-stroke	60
8.19 Fluent node-definition builders	60
8.20 Colour accepted where FillDescriptor expected	61
8.21 Exercises	61
8.22 SDK modules used in this lesson	61
8.23 Further examples	61
8.24 See also	62

8.25 Examples	62
Chapter 9: Lesson 06 — Geometry, Transforms & Curves	65
9.1 What this lesson covers	65
9.2 Coordinate system	66
9.3 Transforms — moving and resizing nodes ..	66
9.4 Adding basic shapes	67
9.5 Shape parameters	67
9.6 Parametric shapes — the full inventory	68
9.7 QR codes	69
9.8 Building curves with CurveBuilder	71
9.9 Bezier curves	72
9.10 Bulge / versine arcs	72
9.11 Grid duplicator	72
9.12 Converting a shape to curves	73
9.13 Rasterising objects	73
9.14 Adding artboards	73
9.15 Knife cut and scissor cut	74
9.16 Boolean operations	74
9.17 TransformBuilder	74
9.18 Key gotchas	75
9.19 Exercises	75
9.20 Combining bounding boxes	75
9.21 SDK modules used in this lesson	76
9.22 Further examples	76
9.23 See also	76
9.24 Examples	77

PART III. PHASE 3 — TOOLS	83
Chapter 10: Phase 3 — Tools	85
10.1 Lessons	86
10.2 What to expect	86
Chapter 11: Lesson 07 — Text & Story Model .	87
11.1 What this lesson covers	88
11.2 How text works in Affinity SDK	88
11.3 Reading text from a node	88
11.4 Replacing all text in a frame	88
11.5 Appending text to a frame	89
11.6 The selection setup order matters	89
11.7 Applying character formatting (StoryDelta)	90
11.8 StoryDelta convenience factories	90
11.9 StoryBuilder — building text from scratch	91
11.10 TableTextNode	92
11.11 GlyphAtts and ParagraphAtts — attribute wrappers	93
11.12 Attribute runs on Story	93
11.13 Reading text and searching words	94
11.14 Glyphs — the typed character model	94
11.15 Batch text update — rename all text nodes	95
11.16 Text frame state	95
11.17 Key gotchas	96
11.18 Exercises	96
11.19 SDK modules used in this lesson	97
11.20 Further examples	97
11.21 See also	97
11.22 Examples	97
Chapter 12: Lesson 08 — Dialog UI	103
12.1 What this lesson covers	104
12.2 The Dialog hierarchy	104
12.3 Minimal dialog example	104
12.4 All control types	104
12.5 Dialog-level properties	109
12.6 Multi-column dialogs	110
12.7 Reactive controls	110
12.8 Full practical example — export dialog ..	111
12.9 Reactive dialog example — shape creator	112
12.10 Live preview dialogs	112
12.11 Attaching controls to the dialog object .	113
12.12 alert()	113
12.13 The canonical example — addGuides.js	114
12.14 Key gotchas	114
12.15 Exercises	114
12.16 SDK modules used in this lesson	115
12.17 Further examples	115
12.18 See also	115
12.19 Examples	115

Chapter 13: Lesson 09 — Export & File I/O	123
13.1 What this lesson covers	123
13.2 Export overview	124
13.3 Getting available export presets	124
13.4 Synchronous export	124
13.5 Export areas	125
13.6 Export every artboard	125
13.7 Per-node export settings (ExportConfig)	126
13.8 Async export with doc.promises	126
13.9 Saving the document	127
13.10 File system — reading and writing text	127
13.11 JSON sidecar pattern	128
13.12 Network requests	128
13.13 Key gotchas	129
13.14 200	129
13.15 Exercises	129
13.16 SDK modules used in this lesson	129
13.17 Further examples	130
13.18 See also	130
13.19 Examples	130
Chapter 14: Lesson 10 — Text Threading	133
14.1 What this lesson covers	133
14.2 How threading works	134
14.3 Linking frames	134
14.4 Unlinking frames	135
14.5 Reading text across linked frames	135
14.6 Putting it together	135
14.7 Key gotchas	136
14.8 Exercises	136
14.9 SDK modules used in this lesson	137
14.10 Further examples	137
14.11 See also	137
14.12 Examples	137

PART IV. PHASE 4 — ADVANCED	141
Chapter 15: Phase 4 — Advanced	143
15.1 Lessons	144
15.2 What to expect	144
Chapter 16: Lesson 11 — AI Commands	147
16.1 What this lesson covers	147
16.2 Why use the SDK AI commands?	148
16.3 Generate an image from a prompt	148
16.4 Generative edit (inpainting)	148
16.5 Remove background	148
16.6 Select subject	148
16.7 Detect depth	149
16.8 Colourise	149
16.9 Image trace (raster to vector)	149
16.10 Batch image generation	149
16.11 Using AI commands with doc.promises (async)	150
16.12 Dialog-driven image generator	150
16.13 Key gotchas	151
16.14 Exercises	151
16.15 SDK modules used in this lesson	151
16.16 Further examples	151
16.17 See also	151
16.18 Examples	152
Chapter 17: Lesson 12 — Layer Effects & Adjustments	153
17.1 What this lesson covers	154
17.2 Effect categories	154
17.3 Outer shadow	154
17.4 The `enableIfDisabled` parameter	155
17.5 The index parameter	155
17.6 Outline (stroke effect)	155
17.7 Inner glow	156
17.8 Gaussian blur (layer effect)	156
17.9 Outer glow	157
17.10 Inner shadow	157
17.11 Colour overlay	157
17.12 Bevel and emboss	158
17.13 Remove all effects	159
17.14 Gradient overlay	159
17.15 Phong bevel	160
17.16 Destructive filters on raster nodes	161
17.17 Drawing scale & margins	162
17.18 Raster brush properties	163
17.19 Key gotchas	163
17.20 Exercises	164
17.21 SDK modules used in this lesson	164
17.22 See also	164
17.23 Examples	164
Chapter 18: Lesson 13 — Pro Patterns: Async, Hints & Network	167
18.1 What this lesson covers	168

18.2 The doc.promises API	168	19.22 `createSetEditable` — argument order	185
18.3 Processing and exporting the current document	168	19.23 `AddChildNodesCommandBuilder` — insertion options	186
18.4 Full async pipeline	169	19.24 SDK native binding modules	186
18.5 Network requests	170	19.25 New node types	186
18.6 Remote data → shapes pattern	170	19.26 Exercises	187
18.7 Timers	171	19.27 SDK modules used in this lesson	188
18.8 search_sdk_hints — community knowledge base	171	19.28 See also	188
18.9 add_sdk_hint — contributing back	171	19.29 Examples	188
18.10 save_script_to_library	171		
18.11 Production script checklist	171	Chapter 20: Lesson 15 — Pixel Processing ..	191
18.12 Key gotchas	172	20.1 What this lesson covers	192
18.13 Exercises	172	20.2 What a pixel is in this context	192
18.14 SDK modules used in this lesson	172	20.3 Why pixel processing rather than filters or adjustments	192
18.15 Further examples	172	20.4 Why pixel processing is slow — and what to do about it	192
18.16 See also	173	20.5 Why pixel processing matters	193
18.17 Examples	173	20.6 The pipeline	193
Chapter 19: Lesson 14 — Special Modules: Pixel Access, Brushes & More	175	20.7 Choosing the right format	193
19.1 What this lesson covers	176	20.8 Core pixel read/write pattern	194
19.2 Modules at a glance	176	20.9 Read-only analysis (no write-back needed)	194
19.3 `/pixellaccessor` — Direct pixel read/write	177	20.10 Creating a bitmap from scratch	195
19.4 `/taginterface` — Per-node custom metadata	179	20.11 Loading a bitmap from a file	196
19.5 `/documentproperties` — Document metadata	179	20.12 Rendering a node to a bitmap with NodeRenderingEngine	196
19.6 `/drawingscale` — Drawing/map scale	179	20.13 The Buffer class	197
19.7 `/marginsinterface` — Spread margins	180	20.14 Performance considerations	198
19.8 `/rasterbrush` — Brush property access	180	20.15 Key gotchas	198
19.9 `/os` — Host operating system info	180	20.16 Raster pixel selection	198
19.10 `/environment` — Runtime permissions & config	181	20.17 Raster auto-fixes and fill	199
19.11 `/logging` — File-based logging	181	20.18 Exercises	200
19.12 `/configuration` — Typed JSON config	181	20.19 SDK modules used in this lesson	200
19.13 `/inspect` — Node.js-style object inspector	181	20.20 Further examples	201
19.14 `/pathbrush` — Vector path brush access	182	20.21 See also	201
19.15 `/imageresourceinterface` — Placed-image metadata	182	20.22 Examples	201
19.16 `/pictureframeinterface` — Picture-frame state	183		
19.17 `/rasterselection` — Pixel-selection state	183	Chapter 21: Lesson 16 — Fonts & Typography	211
19.18 `/timers` — Timer objects	183	21.1 What this lesson covers	211
19.19 `/units` — UnitValue and conversion factors	183	21.2 The fonts module	212
19.20 `/hatch` — Building hatch patterns	184	21.3 Three levels of font data	212
19.21 `DocumentCommand` — full method reference	184	21.4 Enumerating font families	212
		21.5 Inspecting fonts within a family	212
		21.6 Getting a Font object	213
		21.7 Font classification via PANOSE	213
		21.8 Font distance (finding the closest match)	214
		21.9 Font collections	214
		21.10 Practical: check which fonts a document uses	214
		21.11 Practical: font picker dialog	215
		21.12 Font properties reference	216

21.13 FontFamily properties reference	217	PART V. PHASE 5 — OFFICIAL EXAMPLES	241
21.14 Key gotchas	217	Chapter 24: Phase 5 — Official Affinity SDK Examples	243
21.15 Exercises	218	24.1 How to run these examples	243
21.16 SDK modules used in this lesson	218	24.2 Examples at a glance	244
21.17 See also	218	24.3 Attribution & license	247
21.18 Examples	218	Chapter 25: addGuides.js	249
Chapter 22: Lesson 17 — Raster Adjustments	221	Chapter 26: alignToPage.js	251
22.1 What this lesson covers	221	Chapter 27: adjustPageItems.js	253
22.2 Creating an adjustment layer	221	Chapter 28: artboardGrid.js	255
22.3 Reading parameters from an existing adjustment node	222	Chapter 29: setDocumentFormat.js	257
22.4 Brightness / Contrast	222	Chapter 30: addPoints.js	259
22.5 Levels	223	Chapter 31: bulgedPolyline.js	261
22.6 Curves	223	Chapter 32: bulgeVersinePlayground.js	263
22.7 HSL Shift	224	Chapter 33: textOnCurvesAndShapes.js	265
22.8 White Balance	224	Chapter 34: boldItalics.js	267
22.9 Vibrance	224	Chapter 35: tableFromJson.js	269
22.10 Exposure	224	Chapter 36: bitmapWriter.js	271
22.11 Colour Balance	225	Chapter 37: meshBubble.js	273
22.12 Posterise	225	Chapter 38: flexibleLayout.js	275
22.13 Threshold	225	Chapter 39: logToFile.js	277
22.14 Invert	225	Chapter 40: cornerEffects.js	279
22.15 Recolour	225	Chapter 41: divideLength.js	281
22.16 Split Toning	225	Chapter 42: pathEffects.js	283
22.17 Black & White	225	Chapter 43: roundAnyCorner.js	285
22.18 Tone Compression and Tone Stretch ...	226	Chapter 44: cropMarks.js	287
22.19 Normals (normal map rotation)	226	Chapter 45: makeGrid.js	289
22.20 RAW Develop	226	Chapter 46: stepAndRepeat.js	291
22.21 All adjustment types — quick reference	227	Chapter 47: makeNumbersSequence.js	293
22.22 HalftoneFilter	228	Chapter 48: randomise.js	295
22.23 Key gotchas	229	Chapter 49: swapObjects.js	297
22.24 Exercises	229	Chapter 50: breakFrame.js	299
22.25 SDK modules used in this lesson	230	Chapter 51: opticalBackward.js	301
22.26 See also	230	Chapter 52: opticalForward.js	303
22.27 Examples	230	Chapter 53: splitStory.js	305
Chapter 23: Lesson 18 — Align & Distribute	233		
23.1 What this lesson covers	233		
23.2 How it works	234		
23.3 GroupTransformType — what to do	234		
23.4 GroupTransformAnchor — reference frame	235		
23.5 Common operations	235		
23.6 Full example — align & distribute dialog	236		
23.7 Combining align and distribute	237		
23.8 Using DocumentCommand directly	237		
23.9 Key gotchas	237		
23.10 Exercises	238		
23.11 SDK modules used in this lesson	238		
23.12 See also	238		
23.13 Examples	238		

Chapter 54: arrowheads.js	307	60.6 Boundary signals — how to detect what's missing	337
Chapter 55: strokesWeightDown.js	309	60.7 The complete native method call inventory (nodes.js only)	337
Chapter 56: strokesWeightUp.js	311	60.8 Runtime capability detection	338
Chapter 57: countSelectedItems.js	313	60.9 SDK modules	338
Chapter 58: selectObjects.js	315	60.10 Summary counts	347
Chapter 59: Affinity SDK — Quick Reference Cheat Sheet	317	60.11 Official SDK example scripts	348
59.1 Boilerplate — every script starts here	317		
59.2 Imports by task	317		
59.3 Programmatic selection	318		
59.4 The command loop	318		
59.5 Colours	318		
59.6 Apply fill / stroke / opacity	318		
59.7 Text replacement	319		
59.8 Dialog	319		
59.9 Export	320		
59.10 Layer effects (shadow example)	320		
59.11 Async pipeline	320		
59.12 Additional APIs	321		
59.13 `createSetEditable` — argument order	322		
59.14 Document convenience shorthands	322		
59.15 Document line style shorthands	323		
59.16 StoryDelta convenience factories	323		
59.17 Geometry utilities	323		
59.18 HalftoneFilter	324		
59.19 DialogButton and DialogColumnStack	324		
59.20 Fills — selection first	325		
59.21 TableTextNode	325		
59.22 GlyphAtts / ParagraphAtts	325		
59.23 Attribute runs on Story	325		
59.24 Fluent node-definition builders	325		
59.25 Colour → FillDescriptor auto-convert	325		
59.26 `node.spreadBaseBox`	325		
59.27 Spread / artboard document properties	326		
59.28 Boolean operations & shape division	326		
59.29 Text frame threading	326		
59.30 Raster auto-fixes & fill	326		
59.31 System / logging / config	326		
59.32 Per-node export config, glyphs, units & hatch	326		
59.33 Critical rules	327		
Chapter 60: Affinity SDK — C++ Exposure Map	329		
60.1 How the binding works	329		
60.2 The four native C++ binding modules	330		
60.3 The architecture signal: `*Api` classes	330		
60.4 What IS exposed to JavaScript	330		
60.5 What is NOT exposed to JavaScript	335		

Affinity SDK – Complete Learning Tutorial

1

1.1 What this tutorial is

This is a structured tutorial for learning the Affinity by Canva JavaScript SDK – the scripting system that lets you automate Affinity application using JavaScript running directly inside the application.

The tutorial covers the SDK from the ground up: the execution model, the document structure, the command system, visual properties, text, dialogs, export, AI operations, layer effects, async patterns, pixel processing, and typography. It is built from direct inspection of the SDK source code and the official test suite, and every example script has been verified against a running Affinity installation.

1.2 Official SDK documentation

The official Affinity SDK documentation lives at <https://sdk.affinity.studio/>. This tutorial is built from direct inspection of the SDK source code, but the SDK evolves – new modules, commands, and examples ship with Affinity updates. Make it a habit to check the official documentation alongside this tutorial for the latest API reference; when in doubt, the official docs are the canonical source.

1.3 What is the Affinity SDK

The Affinity SDK is a JavaScript scripting environment embedded inside Affinity by Canva. Scripts run in a sandboxed JS engine that has direct access to the open document – its layers, nodes, text, fills, geometry, and pixel data – through a set of native modules.

It is not a browser API. It is not Node.js. It is a purpose-built environment with its own rules:

- Modules are loaded with `require('/module-name.js')` — not npm, not ES imports
- Scripts run synchronously inside the application's main thread unless explicitly made async
- All document modifications go through a command system that integrates with Affinity's undo stack
- The only output mechanism available to scripts is `console.log()`, viewed in the script console
- Scripts are restricted to reading and writing files on the Desktop or to folders you grant access to

Within those constraints, scripts can do a remarkable amount: read and modify any property of any layer, apply fills and effects, manipulate text, export assets in any format, call external APIs via the network module, drive Affinity's AI features, and process images pixel by pixel.

1.4 Who this tutorial is for

This tutorial assumes you can read and write JavaScript — loops, functions, objects, and basic async patterns. You do not need experience with Node.js, browser APIs, or any specific JavaScript framework. SDK scripting is different from all of them, and this tutorial treats it on its own terms.

You do not need any prior experience with Affinity's scripting panel. Lesson 01 covers everything you need to get your first script running.

This tutorial also assumes you are familiar with Affinity by Canva — how to open documents, work with layers, use the export persona, and navigate the UI. The tutorial teaches SDK scripting, not the application itself. If you are new to Affinity, start with the official Affinity tutorials first.

1.5 What you will be able to build

By the end of this tutorial you will be able to write scripts that:

- Iterate every layer in a document, identify node types, and read or modify their properties
- Apply fills, strokes, gradients, and effects to any node or set of nodes in a single undoable command
- Read, replace, and style text in any text frame using the Story API
- Present native dialogs that collect input from the user and drive the rest of the script
- Export every artboard as named files, write JSON manifests, and POST results to an external API
- Generate images from text prompts, remove backgrounds, and chain AI operations with export pipelines
- Process every pixel of a raster layer to implement custom image filters and analysis tools
- Enumerate installed fonts, detect missing typefaces, and substitute them programmatically
- Write async pipelines that chain execute, export, and save operations with proper error handling

1.6 How the tutorial is structured

The tutorial is organised into five phases, each building on the last. Within each phase, lessons are ordered so that earlier lessons are prerequisites for later ones. The phase and lesson READMEs tell you exactly what they depend

on. Phase 5 is a reference collection of official example scripts rather than lessons.

```
affinity-sdk-tutorial/
├── phase-1-foundations/           Lessons 01-03
│   ├── 01-environment-setup/
│   ├── 02-app-and-document/
│   └── 03-node-model/
├── phase-2-core-api/           Lessons 04-06
│   ├── 04-command-pattern/
│   ├── 05-fills-and-strokes/
│   └── 06-geometry/
├── phase-3-tools/             Lessons 07-10
│   ├── 07-text-and-story/
│   ├── 08-dialog-ui/
│   ├── 09-export-and-files/
│   └── 10-text-threading/
├── phase-4-advanced/         Lessons 11-18
│   ├── 11-ai-commands/
│   ├── 12-layer-effects/
│   ├── 13-pro-patterns/
│   ├── 14-special-modules/
│   ├── 15-pixel-processing/
│   ├── 16-fonts-and-typography/
│   ├── 17-raster-adjustments/
│   └── 18-align-distribute/
└── phase-5-official-examples/ Official Affinity SDK example scripts (34
    files)
    └── examples/
```

Each lesson folder contains:

- README.md — a full explanation of the concepts, the correct API patterns, the common failure modes, and a key gotchas section that documents the non-obvious behaviours that cost time to discover independently
- examples/ — runnable .js scripts that you paste directly into Affinity's script panel. Every example is self-contained and includes comments explaining the non-obvious parts

1.7 Phase overview

Phase	What it covers	Lessons
1 – Foundations	The execution model, document entry point, and node tree. Everything else depends on this.	1–3
2 – Core API	The command system, fills, strokes, geometry, and transforms. This is where scripts start making changes.	4–6
3 – Tools	Text automation, dialog UI, file export, and HTTP networking. This is where scripts become useful to other people.	7–10
4 – Advanced	AI commands, layer effects, raster ad-	11–18

Phase	What it covers	Lessons
	justments, async pipelines, pixel processing, typography, and align/distribute. The full capability of the SDK.	
5 – Official examples	Verbatim example scripts shipped by the Affinity team – best-practice references across layout, geometry, paths, text, strokes, selection, raster, dialogs, and logging.	reference

1.8 How to use this tutorial

Read the README before running the examples. Each lesson README explains the concepts, the gotchas, and the argument orders before you encounter them in code. The examples will make more sense – and fail less often – if you understand the model first.

Keep CHEAT-SHEET.md open while coding. It is a single-page reference covering the patterns, argument orders, and critical rules that come up in almost every session. The critical rules table at the bottom is particularly worth memorising – it documents the SDK behaviours that produce silent failures rather than helpful error messages.

Run the examples on a real document. Most examples require an open document and a selection. Start with a simple document – a few shapes on a single page – until you understand how a script interacts with it. Move to more complex documents once the patterns are clear. I would encourage you to type all the examples yourself. Don't just copy-paste. The muscle memory you will gain will serve you well in the future.

Work through phases in order, but not necessarily every lesson. Phases 1 and 2 are prerequisites for everything. Within Phase 3 and 4, individual lessons are largely independent – if you need text automation now, go to Lesson 07. If you need pixel processing, go to Lesson 14. Come back to the others when you need them.

Use SDK-EXPOSURE-MAP.md as a reference. It documents the full API surface – every module, every class, every method – in one place. When you need something the lessons don't cover, start there.

1.9 The three rules everything else builds on

1. `require('/module.js')` – not `./module` The SDK has its own module registry. Paths start with `/`. There is no npm, no ES modules, no `node_modules`. If a `require` call silently returns undefined, the module path is wrong.

2. `console.log()` is your only output Scripts don't return values to the UI. Everything you want to see must be logged. View the output in Affinity's script console panel.

3. All document changes go through `doc.executeCommand()` Never mutate nodes directly. Build a command object and execute it. This is what gives

you undo/redone integration, and it is also what gives the command system its ability to batch multiple changes into a single undoable step via CompoundCommandBuilder.

1.10 Running a script

1. Open a document in Affinity by Canva
2. Open the Script Editor: go to **Window** → **Scripting** → **Script Editor**, or switch to the Scripting Studio for a full range of scripting panels
3. Paste/type a script into the editor
4. Click **Run** — output appears in the console below the editor

For scripts that require a selection, select the relevant objects before clicking Run.

1.11 SDK module reference

Module	Key exports
/application	app — version, platform, Desktop path
/document	Document, DocumentPromises, DocumentPreset, DocumentHistory, DocumentSnapshot, FileExportOptions, FileExportArea, NewDocumentOptions, LoadDocumentOptions
/commands	DocumentCommand, CompoundCommandBuilder, AddChildNodesCommandBuilder
/nodes	All node classes — ShapeNode, TextNode, RasterNode, etc.
/selections	Selection, TextSelection
/colours	RGBA8, Colour, ColourProfile
/fills	FillDescriptor, SolidFill, GradientFill, HatchFill, BitmapFill, No Fill
/linestyle	LineStyle, LineStyleMask, LineType
/geometry	Transform, CurveBuilder, PolyCurve, Bezier
/shapes	Shape definitions and parameters
/dialog	Dialog, all control types, DialogResult
/storybuilder	StoryBuilder — construct text content programmatically
/storydelta	StoryDelta — describe text formatting changes
/glyphatts	Character-level formatting attributes
/paragraphatts	Paragraph-level formatting attributes

Module	Key exports
/pixelaccessor	PixelReaderWriterRGBA8 and all other pixel accessor classes
/rasterobject	Bitmap, PixelBuffer, NodeRenderingEngine, RasterFormat
/fonts	Font, FontFamily, FontWeight, FontWidth
/fs	File — Desktop read/write only
/network	HttpRequest, RequestMethod
/timers	setTimeout(delay, callback), setInterval(delay, callback)
/buffer	Buffer — raw binary data
/units	UnitType, UnitValueConverter

For the complete API surface including all methods and argument signatures, see SDK-EXPOSURE-MAP.md. The official SDK documentation at <https://sdk.affinity.studio/> is the canonical reference — check it for the latest additions and changes.

1.12 Supporting files

File	Purpose
SDK-PRIMER.md	AI project starter — correct patterns, broken APIs, non-obvious behaviours. Read this before writing any code in a new session.
CHEAT-SHEET.md	One-page quick reference — keep this open while coding
SDK-EXPOSURE-MAP.md	Full API surface inventory — modules, classes, methods

Phase 1 — Foundations

PART



Phase 1 — Foundations

2

Every Affinity script, no matter how sophisticated, is built on three pieces of knowledge: how the script runtime works, how a document is structured, and how the objects inside it are organised. Phase 1 covers all three.

This is not a phase you should rush. The patterns established here — getting the current document, guarding against null, iterating the node tree, exploring unfamiliar objects with `inspect` — appear in virtually every script you will ever write. Developers who skip ahead and try to write something complex before understanding these foundations consistently hit the same walls: confusing errors about undefined properties, scripts that work on one document and fail silently on another, loops that miss nodes or visit them twice. The three lessons in this phase exist specifically to prevent those problems.

The Affinity SDK is not a browser API and not a Node.js API. It is a JavaScript environment embedded inside a running design application, with its own rules about what is allowed, what modules exist, and how the document model is exposed. Lesson 01 orients you in that environment and introduces the Application API. Lesson 02 introduces the document itself — the entry point for every script — and explains the spread and page model that underpins every Affinity file. Lesson 03 maps the node tree: the structure that holds every layer, group, shape, and text frame in the document, and teaches `inspect` as the primary discovery tool for any unfamiliar SDK object.

These three lessons have no dependencies on each other — but every later phase depends on them.

2.1 Lessons

Lesson	Topic	Key skill you'll gain
01 — Environment setup	Execution model, module system, Application API	Running your first script; app info, dialogs, document listing
02 — App & document	Entry point, current doc, spreads	Accessing any open document
03 — Node model	Tree traversal, node types, selection, /inspect	Iterating layers; exploring any SDK object

2.2 What to expect

By the end of Phase 1 you should be able to write a script that:

- Runs without throwing errors when pasted into Affinity's script panel
- Guards against no document being open (handles the case where `Document.current` returns null)
- Walks every layer in the document
- Logs the name, type, and visibility of each node
- Uses `inspect` to explore any unknown object and map it back to the SDK primer
- Works correctly on multi-spread (multi-page) documents

You will also understand why every SDK modification requires a command, why the selection must be read before `setCurrentSpread` is called, and why `doc.layers` gives you top-level nodes across all spreads while `doc.layers.all` descends recursively into every group. These are not edge cases — they are core SDK behaviours that every script author encounters within their first hour.

Lesson 01 — Environment Setup & Execution Model

3

Affinity SDK scripts are JavaScript files that run directly inside the Affinity application — not in a browser, not in Node.js, but embedded in the same process as the program itself. This lesson explains exactly how that works: how scripts are loaded and executed, how you access SDK modules with `require()`, and what constraints the environment places on you.

Understanding the execution model before writing a single line of code will save you hours of confusion. The runtime (Affinity's built-in JavaScript engine) has no `fetch`, no `window`, no `process`, and no file access outside the Desktop. Knowing these boundaries upfront means you will never waste time debugging something that simply cannot work in this environment.

3.1 What this lesson covers

- How Affinity SDK scripts execute
- The `require()` module system
- `console.log()` as your output channel
- File system access constraints
- The app singleton — version info, document access, dialog helpers

3.2 The execution model

Affinity SDK scripts are **synchronous JavaScript** running inside the Affinity application process.

Key differences from Node.js or browser JS:

Feature	Affinity SDK	Node.js
Module imports	<code>require('/module.js')</code>	<code>require('./module')</code> or ESM
Package manager	None — built-in modules only	npm
Output	<code>console.log()</code> → Script console	stdout
Return values	Scripts return nothing	Functions return values
File access	Desktop + granted folders	Any path
Network	<code>require('/network.js')</code>	HttpRequest, Request Method

3.3 The module system

All imports use a leading `/`. This is the SDK's internal registry — not the file system.

```
'use strict';
// Correct
const { Document } = require('/document.js');
const { app } = require('/application.js');
const { RGBA8 } = require('/colours.js');

// Wrong — will throw
const { Document } = require('./document.js'); // no relative paths
const { Document } = require('document.js'); // no bare specifiers
```

3.4 console.log() — your only output

Scripts produce no visible return value. Everything you want to inspect must be logged.

```
// Snippet
console.log('simple string');
console.log('label:', someValue);
console.log('multiple:', a, b, c);
console.assert(someCondition, 'message if false');
```

View output in the **Script console** panel in Affinity, below the editor.

3.5 File system access

The SDK can read/write files on the **Desktop** and in folders you explicitly grant access to. Any other path returns `PERMISSION_DENIED`.

```
'use strict';
const { app } = require('/application.js');

// Always use this to get the Desktop path — don't hardcode it
const desktop = app.userDesktopPath;
console.log('Desktop:', desktop);
// macOS: /Users/yourname/Desktop
// Windows: C:\Users\yourname\Desktop
```

3.6 Application API

The `/application` module exposes the `Application` class (app singleton) for app-wide queries and actions — version info, document listing, dialog helpers, and settings.

3.6.1 App info

```
'use strict';
const { Application } = require('/application.js');

console.log('Version: ' + Application.version);
console.log('Build: ' + Application.buildVersion);
console.log('Platform: ' + Application.platformName);
console.log('Major: ' + Application.majorVersion);
console.log('Minor: ' + Application.minorVersion);
console.log('Revision: ' + Application.revisionVersion);
console.log('Product: ' + Application.productFullName);
console.log('Short: ' + Application.productShortName);
console.log('Copyright: ' + Application.productCopyrightMessage);
console.log('Compile: ' + Application.compileDate);
console.log('Args: ' + Application.argC + ' ' +
Application.args.join(' '));
```

3.6.2 Document access

```
// Snippet – assumes Application and Document are available
const { Application } = require('/application.js');
const { Document } = require('/document.js');

const all = Application.documents.all; // all open documents
const current = Application.documents.current; // same as
Document.current

// Open a file from the Desktop
const doc = Application.documents.load(app.userDesktopPath + '/my-
file.afdesign');
```

3.6.3 Dialog helpers

```
// Snippet – assumes Application is available
const { Application } = require('/application.js');

Application.alert('Hello', 'Title');
const ok = Application.confirm('Continue?', 'Confirm');
if (ok) { /* ... */ }

const name = Application.prompt('Your name?', 'Name', 'Default');
console.log('User entered: ' + name);

const picked = Application.chooseFile(); // native file picker, returns
path
Application.chooseFileAsync((path) => { console.log('Picked: ' +
path); });
```

3.6.4 User paths

```
// Snippet – assumes Application is available
const { Application } = require('/application.js');

console.log('Desktop: ' + Application.userDesktopPath);
console.log('Resources: ' + Application.resourcesPath);
```

3.6.5 Settings

app.settings exposes application-level preferences that scripts can read or write:

```
'use strict';
const { Application } = require('/application.js');

// Read-only
console.log('Undo limit: ' + Application.settings.undoLimit);

// Read/write
Application.settings.loadPSDWithEditableText = true;
Application.settings.allowCodeGenerationFromStrings = false;
```

3.7 Key gotchas

No npm, no node_modules. You cannot install external libraries. Only the built-in SDK modules are available.

① **Note**
console.log() is the only output. Scripts don't return values to the UI. If nothing appears in the console, the script either errored silently or has no log statements.

① **Note**
Script storage. Library scripts are stored in a single binary file (scripts.propcol), not as individual .js files. You cannot read them back with fs at runtime. There is no Save to Library button in the Scripts panel — library saving is available via the MCP server.

3.8 Exercises

3.8.1 Exercise 1.1 — Hello Affinity

Write a script that logs the application name, version, and platform.

Expected output:

```
Product: Affinity Designer
Version: x.x
Platform: macOS
```

Hint: Look at app.productShortName, app.shortVersion, app.platformName.

3.8.2 Exercise 1.2 — Desktop path

Write a script that logs the Desktop path and confirms it is a non-empty string.

3.8.3 Exercise 1.3 — All open documents

Write a script that lists every open document's title and whether it has unsaved changes. Use Application.documents.all.

3.8.4 Exercise 1.4 — Module explorer

require several different modules and log what each exports (use console.log(Object.keys(...))). Get familiar with what's available before you need it.

3.9 SDK modules used in this lesson

- /application — app singleton

3.10 See also

- Lesson 02 — App & Document
- examples/ — runnable scripts for this lesson

3.11 Examples

3.11.1 Application info

```
'use strict';

// Example 01a — Application info
// Lesson 01: Environment Setup
//
// Run this first. If it logs cleanly, your environment is working.

const { app } = require('/application.js');

console.log('=== Affinity Application Info ===');
console.log('Product:   ', app.productShortName);
console.log('Full name:  ', app.productFullName);
console.log('Version:    ', app.shortVersion);
console.log('Platform:   ', app.platformName);
console.log('Desktop:    ', app.userDesktopPath);
console.log('Build kind: ', app.buildKind);
console.log('Suite:      ', app.suiteFullName);
```

3.11.2 Module explorer

```
'use strict';

// Example 01b — Module explorer
// Lesson 01: Environment Setup
//
```

```
// Inspect what each core SDK module exports.
// A useful reference when you can't remember where something lives.

const modules = [
  '/application',
  '/document',
  '/commands',
  '/colours',
  '/fills',
  '/linestyle',
  '/geometry',
  '/shapes',
  '/dialog',
  '/units',
];

for (const m of modules) {
  try {
    const exports = require(m);
    const keys = Object.keys(exports);
    console.log(`\n${m} (${keys.length} exports):`);
    console.log(' ', keys.join(', '));
  } catch (e) {
    console.log(`\n${m} - failed to load:`, e.message);
  }
}
```


Lesson 02 — Application & Document Model

4

Every useful Affinity script begins the same way: get the current document, inspect it, and decide what to do with it. This lesson covers the full document model — from the app singleton at the top, down through spreads, pages, and properties like DPI, format, and title.

Before you can move a node, change a colour, or export a file, you need a valid document reference. New developers hit a few key pitfalls: the difference between `Document.current` and `Document.all`, the spread/page distinction, and the selection-vs-spread ordering rule. Mastering these makes your scripts behave predictably across documents.

4.1 What this lesson covers

- The app singleton as entry point
- `Document.current` vs `Document.all`
- Document properties: title, path, format, DPI, dimensions, size and page count
- The spread/page distinction
- Sync vs async document operations (async means the operation runs in the background and returns a result later)
- The critical selection-vs-spread ordering rule
- Document history and snapshots
- Undo/redo and selection convenience methods
- Guide management and drawing scale

4.2 The document hierarchy — JS API vs node tree

It is important to distinguish between the JavaScript API objects and the actual node tree that lives inside Affinity.

JS API `objects`:

```

app
|
Document (doc)
├── rootNode
├── currentSpread
├── selection
└── history

```

Node `tree` (inside `Affinity`):

```

DocumentNode (parent: null)
└── SpreadNode(s)
    └── Layers / children

```

Mapping:

```

doc.rootNode      → DocumentNode
doc.currentSpread → SpreadNode (child of DocumentNode)

```

(The `Document` object itself is not a node — `doc.isDocumentNode` is undefined.)

The `Document` object (`doc`) is the **JS API entry point** — it holds properties like `currentSpread`, `selection`, `history`, and `rootNode`, but it is **not itself a node** (`doc.isDocumentNode` returns undefined).

The **node tree** starts at `rootNode`, a `DocumentNode` (`rootNode.isDocumentNode === true`). It has `parent: null` — it is the true root. Every node in the document, no matter how deeply nested, is a descendant of this `DocumentNode`. It has exactly one type of direct child: `SpreadNode` objects — one per spread in the document.

That is essentially all it exposes. It has no fill, no transform, no visibility — it is purely a container.

The root node exists to give tree traversal a consistent entry point. Every node, including spreads themselves, is a child of something. The `DocumentNode` is the answer to “what is the parent of a spread?” — without it, the tree would have no single root and recursive walkers would need special-case handling for the top level.

```

// Snippet
const rootNode = doc.rootNode;
rootNode.spreadCount // same as doc.spreadCount
rootNode.pageCount   // same as doc.pageCount
rootNode.isDocumentNode // true

```

In practice you almost never use it directly. The SDK gives you more convenient access points that you will use instead. The only time you need `doc.rootNode` directly is when you want to write a recursive walker that starts from the absolute top of the tree and descends through every spread and every layer in one pass — the kind of thing lesson 03 covers.

4.3 Getting the current document

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;

if (!doc) {
  console.log('No document open.');
```

```

} else {
  console.log('Title:      ', doc.title);
  console.log('Path:       ', doc.path);
  console.log('Format:      ', doc.format);
  console.log('DPI:         ', doc.dpi);
  console.log('Width (px): ', doc.widthPixels);

```

```

console.log('Height (px): ', doc.heightPixels);
console.log('Spreads: ', doc.spreadCount);
console.log('Pages: ', doc.pageCount);
console.log('Needs save: ', doc.needsSaving);
console.log('Read only: ', doc.isReadOnly);
}

```

4.4 All open documents

Use `Document.all` to get an array of every document currently open. This is useful for batch operations or multi-document workflows.

```

'use strict';
const { Document } = require('/document.js');

const docs = Document.all;
console.log(`${docs.length} document(s) open`);

for (const d of docs) {
  console.log(d.title, '-', d.path);
}

```

Note

Use `.title` not `.name` — `.name` is undefined on `Document` objects. The SDK test suite uses `.title`.

4.5 Spreads vs pages

- A **spread** is a physical canvas unit. In most documents, one spread = one page.
- In a facing-pages publication, one spread = two pages side by side.
- `doc.spreads` — iterable collection of `SpreadNode` objects
- `doc.pageCount` — total number of pages across all spreads
- `doc.spreadCount` — number of spreads

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  for (const spread of doc.spreads) {
    console.log('Spread:', spread);
  }

  // Shortcuts
  const firstSpread = doc.spreads.first;
  const currentSpread = doc.currentSpread;
}

```

4.6 The selection / currentSpread ordering rule

This is the **most important gotcha in the entire SDK**.

```

// Snippet
// WRONG — selection is lost before you read it
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));
const sel = doc.selection; // always empty!

// CORRECT — capture selection BEFORE changing spread
const sel = doc.selection;
const nodes = sel.nodes; // extract everything you need
doc.executeCommand(DocumentCommand.createSetCurrentSpread(spread));

```

`setCurrentSpread` **clears the selection**. Always read `doc.selection` and extract all needed data before executing that command.

4.7 Document state flags

Property	Meaning
<code>doc.needsSaving</code>	Has unsaved changes

Property	Meaning
<code>doc.isDirty</code>	Same as <code>needsSaving</code>
<code>doc.isReadOnly</code>	Cannot be edited
<code>doc.mustSaveAs</code>	New doc — must be saved to a path first
<code>doc.isEmbedded</code>	Placed inside another document

4.8 Creating a new document

```
'use strict';
const { Document, NewDocumentOptions } = require('/document.js');

const opts = NewDocumentOptions.createDefault();
opts.width = 1920;
opts.height = 1080;
opts.dpi = 96;

const newDoc = Document.create(opts);
console.log('Created:', newDoc.title);
```

Beyond width/height/dpi: `opts.isFacing`, `opts.isVerticalStack`, `opts.isDoublePageStart` control facing-pages layout; `opts.createArtboard`, `opts.isTransparentBackground`, `opts.isMultiPage`, `opts.createMaster` control artboard mode, transparency, multi-page, and master pages; `opts.marginsEnabled`, `opts.viewDpi`, `opts.imagePlacement`, `opts.colourProfile`, `opts.rasterFormat`, `opts.bleed` refine the setup. Validate sizes up front with `NewDocumentOptions.getMaxDpi(unit, dimension)` and `NewDocumentOptions.getMaxSize(unit, dpi)`.

4.9 Async document operations

For long-running work, use `doc.promises`:

```
'use strict';
const { DocumentPromises } = require('/document.js');
const { app } = require('/application.js');

async function main() {
  const doc = await DocumentPromises.getCurrent();
  if (!doc) return;

  // save the file on the desktop
  const desktop = app.userDesktopPath;
  await doc.saveAs(`${desktop}/output.af`);
  console.log('Saved.');
```

}

`main();`

4.10 Document properties

The `DocumentPreset` class lists all built-in templates. `NewDocumentOptions` lets you customise every detail of a new document, including its drawing scale.

```
'use strict';
const { Document, DocumentPreset, NewDocumentOptions } = require('/document.js');
const { DrawingScale } = require('/drawingscale.js');

// Enumerate all built-in document presets
const presets = DocumentPreset.all;
console.log(`${presets.length} presets available`);
for (const p of presets.slice(0, 5)) {
```



```

    console.log(` ${p.name} - ${p.width}x${p.height} @ ${p.dpi}dpi`);
  }

  // Create a document from a preset
  const a4Preset = presets.find(p => p.name === 'A4');
  if (a4Preset) {
    const doc = Document.createFromPreset(a4Preset, false); // false =
    portrait
    console.log('Created:', doc.title);
  }

  // Create with full options including drawing scale
  const opts = NewDocumentOptions.createDefault();
  opts.width = 297; // mm equivalent in document units
  opts.height = 210; // A4 landscape
  opts.dpi = 300;
  opts.useDrawingScale = true;
  opts.drawingScale = DrawingScale.createFromIntegers(1, 50, true); // 1:50
  scale

  'use strict';
  const { Document } = require('/document.js');

  // doc.getFontNames() - all fonts referenced in the document
  const doc = Document.current;
  if (!doc) { console.log('No document open.')} }
  else {
    const names = doc.getFontNames();
    console.log('Document uses ${names.length} font(s):');
    for (const name of names) console.log(' ', name);
  }
}

```

4.10.1 Drawing scale

Affinity supports architectural and engineering drawing scales (e.g. 1:50, 1:100). Create them with `createFromIntegers`, or fetch the built-in defaults for a given unit system.

```

'use strict';
const { DrawingScale } = require('/drawingscale.js');

// Common scales - createFromIntegers(left, right, simplifyFraction)
const scale = DrawingScale.createFromIntegers(1, 100, true); // 1:100
console.log(scale.toString(false, true, false)); // "1 : 100"
console.log('Left:', scale.leftValue, 'Right:', scale.rightValue);

// Identity (no scale, 1:1)
const identity = DrawingScale.identity;

// Available defaults for a unit - pass a raw integer, NOT UnitType.*
// UnitType.Metres etc. return undefined from JS and cause INVALID_ARGS.
// Use these verified integer constants instead:
const DrawingScaleUnit = {
  Pixels: 5, // screen/pixel defaults
  Mixed: 6, // pixels + imperial
  Points: 7, // typographic points
  Millimetres: 8, // mm (includes km map scales)
  Centimetres: 9, // cm (includes km map scales)
  Picas: 10, // picas
  Inches: 11, // inches (includes ft)
  Yards: 12, // yards
  Feet: 13, // feet
  Kilometres: 14, // kilometres
  Metres: 15, // metres
};

const mmDefaults = DrawingScale.getDefaults(DrawingScaleUnit.Millimetres);
console.log(`${mmDefaults.length} mm presets:');
for (const s of mmDefaults) {

```

```

    console.log(' ', s.toString(false, true, false));
  }
  // Output:
  // 1 : 10
  // 1 : 25
  // 1 : 40
  // 1 : 50
  // 1 : 100
  // 1 : 200
  // 1 : 500
  // 1 mm : 1 cm
  // 1 mm : 1 m
  // 1 : 25000 km
  // 1 : 50000 km

```

① **Note**
Do not pass `UnitType.Millimetres` or any `UnitType.*` value to `getDefaults().UnitType`

is a C++ constructor — its named properties return undefined from JavaScript, which causes

`INVALID_ARGS`. Use the integer constants from the `DrawingScaleUnit` table above instead.

4.10.2 Document snapshots

Snapshots let you bookmark the document state — useful for before/after comparisons or checkpointing during long scripts. Create, list, and restore them via `DocumentCommand`.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Add a named snapshot
  doc.executeCommand(DocumentCommand.createAddDocumentSnapshot('Before changes'));

  // List snapshots
  for (const snap of doc.snapshots) {
    console.log(snap.description, snap.format);
  }

  // Restore the first snapshot
  if (doc.snapshots.length > 0) {
doc.executeCommand(DocumentCommand.createRestoreDocumentSnapshot(doc.snapshots[0]));
  }

  // Which snapshot is active, and where it sits in history
  console.log('Current snapshot:', doc.currentSnapshot?.description);
  console.log('Snapshot index:', doc.currentSnapshotIndex, 'history index:', doc.currentSnapshotHistoryIndex);

  // Restore a snapshot into a brand-new document (keeps the current doc untouched)
  const fresh = doc.currentSnapshot.createDocument();
  console.log('Restored into:', fresh.title);
}

```

`doc.enumerateSnapshots(cb)` walks snapshots with a callback instead of the `doc.snapshots` array. `DocumentSnapshot.createDocumentAsync(cb)` is the async variant of `createDocument()`.

4.10.3 Spread & artboard document properties

Use `SpreadDocumentProperties` and `ArtboardDocumentProperties` to read and write per-artboard settings like margins, and to resize spreads and artboards relative to an anchor point.

```

'use strict';
const { Document } = require('/document.js');
const { ArtboardDocumentProperties, SpreadDocumentProperties } =
  require('/documentproperties.js');

```

```

const doc = Document.current;
const spread = doc.currentSpread;
const artboard = spread.artboardInterface;

// Read / write artboard properties
const props = ArtboardDocumentProperties.create()
  .setUseMargin(true)
  .setMargin({ top: 20, left: 20, bottom: 20, right: 20 });

doc.setArtboardDocumentProperties(artboard, props);

// Resize spread about an anchor
doc.setSpreadSizeWithAnchor(spread, 800, 600, SpatialAnchor.TopLeft);

// Resize artboard about an anchor
doc.setArtboardSizeWithAnchor(artboard, 400, 300, SpatialAnchor.Centre);

```

4.10.4 Guide management

Guides are non-printing alignment lines — horizontal (true) or vertical (false) — positioned at a pixel coordinate. Add, move, remove, and recolour them through convenience methods on doc.

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (doc) {
  // Add guides
  doc.addGuide(true, 200); // horizontal guide at y=200px
  doc.addGuide(false, 400); // vertical guide at x=400px

  // Convenience methods
  // doc.addHorizontalGuide(pixels) / doc.addVerticalGuide(pixels)
  // doc.moveGuide(horizontal, index, newPixels)
  // doc.removeGuide(horizontal, index)
  // doc.setGuidesColour(colour)

  console.log('Guides added.');
```

4.10.5 Document size and page count

Quick-access properties for common document dimensions. `sessionUuid` uniquely identifies this document instance within the Affinity session.

```

// Snippet
const doc = Document.current;
console.log('Width:', doc.widthPixels);
console.log('Height:', doc.heightPixels);
console.log('Page count:', doc.pageCount);
console.log('Spread count:', doc.spreadCount);
console.log('Session UUID:', doc.sessionUuid);

```

4.10.6 Document history

The `doc.history` object exposes undo/redo state, and `doc.undo()` / `doc.redo()` let you step through it without building a command.

```

// Snippet
const doc = Document.current;
const h = doc.history;

console.log('Can undo:', h.canUndo, '-', h.undoDescription);
console.log('Can redo:', h.canRedo, '-', h.redoDescription);
console.log('History size:', h.size);

// Undo/redo via document convenience methods
doc.undo();
doc.redo();

```

4.10.7 Undo and selection convenience methods

The Document class exposes many common operations directly, without needing to build a DocumentCommand manually:

```
// Snippet
const doc = Document.current;
const sel = doc.selection;

doc.undo(); // undo last action
doc.redo(); // redo
doc.selectAll(); // select all
doc.deleteSelection(sel); // delete selected nodes
doc.showSelection(sel); // make visible
doc.hideSelection(sel); // hide
doc.lockSelection(sel); // lock (non-editable)
doc.unlockSelection(sel); // unlock
doc.setLayerDescription('My layer', sel); // rename layer
doc.convertToCurves(sel); // convert shape to curves
doc.rasteriseObjects(sel); // rasterise vector to pixels
doc.flatten(); // flatten all layers
doc.mergeVisible(); // merge visible layers

// Additional convenience methods
doc.setText('Hello', sel); // replace text
doc.formatText(delta, sel); // apply text formatting
doc.insertGlyph(0x2665, sel); // insert ♥ glyph
doc.setDashPhase(0, sel); // dash phase offset
doc.setBalancedDashes(true, sel); // balanced dashes at corners

// Saving and closing
doc.save(); // save in place (fails if
mustSaveAs)
doc.saveAs('/path/to/file.af'); // save to new path
doc.saveAsPackage('/path/to/pkg',
  PackageResourcesPolicy.CopyLinks); // package with linked assets
// doc.close(); // NOT_IMPLEMENTED — always throws
```

PackageResourcesPolicy controls how linked resources are handled when packaging. Import it from /document:

```
// Snippet
const { PackageResourcesPolicy } = require('/document.js');
// Values: CopyLinks (copies linked assets into the package)
```

4.11 Document units and page properties

DocumentCommand.createSetDocumentUnits(sel, units) sets the document’s measurement units. DocumentCommand.createSetPageDocumentProperties(sel, props) sets page-level properties like page colour and margins.

```
// Snippet — assumes sel and doc are defined
const { DocumentCommand } = require('/commands.js');
const { UnitType } = require('/units.js');
doc.executeCommand(DocumentCommand.createSetDocumentUnits(sel,
  UnitType.Millimetre));
```

4.12 Key gotchas

mustSaveAs on new documents — a freshly created document has no path. Calling doc.save() on it will fail. Save to the Desktop with doc.saveAs(desktop + '/file.af').

doc.close() and doc.closeAsync() always throw NOT_IMPLEMENTED. You cannot close a document from a script. If your workflow needs a “save and done” pattern, save the document and let the user close it manually.

① **Note**
setCurrentSpread clears selection — always read selection data before calling this command.

4.13 Exercises

4.13.1 Exercise 2.1 — Document dashboard

Write a script that logs a full summary of the current document: title, path, format, DPI, pixel dimensions, spread count, page count, and whether it needs saving.

4.13.2 Exercise 2.2 — Multi-doc lister

Write a script that lists all open documents and, for each, logs the title and whether it's dirty.

4.13.3 Exercise 2.3 — Spread walker

Iterate every spread in the document. For each spread, log its index and artboard count. (Hint: check `spread.artboardCount`.)

4.13.4 Exercise 2.4 — New document

Create a new document programmatically (1920×1080, 96 DPI), log its title, then save it to the Desktop as `exercise-2-4.af`.

4.14 SDK modules used in this lesson

- `/document` — `Document`, `NewDocumentOptions`, `DocumentPreset`, `DocumentPromises`, `PackageResourcesPolicy`
- `/application` — `app`
- `/drawingscale` — `DrawingScale`
- `/commands` — `DocumentCommand`

4.15 See also

- Lesson 01 — Environment Setup
- Lesson 03 — Node Model
- `examples/`

4.16 Examples

4.16.1 Document dashboard

```
'use strict';

// Example 02a — Document dashboard
// Lesson 02: App & Document
//
// Logs a full summary of the current open document.

const { Document } = require('/document.js');
const { app } = require('/application.js');

console.log('=== Application ===');
console.log('Product:', app.productShortName, app.shortVersion);
console.log('Desktop:', app.userDesktopPath);

const doc = Document.current;

if (!doc) {
  console.log('\nNo document is open. Open a file and run again.');
```

```
} else {
  console.log('\n=== Document ===');
  console.log('Title:      ', doc.title);
  console.log('Path:        ', doc.path || '(unsaved)');
  console.log('Format:      ', doc.format);
  console.log('DPI:         ', doc.dpi);
  console.log('View DPI:    ', doc.viewdpi);
  console.log('Width (px):  ', doc.widthPixels);
  console.log('Height (px): ', doc.heightPixels);
  console.log('Spreads:    ', doc.spreadCount);
  console.log('Pages:      ', doc.pageCount);
```

```

    console.log('Snapshots:   ', doc.snapshotCount);
    console.log('Needs save:  ', doc.needsSaving);
    console.log('Read only:   ', doc.isReadOnly);
    console.log('Must save as:', doc.mustSaveAs);
    console.log('Embedded:    ', doc.isEmbedded);
    console.log('UUID:        ', doc.persistentUuid);
  }
}

```

4.16.2 Spread walker

```

'use strict';

// Example 02b — Spread walker
// Lesson 02: App & Document
//
// Iterates every spread and logs its properties.

const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open. '); }
else {
  console.log(`Document: ${doc.title}`);
  console.log(`Total spreads: ${doc.spreadCount}, pages: ${doc.pageCount}`);
}

let i = 0;
for (const spread of doc.spreads) {
  i++;
  console.log(`\nSpread ${i}:`);
  console.log('  Artboards:', spread.artboardCount);
  console.log('  Has artboards:', spread.artboardCount > 0);
}

console.log(`\nCurrent spread:`, doc.currentSpread);
}

```

Lesson 03 — Node Model & Tree Traversal

5

In Affinity, every object you see in the canvas — every shape, text frame, image, group, and curve — is a node in a tree. This lesson teaches you how that tree is structured, how to walk it, and how to identify what kind of node you are looking at.

Understanding the node tree is the foundation of every non-trivial script. Whether you want to rename all layers, find every text frame, apply an effect to shapes only, or build a custom export pipeline, you need to be able to traverse the tree and identify nodes by type. Without this lesson, most of the rest of the tutorial would be inaccessible.

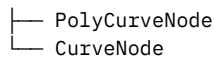
5.1 What this lesson covers

- The node tree structure
- Node type guards (`isShapeNode`, `isTextNode`, etc.)
- Traversing layers recursively
- `/collection` — lazy functional traversal
- Working with selections
- `NodeChildType` — Main vs Enclosure child lists

5.2 The node tree

Every object in an Affinity document is a node. The tree looks like this:

```
DocumentNode (rootNode)
├── SpreadNode
│   ├── GroupNode
│   │   ├── ShapeNode
│   │   └── TextNode
│   ├── RasterNode
│   └── ImageNode
```



Nodes expose typed **interfaces** as properties:

- `node.visibilityInterface` — show/hide
- `node.baseBoxInterface` — bounding box (x, y, width, height)
- `node.transformInterface` — transform matrix (NOT bounding box)
- `node.descriptionInterface` — layer name
- `node.editabilityInterface` — lock/unlock
- `node.blendModeInterface` — blend mode, opacity

5.3 Traversing layers

`doc.layers` iterates top-level nodes across every spread. To descend into group children, use `doc.layers.all`.

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No doc.');
} else {
  // Top-level layers across all spreads (does not descend into groups)
  for (const node of doc.layers) {
    console.log(node.descriptionInterface?.description ?? '(unnamed)');
  }

  // ALL nodes recursively across all spreads
  for (const node of doc.layers.all) {
    console.log(node);
  }
}

```

① **Note**
`doc.layers` covers all spreads but not group children. It returns top-level nodes across every spread. Use `doc.layers.all` to descend into groups. Use `doc.spreads` when you need to know which spread a node belongs to.

5.4 Multi-spread traversal

When you need per-spread context, iterate `doc.spreads` directly and use `getNodeChildren` to access nodes within a specific spread.

```

'use strict';
const { Document } = require('/document.js');
const Nodes = require('/nodes.js');

const doc = Document.current;
if (!doc) return;

let spreadIndex = 0;
for (const spread of doc.spreads) {
  spreadIndex++;
  console.log(`\n— Spread ${spreadIndex} —`);

  for (const node of Nodes.getNodeChildren(spread.handle,
Nodes.NodeChildType.Main, false)) {
    const name = node.descriptionInterface?.description ?? '(unnamed)';
    console.log(' ', name);
  }
}

```

5.5 `/collection` — lazy functional traversal

`doc.layers`, `doc.layers.all`, and the `getNodeChildren*` generators all feed Collection objects — lazy, generator-backed sequences with `map`, `filter`, `take`, `skip`, and `reduce`. Nothing is computed until a terminal call like to `Array()`, `length`, or `first`, so filtering a large tree costs nothing until you actually need the results.

```

'use strict';
const { Document } = require('/document.js');
const { Collection } = require('/collection.js');

```



```

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  const all = doc.layers.all;

  console.log('Total nodes:', all.length);
  console.log('Text nodes: ', all.countIf(n => n.isTextNode));

  const names = all
    .map(n => n.descriptionInterface?.description ?? '(unnamed)')
    .take(5)
    .toArray();

  const pageLabels = Collection.range(1, doc.pageCount)
    .map(n => `Page ${n}`)
    .toArray();
  console.log(pageLabels.join(', '));
}

```

Key methods: `map`, `filter`, `take(n)`, `skip(n)`, `reduce(fn, seed)`, `countIf(pred)`, `some(pred)`, `first`, `last`, `isEmpty`, `length`, `join(sep)`, `toArray`. Static factories: `Collection.range(start, count)`, `Collection.of(...items)`, `Collection.over(seq)`. See the official `adjustPageItems.js` example in phase 5 for `Collection.range` driving a per-page loop.

More chaining ops: `append`, `flat`, `reverse`, `takeWhile` / `skipWhile`, `repeat(n)`, `leftZip` / `rightZip`, `notNull`. More terminals: `all(pred)`, `any(pred)`, `none(pred)`, `forEach(fn)`, `at(i)`, `entries()`, `keys()`. Factories: `Collection.empty()`, `Collection.fibonacci()`, `Collection.random()`.

5.6 Node type guards

Use these boolean properties to identify node types before accessing type-specific APIs:

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  for (const node of doc.layers.all) {
    if (node.isShapeNode) console.log('Shape: ',
node.descriptionInterface?.description);
    if (node.isTextNode) console.log('Text: ',
node.descriptionInterface?.description);
    if (node.isRasterNode) console.log('Raster: ',
node.descriptionInterface?.description);
    if (node.isImageNode) console.log('Image: ',
node.descriptionInterface?.description);
    if (node.isPolyCurveNode) console.log('PolyCurve:',
node.descriptionInterface?.description);
    if (node.isGroupNode) console.log('Group: ',
node.descriptionInterface?.description);
    if (node.isContainerNode) console.log('Container:',
node.descriptionInterface?.description);
    if (node.isCurveNode) console.log('Curve: ',
node.descriptionInterface?.description);
    if (node.isColouredLogicalNode) console.log('Coloured: ',
node.descriptionInterface?.description, `(colour: ${node.layerColour})`);
  }
}

```

Note
Collection methods return new Collections, not arrays. Chain freely, then call `.toArray()` or iterate when you need the values.

Note
`isColouredLogicalNode` detects colour-coded folder groups (set by the user via the Layers panel colour label). Read the colour index via `node.layerColour`.

5.7 Working with the selection

`doc.selection.nodes` gives you the current selection as an iterable. Access each node's interfaces to read its name, bounding box, etc.

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) return;

const sel = doc.selection;
const nodes = sel.nodes;

if (nodes.isEmpty) {
  console.log('Nothing selected.');
```

```
} else {
  console.log(`${nodes.length} node(s) selected: `);
  for (const node of nodes) {
    const name = node.descriptionInterface?.description ?? '(unnamed)';
    const bbox = node.baseBoxInterface?.baseBox; // NOT
transformInterface.baseBox
    console.log(`  ${name} - x:${bbox?.x} y:${bbox?.y} w:${bbox?.width} h:
${bbox?.height}`);
  }
}
```

5.8 Node interfaces reference

Interface	Property on node	What it gives you
Visibility	visibilityInterface	.isVisible, .visibility
Transform	transformInterface	.transform (matrix), .unconstrained Transform
Bounding box	baseBoxInterface	.baseBox (x, y, width, height) — NOT on transformInterface
Spread bounding box	spreadBaseBox (convenience)	.spreadBaseBox — equivalent to get SpreadBaseBox(true)
Description	descriptionInterface	.description (layer name)
Editability	editabilityInterface	.isEditable
Blend mode	blendModeInterface	.blendMode, .opacity
Tag	tagInterface	.tagColour
Artboard	artboardInterface	artboard-specific properties

5.9 NodeChildType

Nodes can have two child lists:

Type	Used for
NodeChildType.Main	Normal layer children

Type	Used for
NodeChildType.Enclosure	Enclosure layers attached to a node (masks, clips) — read via <code>node.enclosures</code>
NodeChildType.MasterSpread / WrapShape / StoryPin	Master-spread, wrap-shape, and story-pin child lists (specialised; values 2/3/4)

Most traversal only needs Main.

5.10 Node navigation shortcuts

```
// Snippet — assumes node is defined
const sel = node.selfSelection;           // one-line Selection for the
node itself
const next = node.nextSibling;           // typed sibling navigation
const prev = node.previousSibling;
const first = node.getFirstChild();      // Main child list by default
const encl = node.enclosures;           // enclosure (mask/clip) child
list
```

`Nodes.getNodeSiblings(handle, reverse)` iterates siblings; `Nodes.getNodesRecursive(handle, childList, reverse)` yields a node plus all its descendants (the self-inclusive variant of `getNodeChildrenRecursive`).

5.11 ContainerNode — named layer groups

`ContainerNode` is a named group type distinct from the anonymous `Group Node`. It appears in the Layers panel as a coloured folder and supports a fill, stroke, and blend mode of its own. The key difference is that `ContainerNode Definition.create(name)` accepts a display name upfront, whereas `Group Node` has no programmatic factory.

```
'use strict';
const { Document } = require('/document.js');
const { ContainerNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Create a named container (coloured layer group)
  const def = ContainerNodeDefinition.create('My Group');
  const builder = AddChildNodesCommandBuilder.create();
  builder.addContainerNode(def);
  const cmd = builder.createCommand(true);
  doc.executeCommand(cmd);

  const container = cmd.newNodes[0];
  console.log('Container created:',
container.descriptionInterface.description);
  console.log('Is container:', container.isContainerNode);
}
```

Use `ContainerNodeDefinition.createDefault()` when you want an unnamed container and will set `def.userDescription` separately.

5.12 Node convenience methods

Every `Node` exposes shorthand methods that wrap the underlying commands:

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
```

```

else {
  const node = doc.selection.nodes.first;
  if (!node) { console.log('Select a node first.')} }
  else {
    // Rename
    node.userDescription = 'Renamed layer';

    // Lock / unlock (sets editability)
    node.lock(); // equivalent to
    DocumentCommand.createSetEditable(sel, false)
    node.unlock();

    // Duplicate - returns the new node
    const copy = node.duplicate();
    console.log('Duplicate created:',
    copy?.descriptionInterface?.description);

    // Delete - use Selection.create(doc, node), then
    doc.deleteSelection()
    // node.delete(); // BROKEN: passes a Node where Selection is
    // expected
    // CORRECT:
    const { Selection } = require('/selections.js');
    const nodeSel = Selection.create(doc, node);
    doc.deleteSelection(nodeSel);
  }
}

```

`node.duplicate()` applies no transform by default, so the copy lands on top of the original. Pass a Transform as the first argument to offset it.

`doc.addNode(nodeDefinition, targetNode)` is a shorthand that wraps `AddChildNodesCommandBuilder` for single-node additions without the full builder setup:

```

'use strict';
const { Document } = require('/document.js');
const { ContainerNodeDefinition } = require('/nodes.js');

const doc = Document.current;
if (doc) {
  const def = ContainerNodeDefinition.create('Quick group');
  doc.addNode(def); // adds to the current spread, top of stack
}

```

5.13 Per-node tag metadata

The SDK lets you store arbitrary string key-value pairs on any node via `DocumentCommand`. These persist inside the `.af` file, making them useful for metadata workflows, asset management systems, and script-to-script communication.

`node.tagInterface` exposes methods for reading tags back at runtime: `hasKey(key)`, `getValueForKey(key)`, `hasPredefinedKey(key)`, and `getValueForPredefinedKey(key)`.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { PredefinedTagKey } = require('/taginterface.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else if (doc.selection.nodes.isEmpty) { console.log('Select a node
first.')} }
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

```

```
// Write a custom tag (persists in the file, adds to undo stack)
doc.executeCommand(
  DocumentCommand.createSetTagValueForKey(sel, 'asset-id', 'IMG-0042')
);
doc.executeCommand(
  DocumentCommand.createSetTagValueForKey(sel, 'approved', 'true')
);

// Write a predefined tag (ScriptLabel, AltTextSource, AltTextTag, etc.)
doc.executeCommand(
  DocumentCommand.createSetTagValueForPredefinedKey(
    sel, PredefinedTagKey.ScriptLabel, 'hero-image')
);

// Read tags back via tagInterface
const tagIface = node.tagInterface;
if (tagIface) {
  console.log('Has asset-id:', tagIface.hasKey('asset-id'));
  console.log('asset-id value:', tagIface.getValueForKey('asset-id'));
  console.log('ScriptLabel:',
tagIface.getValueForPredefinedKey(PredefinedTagKey.ScriptLabel));
  console.log('Is decoration:', tagIface.isMarkAsDecoration);
}
}
```

5.13.1 Use cases

- **Asset management:** tag layers with IDs, status, or project codes that survive export/reimport.
- **Script handoff:** script A tags nodes, external tools read the .af XML to process them.
- **Conditional logic:** write tags that future script runs can detect via other means.
- **Audit trail:** write timestamps or version numbers when a layer was last processed.

5.14 Exploring nodes with `/inspect`

When you encounter an unfamiliar object, `console.log` won't help — most SDK objects are `HandleObject` instances whose state lives behind prototype getters, so `console.log(node)` prints `[Node]` and tells you nothing.

`/inspect` provides a Node.js-compatible `util.inspect` that safely evaluates getters, walks prototype chains, and formats the result as a readable string:

```
'use strict';
const { Document } = require('/document.js');
const { inspect } = require('/inspect.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  console.log(inspect(doc.selection, { depth: 2 }));
}
```

`patchConsole` routes all `console.log` output through `inspect` automatically:

```
'use strict';
const { patchConsole, unpatchConsole } = require('/inspect.js');
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  patchConsole({ depth: 3, getters: true });
  console.log(doc.currentSpread); // auto-formatted
  console.log(doc.selection.nodes.first);
}
```

```
    unpatchConsole();
  }
}
```

Every property in the inspect output corresponds to a getter on the object's prototype. If you see `marginsInterface` on a spread node, that maps directly to the `/marginsinterface` module in the SDK primer — this is the discovery loop: inspect → find property → find module → read the primer.

Three formatting modes for enums:

```
// Snippet — assumes doc is a Document instance
const { inspect } = require('/inspect.js');
const rs = doc.rasterSelection;
inspect(rs.isSelectNone, { enums: 'brief' }); // SelectNone
inspect(rs.isSelectNone, { enums: 'full' }); // RasterSelection.SelectNone
inspect(rs.isSelectNone, { enums: 'raw' }); // the raw enum object
```

Options: `depth` (default 2, null = unlimited), `showHidden`, `getters`, `sorted`, `enums` (`'brief'/'full'/'raw'`), `maxArrayLength`.

5.15 Key gotchas

Null-safe interface access — not every node implements every interface. Always use optional chaining (`node.descriptionInterface?.description`) or check for null first.

Read `doc.selection` before `setCurrentSpread` — switching spreads clears the selection.

① **Note**
`doc.layers` = all spreads, top-level only. It covers every spread but does not descend into groups. Use `doc.spreads` when you need to know which spread a node belongs to.

② **Note**
`doc.layers.all` = all spreads, recursive. Descends into groups and returns every node at any depth across all spreads.

5.16 Exercises

5.16.1 Exercise 3.1 — Layer tree printer

Write a script that prints a tree of all layer names, indented by depth, with the node type shown in brackets.

Expected output:

```
Background (ShapeNode)
  Sky (RasterNode)
  Mountains (GroupNode)
    Peak left (PolyCurveNode)
    Peak right (PolyCurveNode)
  Logo (GroupNode)
  Text (TextNode)
```

5.16.2 Exercise 3.2 — Selection inspector

Write a script that logs the name, type, bounding box, and visibility of every currently selected node.

5.16.3 Exercise 3.3 — Type counter

Write a script that counts how many of each node type exist in the document and logs a summary:

```
ShapeNode:    12
TextNode:     3
RasterNode:   1
GroupNode:    5
```

5.16.4 Exercise 3.4 — Multi-spread walker

For a multi-page document, walk every spread and log how many top-level nodes each spread contains.

5.16.5 Exercise 3.5 — Collection census

Rewrite the type counter from Exercise 3.3 using only Collection methods (countIf, filter, length) — no explicit loops or counters.

5.17 SDK modules used in this lesson

- /document — Document
- /nodes — NodeChildType, ContainerNodeDefinition, getNodeChildren, getNodeChildrenRecursive
- /commands — AddChildNodesCommandBuilder
- /selections — Selection
- /collection — Collection (lazy functional sequences)
- /inspect — inspect, patchConsole, unpatchConsole

5.18 Further examples

Example	What it shows
countSelectedItems.js	Count selected objects by type, including nested children
selectObjects.js	Select objects by type/state through a dialog

5.19 See also

- Lesson 02 — App & Document
- Lesson 04 — Command Pattern
- examples/

5.20 Examples

5.20.1 Layer tree printer

```
'use strict';

// Example 03a — Layer tree printer
// Lesson 03: Node Model
//
// Prints every layer in the document as an indented tree.
// Works across all spreads.

const { Document } = require('/document.js');
const Nodes = require('/nodes.js');

const nodeType = (node) => {
  if (node.isShapeNode)    return 'Shape';
  if (node.isTextNode)    return 'Text';
  if (node.isRasterNode)  return 'Raster';
  if (node.isImageNode)   return 'Image';
  if (node.isPolyCurveNode) return 'PolyCurve';
  if (node.isGroupNode)   return 'Group';
  if (node.isCurveNode)   return 'Curve';
  return 'Node';
};

const printChildren = (parentHandle, depth) => {
  const indent = ' '.repeat(depth);
  for (const node of Nodes.getNodeChildren(parentHandle,
Nodes.NodeChildType.Main, false)) {
    const name = node.descriptionInterface?.description ?? '(unnamed)';
    const type = nodeType(node);
    const visible = node.visibilityInterface?.isVisible !== false ? '' : '
[hidden]';
    console.log(`${indent}${name} (${type})${visible}`);
    printChildren(node.handle, depth + 1);
  }
}
```

```

};

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  console.log(`Layer tree: ${doc.title}\n`);

  let i = 0;
  for (const spread of doc.spreads) {
    i++;
    console.log(`— Spread ${i} —`);
    printChildren(spread.handle, 1);
    console.log('');
  }
}

```

5.20.2 Selection inspector

```

'use strict';

// Example 03b — Selection inspector
// Lesson 03: Node Model
//
// Logs full details of every selected node.

const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const sel = doc.selection;
  const nodes = sel.nodes;

  if (nodes.isEmpty) {
    console.log('Nothing selected. Select some objects and run again.')}
  else {
    console.log(`${nodes.length} node(s) selected:\n`);

    let i = 0;
    for (const node of nodes) {
      i++;
      const name = node.descriptionInterface?.description ??
        '(unnamed)';
      const visible = node.visibilityInterface?.isVisible;
      const bbox = node.baseBoxInterface?.baseBox; // NOT
        transformInterface.baseBox

      console.log(`[${i}] ${name}`);
      if (node.isShapeNode) console.log('    Type: Shape');
      else if (node.isTextNode) console.log('    Type: Text');
      else if (node.isRasterNode) console.log('    Type: Raster');
      else if (node.isImageNode) console.log('    Type: Image');
      else if (node.isGroupNode) console.log('    Type: Group');
      else if (node.isPolyCurveNode) console.log('    Type: PolyCurve');
      else console.log('    Type: (other)');

      console.log('    Visible:', visible);
      if (bbox) {
        console.log(`    Bounds: x=${Math.round(bbox.x)}
y=${Math.round(bbox.y)} w=${Math.round(bbox.width)}
h=${Math.round(bbox.height)}`);
      }
    }
  }
}

```

5.20.3 Collection walk

```

'use strict';

```



```

// Example 03c – Collection walk
// Lesson 03: Node Model
//
// Traverses the node tree with the /collection module's lazy functional
// helpers instead of manual loops and counters.

const { Document } = require('/document.js');
const { Collection } = require('/collection.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const all = doc.layers.all;

  console.log('Total nodes:', all.length);
  console.log('Shape nodes:', all.countIf(n => n.isShapeNode));
  console.log('Text nodes: ', all.countIf(n => n.isTextNode));

  const firstNames = all
    .map(n => n.descriptionInterface?.description ?? '(unnamed)')
    .take(5)
    .toArray();
  console.log('First five layers:', firstNames.join(' | '));

  const pageLabels = Collection.range(1, doc.pageCount)
    .map(n => `Page ${n}`)
    .toArray();
  console.log('Page labels:', pageLabels.join(', '));

  const hidden = all
    .filter(n => !(n.visibilityInterface?.isVisible ?? true))
    .toArray();
  console.log('Hidden layers:', hidden.length);
  hidden.forEach(n => console.log(' ',
n.descriptionInterface?.description ?? '(unnamed)'));
}

```


Phase 2 — Core API

PART

II

Phase 2 – Core API

Phase 1 taught you how to read a document — how to navigate its structure, identify nodes, and understand what is there. Phase 2 teaches you how to change it. This is where scripts stop being inspectors and start being tools.

The three lessons in this phase cover the operations you will reach for in almost every practical script: executing commands, applying visual properties, moving and transforming objects, and creating new vector content. They are ordered deliberately — commands come first because everything else depends on them. You cannot apply a fill, move a node, or create a shape without going through the command system.

What makes this phase demanding is that the SDK rarely lets you do the obvious thing. You cannot set a fill by assigning a property. You cannot move a node by changing its coordinates. You cannot make a line dashed without also changing its type. Each lesson in this phase documents not just the correct patterns but the specific reasons the intuitive approaches fail — because those are the mistakes every developer makes, and knowing them in advance saves significant debugging time.

By the time you complete this phase you will have a working mental model of how Affinity separates reading state from writing state, and why that separation exists. That model will make everything in Phases 3 and 4 considerably easier to reason about.

6.1 The core loop

Every useful script follows this pattern:

1. Get doc → `Document.current`
2. Get nodes → `doc.selection` or `doc.layers.all`
3. Build commands → `DocumentCommand.create*()`
4. Batch into compound → `CompoundCommandBuilder`
5. Execute → `doc.executeCommand()`

Once you internalise this cycle, each subsequent API becomes a question of selecting the appropriate `DocumentCommand.create*()` factory.

6.2 Lessons

Lesson	Topic	Key skill you'll gain
04 — Command pattern	<code>DocumentCommand</code> , <code>CompoundCommandBuilder</code> , <code>undo</code>	Making any change to the document safely
05 — Fills & strokes	<code>FillDescriptor</code> , gradients, <code>LineStyle</code> , dash patterns	Controlling all visual properties of any node
06 — Geometry	<code>Transform</code> , <code>CurveBuilder</code> , bounding boxes, <code>PolyCurve</code> , shapes, QR codes	Moving nodes and drawing vector paths in code

6.3 What to expect

By the end of Phase 2 you should be able to write a script that:

- Applies any fill — solid colour, gradient, or none — to any selected node
- Sets stroke weight, colour, cap style, join style, and dash pattern correctly
- Moves, scales, and rotates nodes using `Transform`
- Draws arbitrary vector paths using `CurveBuilder` and adds them to the document
- Reads a node's bounding box and uses it to compute positions
- Batches multiple changes into a single undoable compound command

You will also understand three of the most counterintuitive SDK behaviours: why `LineStyle.createDefault()` produces a style with zero weight, why setting `style.weight` directly has no effect, and why dash patterns require `LineStyle.Dash` to be set explicitly before they render. These are not bugs — they reflect the SDK's design — but knowing them before you encounter them will save you considerable frustration.

Lesson 04 – The Command Pattern

The most important rule in the Affinity SDK is this: you cannot change anything by setting a property directly. Every modification — moving a node, changing a colour, replacing text — must go through a command that you build and execute. This lesson explains why that is, and how to use the command pattern effectively.

This design gives you undo/redo for free. Every command you execute appears in the undo stack exactly as if the user had made the change manually. It also means scripts are safe — you can build a compound command, inspect it, and only commit it when you are ready. Getting fluent with `DocumentCommand`, `CompoundCommandBuilder`, and `executeCommand` is what separates scripts that feel like first-class Affinity operations from ones that feel like hacks.

7.1 What this lesson covers

- Why every change goes through `executeCommand()`
- `DocumentCommand` static factories
- `CompoundCommandBuilder` — batching changes into one undo step
- `AddChildNodesCommandBuilder` — creating new nodes
- Preview mode
- Async command execution

7.2 The golden rule

This is how Affinity gives you undo/redo for free. Every user action in the UI is a command. Your scripts are no different.

```
// Snippet
// WRONG — direct mutation (won't work, and has no undo)
```

Note
Never mutate nodes directly.
Always build a command
and execute it.

```
node.fill = someColour;

// CORRECT - command pattern (selection first, then fill descriptor)
const cmd = DocumentCommand.createSetBrushFill(selection, fillDescriptor);
doc.executeCommand(cmd);
```

7.3 Basic command execution

Build a command with a DocumentCommand factory, then pass it to doc.executeCommand().

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc) { console.log('No doc.'); }
else {
  const sel = doc.selection;
  if (sel.nodes.isEmpty) { console.log('Select something first.'); }
  else {
    const colour = RGBA8(220, 50, 50, 255); // red
    const fill = FillDescriptor.createSolid(colour);
    const cmd = DocumentCommand.createSetBrushFill(sel, fill);
    doc.executeCommand(cmd);
    console.log('Fill applied.');
  }
}
```

7.4 CompoundCommandBuilder — one undo step for many changes

When your script makes several changes, wrap them in a compound command so the user can undo everything in one step.

```
'use strict';
const { Document } = require('/document.js');
const { CompoundCommandBuilder, DocumentCommand } = require('/commands.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
const nodes = doc.layers.all.filter(n => n.isShapeNode);

const builder = CompoundCommandBuilder.create();

for (const node of nodes) {
  const sel = doc.selection; // or build a selection from node
  const colour = RGBA8(0, 120, 200, 255);
  const fill = FillDescriptor.createSolid(colour);
  const cmd = DocumentCommand.createSetBrushFill(sel, fill);
  builder.addCommand(cmd);
}

// Execute all changes as a single undoable action
doc.executeCommand(builder.createCommand());
console.log('All shapes recoloured. One undo step.');
```

7.5 AddChildNodesCommandBuilder — creating new nodes

To add objects to the document, use AddChildNodesCommandBuilder:

```
'use strict';
const { Document } = require('/document.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const Nodes = require('/nodes.js');

const doc = Document.current;
```



```
// Get a node definition from the SDK (shapes, text, etc.)
// See Lesson 06 (Geometry) — basic shapes section — for ShapeRectangle,
ShapeEllipse, etc.
const builder = AddChildNodesCommandBuilder.create();
builder.addNode(someNodeDefinition);

const cmd = builder.createCommand(
  true, // andSelect — select the new node after
        adding
  Nodes.NodeChildType.Main // which child list to add to
);
doc.executeCommand(cmd);

// Access the newly created node
const newNode = cmd.newNodes[0];
console.log('Created:', newNode.descriptionInterface?.description);
```

Typed add* methods are shorthand for addNode with a specific definition type — same pattern, less setup:

```
// Snippet — assumes builder and definitions are ready
builder.addShapeNode(shapeNodeDef); // shapes
builder.addImageNode(imageNodeDef); // placed images
builder.addTableTextNode(tableTextDef); // tables
builder.addPatternRasterNode(patternDef); // tiled-pattern raster
builder.addExposureAdjustmentRasterNode(exposureDef); //
adjustments...
builder.addGaussianBlurFilterRasterNode(gaussianBlurDef); // ...and live
filters
// ~50 typed adjustment/filter adds follow the same pattern
(BlackAndWhite,
// Posterise, Curves, Vibrance, WhiteBalance, ...): addXRasterNode(xDef).
```

7.6 Reading command results

After executing an add-node command, you can access the newly created nodes:

```
// Snippet — assumes cmd has already been executed via
doc.executeCommand(cmd)
doc.executeCommand(cmd);
const newNodes = cmd.newNodes;
for (const n of newNodes) {
  console.log('New node:', n.descriptionInterface?.description);
}
```

7.7 Preview mode

Preview mode lets you see the effect of a command without committing it to the undo stack. Useful for interactive tools.

```
// Snippet — assumes cmd and doc are already defined
// Apply as preview (not committed to history)
doc.executeCommand(cmd, true);

// Clear all previews
doc.executeCommand(DocumentCommand.createClearPreviews());
```

7.8 Async command execution

For long-running scripts or sequences:

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

else if (doc.selection.nodes.isEmpty) { console.log('Select something first.');
```

```

    }
    else {
      const sel = doc.selection;
      const fill = FillDescriptor.createSolid(rgba8(80, 140, 220, 255));
      const cmd = DocumentCommand.createSetBrushFill(sel, fill);

      doc.executeCommandAsync(cmd, (errorCode) => {
        if (errorCode) {
          console.log('Command failed:', errorCode);
        } else {
          console.log('Done.');
```

```

        }
      });
    }
  }
}

```

Or with `doc.promises` for promise-based workflows:

```

// Snippet — assumes doc is already defined (e.g. const doc =
Document.current)
async function applyAndSave() {
  const docP = doc.promises;
  await docP.executeCommand(cmd);
  // Check mustSaveAs before saving — new documents need saveAs(path)
  first
  if (doc.mustSaveAs) {
    const { app } = require('/application.js');
    await docP.saveAs(app.userDesktopPath + '/' + doc.title + '.af');
  } else {
    await docP.save();
  }
  console.log('Saved.');
```

```

}

```

7.9 Programmatic selection

Use `DocumentCommand.createSetSelection(selection)` to select nodes from a script — replaces the manual selection that a user would make with the Move Tool.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
if (!doc || doc.layers.all.isEmpty) { console.log('No document or no layers.');
```

```

}
else {
  // Select the first top-level layer
  const target = doc.layers.all.first;
  const sel = Selection.create(doc, target);
  doc.executeCommand(DocumentCommand.createSetSelection(sel));
  console.log('Selected:', target.descriptionInterface?.description);
}

```

To select multiple nodes, create a `Selection` that includes them all:

```

// Snippet — assumes doc and several node references exist
const multiSel = Selection.create(doc, node1, node2, node3);
doc.executeCommand(DocumentCommand.createSetSelection(multiSel));

```

7.10 Undo and redo

`doc.undo()` and `doc.redo()` step through the history. `doc.canUndo` and `doc.undoDescription` let you inspect the undo stack first.

```

// Snippet — assumes doc is already defined
// Programmatic undo/redo
doc.undo();
doc.redo();

```

```
// Check what's available
console.log('Can undo:', doc.canUndo);
console.log('Can redo:', doc.canRedo);
console.log('Next undo:', doc.undoDescription);
```

7.11 Common DocumentCommand factories

Factory	What it does
<code>createSetBrushFill(sel, fill)</code>	Set interior fill
<code>createSetPenFill(sel, fill)</code>	Set stroke fill
<code>createSetLineStyle(sel, style)</code>	Set stroke style
<code>createSetVisibility(sel, bool)</code>	Show/hide
<code>createSetEditable(sel, bool)</code>	Lock/unlock — selection first
<code>createDeleteSelection(sel)</code>	Delete nodes
<code>createSelectAll(layerOnly)</code>	Select all
<code>createSetDescription(sel, str)</code>	Rename layer
<code>createSetBlendMode(sel, mode)</code>	Blend mode
<code>createTransform(sel, xf)</code>	Move/scale/rotate
<code>createSetText(sel, text)</code>	Set text content
<code>createSetCurrentSpread(spread)</code>	Change active spread
<code>createSetSelection(sel)</code>	Select nodes programmatically

7.12 Key gotchas

Compound commands = one undo step. If you call `executeCommand()` in a loop, each call creates a separate undo entry. Use `CompoundCommandBuilder` to group them.

`cmd.newNodes` only works after execution. Access the result after `doc.executeCommand(cmd)` returns.

7.13 Exercises

7.13.1 Exercise 4.1 — Batch recolour

Select several shapes. Write a script that sets each one to a different colour using a compound command (one undo step for all changes).

7.13.2 Exercise 4.2 — Undo explorer

Write a script that logs the current undo stack size, makes a visible change, then logs the undo description before calling `doc.undo()`.

7.13.3 Exercise 4.3 — Visibility toggle

Write a script that hides all currently visible nodes in the selection, or shows all hidden ones — toggling their state — in a single compound command.

7.14 SDK modules used in this lesson

- /commands — `DocumentCommand`, `CompoundCommandBuilder`, `AddChildNodesCommandBuilder`
- /document — `Document`
- /nodes — `NodeChildType`

Note
`setCurrentSpread` clears selection. See Lesson 02.

7.15 Further examples

The official Affinity examples show command-based workflows in complete, production-quality scripts:

Example	What it shows
<code>addGuides.js</code>	<code>DocumentCommand.createAddGuide</code> , <code>CompoundCommandBuilder</code> , live-preview dialog
<code>alignToPage.js</code>	<code>DocumentCommand.createSetNodeAlignment</code> , bounding-box computation
<code>adjustPageItems.js</code>	<code>DocumentCommand</code> batching, <code>Collection.range</code> for per-page iteration
<code>setDocumentFormat.js</code>	<code>DocumentCommand.createSetDocumentFormat</code> , <code>DocumentRasterFormat</code>
<code>boldItalics.js</code>	<code>DocumentCommand</code> for text attribute changes

7.16 See also

- Lesson 03 — Node Model
- Lesson 05 — Fills & Strokes
- [examples/](#)

7.17 Examples

7.17.1 Batch recolour (compound command)

```
'use strict';

// Example 04a — Batch recolour (compound command)
// Lesson 04: Command Pattern
//
// Recolours all selected shape nodes to a palette of colours.
// All changes are grouped into one undo step.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { Selection } = require('/selections.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const PALETTE = [
  RGBA8(220, 60, 60, 255), // red
  RGBA8(60, 160, 220, 255), // blue
  RGBA8(80, 200, 120, 255), // green
  RGBA8(240, 160, 40, 255), // amber
  RGBA8(160, 80, 220, 255), // purple
  RGBA8(40, 190, 190, 255), // teal
];

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const shapes = doc.selection.nodes.filter(n => n.isShapeNode);

  if (shapes.isEmpty) {
    console.log('Select some shape nodes and run again.')}
  else {
    const compound = CompoundCommandBuilder.create();
```

```
let i = 0;

for (const node of shapes) {
  const colour = PALETTE[i % PALETTE.length];
  const fill = FillDescriptor.createSolid(colour);
  const sel = Selection.create(doc, node);
  const cmd = DocumentCommand.createSetBrushFill(sel, fill);
  compound.addCommand(cmd);
  i++;
}

doc.executeCommand(compound.createCommand());
console.log(`Recoloured ${i} shape(s) – one undo step.`);
}
```


Lesson 05 — Fills & Strokes

Fills and strokes are the visual language of vector design. This lesson covers how the SDK represents colour, how to construct any fill type — solid, gradient, or none — and how to apply stroke styles including weight, caps, joins, and dash patterns to selected nodes.

Almost every script that does something visible will need to set a fill or a stroke. The SDK's approach is more structured than you might expect: fills are wrapped in a `FillDescriptor`, strokes are described by a `LineStyle`, and partial updates require a `LineStyleMask`. Understanding this structure means you can apply exactly the visual properties you want without accidentally overwriting ones you meant to keep.

8.1 What this lesson covers

- Creating colours with `RGBA8()`
- `FillDescriptor` — the wrapper for any fill type
- Solid, gradient, hatch, and bitmap fills
- Line styles — weight, cap, join, dash patterns
- `LineStyleMask` — partial style updates
- Brush fill (interior) vs pen fill (stroke) vs transparency fill
- Setting opacity and blend mode
- Convenience fill methods on doc directly

8.2 Colours

Create colours with the `RGBA8(r, g, b, a)` convenience function — values are 0–255 integers.

```
'use strict';  
const { RGBA8, Colour } = require('/colours.js');
```

```
// RGBA8(red, green, blue, alpha) – values 0–255
const red    = RGBA8(220, 50, 50, 255);
const blue   = RGBA8(40, 120, 220, 255);
const semiBlack = RGBA8(0, 0, 0, 128);    // 50% opacity
const clear   = RGBA8(0, 0, 0, 0);        // fully transparent

// Alternative – same result
const red2 = RGBA8(220, 50, 50, 255);
```

ⓘ **Note**
Never use `Colour.fromRGBA()` – it does not exist in the SDK. Use `RGBA8()` instead.

8.3 FillDescriptor

Wrap a fill type in `FillDescriptor` so it can be passed to command factories like `createSetBrushFill`.

```
'use strict';
const { FillDescriptor, SolidFill } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

// Solid colour fill
const colour = RGBA8(200, 80, 40, 255);
const fill = FillDescriptor.createSolid(colour);

// No fill (transparent)
const none = FillDescriptor.createNone();
```

8.4 Gradient fills

Create a `GradientFill` from its default black→white state, then switch gradient shape with `cloneWithNewGradientFillType(type)`. Affinity supports four gradient types:

Integer	Type	Description
0	Linear	Straight line from stop to stop
1	Elliptical	Ellipse radiation from centre
2	Radial	Circular radiation from centre
3	Conical	Cone sweeping around a point

Prefer the `GradientFillType` enum over raw integers where available. Values runtime-verified: Linear=0, Elliptical=1, Radial=2, Conical=3 (see SDK-EXPOSURE-MAP.md enum reference). Custom stop colours need a pre-built Gradient from `/colours.js`:

```
// Snippet – assumes RGBA8 is imported
const { Gradient } = require('/colours.js');
const { GradientFill } = require('/fills.js');

const gradient = Gradient.create([
  { colour: RGBA8(255, 0, 0, 255), position: 0 },
  { colour: RGBA8(0, 0, 255, 255), position: 1 },
]);
const gradFill = GradientFill.create(gradient, 0); // 0 = linear

'use strict';
const { GradientFill, FillDescriptor } = require('/fills.js');
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
```



```

if (!doc) { console.log('No document open.')} }
else if (doc.selection.nodes.isEmpty) { console.log('Select something
first.')} }
else {
  const sel = doc.selection;

  // Start from the default gradient (black → white, linear)
  const baseGf = GradientFill.createDefault();

  // Apply a linear gradient (type 0)
  const linearFd = FillDescriptor.create(baseGf, true);
  doc.executeCommand(DocumentCommand.createSetBrushFill(sel, linearFd));
  console.log('Linear gradient applied.');
```

```

  // Switch to radial (type 2) – same gradient, different shape
  const radialGf = baseGf.cloneWithNewGradientFillType(2);
  const radialFd = FillDescriptor.create(radialGf, true);
  doc.executeCommand(DocumentCommand.createSetBrushFill(sel, radialFd));
  console.log('Radial gradient applied.');
```

```

  // Conical (type 3)
  const conicalGf = baseGf.cloneWithNewGradientFillType(3);
  const conicalFd = FillDescriptor.create(conicalGf, true);
  doc.executeCommand(DocumentCommand.createSetBrushFill(sel, conicalFd));
  console.log('Conical gradient applied.');
```

8.4.1 Reading gradient stop data

Inspect the gradient stops — their positions, midpoints, and colours — via `GradientFill.gradient`.

```

'use strict';
const { GradientFill } = require('/fills.js');

const gf = GradientFill.createDefault();
const grad = gf.gradient;

console.log('Stop count:', grad.stopCount);

for (let i = 0; i < grad.stopCount; i++) {
  const stop = grad.getStop(i);
  console.log(`Stop ${i}: position=${stop.position}
midpoint=${stop.midpoint}`);
}
```

The SDK exposes `GradientFill.createDefault()` for a basic black-to-white gradient, and `cloneWithNewGradientFillType(type)` to switch the shape. Fine-grained stop colour modification via `Gradient.getStop(i).colour` is readable but setting stop colours programmatically requires passing a pre-built Gradient object to `GradientFill.create(gradient, type)`, which in turn requires the `colours.js` Gradient class stop-building API.

8.5 Applying fills to selected nodes

Use `createSetBrushFill` for interior fill and `createSetPenFill` for stroke fill — both take the selection first, then the fill.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc || doc.selection.nodes.isEmpty) {
  console.log('Select something first.');
```

Note
Gradient colours are controlled interactively in the Affinity UI gradient editor.

```

const sel    = doc.selection;
const colour = RGBA8(255, 140, 0, 255); // orange
const fill   = FillDescriptor.createSolid(colour);

// Brush fill = interior fill (options object first)
doc.executeCommand(DocumentCommand.createSetBrushFill(sel, fill));

// Pen fill = stroke fill (use a different colour)
const strokeColour = RGBA8(40, 40, 40, 255);
const strokeFill   = FillDescriptor.createSolid(strokeColour);
doc.executeCommand(DocumentCommand.createSetPenFill(sel, strokeFill));
}

```

8.6 Three fill channels

Channel	Command	What it controls
Brush	createSetBrushFill	Interior / body fill
Pen	createSetPenFill	Stroke / outline fill
Transparency	createSetTransparencyFill	Transparency mask fill

8.7 Line styles

To change a line's **weight**, clone the node's current line style via `LineStyle` Interface rather than using `LineStyle.createDefault()` — see the weight note below.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { LineStyleMask } = require('/linestyle.js');

const doc = Document.current;
if (!doc) { console.log('No document open. '); }
else if (doc.selection.nodes.isEmpty) { console.log('Select something first. '); }
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

  // Clone the node's current line style — do NOT use
  // LineStyle.createDefault()
  // because its weight is 0 and the setter is silently ignored.
  // Use LineStyle.createDefaultWithWeight(n) instead if you need a fresh
  // style.
  const currentStyle =
node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
  if (currentStyle) {
    const style = currentStyle.clone();
    style.weight = 3 * (doc.dpi / 72); // 3pt in document pixels
    doc.executeCommand(DocumentCommand.createSetLineStyle(sel, style,
      { lineStyleMask: LineStyleMask.Weight }));
    console.log('Line weight set to 3pt. ');
  }
}
}

```

`createDefault()` returns a style with `weight = 0`, and the `.weight` setter is silently ignored — setting it has no effect. Use `LineStyle.createDefaultWithWeight(n)` if you need a fresh style with a specific weight, or clone the existing style from `node.lineStyleInterface.lineStyleDescriptor.lineStyle` to modify the current weight.

① **Note**
For weight, clone the node's current line style or use `LineStyle.createDefaultWithWeight(n)` — `LineStyle.createDefault()` can't set a usable weight. For cap/join/dash properties (which aren't affected), `createDefault()` is fine.

8.7.1 Converting points to pixels

Affinity works in 96dpi pixels internally. If you want a weight in points (like in the UI):

```
// Snippet – assumes doc is already defined (e.g. const doc =
Document.current)
const weightInPts = 2;
const weightInPx = weightInPts * (doc.dpi / 72);
style.weight = weightInPx;
```

Or use the doc shorthand which handles the pts-to-pixels conversion automatically:

```
'use strict';
const { Document } = require('/document.js');
const { LineStyleMask } = require('/linestyle.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something
first.')}
else {
  // doc.setLineWeightPts converts points to document pixels automatically
  doc.setLineWeightPts(2, doc.selection);
  console.log('Weight set to 2pt.')}
}
```

8.8 Line caps and joins

Set .cap and .join to the desired integer value and apply with LineStyleMask.Cap/LineStyleMask.Join. Because cap/join are independent of weight, LineStyle.createDefault() is fine here — the weight caveat above doesn't apply.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/
commands.js');
const { LineStyle, LineStyleMask } = require('/linestyle.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something
first.')}
else {
  const sel = doc.selection;
  const compound = CompoundCommandBuilder.create();

  // Set round caps
  const capStyle = LineStyle.createDefault();
  // Cap values: 0 = Butt, 1 = Square, 2 = Round
  capStyle.cap = 2;
  compound.addCommand(DocumentCommand.createSetLineStyle(sel, capStyle,
    { lineStyleMask: LineStyleMask.Cap }));

  // Set round joins
  const joinStyle = LineStyle.createDefault();
  // Join values: 0/1 = Miter, 2 = Round, 3 = Bevel
  joinStyle.join = 2;
  compound.addCommand(DocumentCommand.createSetLineStyle(sel, joinStyle,
    { lineStyleMask: LineStyleMask.Join }));

  doc.executeCommand(compound.createCommand());
  console.log('Round caps and joins applied.')}
}
```

8.9 Dash patterns

For dashes to be visible you must set **three things**: the line type to Dash, a non-zero weight, and the dash pattern. Setting only DashPattern stores the values but Affinity continues to render a solid line. Apply all three in one command using `LineStyleMask.All`.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { LineStyleMask, LineType } = require('/linestyle.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;
  const compound = CompoundCommandBuilder.create();

  // Set a visible stroke colour (selection first)
  compound.addCommand(DocumentCommand.createSetPenFill(
    sel, FillDescriptor.createSolid(RGBA8(30, 30, 30, 255))));

  // Clone current style – set weight, type AND dash together, apply with All mask
  const node = sel.nodes.first;
  const currentStyle = node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
  if (currentStyle) {
    const dashStyle = currentStyle.clone();
    dashStyle.weight = 2 * (doc.dpi / 72); // 2pt in document pixels
    dashStyle.type = LineType.Dash; // REQUIRED – renders as solid without this
    dashStyle.dashPattern = [8, 4]; // 8px dash, 4px gap
    dashStyle.hasBalancedDashes = true;

    compound.addCommand(DocumentCommand.createSetLineStyle(sel, dashStyle, {
      lineStyleMask: LineStyleMask.All }));
  }

  doc.executeCommand(compound.createCommand());
  console.log('Dashed stroke applied.')}
}
```

① **Note**
LineStyleMask.Dash is required for dashes to appear.

Without it the pattern is stored but Affinity renders solid. Always use `LineStyleMask.All` when setting type, weight, and pattern together.

8.10 LineStyleMask – partial updates

Apply only specific parts of a line style, leaving the rest unchanged:

```
// Snippet – assumes doc, sel and style are already defined
// Only change the weight – cap, join, dash stay as they are
doc.setLineStyle(style, sel, { lineStyleMask: LineStyleMask.Weight });

// Available masks:
// LineStyleMask.Weight, .Cap, .Join, .DashPattern, .DashPhase,
// .BalancedDashes, .Type, .All
```

8.11 Stroke alignment

Control whether the stroke sits inside, outside, or centred on the path using `doc.setStrokeAlignment()`.

Value	Name	Effect
0	Centre	Stroke sits evenly on both sides of the path (default)
1	Inside	Stroke sits entirely inside the path boundary
2	Outside	Stroke sits entirely outside the path boundary

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { StrokeAlignment, LineStyleMask } = require('/linestyle.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something first.')}
else {
  const sel = doc.selection;
  const style = require('/linestyle.js').LineStyle.createDefault();

  // StrokeAlignment is in /linestyle – not in affinity:dom
  doc.setStrokeAlignment(StrokeAlignment.Centre, sel);
  console.log('Stroke alignment set to Centre.')}
}
```

8.12 Opacity and blend mode

Set overall layer opacity or fill-specific opacity, and apply blend modes like Multiply or Screen.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
const sel = doc.selection;

// Layer opacity (0.0–1.0)
doc.executeCommand(DocumentCommand.createSetOpacity(sel, 0.5));

// Or use the doc convenience method:
doc.setOpacity(0.5, sel);

// Blend mode – BlendMode enum from affinity:blendmode
// doc.setBlendMode(blendMode, setPassthrough, selection)
// common values: Normal, Multiply, Screen, Overlay, SoftLight, HardLight,
// Darken, Lighten

// Fill-level opacity (separate from layer opacity)
doc.setBrushFillOpacity(0.7, sel); // interior fill opacity
doc.setLineFillOpacity(0.5, sel); // stroke fill opacity
```

8.13 Hatch fills

Hatch fills tile a repeating line pattern over the shape. Start from HatchPattern.createDefault(), then build a HatchFill and wrap it in a FillDescriptor.

```
'use strict';
const { HatchFill, FillDescriptor } = require('/fills.js');
const { HatchPattern } = require('/hatch.js');
const { Colour } = require('/colours.js');
```

```

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
const sel = doc.selection;

// Start from the default hatch pattern (customise lines via
appendHatchLine)
const pattern = HatchPattern.createDefault();
const penColor = new Colour(/* ... */);

const hatch = HatchFill.create(
  pattern,          // HatchPattern
  1,                // units (1 = document units)
  penColor,         // line colour
  null,             // background colour (null = transparent)
  1.0               // line weight
);

const fill = FillDescriptor.create(hatch, false, null, null, false);
doc.executeCommand(DocumentCommand.createSetBrushFill(sel, fill));

```

8.14 Bitmap fills

Load a raster image from disk with `Bitmap.loadFromFile`, then wrap it in a `BitmapFill` and apply as a fill pattern.

```

'use strict';
const { BitmapFill, FillDescriptor } = require('/fills.js');
const { Bitmap } = require('/rasterobject.js');
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { app } = require('/application.js');

const doc = Document.current;
const bitmap = Bitmap.loadFromFile(app.userDesktopPath + '/texture.png');

if (bitmap) {
  const bitmapFill = BitmapFill.create(bitmap, 0, 0, false);
  const fill = FillDescriptor.create(bitmapFill, true, null, null, false);
  doc.executeCommand(DocumentCommand.createSetBrushFill(doc.selection, fill));
  console.log('Bitmap fill applied.');
```

8.15 Mesh fills

A mesh fill places a grid of colour nodes over a shape and interpolates smoothly between them — the same technology as the Mesh Gradient tool in the UI. Build a `ColourMesh` with `createFromColour(colour, nodes)`, set each node's position and colour, then wrap in a `MeshFill`.

```

'use strict';
const { ColourMesh, MeshFill, FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc || doc.selection.nodes.isEmpty) { console.log('Select a shape.');
```

```

  }
  else {
    const mesh = ColourMesh.createFromColour(
      RGBA8(255, 255, 255, 255), 4);

    mesh.setNodeColour(0, 0, RGBA8(255, 0, 0, 255)); // top-left: red
    mesh.setNodeColour(4, 0, RGBA8(0, 0, 255, 255)); // top-right: blue
    mesh.setNodeColour(0, 4, RGBA8(0, 255, 0, 255)); // bottom-left:
    green
  }
}

```

```

    mesh.setNodeColour(4, 4, RGBA8(255, 255, 0, 255)); // bottom-right:
    yellow

    const fill = FillDescriptor.create(MeshFill.create(mesh), true);
    doc.executeCommand(DocumentCommand.createSetBrushFill(doc.selection,
    fill));
    console.log('Mesh fill applied.');
```

ColourMesh exposes `setNodePosition(x, y, {x, y})`, `setNodeColour(x, y, colour)`, `setCurveNodePosition(x, y, MeshDirection.X, {x, y})`, and a `.size` property.

8.16 Diffusion fills

Diffusion curves use overlapping geometric curves to define smooth colour regions (like the UI's diffusion-curve fill). Build a `DiffusionCurveSet`, add parametric curves, then wrap in a `DiffusionFill`.

```

'use strict';
const { DiffusionFill, DiffusionCurveSet, DiffusionCurveParametric,
FillDescriptor } = require('/fills.js');
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc || doc.selection.nodes.isEmpty) { console.log('Select a
shape.');
```

Supporting enums:

Enum	Values
DiffusionCurveKind	Bezier, Ellipse, Arc, Line
DiffusionCurveSide	Left, Right

8.17 Doc convenience fill methods

Document exposes shorthand methods that accept a `Colour`, `SolidFill`, or `FillDescriptor` directly — no need to call `DocumentCommand.createSetBrushFill` explicitly:

```

// Snippet
const doc = Document.current;
const sel = doc.selection;

// All three accept Colour, SolidFill, or FillDescriptor
doc.setBrushFillDescriptor(myColour, sel); // interior fill
doc.setPenFillDescriptor(myFillDescriptor, sel); // stroke fill
doc.setTransparencyFillDescriptor(myFill, sel); // transparency

// Line style convenience — no LineStyle object needed
doc.setLineWeight(4, sel); // weight in pixels
doc.setLineWeightPts(2, sel); // weight in points (auto-converts to
pixels)
doc.setLineCap(LineCap.Round, sel);
doc.setLineJoin(LineJoin.Round, sel);
doc.setStrokeAlignment(StrokeAlignment.Inside, sel);
```

```
// Dash enhancements
doc.setDashPhase(0, sel);           // dash phase offset
doc.setBalancedDashes(true, sel);   // balanced dashes at corners
```

8.18 VectorNodeDefinition — multi-fill and multi-stroke

When creating vector nodes programmatically, VectorNodeDefinition supports stacking multiple fill descriptors and line descriptor pairs. This is how Affinity represents nodes that have more than one fill (e.g. a shape with both a solid fill and a gradient fill layered).

```
'use strict';
const { Document } = require('/document.js');
const { PolyCurveNodeDefinition, VectorNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { FillDescriptor, SolidFill, GradientFill, NoFill } = require('/fills.js');
const { LineStyle, LineStyleDescriptor } = require('/linestyle.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (doc) {
  const def = PolyCurveNodeDefinition.createDefault();

  // --- Multiple brush fills ---
  const solid = FillDescriptor.createSolid(rgba8(200, 80, 40, 255));
  const gradient = FillDescriptor.create(GradientFill.createDefault(), true);
  def.addBrushFillDescriptor(solid); // fill slot 0
  def.addBrushFillDescriptor(gradient); // fill slot 1

  console.log('Fill count:', def.brushFillDescriptorCount); // 2
  def.currentBrushFillIndex = 0; // which fill is active when rendering

  // Replace or remove a fill slot
  // def.setBrushFillDescriptor(newFillDescriptor, 0); // fill, then
  index
  // def.removeBrushFillDescriptor(1);

  // --- Multiple line (stroke) descriptor pairs ---
  const ls = LineStyle.createDefault();
  ls.weight = 2;
  const lsd = LineStyleDescriptor.createDefault(ls);
  const lineFill = FillDescriptor.createSolid(rgba8(40, 40, 40, 255));
  def.addLineDescriptors(lineFill, lsd); // stroke slot 0

  console.log('Stroke count:', def.lineDescriptorsCount); // 1
}
```

For most scripts you will only use one fill and one stroke — addBrushFillDescriptor and addLineDescriptors with a single call each. The multi-fill API is primarily useful when reproducing a node's visual state exactly from an existing document.

8.19 Fluent node-definition builders

Node-definition property setters return the definition, enabling fluent chains:

```
// Snippet
const def = PolyCurveNodeDefinition.createDefault()
  .setUserDescription('My Shape')
  .setTransform(someTransform);

const adjDef =
  BrightnessContrastAdjustmentRasterNodeDefinition.create(params)
    .setParameters(params)
    .setUserDescription('Brightness fix')
    .setTransform(xf);
```


8.20 Colour accepted where FillDescriptor expected

VectorNodeDefinition fill/line methods accept a Colour directly — a solid fill is created automatically:

```
// Snippet
const def = PolyCurveNodeDefinition.createDefault();
const { RGBA8 } = require('/colours.js');

def.addBrushFillDescriptor(RGBA8(200, 80, 40, 255));
def.setBrushFillDescriptor(0, RGBA8(100, 180, 60, 255));
```

LineStyle.Dash is required for dashes to render. Setting dashPattern alone stores the values but Affinity continues to render a solid line. Always set style.type = LineType.Dash and apply with LineStyleMask.All.

createSetBrushFill and createSetPenFill take selection first. The signature is createSetBrushFill(selection, fill, options) — selection first, then the fill (a FillDescriptor, SolidFill, or Colour), then an optional options object. Use createSetBrushFill(sel, myFill).

copyTransform option in createSetBrushFill — when true (default), gradient fill transforms are copied from the selection's bounding box. Set to false if you want the gradient anchored independently.

8.21 Exercises

8.21.1 Exercise 5.1 — Solid fill batch

Select several shapes. Apply a different solid colour from a predefined palette to each one.

8.21.2 Exercise 5.2 — Stroke styler

Write a script that gives every selected node a 2pt round-capped dashed stroke, without changing their fill colours. Use LineStyleMask to target only the relevant properties.

8.21.3 Exercise 5.3 — Opacity fade

Apply decreasing opacity to each node in the selection (100%, 80%, 60%, 40%, 20%) using setTransparencyFillDescriptor.

8.22 SDK modules used in this lesson

- /colours — Colour, RGBA8
- /fills — FillDescriptor, SolidFill, GradientFill, HatchFill, Bitmap Fill, MeshFill, DiffusionFill, NoFill
- /hatch — HatchPattern (for hatch fills)
- /linestyle — LineStyle, LineStyleMask, LineType, StrokeAlignment
- /commands — DocumentCommand, CompoundCommandBuilder
- /document — Document

8.23 Further examples

Example	What it shows
meshBubble.js	ColourMesh, MeshFill, FillDescriptor.create, createSetBrushFill — full mesh-gradient workflow
arrowheads.js	ArrowHead, ArrowHeadStyle, LineStyleDescriptor.cloneWithNew ArrowHeads

Note
LineStyle.createDefault() has weight = 0 and the setter is silently ignored. Use LineStyle.createDefaultWithWeight(n) for a fresh style with a specific weight.

Example	What it shows
strokesWeightDown.js	Decrease stroke weight across a selection with <code>LineStyle</code> / <code>LineStyleMask</code>
strokesWeightUp.js	Increase stroke weight across a selection with <code>LineStyle</code> / <code>LineStyleMask</code>

8.24 See also

- Lesson 04 — Command Pattern
- Lesson 06 — Geometry
- examples/

8.25 Examples

8.25.1 Fills and strokes

```
'use strict';

// Example 05a — Fills and strokes
// Lesson 05: Fills & Strokes
//
// Applies a solid fill and a visible dashed stroke to all selected nodes.
//
// KEY INSIGHTS:
// 1. Clone the node's CURRENT line style — LineStyle.createDefault() has
//    weight=0 and the weight setter is silently ignored.
// 2. Set style.type = LineType.Dash — without this the dash pattern is
//    stored but Affinity renders a solid line.
// 3. Apply weight + type + dash in one command using LineStyleMask.All.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { Selection } = require('/selections.js');
const { FillDescriptor } = require('/fills.js');
const { LineStyle, LineStyleMask, LineType } = require('/linestyle.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.');
```

```
  } else {
    const compound = CompoundCommandBuilder.create();

    for (const node of doc.selection.nodes) {
      const nodeSel = Selection.create(doc, node);

      // 1. Solid orange fill (selection first)
      compound.addCommand(DocumentCommand.createSetBrushFill(
        nodeSel, FillDescriptor.createSolid(rgba8(255, 140, 0, 255))));

      // 2. Dark stroke colour (selection first)
      compound.addCommand(DocumentCommand.createSetPenFill(
        nodeSel, FillDescriptor.createSolid(rgba8(30, 30, 30, 255))));

      // 3. Stroke: clone current style, set weight + type + dash, apply all
      // at once
      const currentStyle =
node.lineStyleInterface?.lineStyleDescriptor?.lineStyle;
      if (currentStyle) {
        const strokeStyle = currentStyle.clone();
```

```
        strokeStyle.weight          = 2 * (doc.dpi / 72); // 2pt in document
pixels
        strokeStyle.type            = LineType.Dash;      // REQUIRED for
dashes to show
        strokeStyle.dashPattern     = [6, 3];             // 6px dash, 3px
gap
        strokeStyle.hasBalancedDashes = true;

        compound.addCommand(DocumentCommand.createSetLineStyle(nodeSel,
strokeStyle,
            { lineStyleMask: LineStyleMask.All }));
    }
}

doc.executeCommand(compound.createCommand());
console.log('Fill and dashed stroke applied.');
```


Lesson 06 — Geometry, Transforms & Curves

This lesson covers the geometric side of the SDK — how to move, scale, and rotate nodes using transforms, how to draw vector paths programmatically with `CurveBuilder`, and how to read a node's bounding box. It also covers the coordinate system all SDK measurements use.

Geometry operations unlock a whole class of scripts that go beyond simple property changes: grid layouts, pattern generators, custom shape creation, alignment tools, and anything that needs to know where things are on the canvas. The `Transform` class and `CurveBuilder` are the two workhorses here — once you are comfortable with them you can build almost any spatial manipulation imaginable.

9.1 What this lesson covers

- Adding basic shapes — rectangles, ellipses, polygons, stars, triangles
- Shape parameters — corner radius, number of sides, star inner radius
- The full parametric shape inventory (cogs, spirals, hearts, callouts...)
- QR codes and their payload (the data the code encodes) types
- `Transform` — moving, scaling, rotating nodes
- `CurveBuilder` — programmatically drawing vector paths, beziers and arcs
- `PolyCurve` — multi-subpath curves
- Unit conversions (document pixels vs points)
- Duplicating nodes in a grid layout
- Converting shapes to curves and rasterising objects
- Knife cut and scissor cut
- Boolean operations and dividing shapes
- Reading back a boolean compound operation
- Adding artboards

- TransformBuilder and combining bounding boxes

9.2 Coordinate system

The SDK works in **document pixels** at 96dpi. All coordinates and sizes are in this unit.

To convert from points (as shown in the UI):

```
// Snippet – assumes doc is already defined (e.g. const doc =
Document.current)
const PTS_TO_PX = doc.dpi / 72;
const tenPtInPx = 10 * PTS_TO_PX;
```

9.3 Transforms — moving and resizing nodes

Build a Transform via static factories like createTranslate, then pass it to DocumentCommand.createTransform().

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Transform } = require('/geometry.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select something
first.')}
else {
  const sel = doc.selection;

  // Pattern 1: instance method – createIdentity, then mutate
  const xf = Transform.createIdentity();
  xf.translate(100, 50); // translate(x, y) – mutates in place
  doc.executeCommand(DocumentCommand.createTransform(sel, xf));

  // Pattern 2: static constructor (most concise)
  const xf2 = Transform.createTranslate(100, 50);
  doc.executeCommand(DocumentCommand.createTransform(sel, xf2));

  console.log('Nodes translated.')}
}
```

9.3.1 Transform properties and chainable methods

Access matrix components via .xAxis, .yAxis, .origin, and chain mutating calls like .translate(), .rotate(), .scale().

```
// Snippet – assumes xf is a Transform
xf.xAxis; // Vector – first column of the matrix
xf.yAxis; // Vector – second column
xf.origin; // Point – translation component
xf.inverted; // Transform – non-mutating inverse

// Chainable mutating methods
xf.translate(50, 30)
  .rotate(Math.PI / 4)
  .scale(2, 1.5);

// Non-mutating variants return a new Transform
const scaled = xf.scaled(2); // new Transform, xf unchanged
const rotated = xf.rotated(Math.PI / 4);
const combined = xf.translated(50, 0);

// around(originX, originY) – transform about a point, then translate
back.
xf.around(100, 100).rotate(Math.PI / 4);

// Compose with another transform
const a = Transform.createTranslate(100, 0);
const b = Transform.createRotate(Math.PI / 4);
```

```

const c = a.multiply(b);    // a * b — new Transform
a.postmultiplyBy(b);        // a = a * b — mutate a in place
a.premultiplyBy(b);        // a = b * a — mutate a in place

// Static multiply
const d = Transform.multiply(a, b);    // same as a.multiply(b)

// Decompose into components
const data = xf.decompose();
console.log(data.translation, data.rotation, data.scaling, data.shear);

```

9.4 Adding basic shapes

Shapes are created using static `.create()` factory methods from the `/shapes` module — never with `new`. Each shape type has its own class: `ShapeRectangle`, `ShapeEllipse`, `ShapePolygon`, `ShapeTriangle`, `ShapeStar`, and more.

The pattern is always the same: create the shape, wrap it in a `ShapeNode Definition`, set its bounding rectangle, then add it to the document.

```

'use strict';
const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { ShapeRectangle, ShapeEllipse, ShapePolygon, ShapeTriangle,
  ShapeStar } = require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  function addShape(shape, x, y, w, h) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    doc.executeCommand(builder.createCommand(true));
  }

  addShape(ShapeRectangle.create(), 100, 100, 200, 150); // rectangle
  addShape(ShapeEllipse.create(), 350, 100, 200, 150); // ellipse
  addShape(ShapePolygon.create(), 600, 100, 200, 200); // hexagon
  (default 6 sides)
  addShape(ShapeTriangle.create(), 100, 300, 200, 200); // triangle
  addShape(ShapeStar.create(), 350, 300, 200, 200); // star

  console.log('Shapes added.')}
}

```

9.5 Shape parameters

Once a shape is on the canvas, its parameters can be modified using `Document Command`. Select the shape first or use a per-node selection:

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select a shape first.')}
else {
  const sel = doc.selection;

  // Corner radius on a rectangle (0 = sharp corners)
  doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
    'CornerRadius', 20));
}

```

Note
ShapeRectangle.create()
not new ShapeRectangle().
 All shape classes require a native handle — instantiating with `new` throws `Invalid handle`. Always use the static `.create()` factory method.

```
// Number of sides on a polygon (integer param)
doc.executeCommand(DocumentCommand.createSetShapeIntParam(sel,
'NumberOfPoints', 3));

// Star inner radius – controls how pointy the star is (0.0–1.0)
doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
'InnerRadius', 0.3));
}
```

Type	Params
Float	CornerRadius, InnerRadius, sweep, startAngle
Int	NumberOfPoints

9.6 Parametric shapes – the full inventory

/shapes exposes 24 shape classes, far beyond the five basics above. Each is created with its static `.create()` factory and configured through properties before being wrapped in a `ShapeNodeDefinition`:

Class	Tunable properties
ShapeArrow	leftEnd, rightEnd, thickness
ShapeCalloutEllipse / ShapeCalloutRectangle	tail, absoluteSizes, corner radius (rectangle)
ShapeCat ... ShapeCat4	(no parameters)
ShapeCloud	bubbles, innerRadius
ShapeCog	teeth, innerRadius, holeRadius, toothSize, notchSize, curvature
ShapeCrescent	leftArc, rightArc
ShapeDiamond / ShapeTriangle	position
ShapeDoubleStar	points, innerRadius, pointRadius
ShapeEllipse / ShapeRectangle	rectangle corners via topLeft...bottomRight
ShapeHeart	spread
ShapePie	startAngle, endAngle, sweep, innerRadius, closePie()
ShapePolygon	sides, smoothPoints
ShapeSegment	angle, lowerLine, upperLine
ShapeSpiral	turns, partTurn, decay, style, cusped
ShapeSquareStar	sides, cutout
ShapeStar	points, innerRadius, curvedEdges
ShapeTear	ball, curve, bend, tailPosition
ShapeTrapezoid	left, right (via setPositions)
ShapeQRCode	payload – see below

The generic factory `Shape.create(ShapeType.X)` also works and returns the correctly-typed subclass – useful when the shape type is chosen at runtime.

Worked property values for the most common shapes:

```
// Snippet – assumes the Shape* classes are imported from /shapes.js
const star = ShapeStar.create();
star.innerRadius = 0.4;           // 0-1 as fraction of outer
star.points = 5;
star.leftCurve = 0.5;
star.rightCurve = 0.5;

const rect = ShapeRectangle.create();
rect.topLeft.setRadius(10, 200, 200);
rect.topRight.setRadius(10, 200, 200);
rect.useSingleRadius = false;    // true = all corners equal

const arrow = ShapeArrow.create();
arrow.leftEnd = ShapeArrowEndStyle.None;
arrow.rightEnd = ShapeArrowEndStyle.Arrow;
arrow.thickness = 0.3;

const pie = ShapePie.create();
pie.innerRadius = 0.2;
pie.startAngle = 0;
pie.sweep = 270;                 // degrees
pie.isClosed = true;

'use strict';
const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { ShapeCog, ShapeHeart, ShapeSpiral, ShapeQRCode, QRPayloadURL } =
  require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  function addShape(shape, x, y, w, h, name) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.userDescription = name;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    doc.executeCommand(builder.createCommand(true));
  }

  const cog = ShapeCog.create();
  cog.teeth = 12;
  addShape(cog, 100, 100, 150, 150, 'Cog');

  const heart = ShapeHeart.create();
  heart.spread = 0.6;
  addShape(heart, 300, 100, 150, 150, 'Heart');

  const spiral = ShapeSpiral.create();
  spiral.turns = 4;
  addShape(spiral, 500, 100, 150, 150, 'Spiral');

  console.log('Shapes added.')}
}
```

9.7 QR codes

ShapeQRCode is a parametric shape like any other — the difference is its payload, a QRPayload subclass describing what the code encodes. Ten payload types ship with the SDK: QRPayloadURL, QRPayloadWifi, QRPayloadVCard, QRPayloadEmail, QRPayloadPhone, QRPayloadSMS, QRPayloadText, QRPayloadLocation, QRPayloadFaceTime, and QRPayloadWhatsApp.

Note
Set shape properties before adding the node. Property changes on a shape object are applied when the node is created; changing them afterwards requires createSet Shape*Param commands (see Shape parameters).

```

'use strict';
const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { ShapeQRCode, QRPayloadURL, QRPayloadWifi, WifiEncryptionType } =
require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  function addQR(payload, x, y, name) {
    const qr = ShapeQRCode.create();
    qr.payload = payload;
    const def = ShapeNodeDefinition.createDefault();
    def.shape = qr;
    def.userDescription = name;
    def.setBoundingBox({ x, y, width: 150, height: 150 });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    doc.executeCommand(builder.createCommand(true));
  }

  const url = QRPayloadURL.create();
  url.url = 'https://example.com';
  addQR(url, 100, 300, 'Website QR');

  const wifi = QRPayloadWifi.create();
  wifi.ssid = 'Studio-5G';
  wifi.password = 'hunter2';
  wifi.encryption = WifiEncryptionType.WPA2;
  addQR(wifi, 300, 300, 'Wifi QR');

  console.log('QR codes added.')}
}

```

`QRPayload.parse(payloadString)` decodes a payload string back into a typed object, and `isPayloadValid` reports whether the fields are complete enough to encode. Payload data is baked into the node — changing the URL later means replacing the payload and re-applying via `shapeInterface.setShape()`.

9.7.1 QR payload types

Class	Factory	Key fields
URL	<code>QRPayloadURL.create(url)</code>	url
VCARD	<code>QRPayloadVCARD.create()</code>	name, phone, email, company, address, birthday, etc.
Email	<code>QRPayloadEmail.create(address, subject, body)</code>	address, subject, body
SMS	<code>QRPayloadSMS.create(number, content)</code>	number, content
Phone	<code>QRPayloadPhone.create(number)</code>	number

Class	Factory	Key fields
WiFi	QRPayload Wifi.create(ssid, password, encryp- tion)	ssid, password, en- ryption (open/wpa2/ wpa3)
Location	QRPayloadLocation.create(latitude, long, elevation)	latitude, longitude, ele- vation
FaceTime	QRPayloadFace Time.create(recipi- ent)	recipient
WhatsApp	QRPayloadWhats App.create(number, content)	number, content
Text	QRPayload Text.create(text)	text

QRPayloadType identifies the payload kind (payload.payloadType also accepted by QRPayload.create(type)). ShapeMajorAxis picks a shape's major axis (shape.majorAxis); ShapeSpiralStyle selects the spiral rendering style.

9.7.2 VCard fields

```
// Snippet – assumes QRPayloadVCard is available
const vcard = QRPayloadVCard.create();
vcard.name = 'Jane Doe';
vcard.firstName = 'Jane';
vcard.lastName = 'Doe';
vcard.email = 'jane@example.com';
vcard.phone = '+1 555 0100';
vcard.company = 'Acme Corp';
vcard.title = 'Engineer';
vcard.address = '123 Main St, City';
vcard.city = 'City';
vcard.postcode = '12345';
vcard.country = 'Country';
vcard.birthday = '1990-01-15';
```

9.8 Building curves with CurveBuilder

Use CurveBuilder to draw vector paths programmatically – beginXY(), lineToXY(), then createCurve().

```
'use strict';
const { Document } = require('/document.js');
const { CurveBuilder, PolyCurve } = require('/geometry.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { PolyCurveNodeDefinition } = require('/nodes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  // Build a triangle
  const builder = CurveBuilder.create();
  builder.beginXY(300, 100); // NOT begin({x,y}) – use beginXY
  builder.lineToXY(500, 300); // NOT addLineXY – use lineToXY
  builder.lineToXY(100, 300);
  builder.close();

  const curve = builder.createCurve();
  const poly = PolyCurve.create();
  poly.addCurve(curve);

  // Use createDefault() not create() – create() requires arguments
```

```

const nodeDef = PolyCurveNodeDefinition.createDefault();
nodeDef.polyCurve = poly;

const addBuilder = AddChildNodesCommandBuilder.create();
addBuilder.addPolyCurveNode(nodeDef);
doc.executeCommand(addBuilder.createCommand(true));
console.log('Triangle added.');
```

9.9 Bezier curves

For curved segments, use `addBezierXY(cp1x, cp1y, cp2x, cp2y, endX, endY)`.

```

'use strict';
const { Document } = require('/document.js');
const { CurveBuilder, PolyCurve } = require('/geometry.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { PolyCurveNodeDefinition } = require('/nodes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const builder = CurveBuilder.create();
  builder.beginXY(100, 200);
  builder.addBezierXY(
    150, 100, // control point 1
    250, 100, // control point 2
    300, 200 // end point
  );
  builder.close();

  const curve = builder.createCurve();
  const poly = PolyCurve.create();
  poly.addCurve(curve);

  const nodeDef = PolyCurveNodeDefinition.createDefault();
  nodeDef.polyCurve = poly;

  const addBuilder = AddChildNodesCommandBuilder.create();
  addBuilder.addPolyCurveNode(nodeDef);
  doc.executeCommand(addBuilder.createCommand(true));
  console.log('Bezier curve added.');
```

9.10 Bulge / versine arcs

`bulgeToXY(x, y, bulge)` appends an arc segment described by its bulge factor (positive bulges one way, negative the other — the sweep angle is $4 \cdot \text{atan}(\text{bulge})$). `versineTo(end, versine)` is the equivalent sagitta-based form (sagitta = perpendicular distance from the chord midpoint to the arc). Both forms are supported.

```

// Snippet — assumes builder already begun with beginXY
builder.bulgeToXY(100, 0, 0.5); // 90° arc, bulge 0.5
builder.versineTo({x:200, y:0}, 50); // arc with 50px sagitta
```

9.11 Grid duplicator

A common pattern — take a shape and tile it into a grid:

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { Transform } = require('/geometry.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
```

```

const sel = doc.selection;
const node = sel.nodes.first;
if (!node) { console.log('Select one shape.');
```

9.12 Converting a shape to curves

Turn a parametric shape (rectangle, ellipse, etc.) into raw editable vector curves.

```

// Snippet
// Via DocumentCommand
doc.executeCommand(DocumentCommand.createConvertToCurves(sel));

// Or via doc convenience method
doc.convertToCurves(sel);
```

9.13 Rasterising objects

Convert vector layers into pixel rasters, optionally clipping to spread bounds.

```

// Snippet
// Convert vector to raster pixels
doc.rasteriseObjects(sel);

// rasteriseContentsOnly = true - rasterise contents but keep layer
// structure
// clipToSpread = true - clip to spread bounds
doc.rasteriseObjects(sel, false, true);
```

9.14 Adding artboards

Use doc.addRectangularArtboard(rect) to add a new artboard at a given position and size.

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (doc) {
  // Add a rectangular artboard at a specific position
  doc.addRectangularArtboard({ x: 0, y: 0, width: 1920, height: 1080 });
```

```
    console.log('Artboard added.');
```

```
}
```

9.15 Knife cut and scissor cut

Split nodes along a curve (knife) or at a parameterised point (scissor).

```
// Snippet – assumes doc, sel, and cutCurve are already defined
// Knife cut along a polyCurve
doc.executeCommand(DocumentCommand.createKnifeCut(cutCurve, sel));

// Scissor – split a curve at a specific point
doc.executeCommand(DocumentCommand.createScissorCut(node, polyCurveIndex,
curveIndex, true, 0.5));
```

9.16 Boolean operations

Combine or divide overlapping vector nodes. Union merges into one shape, subtract cuts the top shape out of the one below, intersect keeps only the overlap, and xor keeps everything except the overlap. `divideShapes` splits the selection into separate pieces along every intersection.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc || doc.selection.nodes.length < 2) {
  console.log('Select 2+ overlapping vector nodes first.');
```

```
} else {
```

```
  const sel = doc.selection;
```

```
  // Merge into a single shape
```

```
  doc.executeCommand(DocumentCommand.createBoolOpUnion(sel));
```

```
  // Or via doc convenience methods
```

```
  // doc.boolOpUnion();      doc.boolOpSubtract();
```

```
  // doc.boolOpIntersect(); doc.boolOpXor(); doc.divideShapes();
```

```
  console.log('Boolean operation applied.');
```

```
}
```

① **Note**
Boolean ops need 2+ overlapping vector nodes. They operate on the current selection; a single node is a no-op. The result is a new merged/trimmed curve.

9.16.1 Reading back a boolean result

A node produced by a boolean operation keeps a reference to the operation it came from. Read it via `node.compoundOperationInterface.compoundOperation` — useful for detecting which layers are live boolean groups before destructive edits like `convertToCurves`:

```
// Snippet – assumes node is a vector node
const op = node.compoundOperationInterface?.compoundOperation;
if (op) console.log('Live boolean compound:', op);
```

9.17 TransformBuilder

A fluent builder that accumulates transforms in reading order:

```
// Snippet
const { TransformBuilder } = require('/geometry.js');

const tb = new TransformBuilder();
tb.translateXY(100, 50) // move right 100, down 50
  .rotate(Math.PI / 4) // rotate 45°
  .scaleXY(2, 1.5);    // scale 2x horizontally, 1.5x vertically
const finalXf = tb.transform; // built transform
```

Each builder method appends to the accumulated transform. Use `.transform` to retrieve the result.

9.18 Key gotchas

node.baseBoxInterface?.baseBox not node.transformInterface?.baseBox. transformInterface.baseBox returns undefined — the bounding box (x, y, width, height) lives on baseBoxInterface.baseBox. transformInterface provides the transform matrix, not the box.

xf.translate(x, y) not xf.translateBy(x, y). translateBy does not exist — the correct instance method is translate. Prefer the static constructors: Transform.createTranslate(x, y), Transform.createRotate(angle), Transform.createScale(x, y). Use Transform.createIdentity() to get a clean transform — makeIdentity() is safe but less idiomatic.

builder.lineToXY not builder.addLineXY. The method to draw a straight line segment is lineToXY(x, y). Similarly, beginXY(x, y) not begin({x, y}) — the XY variant takes two numbers, not an object.

createSetCurves replaces the entire polycurve. Build the full replacement, including any subpaths you want to keep.

Geometry types are proper wrappers. Rectangle, Point, Vector, Size, and similar geometry structs are returned as documented types — not anonymous objects. All expected properties (.x, .y, .width, .height, etc.) behave identically.

9.19 Exercises

9.19.1 Exercise 6.1 — Shape creator

Write a script that creates a regular hexagon at a given position using Curve Builder.

9.19.2 Exercise 6.2 — Grid tiler

Select one shape. Generate a 5×3 grid of copies with 10px gaps. Use a compound command.

9.19.3 Exercise 6.3 — Corner rounder

For every selected rectangle, set its corner radius to 12px using setShape FloatParam.

9.19.4 Exercise 6.4 — Spiral

Use CurveBuilder and bezier segments to draw an approximate Archimedean spiral.

9.19.5 Exercise 6.5 — Wifi QR generator

Build a dialog asking for a network name, password, and encryption type, then create a ShapeQRCode with a matching QRPayloadWifi payload.

9.20 Combining bounding boxes

The /geometry module exports unionRects — combines two Rectangle objects into their union:

```
'use strict';
const { Document } = require('/document.js');
const { unionRects } = require('/geometry.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select at least one node.')}
else {
  let combined = null;
```

Note
All coordinates are in document pixels at 96dpi. The UI shows points — convert with pts * (doc.dpi / 72).

Note
duplicateNodes: true in createTransform — without this, the transform moves the original; with it, it clones and moves the clone.

```

for (const node of doc.selection.nodes) {
  const box = node.getSpreadBaseBox();
  combined = combined ? unionRects(combined, box) : box;
}
if (combined) {
  console.log('Combined bounds:',
    Math.round(combined.x), Math.round(combined.y),
    Math.round(combined.width), 'x', Math.round(combined.height));
}
}

```

9.21 SDK modules used in this lesson

- /shapes — ShapeRectangle, ShapeEllipse, ShapePolygon, ShapeTriangle, ShapeStar, ShapeCog, ShapeHeart, ShapeSpiral, ShapeQRCode, QRPayload URL, QRPayloadWifi, ShapeType, and all other shape classes
- /geometry — Transform, TransformBuilder, CurveBuilder, PolyCurve, Rectangle, unionRects, CubicBezier, Vector, Point, Size
- /commands — DocumentCommand, AddChildNodesCommandBuilder, CompoundCommandBuilder
- /nodes — PolyCurveNodeDefinition

9.22 Further examples

Example	What it shows
addPoints.js	Subdividing curves with CurveBuilder, midpoint insertion
bulgedPolyline.js	CurveBuilder.bulgeToXY, building a polyline with bulge segments
bulgeVersinePlayground.js	Interactive bulge/versine explorer, bulgeTo, versineTo
textOnCurvesAndShapes.js	The four curve- and shape-backed text node types
cornerEffects.js	Corner effects on curves: rounded, bevel, inset, fancy
divideLength.js	Divide every curve segment into equal lengths
pathEffects.js	Punk, bloat, twirl and retract path effects
roundAnyCorner.js	Round corners of selected curves with radius clamping
makeGrid.js	Grid duplication with Transform and AddChildNodesCommandBuilder
stepAndRepeat.js	Progressive duplicate/offset/rotate/scale with Transform
randomise.js	Randomise position, rotation, scale and opacity
swapObjects.js	Swap positions of two selected objects via Transform

9.23 See also

- Lesson 05 — Fills & Strokes
- Lesson 07 — Text & Story

- examples/

9.24 Examples

9.24.1 Grid duplicator

```
'use strict';

// Example 06a - Grid duplicator
// Lesson 06: Geometry
//
// Takes the selected shape and creates a grid of copies.
// Select ONE shape before running.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { Transform } = require('/geometry.js');

const COLS = 4;
const ROWS = 3;
const GAP = 16; // pixels between copies

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

  if (!node) {
    console.log('Select exactly one shape and run again.')}
  else {
    // baseBoxInterface.baseBox - NOT transformInterface.baseBox (that is
    undefined)
    const bbox = node.baseBoxInterface?.baseBox;
    if (!bbox) { console.log('Could not get bounding box.')}
    else {
      const stepX = bbox.width + GAP;
      const stepY = bbox.height + GAP;

      const compound = CompoundCommandBuilder.create();
      let count = 0;

      for (let row = 0; row < ROWS; row++) {
        for (let col = 0; col < COLS; col++) {
          if (row === 0 && col === 0) continue; // skip original position

          const xf = Transform.createTranslate(col * stepX, row * stepY);

          compound.addCommand(
            DocumentCommand.createTransform(sel, xf, { duplicateNodes:
true })
          );
          count++;
        }
      }

      doc.executeCommand(compound.createCommand());
      console.log(`Grid created: ${COLS}x${ROWS} (${count} copies added).`);
    }
  }
}
```

9.24.2 Basic shapes

```
'use strict';

// Example 06b - Basic shapes
// Lesson 06: Geometry, Transforms & Curves
//
```

```

// Creates a rectangle, ellipse, polygon, triangle, and star on the
current
// document using ShapeNodeDefinition and the shape factory methods.
//
// Key rule: always use ShapeRectangle.create() – never new
ShapeRectangle().
// The static factory provides the native handle required by the SDK.

const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder, DocumentCommand } = require('/
commands.js');
const {
  ShapeRectangle,
  ShapeEllipse,
  ShapePolygon,
  ShapeTriangle,
  ShapeStar
} = require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Helper – create and add a shape at a given position and size
  function addShape(shape, x, y, w, h, label) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    const cmd = builder.createCommand(true);
    doc.executeCommand(cmd);
    const node = cmd.newNodes?.[0];
    if (node) node.lineWidthPts = 1;
    console.log(`${label} added at (${x}, ${y}) ${w}x${h}`);
    return cmd.newNodes?.[0];
  }

  const W = 180; // shape width
  const H = 150; // shape height
  const GAP = 20; // gap between shapes

  // Row 1 – basic shapes
  const rect = addShape(ShapeRectangle.create(), 100, 100,
W, H, 'Rectangle');
  const ellipse = addShape(ShapeEllipse.create(), 100 + W + GAP, 100,
W, H, 'Ellipse');
  const polygon = addShape(ShapePolygon.create(), 100 + (W+GAP)*2, 100,
W, H, 'Polygon');

  // Row 2
  const triangle = addShape(ShapeTriangle.create(), 100, 100 +
H + GAP, W, H, 'Triangle');
  const star = addShape(ShapeStar.create(), 100 + W + GAP, 100 + H
+ GAP, W, H, 'Star');

  // Customise the polygon to a triangle (3 sides) using shape params
  if (polygon) {
    const sel = doc.selection; // polygon is selected after being added
    doc.executeCommand(DocumentCommand.createSetShapeIntParam(sel,
'NumberOfPoints', 3));
    console.log('Polygon set to 3 sides');
  }

  // Add a rounded rectangle by setting corner radius on the rect
  if (rect) {
    // Re-add a fresh rectangle at a new position with rounded corners
    const rounded = addShape(ShapeRectangle.create(), 100 + (W+GAP)*2, 100

```

```
+ H + GAP, W, H, 'Rounded rectangle');
  if (rounded) {
    const sel = doc.selection;
    doc.executeCommand(DocumentCommand.createSetShapeFloatParam(sel,
'CornerRadius', 25));
    console.log('Corner radius set to 25px');
  }
}

console.log('Done.');
```

9.24.3 Shapes and QR codes

```
'use strict';

// Example 06c – Shapes and QR codes
// Lesson 06: Geometry
//
// Creates parametric shapes beyond the basics (cog, heart, spiral) and
// a QR code with a URL payload, then reports each node's shape type.

const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const {
  ShapeCog, ShapeHeart, ShapeSpiral, ShapeQRCode,
  QRPayloadURL, ShapeType
} = require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  function addShape(shape, x, y, w, h, name) {
    const def = ShapeNodeDefinition.createDefault();
    def.shape = shape;
    def.userDescription = name;
    def.setBoundingRectangle({ x, y, width: w, height: h });
    const builder = AddChildNodesCommandBuilder.create();
    builder.addShapeNode(def);
    const cmd = builder.createCommand(true);
    doc.executeCommand(cmd);
    return cmd.newNodes?.[0];
  }

  const cog = ShapeCog.create();
  cog.teeth = 12;
  const cogNode = addShape(cog, 100, 100, 150, 150, 'Cog');
  if (cogNode) cogNode.lineWeightPts = 1;

  const heart = ShapeHeart.create();
  heart.spread = 0.6;
  const heartNode = addShape(heart, 300, 100, 150, 150, 'Heart');
  if (heartNode) heartNode.lineWeightPts = 1;

  const spiral = ShapeSpiral.create();
  spiral.turns = 4;
  const spiralNode = addShape(spiral, 500, 100, 150, 150, 'Spiral');
  if (spiralNode) spiralNode.lineWeightPts = 1;

  const url = QRPayloadURL.create();
  url.url = 'https://example.com';
  const qr = ShapeQRCode.create();
  qr.payload = url;
  const qrNode = addShape(qr, 100, 300, 150, 150, 'QR code');
  if (qrNode) qrNode.lineWeightPts = 1;

  for (const node of doc.layers.all) {
    const si = node.shapeInterface;
```

```

        if (si) {
            console.log(node.descriptionInterface?.description ?? '(unnamed)',
                '→', String(si.type));
        }
    }
}

```

9.24.4 06d-shapes-and-qr

```

/*
BSD 3-Clause License

```

Copyright (c) 2026, Canva Pty Ltd.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

Redistributions of source code must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

```

*/

```

```

'use strict';

```

```

const { Dialog, DialogResult } = require('/dialog.js');
const { Document, DocumentCommand } = require('/document.js');
const { ShapeStar, ShapePolygon, ShapeRectangle, ShapeArrow,
ShapeArrowEndStyle, ShapePie, ShapeQRCode, QRPayloadURL, QRPayloadVCard,
QRPayloadEmail, QRPayloadSMS, QRPayloadWifi } = require('/shapes.js');

```

```

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else { main(); }

```

```

function applyShapeToSelection(shape) {
    const sel = doc.selection;
    if (!sel || sel.nodes.length === 0) {
        console.log('Select a shape first.');
```

```
        return;
    }
    doc.executeCommand(DocumentCommand.createSetShape(sel, shape));
}

```

```

function demoBuiltInShapes() {
    const star = ShapeStar.create();

```

```

    star.innerRadius = 0.4;
    star.points = 5;
    star.leftCurve = 0.5;
    star.rightCurve = 0.5;
    applyShapeToSelection(star);
    console.log('Applied Star shape');
}

function demoPolygon() {
    const poly = ShapePolygon.create();
    poly.sides = 6;
    poly.curve = 0.2;
    poly.smoothPoints = true;
    applyShapeToSelection(poly);
    console.log('Applied Polygon shape');
}

function demoRectangle() {
    const rect = ShapeRectangle.create();
    const node = doc.selection.nodes.first;
    const box = node ? node.getSpreadBaseBox() : null;
    const w = box ? box.width : 200;
    const h = box ? box.height : 200;
    rect.topLeft.setRadius(20, w, h);
    rect.topRight.setRadius(20, w, h);
    rect.useSingleRadius = false;
    applyShapeToSelection(rect);
    console.log('Applied rounded Rectangle shape');
}

function demoArrow() {
    const arrow = ShapeArrow.create();
    arrow.leftEnd = ShapeArrowEndStyle.None;
    arrow.rightEnd = ShapeArrowEndStyle.Arrow;
    arrow.thickness = 0.3;
    applyShapeToSelection(arrow);
    console.log('Applied Arrow shape');
}

function demoPie() {
    const pie = ShapePie.create();
    pie.innerRadius = 0.2;
    pie.startAngle = 0;
    pie.sweep = 270;
    pie.isClosed = true;
    applyShapeToSelection(pie);
    console.log('Applied Pie shape');
}

function demoQRCode() {
    const url = QRPayloadURL.create('https://affinity.serif.com');
    const qrShape = ShapeQRCode.create();
    qrShape.payload = url;
    applyShapeToSelection(qrShape);
    console.log('Applied QR code shape');
}

function demoVCard() {
    const vcard = QRPayloadVCard.create();
    vcard.name = 'Jane Doe';
    vcard.firstName = 'Jane';
    vcard.lastName = 'Doe';
    vcard.email = 'jane@example.com';
    vcard.phone = '+1 555 0100';
    vcard.company = 'Acme Corp';
    const qrShape = ShapeQRCode.create();
    qrShape.payload = vcard;
    applyShapeToSelection(qrShape);
}

```

```

        console.log('Applied vCard QR code');
    }

    function demoWifi() {
        const wifi = QRPayloadWifi.create('MyNetwork', 'secretpass', 2);
        const qrShape = ShapeQRCode.create();
        qrShape.payload = wifi;
        applyShapeToSelection(qrShape);
        console.log('Applied WiFi QR code');
    }

    function buildDialog() {
        const dlg = Dialog.create('Shapes & QR Codes');
        const col = dlg.addColumn();
        const grp = col.addGroup('Built-in Shapes');
        grp.addButton('Star').onClickHandler = () => { demoBuiltInShapes(); };
        grp.addButton('Polygon').onClickHandler = () => { demoPolygon(); };
        grp.addButton('Rounded Rect').onClickHandler = () =>
    { demoRectangle(); };
        grp.addButton('Arrow').onClickHandler = () => { demoArrow(); };
        grp.addButton('Pie').onClickHandler = () => { demoPie(); };
        const qrGrp = col.addGroup('QR Codes');
        qrGrp.addButton('URL QR').onClickHandler = () => { demoQRCode(); };
        qrGrp.addButton('vCard QR').onClickHandler = () => { demoVCard(); };
        qrGrp.addButton('WiFi QR').onClickHandler = () => { demoWifi(); };
        col.addGroup('').addButton('Close').onClickHandler = () =>
    { dlg.isRunningModal = false; };
        return dlg;
    }

    function main() {
        const dlg = buildDialog();
        dlg.runModal();
    }

```

Phase 3 — Tools

PART



Phase 3 — Tools

10

Phase 2 gave you the mechanical skills — commands, fills, geometry, curves. Phase 3 is where those skills become useful in real world scenarios. This phase covers the four areas that turn a working script into a finished tool: text automation, interactive dialogs, file export, and external communication.

These are the lessons most directly connected to the kinds of tasks designers actually need automating. Text operations — finding, replacing, styling, and generating content — are among the most requested scripting use cases in any design tool. Dialogs transform a hardcoded script into something configurable, where a colleague who has never opened a code editor can supply the inputs and get a result. Export pipelines replace the most time-consuming part of a production workflow with a single button press. And networking means scripts can connect to the wider world — pulling in data, pushing results to APIs, and integrating with the systems a team already uses.

Each lesson in this phase also introduces a subtlety that is easy to miss. The Story model is richer and more structured than a simple string API, and getting text operations wrong produces silent failures rather than errors. Dialogs require understanding the control lifecycle — controls are configured before the dialog is shown and read after it closes. Export has a matrix of area types, format presets, and result records that all need to be handled correctly. And networking requires both user permission and careful attention to error handling.

By the end of this phase your scripts will be capable of doing things that were previously only possible through manual work or complex external tooling.

10.1 Lessons

Lesson	Topic	Key skill you'll gain
07 — Text & story	Story model, storyRange, formatting, StoryBuilder	Reading, replacing, and styling text in any document
08 — Dialog UI	Dialog, controls, columns, groups, reactive layout	Building native dialogs that collect user input
09 — Export & files	FileExportOptions, FileExportArea, File, HttpRequest	Exporting assets, writing files, calling external APIs
10 — Text threading	TextFrameInterface, createLinkTextFrame, createUnlinkTextFrame	Linking text frames so overflow flows between them

10.2 What to expect

By the end of Phase 3 you should be able to write a script that:

- Reads and replaces text in any text frame using the Story API
- Applies character styles and formatting to a precise range of characters
- Links text frames together so overflow flows across a multi-frame chain
- Presents a dialog with multiple controls and uses the values to drive the script
- Exports every artboard in a document as named PNG files with a JSON manifest
- Reads and writes text files on the Desktop
- Makes an HTTP GET or POST request to an external API

You will also understand why `si.storyRange` lives on the `storyInterface` directly and not on `si.story`, why dialog controls must be configured before `runModal()` is called and read after it returns, and why error handling matters in network requests. These are the most common sources of confusion in this phase — and knowing them in advance means you will recognise them immediately if you encounter them.

Lesson 07 — Text & Story Model

11

Text in Affinity is not a simple string. Every text node contains a **Story** — a structured sequence of characters with formatting attached, like a miniature word processor embedded in the document. This lesson explains that model and shows you how to read, replace, style, and build text programmatically.

Automating text is one of the most common scripting use cases — batch-renaming layers, applying consistent styles, generating text from data, or checking for missing fonts. The Story model is more complex than a plain string API, but that complexity is what lets you format a precise range of characters without touching anything else.

Two details trip up most newcomers, so they're worth knowing up front:

- A `TextNode` exposes a `storyInterface`, and the full text range lives on `storyInterface.storyRange` — not on the `story` property. Confusing these is the most common mistake in this lesson.
- Every story ends with a hidden paragraph marker. When selecting the full text, use `range.end - 1` to exclude it.

By the end of this lesson you'll be able to read text from any frame, replace it, apply character and paragraph formatting to a precise range, and generate structured text from scratch with `StoryBuilder`.

A note on scope: the SDK's scripting surface currently has limited support for master-page content. Text frames that originate from a master page and appear on ordinary pages may not respond to the same commands as regular text frames — some operations work, others silently fail. Because of this, text frames derived from master pages are not dealt with in this lesson.

11.1 What this lesson covers

- The Story model — how text is stored internally
- StoryBuilder — constructing text content
- StoryDelta — describing formatting changes
- TextSelection — targeting ranges of text
- Constructing text selection ranges correctly (the end - 1 convention)
- Applying character styles by name
- Paragraph attributes
- Glyphs — the typed character model and /glyphs taxonomy

11.2 How text works in Affinity SDK

Text in Affinity is not a simple string. It is a **Story** — a structured sequence of glyphs with formatting attached. Think of it like a mini word processor document embedded inside each text frame.

```
TextNode
├─ storyInterface
│   └─ story
│       └─ storyRange { begin: 0, end: N }
│           └─ glyphs (characters + formatting)
```

To change text, you either:

1. **Replace the whole story** with StoryBuilder
2. **Apply formatting** to a range with StoryDelta

11.3 Reading text from a node

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No doc.'); }
else {
  for (const node of doc.layers.all) {
    if (!node.isTextNode) continue;

    const si = node.storyInterface;
    if (!si) continue;

    const range = si.storyRange;
    console.log('Text node:', node.descriptionInterface?.description);
    console.log('Range: begin=', range.begin, 'end=', range.end);
  }
}
```

11.4 Replacing all text in a frame

This is the most common operation — and has a critical gotcha.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');
const { StoryBuilder } = require('/storybuilder.js');

const doc = Document.current;
const sel = doc.selection;
const node = sel.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text frame.'); }
else {
  // CRITICAL: set current spread before any text work
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  const si = node.storyInterface;
  const range = si.storyRange;
```

```
// Build a text selection that covers all editable text.
// end - 1 excludes the story terminator. To append instead of replace,
// see the "Appending text" section below and example 07c.
const textSel = TextSelection.create([
  begin: range.begin,
  end:   range.end - 1    // <-- CRITICAL: -1 excludes the story
terminator
});

const selection = Selection.create(doc, node);
selection.addSubSelectionForNode(node, textSel);

const cmd = DocumentCommand.createSetText(selection, 'New text
content');
doc.executeCommand(cmd);
console.log('Text replaced.');
```

11.5 Appending text to a frame

To append without overwriting existing content, use a zero-length Text Selection — one where begin and end are the same value. This is an insertion point rather than a range. Text passed to `createSetText` is inserted at that position, leaving everything before it untouched.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');

const doc = Document.current;
const node = doc.selection.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text frame. '); }
else {
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  const si      = node.storyInterface;
  const range    = si.storyRange;

  // Insert at end - 1: just before the story terminator.
  // begin == end makes this an insertion point, not a selection.
  const insertAt = range.end - 1;
  const textSel  = TextSelection.create([
    { begin: insertAt, end: insertAt }
  ]);

  const sel = Selection.create(doc, node);
  sel.addSubSelectionForNode(node, textSel);

  doc.executeCommand(DocumentCommand.createSetText(sel, ' (draft)'));
  console.log('Suffix appended.');
```

The contrast with replacement is just in the TextSelection:

Intent	begin	end
Replace all	range.begin	range.end - 1
Append	range.end - 1	range.end - 1

11.6 The selection setup order matters

`Selection.createEmpty(doc)` causes `addSubSelectionForNode` to silently fail — always use `Selection.create(doc, node)` instead.

Note

The end - 1 convention.

The story's final character is a hidden paragraph marker that is part of the story's internal structure. Including it in your selection (`range.end`) may work in practice, but the correct form is always `range.end - 1` — it keeps the selection within the editable content and avoids touching the terminator. Use it consistently.

```
// Snippet
// WRONG — using Selection.createEmpty causes addSubSelectionForNode to
// silently fail
const selection = Selection.createEmpty(doc);
selection.addSubSelectionForNode(node, textSel); // silently does nothing

// CORRECT — use Selection.create(doc, node) as the base
const selection = Selection.create(doc, node);
selection.addSubSelectionForNode(node, textSel); // works
```

11.7 Applying character formatting (StoryDelta)

Build a StoryDelta with a factory like createGlyphString, then pass it to DocumentCommand.createFormatText() to apply formatting to a text selection.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');
const { StoryDelta } = require('/storydelta.js');
const { GlyphAttStringType } = require('/glyphatts.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const textNode = doc.selection.nodes.filter(n => n.isTextNode).first;
  if (!textNode) { console.log('Select a text node first.')}
  else {
    doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

    const si = textNode.storyInterface;
    const range = si.storyRange;
    const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
    const sel = Selection.create(doc, textNode);
    sel.addSubSelectionForNode(textNode, textSel);

    // Apply a named character style to the text selection
    const delta =
StoryDelta.createGlyphString(GlyphAttStringType.StyleName, 'Heading
Bold');
    const cmd = DocumentCommand.createFormatText(sel, delta);
    doc.executeCommand(cmd);
    console.log('Style applied.')}
  }
}
```

11.8 StoryDelta convenience factories

StoryDelta provides higher-level factories for common formatting operations — no need to build raw glyph attribute strings:

```
'use strict';
const { Document } = require('/document.js');
const { StoryDelta } = require('/storydelta.js');
const { FontWeight } = require('/fonts.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Apply bold and italic to current text selection in one composite
  delta
    const makeBold = StoryDelta.createWeight(FontWeight.Bold);
    const makeItalic = StoryDelta.createItalic(true);
    const composite = StoryDelta.createComposite([makeBold, makeItalic]);

  // doc.formatText() applies to current selection without needing an
  explicit Selection object
  doc.formatText(composite);
```

```

    console.log('Bold italic applied.');
```

```

}
```

11.8.1 More StoryDelta factories

Factory	What it applies
createUnderlineType / createStrikeoutType / createSuperSubType / createCapsType	Underline, strikethrough, super/subscript, small caps
createBrushFill / createPenFill / createTransparency / createHighlightFill	Text colour, stroke, transparency, highlight
createFont(fields, font)	A Font object directly (vs the postscript-name string path)
createPostscriptName / createFamilyName / createWidth	Font selection (companions to createWeight/createItalic)
createLeftIndent / createRightIndent	Paragraph indents
createLeadingType	Leading behaviour
createIsKeepWithPrevious / createIsKeepTogether / createIsPreventWidows / createIsPreventOrphans	Keep-together / widow / orphan control
createIsAutoHyphenate / createHyphenateMinLength / createMaxConsecutiveHyphens	Hyphenation control
createIsNoBreak	No-break runs
createParagraphDouble / createParagraphString	Generic paragraph-attribute escape hatches (companions to createGlyphDouble/createGlyphString)

Note
`doc.formatText(delta, selection?)` is a shorthand that applies the delta to any selection without manually building a Selection and Text Selection. Use it for quick formatting operations. Similarly, `doc.setText(text, selection)` replaces text in one call.

11.9 StoryBuilder – building text from scratch

Use `StoryBuilder.create()` to construct styled text programmatically, then set the default style for artistic or frame text.

```

'use strict';
const { Document } = require('/document.js');
const { StoryBuilder } = require('/storybuilder.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // For artistic text (single-line, not in a frame)
  const sb = StoryBuilder.create();
  sb.setToArtisticTextDefaultStyle(doc.dpi, doc.format); // INSTANCE
  // method, not static
  sb.addText('Hello, Affinity!');
  console.log('StoryBuilder ready – use with AddChildNodesCommandBuilder
  to add to doc.');
```

```

  // For frame text use:
  // sb.setToFrameTextDefaultStyle(doc.dpi, doc.format);
}
```

`sb.addParagraphBreak()` starts a new paragraph — required for multi-paragraph stories (a raw `\n` in `addText` is not equivalent). `sb.clearText()` resets the builder for reuse. `sb.setGlyphAtts(atts)` / `sb.setParagraphAtts(atts)` apply full attribute bundles to subsequently added text, and the `sb.glyphAtts` / `sb.paragraphAtts` getters read the pending style back.

11.10 TableTextNode

`TableTextNode` is a grid-based text node. Cells are separated by `HardBreakType`. `TableCell` glyphs in the node's story.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');
const { TableTextNodeDefinition } = require('/nodes.js');
const { StoryDelta } = require('/storydelta.js');
const { GlyphAttDoubleType } = require('/glyphatts.js');
const { FontWeight } = require('/fonts.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Create a 3x3 table
  const box = { x: 100, y: 100, width: 300, height: 200 };
  const def = TableTextNodeDefinition.create(box, { width: 3, height:
3 });
  doc.addNode(def);
  const table = doc.layers.first; // newly added table

  // Walk cells via HardBreakType
  const story = table.storyInterface.story;
  const len = story.length;
  const starts = [0];
  for (let i = 0; i < len; i++) {
    if (story.getHardBreakType(i)?.value === 2) { //
HardBreakType.TableCell
      starts.push(i + 1);
    }
  }

  // Fill cells in REVERSE order so positions stay stable as text grows
  const data = [['A1', 'B1', 'C1'], ['A2', 'B2', 'C2'], ['A3', 'B3', 'C3']];
  for (let k = data.length - 1; k >= 0; k--) {
    for (let j = data[k].length - 1; j >= 0; j--) {
      const begin = starts[k * 3 + j];
      const end = (k * 3 + j + 1 < starts.length) ? starts[k * 3 + j + 1]
- 1 : len;
      const textSel = TextSelection.create([begin, end]);
      const sel = Selection.create(doc, table);
      sel.addSubSelectionForNode(table, textSel);
      doc.executeCommand(DocumentCommand.createSetText(sel, data[k][j]));
    }
  }

  // Bold the header row
  const headerSel = TextSelection.create([begin: 0, end: starts[3]]);
  const sel = Selection.create(doc, table);
  sel.addSubSelectionForNode(table, headerSel);
  const boldDelta = StoryDelta.createComposite([
    StoryDelta.createWeight(FontWeight.Bold),
    StoryDelta.createGlyphDouble(GlyphAttDoubleType.Height, 14 *
(doc.dpi / 72))
  ]);
  doc.executeCommand(DocumentCommand.createFormatText(sel, boldDelta));
  console.log('Table created and populated.')}
}
```

Note
TableTextNode
Definition.create(box, {width, height}) creates a grid. Cells are indexed left-to-right, top-to-bottom. Always fill cells in reverse order so earlier cell positions stay stable as the story grows.

11.11 GlyphAtts and ParagraphAtts – attribute wrappers

GlyphAtts and ParagraphAtts provide proper wrapper types for reading and building text attributes:

```
'use strict';
const { GlyphAtts, GlyphAttStringType, GlyphAttDoubleType } = require('./glyphatts.js');
const { ParagraphAtts } = require('./paragraphatts.js');
const { FontWeight } = require('./fonts.js');
const { Document } = require('./document.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (node) {
  // Build glyph attributes
  const glyphAtts = GlyphAtts.create();
  glyphAtts.setStringValue(GlyphAttStringType.StyleName, 'Heading');
  glyphAtts.setDoubleValue(GlyphAttDoubleType.Height, 24);

  // Build paragraph attributes
  const { ParagraphAlignXType } = require('./paragraphatts.js');
  const paraAtts = ParagraphAtts.create();
  paraAtts.alignXType = ParagraphAlignXType.Centre; // centre alignment
  paraAtts.isPreventWidows = true; // keep lines together
  paraAtts.isAutoHyphenate = false; // no hyphens

  // Read existing attributes from a story
  const si = node.storyInterface;
  const story = si.story;
  const existing = story.getGlyphAtts(0);
  console.log('Font loaded:',
existing.getDoubleValue(GlyphAttDoubleType.Height));
}
```

11.12 Attribute runs on Story

Story can enumerate contiguous runs of formatting, making it easy to walk the styling of a story without inspecting every glyph:

```
'use strict';
const { Document } = require('./document.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (node) {
  const story = node.storyInterface.story;

  // Enumerate all glyph attribute runs
  for (const run of story.glyphAttRuns) {
    console.log(`Glyphs ${run.begin}-${run.end}`, run.glyphAtts);
  }

  // Enumerate all paragraph attribute runs
  for (const run of story.paragraphAttRuns) {
    console.log(`Paragraphs ${run.begin}-${run.end}`, run.paragraphAtts);
  }

  // Combined glyph + paragraph runs
  for (const run of story.attRuns) {
    console.log(`Combined run ${run.begin}-${run.end}`, run);
  }

  // Partial range queries – start position only, runs are enumerated to
  end of story
  const partialRuns = story.getGlyphAttRunsFrom(5);
```

```

    for (const run of partialRuns) {
      console.log(`Sub-run ${run.begin}-${run.end}`);
    }
  }
}

```

Each run yields { begin, end, glyphAtts, paragraphAtts } so you can process homogeneous spans in one step.

11.13 Reading text and searching words

story.getText(startPos, maxLength, format) and story.getTextRange(range, format) extract text directly — no range arithmetic needed. story.getParagraphAtts(pos) reads paragraph attributes at a position (companion to getGlyphAtts).

```

// Snippet — assumes story is a Story
const text = story.getText(0, story.length - 1);
const begin = story.findWordBegin(10); // word/paragraph search family:
const end = story.findWordEnd(10);     // findWordBegin/End,
getWordRange,
// range helpers: getWordRange(pos), getParagraphRange(pos)
// enumerators: story.wordRanges, story.paragraphRanges
// predicates: story.isWordBegin(pos), story.isWordEnd(pos)
console.log(text.slice(begin, end));

```

11.14 Glyphs — the typed character model

Every position in a story holds a typed glyph, not a plain character. The /glyphs module exposes the full taxonomy — around 28 glyph classes including CharGlyph, HardBreakGlyph, tab state on the base Glyph class (isTab), Page NumberGlyph, FieldGlyph, PinGlyph, ListNumberGlyph, and IndexMarkGlyph — plus the GlyphType, HardBreakType, SoftBreakType, and WordPartType enums. TableTextNode cells, covered earlier in this lesson, are delimited by exactly these HardBreakType.TableCell glyphs.

Story hands out pre-filtered glyph collections so you never need to classify manually: story.glyphs(all), charGlyphs, hardBreakGlyphs, pageNumberGlyphs, fieldGlyphs, anchorGlyphs, pinGlyphs, and more. Every glyph also answers predicate questions directly:

Predicate	Answers
isParagraphBreak / isSoftBreak	Ends a paragraph / a wrapped line
isTab	Tab character
isAlphaNumeric	Ordinary letter or digit (CharGlyph)
isZeroWidth	Takes no horizontal space (anchors, marks)
isDeletable	Safe to remove from the story
isCombining	Combining diacritic

```

'use strict';
const { Document } = require('/document.js');
const { CharGlyph } = require('/glyphs.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text node.')}
else {
  const story = node.storyInterface.story;

  let chars = 0, breaks = 0, tabs = 0, other = 0;
  for (const glyph of story.glyphs) {

```

```

    if (glyph.isParagraphBreak || glyph.isSoftBreak) breaks++;
    else if (glyph.isTab) tabs++;
    else if (glyph.isAlphaNumeric) chars++;
    else other++;
  }
  console.log('Alphanumeric:', chars, '| Breaks:', breaks, '| Tabs:',
    tabs, '| Other:', other);

  console.log('Hard breaks in story:');
  for (const hb of story.hardBreakGlyphs) {
    console.log(' ', hb.description ?? '(hard break)');
  }

  const heart = CharGlyph.create(0x2665);
  console.log('Built CharGlyph:', heart.string, '(char32', heart.char32 +
    ')');
}

```

CharGlyph.create(char32) builds a character glyph (code point, not a one-character string) for use with doc.insertGlyph(glyph, sel) or DocumentCommand.createInsertGlyph(sel, glyph).

11.15 Batch text update — rename all text nodes

Iterate all text nodes in the document and replace their content using a CompoundCommandBuilder for a single undo step.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/
commands.js');
const { Selection, TextSelection } = require('/selections.js');

const doc = Document.current;
const compound = CompoundCommandBuilder.create();

// Capture selection BEFORE setCurrentSpread
const spreadCmd =
  DocumentCommand.createSetCurrentSpread(doc.currentSpread);

let count = 0;
for (const node of doc.layers.all) {
  if (!node.isTextNode) continue;

  const si = node.storyInterface;
  if (!si) continue;
  const range = si.storyRange;

  const textSel = TextSelection.create([ { begin: range.begin, end:
    range.end - 1 } ]);
  const nodeSel = Selection.create(doc, node);
  nodeSel.addSubSelectionForNode(node, textSel);

  compound.addCommand(DocumentCommand.createSetText(nodeSel, `Updated text
    ${count}`));
}

if (count > 0) {
  doc.executeCommand(spreadCmd);
  doc.executeCommand(compound.createCommand());
  console.log(`Updated ${count} text node(s).`);
} else {
  console.log('No text nodes found.');
```

11.16 Text frame state

The /textframeinterface module gives read-only access to text frame properties via node.textFrameInterface: ignoreBaselineGrid, ignoreTextWraps,

isMultiFrameTextFlow, isTextFlowBack, isTextFlowFront, isWrappingText, storyToDomainTransform, and more.

```
// Snippet — assumes node is a text node
const tfi = node.textFrameInterface;
if (tfi) {
  console.log('Ignore baseline grid:', tfi.ignoreBaselineGrid);
  console.log('Multi-frame flow:', tfi.isMultiFrameTextFlow);
}
```

Threading text frames. Link two FrameTextNodes so overflowing text flows from one into the next — the SDK provides DocumentCommand.createLinkTextFrame(srcNode, destNode) and DocumentCommand.createUnlinkTextFrame(node) as command factories. Both require two existing FrameTextNodes on the spread.

For a complete workflow with querying thread state, linking/unlinking, and reading across linked stories, see Lesson 10: Text Threading.

11.17 Key gotchas

end - 1 to stay within editable content. The final character in any story is a hidden paragraph marker. Including it in a TextSelection(range.end) may work without error in practice, but the correct and safe form is range.end - 1. Use it consistently rather than testing where the boundary actually bites.

setToArtisticTextDefaultStyle() is an instance method. Call it as sb.setToArtisticTextDefaultStyle(dpi, format) on a StoryBuilder instance, not as a static.

createSetCurrentSpread before text editing. Always set the spread before any text commands — even if you think it’s already the current spread.

Fill cells in reverse order. When populating a TableTextNode, fill cells from the last cell to the first. Inserting text at earlier positions shifts later positions; reverse order avoids position drift.

Format each cell individually. formatText across cell separators (HardBreakType.TableCell) only partially applies — the SDK stops at the separator boundary. When styling multiple cells (e.g. a header row), loop over each cell and format it separately.

① **Note**
Capture doc.selection BEFORE setCurrentSpread. See Lesson 02.

11.18 Exercises

11.18.1 Exercise 7.1 — Text reader

Write a script that finds every text node in the document and logs its layer name and current text content.

11.18.2 Exercise 7.2 — Suffix appender

For every selected text node, append (draft) to its text content. Use a zero-length TextSelection at range.end - 1. See 07c-append-text.js for a working implementation.

11.18.3 Exercise 7.3 — Style applier

Build a script that applies a named character style (e.g. “Bold”) to the full text of each selected text node.

11.18.4 Exercise 7.4 — Text census

Count how many characters total are across all text nodes in the document (use range.end - range.begin).

11.19 SDK modules used in this lesson

- /story — Story, StoryRange
- /storybuilder — StoryBuilder
- /storydelta — StoryDelta
- /storyinterface — StoryInterface
- /glyphatts — GlyphAtts, GlyphAttStringType, GlyphAttDoubleType
- /glyphs — CharGlyph, GlyphType, HardBreakType, SoftBreakType, and the glyph class taxonomy
- /paragraphatts — ParagraphAtts, ParagraphAttDoubleType, ParagraphAttStringType
- /selections — Selection, TextSelection
- /commands — DocumentCommand
- /nodes — TableTextNodeDefinition, FrameTextNodeDefinition, ArtTextNodeDefinition

11.20 Further examples

Example	What it shows
<code>boldItalics.js</code>	GlyphAtts, ParagraphAtts for bold/italic; DocumentCommand for text formatting
<code>textOnCurvesAndShapes.js</code>	The four curve- and shape-backed text node types; createSetTextFrameShape
<code>tableFromJson.js</code>	TableTextNodeDefinition, StoryBuilder, populating a table from JSON data
<code>makeNumbersSequence.js</code>	Number selected text objects with prefix/suffix/padding

11.21 See also

- Lesson 06 — Geometry
- Lesson 08 — Dialog UI
- examples/

11.22 Examples**11.22.1 Text reader**

```
'use strict';

// Example 07a — Text reader
// Lesson 07: Text & Story
//
// Finds every text node in the document and logs its content.

const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  let count = 0;

  for (const node of doc.layers.all) {
    if (!node.isTextNode) continue;

    const name = node.descriptionInterface?.description ?? '(unnamed)';
    const si = node.storyInterface;

    if (!si) {
```

```

        console.log(`[${name}] - no storyInterface`);
        continue;
    }

    const range = si.storyRange;
    const charCount = range.end - range.begin;

    console.log(`\n[${name}]`);
    console.log('  Chars:', charCount);
    console.log('  Range: begin=' + range.begin + ' end=' + range.end);
    count++;
}

if (count === 0) {
    console.log('No text nodes found in document.');
```

11.22.2 Replace text in selected frame

```

'use strict';

// Example 07b - Replace text in selected frame
// Lesson 07: Text & Story
//
// Replaces ALL text in the selected text node.
// Demonstrates the critical end - 1 rule.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');

const NEW_TEXT = 'This text was set by the Affinity SDK.';

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

11.22.3 Append text to selected frame

```
'use strict';

// Example 07c – Append text to selected frame
// Lesson 07: Text & Story
//
// Appends a suffix to the existing text in the selected text node
// without disturbing the content already there.
//
// The key difference from replacing:
//   Replace – TextSelection covering begin to end - 1 (selects all
//   editable content)
//   Append – TextSelection with begin == end == end - 1 (zero-length,
//   at the end)
//
// A zero-length selection is an insertion point. createSetText at that
// position inserts text without overwriting anything before it.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection, TextSelection } = require('/selections.js');

const SUFFIX = ' (draft)';

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const textNodes = doc.selection.nodes.filter(n => n.isTextNode);

  if (textNodes.isEmpty) {
    console.log('Select a text frame and run again.')}
  else {
    const node = textNodes.first;
    const name = node.descriptionInterface?.description ?? '(unnamed)';

    // Always set current spread before text operations
    doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

    const si = node.storyInterface;
    const range = si.storyRange;

    // Insertion point: begin and end both set to range.end - 1.
    // This places the cursor just before the story terminator – text
    // is inserted here, leaving everything before it intact.
    const insertAt = range.end - 1;
    const textSel = TextSelection.create([ { begin: insertAt, end:
insertAt } ]);

    const sel = Selection.create(doc, node);
    sel.addSubSelectionForNode(node, textSel);

    doc.executeCommand(DocumentCommand.createSetText(sel, SUFFIX));
    console.log(`Appended "${SUFFIX}" to "${name}".`);
  }
}
```

11.22.4 Data-driven table

```
'use strict';

// Example 07d – Data-driven table
// Lesson 07: Text & Story
//
// Builds a table from a hardcoded dataset.
// Cells are filled in reverse order so earlier positions stay stable.
// A bold header row is applied via StoryDelta.

const { Document } = require('/document.js');
```

```

const { Selection, TextSelection } = require('/selections.js');
const { TableTextNodeDefinition } = require('/nodes.js');
const { StoryDelta, GlyphAttDoubleType } = require('/storydelta.js');
const { FontWeight } = require('/fonts.js');
const { HardBreakType } = require('/glyphs.js');
const { FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const DATA = [
  ['Country', 'Capital', 'Population', 'Area km²'],
  ['France', 'Paris', '67M', '551_695'],
  ['Germany', 'Berlin', '83M', '357_022'],
  ['Japan', 'Tokyo', '125M', '377_975'],
  ['Brazil', 'Brasília', '213M', '8_515_767'],
];

const COLS = DATA[0].length;
const ROWS = DATA.length;

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const box = { x: 40, y: 40, width: 700, height: 40 * ROWS };
  const def = TableTextNodeDefinition.create(box, { width: COLS, height:
ROWS });
  doc.addNode(def);
  const table = doc.layers.first;

  const story = table.storyInterface.story;

  // Set default font size and text colour so the table is visible
  const allSel = TextSelection.create([ { begin: 0, end: story.length } ]);
  const allNodeSel = Selection.create(doc, table);
  allNodeSel.addSubSelectionForNode(table, allSel);
  doc.formatText(StoryDelta.createComposite([
    StoryDelta.createGlyphDouble(GlyphAttDoubleType.Height, 24),
    StoryDelta.createBrushFill(FillDescriptor.createSolid(RGBA8(0, 0, 0,
255))),
  ]), allNodeSel);

  // Walk cells via HardBreakType
  const starts = [0];
  for (let i = 0; i < story.length; i++) {
    const hb = story.getHardBreakType(i);
    if (hb && hb.value === HardBreakType.TableCell.value) starts.push(i +
1);
  }

  // Fill cells in REVERSE order so positions stay stable
  const texts = DATA.flat();
  for (let k = texts.length - 1; k >= 0; k--) {
    const begin = starts[k];
    const end = (k + 1 < starts.length) ? starts[k + 1] - 1 :
story.length;
    const textSel = TextSelection.create([ { begin, end } ]);
    const sel = Selection.create(doc, table);
    sel.addSubSelectionForNode(table, textSel);
    doc.setText(String(texts[k]), sel);
  }

  // Bold and enlarge each header cell individually
  // (formatting across cell separators only partially applies)
  for (let c = 0; c < COLS; c++) {
    const begin = starts[c];
    const end = (c + 1 < COLS) ? starts[c + 1] - 1 : starts[COLS] - 1;
    const hSel = Selection.create(doc, table);
    hSel.addSubSelectionForNode(table, TextSelection.create([ { begin,
end } ]));
  }
}

```



```

    doc.formatText(StoryDelta.createComposite([
      StoryDelta.createWeight(FontWeight.Bold),
      StoryDelta.createGlyphDouble(GlyphAttDoubleType.Height, 30),
    ]), hSel);
  }

  console.log(`${ROWS}x${COLS} table created and populated.`);
}

```

11.22.5 Glyph inspector

```

'use strict';

// Example 07e – Glyph inspector
// Lesson 07: Text & Story
//
// Classifies every glyph in a story (characters, tabs, breaks, specials)
// and reports the census. Also builds a CharGlyph ready for insertion.

const { Document } = require('/document.js');
const { CharGlyph } = require('/glyphs.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isTextNode).first;

if (!node) { console.log('Select a text node.')}
else {
  const story = node.storyInterface.story;

  let chars = 0, breaks = 0, tabs = 0, other = 0;
  for (const glyph of story.glyphs) {
    if (glyph.isParagraphBreak || glyph.isSoftBreak) breaks++;
    else if (glyph.isTab) tabs++;
    else if (glyph.isAlphaNumeric) chars++;
    else other++;
  }
  console.log('Alphanumeric:', chars, '| Breaks:', breaks, '| Tabs:',
    tabs, '| Other:', other);

  console.log('Hard breaks in story:');
  for (const hb of story.hardBreakGlyphs) {
    console.log(' ', hb.description ?? '(hard break)');
  }

  const heart = CharGlyph.create(0x2665);
  console.log('Built CharGlyph:', heart.string, '(char32', heart.char32 +
    ')');
}

```


Lesson 08 — Dialog UI

12

Scripts become dramatically more useful when they can ask the user a question before running. This lesson covers Affinity's built-in dialog system — how to build modal dialogs with columns, groups, and controls, how to read the values the user enters, and how to use those values to drive the rest of your script.

A dialog is the difference between a script that always does the same thing and one that adapts to what the user needs. Instead of hardcoding a colour, you present a colour picker. Instead of a fixed export format, you offer a dropdown. The dialog API is self-contained — no HTML, no CSS, no web views — and produces native dialogs that look and feel like the rest of the application.

The dialog system has a lifecycle that matters: controls are configured before the dialog is shown, and values are read after it closes. Attempting to read a control value before calling `runModal()`, or configuring a control after it, produces either incorrect results or silent failures. Understanding the build-then-show-then-read model before writing your first dialog will save a significant amount of debugging.

The layout system is column-based. Controls live inside groups, groups live inside columns, and columns live inside the dialog. This hierarchy is stricter than a web layout, but it means dialogs render consistently without any CSS or measurement code on your part. Reactive behaviour — enabling one control based on the state of another — is handled through per-control or per-dialog value changed callbacks.

By the end of this lesson you will be able to build dialogs that collect text, numbers, colours, fills, strokes, fonts, and dropdown selections from the user, handle OK and Cancel correctly, and use the resulting values to parameterise the rest of your script.

12.1 What this lesson covers

- Creating modal dialogs with `Dialog.create()`
- The column → group → control hierarchy
- All available control types including `FillEditor`, `StrokeEditor`, `Font Picker`, and `SpatialAnchor`
- Reading values after the user clicks OK
- Reactive controls — `onValueChangedHandler` (per-control) and `onControl ValueChangedHandler` (dialog-level)
- Live preview dialogs — the `while (runModal() == Ok)` loop, preview flag, and `clearPreviews()`
- Mixed units in `UnitValueEditor` — store in pixels, display in mm
- `UnitType.Number` for unitless integer editors
- Resizable dialogs, `findControl()`, and attaching controls to the dialog object
- `DialogResult` — handling OK vs Cancel
- `alert()` for simple user messages

12.2 The Dialog hierarchy

```

Dialog
├─ Column (one or more)
│   └─ Group (one or more, with optional label)
│       ├── Controls (CheckBox, ComboBox, TextBox, FillEditor, etc.)
│       └─ ColumnStack (sub-layout inside a group)
│           └─ Column
│               └─ Group
│                   └─ Controls

```

```

// All items inherit from DialogItem base
// DialogItem exposes: .itemID (number), .itemType (string)
// Legacy aliases retained: .controlID, .controlType

```

12.3 Minimal dialog example

```

'use strict';
const { Dialog, DialogResult } = require('/dialog.js');

const dlg = Dialog.create('My Script');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

const nameBox = grp.addTextBox('Name', 'Default text');
const enableCb = grp.addCheckBox('Enable feature', true);

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  console.log('Name: ', nameBox.text);
  console.log('Enabled: ', enableCb.value);
} else {
  console.log('User cancelled.');
```

12.4 All control types**12.4.1 TextBox**

```

// Snippet
const tb = grp.addTextBox('Label', 'initial text');
// Read: tb.text
tb.isMultiline = true;           // allow multiple lines
tb.rowSpan = 3;                  // height in rows
tb.textHorizontalAlignment = 1;  // 0=Left, 1=Centre, 2=Right

```

12.4.2 CheckBox

```

// Snippet
const cb = grp.addCheckBox('Enable option', false);

```

```
// Read: cb.value (boolean)
cb.isFullWidth = true;
```

12.4.3 Switch (toggle)

```
// Snippet
const sw = grp.addSwitch('Dark mode', false);
// Read: sw.value (boolean)
```

12.4.4 ComboBox (dropdown)

```
// Snippet
const FORMATS = ['PNG', 'JPEG', 'PDF', 'SVG'];
const combo = grp.addComboBox('Format', FORMATS, 0);
// Read: FORMATS[combo.selectedIndex]
// Note: combo.value returns undefined – always use selectedIndex
combo.isFullWidth = true;
```

12.4.5 RadioGroup

```
// Snippet
const OPTIONS = ['Low', 'Medium', 'High'];
const radio = grp.addRadioGroup('Quality', OPTIONS, 1);
// Read: OPTIONS[radio.selectedIndex]
radio.isFullWidth = true;
```

12.4.6 ButtonSet (segmented control)

```
// Snippet
const ALIGN = ['Left', 'Centre', 'Right'];
const btns = grp.addButtonSet('Align', ALIGN, 0);
btns.isFullWidth = true;
// Read: ALIGN[btns.selectedIndex]
```

12.4.7 ColourPicker

```
// Snippet
const { Colour } = require('/colours.js');
const picker = grp.addColourPicker('Colour'); // no initial colour
// Or with an initial colour:
// const picker = grp.addColourPicker('Colour', initialColour);
picker.allowPickNone = true; // allow "no colour" selection
picker.allowNoise = true; // allow noise fills
// Read: picker.value (Colour object or null)
```

12.4.8 FillEditor

```
// Snippet
const { SolidFill } = require('/fills.js');
const { Colour } = require('/colours.js');
const initialFill = SolidFill.create(new Colour(/* ... */));
const fillEditor = grp.addFillEditor('Fill', initialFill);
fillEditor.isFullWidth = true;
fillEditor.isStrokeFill = false; // true = stroke fill mode
// Read: fillEditor.fill (returns a fill descriptor)
```

12.4.9 StrokeEditor

```
// Snippet
const strokeEditor = grp.addStrokeEditor('Stroke');
strokeEditor.isFullWidth = true;
// Read: strokeEditor.stroke
```

12.4.10 SpatialAnchor

A spatial anchor picker lets the user choose a reference point (top-left, centre, bottom-right, etc.) for alignment, resize, or transform operations. It renders as a 3x3 grid of position buttons.

```
// Snippet – assumes grp is defined
const anchor = grp.addSpatialAnchor('Anchor Point');
// Read: anchor.value returns a SpatialAnchor enum value (Centre, TopLeft,
```

etc.)

```
// Import from: const { SpatialAnchor } = require('/dialog.js');
```

SpatialAnchor is re-exported from /dialog — no need to import from affinity:dom directly. The initial value defaults to SpatialAnchor.None — pass a different value as the second argument.

12.4.11 Button

A standalone push button with an onClickHandler. Unlike other controls it doesn't hold a value — it fires an action.

```
// Snippet
const btn = grp.addButton('Apply');
btn.isFullWidth = true;
btn.alignment = HorizontalAlignment.Centre; // Centre, Left, Right
```

```
btn.onClickHandler = () => {
  console.log('Button clicked — running action');
};
```

```
// Fluent chaining — all setters return the item
grp.addCheckBox('Dark mode', false)
  .setIsFullWidth(true)
  .setIsEnabled(false);
```

12.4.12 Control quick reference

Control	Factory	Read value	Key properties
TextBox	grp.addText Box(label, text)	.text	.isMulti Line, .row Span, .isFull Width, .isEn- abled, .de- scrip- tion, .textHor- izontalAlign- ment
CheckBox	grp.addCheck Box(label, checked)	.value (bool)	.isFull Width, .isEn- abled
Switch	grp.add Switch(label, checked)	.value (bool)	.isEnabled
ComboBox	grp.addCombo Box(label, items[], in- dex)	items[.se- lectedIndex] (.value returns undefined)	.selectedIndex, .isFull Width, .isEn- abled
RadioGroup	grp.addRadio Group(label, items[], in- dex)	items[.se- lectedIndex]	.selectedIndex, .isFull Width, .isEn- abled
ButtonSet	grp.addBut- tonSet(label, items[], in- dex)	items[.se- lectedIndex]	.selectedIndex, .isFull Width, .isEn- abled
ColourPicker	grp.addColour Picker(la-	.value (Colour or null)	.allowPick None, .allow

Control	Factory	Read value	Key properties
	bel, initial Colour?)		Noise, .isEnabled
FillEditor	grp.addFillEditor(label, initialFill?)	.fill (FillDescriptor)	.isFullWidth, .isStrokeFill, .isEnabled
StrokeEditor	grp.addStrokeEditor(label, initialStroke?)	.stroke	.isFullWidth, .isEnabled
Button	grp.addButton(label)	— (fires .onClickHandler)	.isFullWidth, .alignment (Left/Centre/Right)
UnitValueEditor	grp.addUnitValueEditor(label, storeUnit, dispUnit, value, min?, max?) grp.addUserUnitValueEditor(label, storeUnit, dispUnit, value, min?, max?)	.value (in store unit)	.showPopupSlider, .precision, .isEnabled, .isFullWidth, .noMinValue, .noMaxValue
FontPicker	grp.addFontPicker(label)	.text(name), .fontFamily, .fontCollection	.isEnabled
SpatialAnchor	grp.addSpatialAnchor(label)	.value (SpatialAnchor enum)	.isEnabled
StaticText	grp.addStaticText(label, text)	.text	.isFullWidth (read-only display)

Common to all controls: .isEnabled, .isVisible, .itemID, .itemType, .description, .onValueChangedHandler, .setIsVisible(v), .setEnabled(v), .setIsFullWidth(v)

Common to groups: .addColumnStack(), .label, .column, .controls, .isVisible, .setIsVisible(v), .enableSeparator, .setEnabledSeparator(v)

Common to columns: .addGroup(name), .groups, .controls, .columnIndex, .widthProportion, .paddingFactor, .setPaddingFactor(v), .isVisible, .setIsVisible(v)

Common to dialog: .columns, .groups, .controls, .isResizable, .setIsResizable(v), .initialWidth, .isRunningModal, .findControl(id), .setItemVisibility(showIDs, hideIDs), .onControlValueChangedHandler, .setOnControlValueChangedHandler(fn)

12.4.13 ColumnStack

Columns added directly to the dialog (`dlg.addColumn()`) sit at the top level and affect the whole dialog width. `ColumnStack` lets you create a multi-column sub-layout inside a single group — useful when one section of your dialog needs a side-by-side layout while the rest uses a single column:

```
// Snippet
const stack = grp.addColumnStack();
const leftCol = stack.addColumn();
const rightCol = stack.addColumn();

leftCol.addGroup('Left').addCheckBox('Option A', true);
rightCol.addGroup('Right').addCheckBox('Option B', false);
```

12.4.14 UnitValueEditor (numeric with units)

```
// Snippet
const { UnitType } = require('/units.js');

// Basic — store and display in the same unit
const editor = grp.addUnitValueEditor(
  'Size', // label
  UnitType.Pixel, // storage unit — values read/written in this unit
  UnitType.Pixel, // display unit — what the user sees
  100, // initial value (in storage units)
  1, // min (null = no minimum)
  2000 // max (null = no maximum)
);
editor.showPopupSlider = true; // show a slider popup on the field
editor.precision = 1; // decimal places
editor.noMinValue = false; // allow values below the original min
editor.noMaxValue = false; // allow values above the original max
// Read: editor.value (always in storage units)

// Mixed units — store in pixels, display in millimetres
// The user sees and types mm; .value returns pixels
const mmEditor = grp.addUnitValueEditor('Offset', UnitType.Pixel,
UnitType.Millimetre, 0);

// Unitless integer — for counts, indices, sides, etc.
const sidesEditor = grp.addUnitValueEditor('Sides', UnitType.Number,
UnitType.Number, 6, 3, 20);

// UserUnitType variant — accepts UserUnitType enums instead of UnitType
// const userEditor = grp.addUserUnitValueEditor(label, storeUnit,
displayUnit, value, min?, max?);
```

12.4.15 FontPicker

`DialogFontPicker` lets the user choose a font interactively. It exposes three read properties after the dialog closes: `text` (a plain name string), `font` (a `Font` object), and `fontFamily` / `fontCollection` for family- or collection-level access.

```
'use strict';
const { Dialog, DialogResult } = require('/dialog.js');
const { FontFamily } = require('/fonts.js');
const { Document } = require('/document.js');

const doc = Document.current;
if (doc) {
  const dlg = Dialog.create('Choose font');
  const col = dlg.addColumn();
  const grp = col.addGroup('');

  const fp = grp.addFontPicker('Font');
  fp.text = 'Arial'; // optional: set initial font by name

  if (dlg.runModal() == DialogResult.Ok) {
```



```

    console.log('Name:      ', fp.text);
    console.log('Font:      ', fp.font?.postscriptName);
    console.log('Family:    ', fp.fontFamily?.name);
    console.log('Collection: ', fp.fontCollection?.displayName ??
'(none)');

    // Walk all variants in the chosen family
    const fam = fp.fontFamily;
    if (fam) {
        for (const f of fam.fonts) {
            console.log('  Variant:', f.postscriptName, 'bold:', f.isBold,
'italic:', f.isItalic);
        }
    }
}
}
}

```

fontCollection is null when the font is not part of a named collection (e.g. system fonts). Always null-guard before using it.

12.4.16 StaticText (label)

```

// Snippet
const label = grp.addStaticText('Info', 'This is a read-only label.');
```

label.isFullWidth = true;

// Update text reactively:

label.text = 'Updated text';

12.4.17 Per-item visibility

Every dialog item exposes isVisible (read/write) and a fluent setIsVisible():

```

// Snippet
const showAdvanced = grp.addCheckBox('Show advanced', false);
const advanced      = grp.addGroup('Advanced').setIsVisible(false);

showAdvanced.onValueChangedHandler = () => {
    advanced.isVisible = showAdvanced.value;
};

```

For bulk changes, use setItemsVisibility(showIDs, hideIDs) — pass arrays of itemID values:

```

// Snippet
const dlg = Dialog.create('My Dialog');
const col = dlg.addColumn();
const grp = col.addGroup('Options');
const rgbIds = [redEd.itemID, greenEd.itemID, blueEd.itemID];
const cmykIds = [cyanEd.itemID, magentaEd.itemID, yellowEd.itemID,
keyEd.itemID];

modeSelector.onValueChangedHandler = () => {
    if (modeSelector.selectedIndex === 0)
        dlg.setItemsVisibility(rgbIds, cmykIds); // show RGB, hide CMYK
    else
        dlg.setItemsVisibility(cmykIds, rgbIds); // show CMYK, hide RGB
};

```

12.5 Dialog-level properties

```

// Snippet
const dlg = Dialog.create('My Script');

// Make dialog resizable
dlg.isResizable = true;
dlg.initialWidth = 400; // pixels

// Find a control by ID after building the dialog
const ctrl = dlg.findControl(someControl.controlID);

```

ⓘ Note

Use itemID (not control ID) with setItemsVisibility. The controlId property is an alias for itemID but itemID is the canonical name.

```
// findControl returns an untyped control — narrow it with DialogItemType
// (DialogSwitch, DialogCheckBox, DialogComboBox, DialogRadioGroup,
// DialogButtonSet, DialogStaticText, DialogGroup, DialogColumn all
// inherit
// from DialogItem / DialogControl, which expose itemID/itemType/
// description)
// if (ctrl?.itemType === DialogItemType.CheckBox)
// { console.log(ctrl.value); }
// Full DialogItemType member list: SDK-EXPOSURE-MAP.md enum reference.

// Iterate all controls across all groups
for (const ctrl of dlg.controls) {
  console.log(ctrl.label);
}

// Check if dialog is currently showing
console.log('Is modal open?', dlg.isRunningModal);

// Column padding
col.paddingFactor = 0.5; // finer spacing control
col.setPaddingFactor(0.8); // fluent alternative

// Control description — now readable
tb.description = 'Enter the filename';
console.log(tb.description); // readable description
```

12.6 Multi-column dialogs

```
'use strict';
const { Dialog, DialogResult } = require('/dialog.js');
const { UnitType } = require('/units.js');

const dlg = Dialog.create('Export Options');

const col1 = dlg.addColumn();
col1.widthProportion = 1;
const grp1 = col1.addGroup('Format');
const FORMATS = ['PNG', 'JPEG', 'PDF'];
const formatCombo = grp1.addComboBox('Type', FORMATS, 0);

const col2 = dlg.addColumn();
col2.widthProportion = 1;
const grp2 = col2.addGroup('Quality');
const qualityEdit = grp2.addUnitValueEditor('DPI', UnitType.Pixel,
UnitType.Pixel, 300, 72, 600);

const result = dlg.runModal();

if (result === DialogResult.Ok) {
  console.log('Format:', FORMATS[formatCombo.selectedIndex]);
  console.log('DPI: ', qualityEdit.value);
} else {
  console.log('Cancelled.');
```

① **Note**
Handlers fire on user interaction only. Setting a control value from script code (`ctrl.value = x`) does not trigger `onValueChangedHandler` — only genuine user interaction does. This prevents accidental handler recursion.

`setEnabled()` updates is Enabled correctly. It correctly updates `isEnabled`.

Values modified inside a handler are reflected immediately. Changing a control's value inside an `onValueChangedHandler` updates the UI correctly.

12.7 Reactive controls

Two approaches are available. Per-control handlers use `onValueChangedHandler` set directly on each control — no argument, just read the control's current value. The dialog-level handler uses `onControlValueChangedHandler` on the dialog and receives the changed control as its argument.

Per-control (simpler for most cases):

```
'use strict';
const { Dialog, DialogResult } = require('/dialog.js');
const { UnitType } = require('/units.js');
```

```

const dlg = Dialog.create('Export Size');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

const useCustom = grp.addCheckBox('Use custom size', false);
const widthEdit = grp.addUnitValueEditor('Width', UnitType.Pixel,
UnitType.Pixel, 800, 1, 4096);
const heightEdit = grp.addUnitValueEditor('Height', UnitType.Pixel,
UnitType.Pixel, 600, 1, 4096);

widthEdit.isEnabled = false;
heightEdit.isEnabled = false;

// onValueChangedHandler on the control – no argument, read control value
directly
useCustom.onValueChangedHandler = () => {
  widthEdit.isEnabled = useCustom.value;
  heightEdit.isEnabled = useCustom.value;
};

const result = dlg.runModal();
if (result == DialogResult.Ok) {
  if (useCustom.value) {
    console.log('Width:', widthEdit.value, 'Height:', heightEdit.value);
  } else {
    console.log('Using default size.');
```

Dialog-level (useful when many controls share logic):

```

// Snippet
// onControlValueChangedHandler on the dialog – receives changed control
as argument
dlg.onControlValueChangedHandler = (ctrl) => {
  if (ctrl === useCustom) {
    widthEdit.isEnabled = useCustom.value;
    heightEdit.isEnabled = useCustom.value;
  }
};
```

12.8 Full practical example – export dialog

```

'use strict';
const { Document, FileExportOptions } = require('./document.js');
const { Dialog, DialogResult } = require('./dialog.js');
const { app } = require('./application.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

const presets = FileExportOptions.allPresetNames;

const dlg = Dialog.create('Export Document');
const col = dlg.addColumn();
const grp = col.addGroup('Settings');

const presetCombo = grp.addComboBox('Format', presets, 0);
const suffixBox = grp.addTextBox('Filename suffix', '');

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  const presetName = presets[presetCombo.selectedIndex];
  const suffix = suffixBox.text || '';
  const desktop = app.userDesktopPath;
  const outPath = `${desktop}/${doc.title}${suffix}.
${presetName.toLowerCase()}`;

  const opts = FileExportOptions.createWithPresetName(presetName);
```

```

    const records = doc.export(outPath, opts, null, null);

    for (const rec of records.all) {
        console.log(rec.isSuccess ? 'Exported: ' + rec.path : 'Failed: ' +
rec.errorMessage);
    }
}
}

```

12.9 Reactive dialog example — shape creator

08d-reactive-dialog.js shows a complete reactive dialog in practice. It builds a shape creation tool where:

- A ComboBox selects the shape type — changing it enables the Sides field only when Polygon is selected
- Width and Height fields each have their own onControlValueChangedHandler — when “Lock aspect ratio” is ticked, changing one syncs the other
- A static text label updates in real time as values change, showing a preview of what will be created before the user clicks OK

The key pattern is that each control has its own onValueChangedHandler set before runModal(). The function takes no argument — read the control’s value directly. The function runs inside the modal without closing it.

```

// Snippet
// Per-control handler — no argument needed
useCustom.onValueChangedHandler = () => {
    widthEdit.isEnabled = useCustom.value;
    heightEdit.isEnabled = useCustom.value;
};

```

See 08d-reactive-dialog.js for the full working implementation.

12.10 Live preview dialogs

The most powerful dialog pattern runs preview commands while the dialog is open, so the user sees the effect of their settings in real time before committing. This uses three elements working together:

- **doc.executeCommand(cmd, true)** — the second argument true marks this as a preview. It renders immediately but is not added to the undo history.
- **doc.clearPreviews()** — removes all active preview commands. Must always be called when the dialog loop ends, whether the user clicks OK or Cancel.
- **while (dlg.runModal() == DialogResult.Ok)** — keeps the dialog open if the user clicks OK but validation fails, allowing another attempt.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { UnitType } = require('/units.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
    const dlg = Dialog.create('Live Preview Example');
    const col = dlg.addColumn();
    const grp = col.addGroup('Settings');

    // Attach controls directly to dlg for convenient access in helper
    functions
    dlg.opacity = grp.addUnitValueEditor('Opacity %', UnitType.Number,
UnitType.Number, 100, 0, 100);

    function update() {

```

```

// Build and execute a preview command – rendered but not committed
const opacity = dlg.opacity.value / 100;
const cmd = DocumentCommand.createSetOpacity(doc.selection, opacity);
doc.executeCommand(cmd, true); // true = preview
}

// Update preview every time a control changes
dlg.onControlValueChangedHandler = update;

// Loop keeps dialog open if validation fails
while (dlg.runModal() == DialogResult.Ok) {
  if (dlg.opacity.value < 0 || dlg.opacity.value > 100) {
    alert('Opacity must be between 0 and 100.');
```

continue;

// re-show dialog

 }
 // Commit – run without preview flag so it goes into undo history
 const opacity = dlg.opacity.value / 100;
 doc.executeCommand(DocumentCommand.createSetOpacity(doc.selection,
opacity));
 break;
}

// Always clear previews – whether OK or Cancel
doc.clearPreviews();
}

12.11 Attaching controls to the dialog object

Rather than storing all controls in separate variables, you can attach them as properties directly on the dialog object. This keeps related controls together and makes helper functions cleaner – they only need the `dlg` argument:

```

// Snippet
const dlg = Dialog.create('Add Guides');
const col = dlg.addColumn();
const grp = col.addGroup('Position');

// Attach controls as properties on dlg itself
dlg.left = grp.addSwitch('Left');
dlg.right = grp.addSwitch('Right');
dlg.top = grp.addSwitch('Top');
dlg.bottom = grp.addSwitch('Bottom');

// You can also attach helper methods
dlg.anySelected = () => dlg.left.value || dlg.right.value || dlg.top.value
|| dlg.bottom.value;

while (dlg.runModal() == DialogResult.Ok) {
  if (!dlg.anySelected()) {
    alert('Please select at least one side.');
```

continue;

 }
 // use dlg.left.value, dlg.right.value etc.
 break;
}
doc.clearPreviews();

12.12 alert()

For simple validation messages, `alert()` is available as a global:

```

// Snippet
if (!doc) {
  alert('This script requires an open document.');
```

}

It blocks execution and shows a native system alert. Use it for error conditions only – not as a substitute for `console.log`.

Note
`doc.clearPreviews()` must always be called after the loop. If the user cancels, previews would otherwise remain visible on the canvas permanently.

Note
Previews don't stack. Only one preview command is active at a time. Rebuild everything in a single command (or compound command) on each change rather than layering preview commands on top of each other. The SDK replaces the previous preview with the new one automatically.

12.13 The canonical example — addGuides.js

The SDK ships a built-in example at `examples/addGuides.js` (see source) that demonstrates the full production-quality dialog pattern: live preview with the while loop, per-control `onValueChangedHandler`, mixed units (`UnitType.Pixel` stored / `UnitType.Millimetre` displayed), controls attached to the dialog object, and `clearPreviews()` always called after the loop. Reading that script alongside this lesson is the most direct way to see all the patterns working together.

12.14 Key gotchas

combo.selectedIndex not combo.value. `combo.value` returns undefined. Always read the selection as `combo.selectedIndex` and look up the value in your options array.

Per-control handler is onValueChangedHandler, dialog handler is onControlValueChangedHandler. These are different properties — `onValueChangedHandler` lives on each control and takes no argument. `onControlValueChangedHandler` lives on the dialog and receives the changed control.

UnitType.Pixel not UnitType.Pixels, UnitType.Number for unitless. All enum values use singular spelling. Use `UnitType.Number` when you want a plain integer with no unit — e.g. a count of sides or a percentage value.

Storage unit vs display unit in addUnitValueEditor. The second argument is the storage unit (what `.value` returns), the third is the display unit (what the user sees). They can differ — e.g. (`UnitType.Pixel`, `UnitType.Millimetre`, `0`) stores pixels, displays mm.

Previews don't stack — rebuild everything each time. `doc.executeCommand(cmd, true)` replaces the previous preview, not adds to it. Build a single compound command covering all changes on each update.

Always call doc.clearPreviews() after the dialog loop. If the user cancels, previews remain on the canvas permanently without this call.

Read control values only after runModal() returns. Values are set by user interaction — reading them before the dialog is shown returns initial values only.

Fill/Stroke editors and buttons return proper typed wrappers. `addFillEditor`, `addStrokeEditor` and `addButton` return their correct types — use them directly without casting. `createTypedDialogControl` is not needed.

① **Note**
result == DialogResult.Ok — title-case Ok not OK. The simpler `==` comparison works directly. Wrong case returns undefined and your check silently fails.

① **Note**
There is no addSlider. The SDK dialog has no slider control. Use `addComboBox` for a fixed set of options, `addUnitValueEditor` for a free numeric input, or set `showPopupSlider = true` on a `UnitValueEditor`.

12.15 Exercises

12.15.1 Exercise 8.1 — Colour picker dialog

Build a dialog with a `ColourPicker` and a `Switch` for “apply to stroke”. Apply the chosen colour to all selected nodes when confirmed.

12.15.2 Exercise 8.2 — Fill editor dialog

Build a dialog with a `FillEditor`. Apply the chosen fill to all selected nodes when confirmed.

12.15.3 Exercise 8.3 — Export preset picker

Show a `ComboBox` with all available export preset names. Export the current document using the chosen preset on OK.

12.15.4 Exercise 8.4 — Reactive form

Build a dialog where a `CheckBox` controls whether two `UnitValueEditor` fields (width and height) are enabled. Use `onValueChangedHandler` on the checkbox. Use `UnitType.Pixel` for storage and `UnitType.Millimetre` for display.

12.15.5 Exercise 8.5 — Reactive shape creator

See `08d-reactive-dialog.js` for a full implementation. Extend it by adding a `ColourPicker` control and applying the chosen fill colour to the newly created shape after it is added to the document.

12.15.6 Exercise 8.6 — Live preview opacity

See `08e-live-preview-dialog.js` for a full working implementation. Extend it by adding a second `UnitValueEditor` for blur radius, and applying both opacity and a Gaussian blur as a combined preview command using `CompoundCommandBuilder`.

12.16 SDK modules used in this lesson

- `/dialog` — `Dialog`, `DialogResult`, all control classes
- `/colours` — `Colour`
- `/fills` — `SolidFill`
- `/units` — `UnitType`
- `/document` — `FileExportOptions`
- `/shapes` — `ShapeRectangle`, `ShapeEllipse`, etc. (used in `08d`)
- `/nodes` — `ShapeNodeDefinition` (used in `08d`)

12.17 Further examples

Example	What it shows
<code>addGuides.js</code>	Live-preview dialog pattern: <code>runModal</code> , <code>preview flag</code> , <code>clearPreviews</code> <code>CompoundCommandBuilder</code> for multi-guide batching
<code>flexibleLayout.js</code>	Column stacks, <code>DialogColourPicker</code> , dynamic visibility via <code>setItemsVisibility</code>

12.18 See also

- Lesson 07 — Text & Story
- Lesson 09 — Export & Files
- `examples/08d-reactive-dialog.js` — reactive shape creator
- `examples/08e-live-preview-dialog.js` — live preview with opacity

12.19 Examples**12.19.1 Colour picker dialog**

```
'use strict';

// Example 08a — Colour picker dialog
// Lesson 08: Dialog UI
//
// Shows a dialog with a colour picker and stroke toggle.
// Applies the chosen colour to selected nodes on OK.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { FillDescriptor } = require('/fills.js');
const { Colour } = require('/colours.js');
```

```

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.')}
else {
  const dlg = Dialog.create('Apply Colour');
  const col = dlg.addColumn();
  const grp = col.addGroup('Colour');

  // addColourPicker accepts a Colour object or null (no initial colour)
  const picker = grp.addColourPicker('Fill colour');
  const strokeSw = grp.addSwitch('Also apply to stroke', false);

  // Capture the selection BEFORE runModal() - runModal clears it
  const sel = doc.selection;

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const colour = picker.value;
    if (!colour) { console.log('No colour selected.')}
    else {
      const fill = FillDescriptor.createSolid(colour);
      const compound = CompoundCommandBuilder.create();

      compound.addCommand(DocumentCommand.createSetBrushFill(sel, fill));
      if (strokeSw.value) {
        compound.addCommand(DocumentCommand.createSetPenFill(sel, fill));
      }

      doc.executeCommand(compound.createCommand());
      console.log('Colour applied' + (strokeSw.value ? ' to fill and
stroke.' : ' to fill.'));
    }
  } else {
    console.log('Cancelled.')}
  }
}

```

12.19.2 Batch rename dialog

```

'use strict';

// Example 08b - Batch rename dialog
// Lesson 08: Dialog UI
//
// Renames all selected layers by wrapping their current names
// with a prefix and/or suffix entered by the user.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/
commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) {
  console.log('Select some layers and run again.')}
else {
  const dlg = Dialog.create('Batch Rename Layers');
  const col = dlg.addColumn();
  const grp = col.addGroup('Name pattern');

  const prefixBox = grp.addTextBox('Prefix', '');
  const suffixBox = grp.addTextBox('Suffix', '');

  // Capture the selection BEFORE runModal() - runModal clears it

```



```

const nodes = doc.selection.nodes.toArray();

const result = dlg.runModal();

if (result == DialogResult.Ok) {
  const prefix = prefixBox.text;
  const suffix = suffixBox.text;
  const compound = CompoundCommandBuilder.create();
  let count = 0;

  for (const node of nodes) {
    const currentName = node.descriptionInterface?.description ?? '';
    const newName = `${prefix}${currentName}${suffix}`;
    const nodeSel = Selection.create(doc, node);
    compound.addCommand(DocumentCommand.createSetDescription(nodeSel,
newName));
    count++;
  }

  doc.executeCommand(compound.createCommand());
  console.log(`Renamed ${count} layer(s).`);
} else {
  console.log('Cancelled.');
```

12.19.3 Fill and stroke editor dialog

```

'use strict';

// Example 08c – Fill and stroke editor dialog
// Lesson 08: Dialog UI
//
// Shows a dialog with a FillEditor and StrokeEditor – the most capable
// fill/stroke controls available in the dialog API. Applies the chosen
// fill and stroke to all selected nodes on OK.
//
// FillEditor supports solid, gradient, and pattern fills.
// StrokeEditor supports full stroke style configuration.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/
commands.js');
const { Dialog, DialogResult } = require('/dialog.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

    const fill    = fillEditor.fill;
    const stroke  = strokeEditor.stroke;

    const compound = CompoundCommandBuilder.create();
    if (fill) compound.addCommand(DocumentCommand.createSetBrushFill(sel,
fill));
    if (stroke) {
        // StrokeEditor returns a LineStyleDescriptor with weight, colour,
etc.
        // Apply it via createSetLineStyleDescriptor
        compound.addCommand(DocumentCommand.createSetLineStyleDescriptor(sel,
stroke));
    }
    doc.executeCommand(compound.createCommand());

    console.log('Fill and stroke applied.');
```

12.19.4 Reactive dialog with value changed callbacks

```

'use strict';

// Example 08d — Reactive dialog with value changed callbacks
// Lesson 08: Dialog UI
//
// Demonstrates onValueChangedHandler on individual controls.
// Each control has its own handler — no need to branch on which control
fired.
//
// This example builds a shape creation dialog where:
// - A ComboBox selects the shape type
// - Sides field enables only when Polygon is selected
// - A "lock aspect ratio" checkbox keeps width and height in sync
// - A live preview label updates as values change

const { Document } = require('/document.js');
const { ShapeNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder, DocumentCommand } = require('/
commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { UnitType } = require('/units.js');
const {
    ShapeRectangle,
    ShapeEllipse,
    ShapePolygon,
    ShapeStar,
    ShapeTriangle
} = require('/shapes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

    // — Build dialog
```

```

    const dlg = Dialog.create('Add Shape');
    dlg.isResizable = true;

    const col = dlg.addColumn();

    const typeGrp = col.addGroup('Shape');
```

```

const typeCombo = typeGrp.addComboBox('Type', SHAPES, 0);

const sizeGrp = col.addGroup('Size');
const widthEdit = sizeGrp.addUnitValueEditor('Width', UnitType.Pixel,
UnitType.Pixel, 200, 10, 2000);
const heightEdit = sizeGrp.addUnitValueEditor('Height', UnitType.Pixel,
UnitType.Pixel, 200, 10, 2000);
widthEdit.showPopupSlider = true;
heightEdit.showPopupSlider = true;

const optGrp = col.addGroup('Options');
const lockAR = optGrp.addCheckBox('Lock aspect ratio', false);
const sidesEdit = optGrp.addUnitValueEditor('Sides', UnitType.Number,
UnitType.Number, 6, 3, 20); // unitless integer
sidesEdit.isEnabled = false;

const infoGrp = col.addGroup('');
const infoLabel = infoGrp.addStaticText(' ', `${SHAPES[0]} 200 × 200
px`);
infoLabel.isFullWidth = true;

// — Helper

function updateLabel() {
  const name = SHAPES[typeCombo.selectedIndex];
  const w = Math.round(widthEdit.value);
  const h = Math.round(heightEdit.value);
  infoLabel.text = `${name} ${w} × ${h} px`;
}

// — Per-control handlers

// onValueChangedHandler is set on each control individually.
// The dialog-level onControlValueChangedHandler is also available
// if you prefer a single entry point.

typeCombo.onValueChangedHandler = () => {
  sidesEdit.isEnabled = (SHAPES[typeCombo.selectedIndex] === 'Polygon');
  updateLabel();
};

let updatingFromLock = false;

widthEdit.onValueChangedHandler = () => {
  if (lockAR.value && !updatingFromLock) {
    updatingFromLock = true;
    heightEdit.value = widthEdit.value;
    updatingFromLock = false;
  }
  updateLabel();
};

heightEdit.onValueChangedHandler = () => {
  if (lockAR.value && !updatingFromLock) {
    updatingFromLock = true;
    widthEdit.value = heightEdit.value;
    updatingFromLock = false;
  }
  updateLabel();
};

lockAR.onValueChangedHandler = () => {
  if (lockAR.value) {
    heightEdit.value = widthEdit.value;
    updateLabel();
  }
};

```

```
// — Run

const result = dlg.runModal();

if (result !== DialogResult.Ok) {
  console.log('Cancelled.');
```

```

} else {
  const idx = typeCombo.selectedIndex;
  const shape = SHAPE_CTORS[idx].create();
  const w = widthEdit.value;
  const h = heightEdit.value;

  const def = ShapeNodeDefinition.createDefault();
  def.shape = shape;
  def.setBoundingBox({ x: 100, y: 100, width: w, height: h });

  const builder = AddChildNodesCommandBuilder.create();
  builder.addShapeNode(def);
  const cmd = builder.createCommand(true);
  doc.executeCommand(cmd);

  if (SHAPES[idx] === 'Polygon') {
    const sides = Math.round(sidesEdit.value);
    doc.executeCommand(DocumentCommand.createSetShapeIntParam(doc.selection,
      'NumberOfPoints', sides));
  }

  console.log(`${SHAPES[idx]} added — ${Math.round(w)} ×
    ${Math.round(h)} px`);
  }
}
}
```

12.19.5 Live preview dialog

```
'use strict';

// Example 08e — Live preview dialog
// Lesson 08: Dialog UI
//
// Demonstrates the live preview pattern:
// - doc.executeCommand(cmd, true) — preview flag: renders but not in
//   undo history
// - doc.clearPreviews() — removes all previews (always call
//   after loop)
// - while (runModal() == Ok) — loop for validation, re-showing
//   dialog on failure
// - onControlValueChangedHandler — dialog-level reactive callback
//
// Select one or more objects and run.
// The opacity changes live as you move the slider — cancel restores the
// original.

const { Document } = require('/document.js');
const { DocumentCommand, CompoundCommandBuilder } = require('/
  commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { UnitType } = require('/units.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else if (doc.selection.nodes.isEmpty) {
  console.log('Select some objects and run again.');
```

```

} else {
  const sel = doc.selection;

  // — Build dialog
```

```

const dlg = Dialog.create('Set Opacity');
const col = dlg.addColumn();
const grp = col.addGroup('Settings');

// UnitType.Number = unitless integer, shown without unit suffix
dlg.opacity = grp.addUnitValueEditor('Opacity %', UnitType.Number,
UnitType.Number, 100, 0, 100);
dlg.opacity.showPopupSlider = true;
dlg.opacity.precision = 0;

dlg.infoLabel = grp.addStaticText('', 'Opacity: 100%');
dlg.infoLabel.isFullWidth = true;

// — Live preview


---


// Dialog-level handler fires on every control change.
// executeCommand with preview=true renders immediately but is not
committed.

function update() {
  const pct = Math.round(dlg.opacity.value);
  dlg.infoLabel.text = `Opacity: ${pct}%`;
  const cmd = DocumentCommand.createSetOpacity(sel, pct / 100);
  doc.executeCommand(cmd, true); // true = preview
}

dlg.onControlValueChangedHandler = update;

// — Run loop


---


// while loop allows re-showing the dialog with an alert if validation
fails.

while (dlg.runModal() == DialogResult.Ok) {
  const pct = dlg.opacity.value;
  if (pct < 0 || pct > 100) {
    alert('Opacity must be between 0 and 100.');
```

continue;

```
  }
  // Commit – run without preview flag so it goes into the undo history
  doc.executeCommand(DocumentCommand.createSetOpacity(sel, pct / 100));
  console.log(`Opacity set to ${Math.round(pct)}%`);
  break;
}

// ALWAYS clear previews – if user cancels, previews stay on canvas
without this
doc.clearPreviews();
}
```


Lesson 09 — Export & File I/O

13

Getting content out of Affinity — as PNGs, PDFs, SVGs, or any of the 60+ supported formats — is one of the most valuable things a script can do. This lesson covers the full export pipeline: choosing a format, defining what to export, writing to disk, and optionally generating a JSON manifest. It also covers plain-text file I/O and HTTP requests.

Export automation is where scripts deliver the most time savings. A script that exports every artboard to a named PNG, writes a manifest, and uploads to an API can replace hours of manual work. The pieces are `FileExportOptions`, `FileExportArea`, `doc.export()`, the `File` class, and `HttpRequest`.

Exporting involves two independent choices: the **format preset** (PNG, PDF, SVG, etc.) and the **export area** (the whole document, a single artboard, the current spread, or just the selection). This lesson walks through the most common combinations.

Two things to remember:

- `fs` writes files to the Desktop or folders you have granted access to (JSON manifests, config, any text output).
- HTTP requests are executed with `req.do()`.

By the end you'll have a complete export pipeline: format selection, area targeting, file writing, manifest generation, and external communication. These example scripts are among the most directly reusable in the tutorial.

13.1 What this lesson covers

- `FileExportOptions` — choosing and configuring export formats
- `FileExportArea` — what part of the document to export
- `doc.export()` — synchronous export

- `doc.promises.export()` — async export
- Checking export results
- Per-node export settings — `ExportConfig`, `ExportFormat`, `ExportScale`
- `fs` module — reading and writing files
- `network` module — HTTP requests from scripts

13.2 Export overview

Three things combine to describe an export operation:

Object	What it defines
<code>FileExportOptions</code>	Format (PNG, JPEG, PDF, SVG, etc.) and format-specific settings
<code>FileExportArea</code>	Which part of the document (whole doc, current spread, artboard, selection)
Path string	Where the file goes (Desktop + granted folders)

13.3 Getting available export presets

`FileExportOptions.allPresetNames` lists every format preset available in the current Affinity session.

```
'use strict';
const { FileExportOptions } = require('/document.js');

const presets = FileExportOptions.allPresetNames;
console.log('Available presets:');
for (const name of presets) {
  console.log(' ', name);
}
// PNG, JPEG, PDF, SVG, EPS, TIFF, ...
```

13.4 Synchronous export

Call `doc.export()` with a format preset, area, and output path — returns an array of result records.

```
'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document.js');
const { app } = require('/application.js');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const outPath = `${desktop}/${doc.title}.png`;

  const records = doc.export(outPath, opts, area, null);

  for (const rec of records.all) {
    if (rec.isSuccess) {
      console.log('Exported to:', rec.path);
    } else {
      console.log('Export failed:', rec.errorMessage);
    }
  }
}
```


13.5 Export areas

FileExportArea provides factory methods for targeting different parts of the document — whole doc, current spread, specific artboards, pages, or selected nodes.

```
'use strict';
const { Document, FileExportArea } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

}

```
  // Whole document
  const wholeDoc = FileExportArea.createForWholeDocument();

  // Current spread only
  const spread = FileExportArea.createForCurrentSpread();

  // Current page only
  const page = FileExportArea.createForCurrentPage();

  // Specific artboard — ai comes from spread.getArtboard(i)
  // const ai      = doc.spreads.first.getArtboard(0);
  // const artboard = FileExportArea.createForArtboard(ai);

  // Selection bounding box
  const selArea = FileExportArea.createForSelectionArea(doc.selection);

  // Specific pages by string (e.g. "1-3,5")
  const pages = FileExportArea.createForPages('1-3');
```

}

```
  // Specific spreads by string
  const spreads = FileExportArea.createForSpreads('1-2');
```

}

```
  // Exact selection bounding box (vs selectionArea which pads to page)
  const sel = FileExportArea.createForSelection(doc.selection);

  console.log('Export areas ready.');
```

13.6 Export every artboard

Iterate spreads and artboards, exporting each one as a separate file named after the artboard.

```
'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/
document.js');
const { app } = require('/application.js');
```

}

```
const doc      = Document.current;
const desktop  = app.userDesktopPath;
```

}

```
if (!doc) { console.log('No document open.');
```

}

```
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
```

directly

```
  for (const spread of doc.spreads) {
    for (let i = 0; i < spread.artboardCount; i++) {
      const ai      = spread.getArtboard(i);    // returns ArtboardInterface
      const name     = ai.description ?? `artboard-${i}`;
      const area     = FileExportArea.createForArtboard(ai);
      const path     = `${desktop}/${doc.title}_${name}.png`;

      const records  = doc.export(path, opts, area, null);
      const rec      = records.all[0];
      console.log(rec?.isSuccess ? `OK: ${path}` : `FAIL: ${name}`);
    }
  }
}
```

```

    }
  }

```

13.7 Per-node export settings (ExportConfig)

Beyond scripting a one-off `doc.export()` call, each node can carry its own export configuration — the same settings the Export persona writes. These live on `node.exportableInterface.exportConfig` and follow a three-level hierarchy: **config** → **formats** → **sizes**, where each size wraps an `ExportScale` (multiplier, fixed width/height, square, or preset).

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { ExportScale, ExportSize } = require('/exportconfig.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node) { console.log('Select a node.'); }
else {
  const config = node.exportableInterface?.exportConfig;
  if (!config) {
    console.log('No export config on this node yet.');
  } else {
    for (const format of config.formats) {
      for (const size of format.sizes) {
        const s = size.exportScale;
        console.log('size — multiplier:', s.hasMultiplier ? s.multiplier :
`$${s.width}x$${s.height}`);
      }
    }

    if (config.formatCount > 0) {
      const twoX =
ExportSize.createWithExportScale(ExportScale.createWithMultiplier(2));
      config.formats[0].appendSize(twoX);
    }
  }
doc.executeCommand(DocumentCommand.createSetExportConfig(doc.selection,
config));
  console.log('2x size appended.');
}
}

```

`ExportScale` factories: `createWithMultiplier(m)`, `createWithWidth(w)`, `createWithHeight(h)`, `createWithSquare(size)`, `createWithWidthHeight(w, h)`, `createWithMultiplierSquare(m, size)`, and `createWithPreset(preset)` for the named presets. `ExportFormat.createWithFileExportOptions(options)` derives a format from the same `FileExportOptions` used by `doc.export()`. Read scales back via `sizeType` / `width` / `height` (companions to `hasMultiplier` / `multiplier` / `size`).

① **Note**
ExportConfig has no constructor. It can only be obtained from a node that already has one — set it up once via the UI Export persona, then read, modify (appendSize / replaceSize / deleteSize / enumerateSizes / sizes on formats; appendFormat / replaceFormat / deleteFormat / enumerateFormats / formats on the config; setExportScale / createWithExportScale on sizes), and re-apply with `DocumentCommand.createSetExportConfig(selection, config)`. For one-off scripted exports without per-node settings, plain `doc.export()` is simpler.

13.8 Async export with doc.promises

Use `DocumentPromises.getCurrent()` and `docP.export()` for async export workflows that don't block the UI.

```

'use strict';
async function exportAsync() {
  const { DocumentPromises, FileExportOptions, FileExportArea } =
require('/document.js');
  const { app } = require('/application.js');

  const docP = await DocumentPromises.getCurrent();
  if (!docP) { console.log('No doc.'); return; }
}

```

```

const opts    = FileExportOptions.createWithPresetName('PDF');
const area    = FileExportArea.createForWholeDocument();
const desktop = app.userDesktopPath;
const path    = `${desktop}/output.pdf`;

try {
  const records = await docP.export(path, opts, area, null);
  for (const rec of records.all) {
    console.log(rec.isSuccess ? 'Exported: ' + rec.path : 'Failed.');
```

```

  }
} catch (err) {
  console.log('Export error:', err);
}
}

exportAsync();
```

13.9 Saving the document

`doc.save()` writes in place; `doc.saveAs(path)` writes to a new location. New documents require `saveAs()` first.

```

'use strict';
const { Document } = require('/document.js');
const { app } = require('/application.js');

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  const desktop = app.userDesktopPath;

  // Save in place (fails if mustSaveAs is true)
  doc.save();

  // Save to a new path
  doc.saveAs(`${desktop}/my-file.af`);

  // Async versions
  // doc.promises.save();
  // doc.promises.saveAs(`${desktop}/my-file.af`);
}
```

13.10 File system — reading and writing text

Use `File` from `/fs` to read and write text files. Only the Desktop and explicitly granted folders are accessible.

```

'use strict';
const { File } = require('/fs.js');
const { Buffer } = require('/buffer.js');
const { app } = require('/application.js');

const desktop = app.userDesktopPath;

// Write a text file
const wf = new File(`${desktop}/output.txt`, 'w');
wf.writeString('Hello from Affinity SDK');
wf.close();
console.log('File written.');
```

```

// Read a text file
const rf = new File(`${desktop}/output.txt`, 'r');
const buf = Buffer.create(rf.length);
rf.read(buf, buf.length);
rf.close();
console.log(buf.toString());
```

Note
Desktop and explicitly granted folders. Paths outside the Desktop or a folder you have granted access to return `PERMISSION_DENIED`.

13.11 JSON sidecar pattern

A common pattern — export assets and write a JSON manifest alongside them:

```
'use strict';
const { Document, FileExportOptions, FileExportArea } = require('./
document.js');
const { app } = require('./application.js');
const { File } = require('./fs.js');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const manifest = [];

  for (const spread of doc.spreads) {
    for (let i = 0; i < spread.artboardCount; i++) {
      const ai = spread.getArtboard(i); // returns ArtboardInterface
      directly
      const name = ai.description ?? `artboard-${i}`;
      const path = `${desktop}/${name}.png`;

      const area = FileExportArea.createForArtboard(ai);
      const records = doc.export(path, opts, area, null);
      if (records.all[0]?.isSuccess) {
        manifest.push({ name, path, exported: new Date().toISOString() });
      }
    }
  }

  if (manifest.length === 0) {
    console.log('No artboards found — add artboards to your document
first.');
```

13.12 Network requests

① **Note**
Network access requires permission. In Affinity Settings, the user must enable network access for scripts. If not enabled, `HttpRequest.create()` throws `PERMISSION_DENIED`.

```
'use strict';
const { HttpRequest, RequestMethod } = require('./network.js');

// GET request — uses httpbin.org, a public HTTP testing service
try {
  const req = HttpRequest.create('https://httpbin.org/get',
  RequestMethod.Get);
  const result = req.do();
  const response = result.response;
  console.log('Status:', response.statusCode); // statusCode is an enum
  console.log('Body: ', response.content);
} catch (e) {
  console.log('Network error:', e.message);
  // PERMISSION_DENIED → enable network access in Affinity Settings
  // INVALID_HANDLE → the server could not be reached (bad URL or no
  connectivity)
}

// POST with JSON body
try {
  const req = HttpRequest.create('https://httpbin.org/post',
```

```

RequestMethod.Post);
req.setHeaderValue('Content-Type', 'application/json');
const result = req.do();
console.log('POST status:', result.response.statusCode); // statusCode
is an enum object – use == 200 not === 200
} catch(e) {
  console.log('Network error:', e.message);
}

```

13.13 Key gotchas

response.statusCode == 200 not

13.14 200

.statusCode is an enum object, not a number or string. Strict equality always returns false. Use loose equality (==) or statusCode.equals(200).

Paths outside the Desktop or a granted folder fail with PERMISSION_DENIED.

mustSaveAs on new documents. Call doc.saveAs() first before doc.save() or export – a fresh document has no path.

Check rec.isSuccess on every record. Export can partially succeed with warnings. Always iterate records.all and check each record.

13.15 Exercises

13.15.1 Exercise 9.1 – Export current spread

Export the current spread as a PNG to the Desktop, named after the document title.

13.15.2 Exercise 9.2 – Export all artboards

Export every artboard in the document as separate PNG files. Name each file after the artboard.

13.15.3 Exercise 9.3 – JSON manifest

After exporting all artboards, write a manifest.json to the Desktop listing each exported file path, artboard name, and timestamp.

13.15.4 Exercise 9.4 – Export with dialog

Combine Lesson 08 (Dialog) with this lesson: show a dialog to choose the export format and output path suffix, then export.

13.15.5 Exercise 9.5 – Export config inspector

Log the export configuration of every selected node (formats, sizes, scales). Where a config exists, append a 3x size and re-apply it with createSetExport Config.

13.16 SDK modules used in this lesson

- /document – Document, FileExportOptions, FileExportArea, Document Promises
- /exportconfig – ExportConfig, ExportFormat, ExportScale, ExportSize, ExportScalePreset
- /application – app
- /fs – File, Buffer, Directory, FileSystemPromises
- /network – HttpRequest, RequestMethod, HttpStatusCode

Note
Export area is optional – pass null to export the whole document by default.

13.17 Further examples

Example	What it shows
bitmapWriter.js	ExportConfig, ExportSize, pixel-level bitmap writing via /buffer and /rasterobject
logToFile.js	/fs file writing with LogFile, Directory, FileSystemPromises

13.18 See also

- Lesson 08 — Dialog UI
- Lesson 11 — AI Commands
- examples/

13.19 Examples

13.19.1 Export all artboards with JSON manifest

```
'use strict';

// Example 09a — Export all artboards with JSON manifest
// Lesson 09: Export & Files
//
// Exports every artboard in the document as PNG files to the Desktop.
// Writes a manifest.json alongside them.
//
// NOTE: spread.getArtboard(i) returns the ArtboardInterface directly.
// Pass it straight to FileExportArea.createForArtboard().
// The artboard name is in .description, not .name.

const { Document, FileExportOptions, FileExportArea } = require('./document.js');
const { app } = require('./application.js');
const { File } = require('./fs.js');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  const opts = FileExportOptions.createWithPresetName('PNG');
  const manifest = [];
  let exported = 0;
  let failed = 0;
  let total = 0;

  for (const spread of doc.spreads) total += spread.artboardCount;

  if (total === 0) {
    console.log('No artboards found. Add artboards to your document first.');
```

```
  } else {
    for (const spread of doc.spreads) {
      for (let i = 0; i < spread.artboardCount; i++) {
        const ai = spread.getArtboard(i); // returns
        ArtboardInterface directly
        const name = (ai.description ?? `artboard-${exported + failed + 1}`)
          .replace(/[^a-zA-Z0-9_-]/g, '_');
        const path = `${desktop}/${doc.title}_${name}.png`;

        const area = FileExportArea.createForArtboard(ai);
        const records = doc.export(path, opts, area, null);
        const rec = records.all[0];
```

```

    if (rec?.isSuccess) {
        console.log('OK:', path);
        manifest.push({
            artboard: ai.description ?? name,
            file:      path,
            timestamp: new Date().toISOString(),
        });
        exported++;
    } else {
        console.log('FAIL:', name, rec?.errorMessage ?? '');
        failed++;
    }
}
}
}

const manifestPath = `${desktop}/${doc.title}_manifest.json`;
const mf = File.create(manifestPath, 'w');
mf.writeString(JSON.stringify(manifest, null, 2));
mf.close();

console.log(`\nDone. ${exported} exported, ${failed} failed.`);
console.log('Manifest:', manifestPath);
}
}

```

13.19.2 Network requests

```

'use strict';

// Example 09b – Network requests
// Lesson 09: Export & Files
//
// Demonstrates GET and POST requests using the /network module.
// Uses httpbin.org – a public HTTP testing service that echoes requests
// back.
//
// Before running: enable network access in Affinity Settings > Scripting.

const { HttpRequest, RequestMethod } = require('/network.js');

// GET request
try {
    const req = HttpRequest.create('https://httpbin.org/get',
    RequestMethod.Get);
    const result = req.do();
    const response = result.response;
    console.log('GET status:', response.statusCode); // statusCode is an
enum object – use == 200 not === 200
    console.log('GET body: ', response.content);
} catch (e) {
    console.log('Network error:', e.message);
    // PERMISSION_DENIED → enable network access in Affinity Settings
    // INVALID_HANDLE → server could not be reached (bad URL or no
connectivity)
}

// POST with JSON body
try {
    const req = HttpRequest.create('https://httpbin.org/post',
    RequestMethod.Post);
    req.setHeaderValue('Content-Type', 'application/json');
    const result = req.do();
    console.log('POST status:', result.response.statusCode); // statusCode
is an enum object – use == 200 not === 200
    console.log('POST body: ', result.response.content);
} catch (e) {
    console.log('Network error:', e.message);
}

```

13.19.3 Node export config

```

'use strict';

// Example 09c — Node export config
// Lesson 09: Export & Files
//
// Reads the per-node export settings on a selected node, appends a 2x
// PNG size, and re-applies the config. Note: ExportConfig has no
// constructor — it can only be read from a node that already has one
// (set via the UI Export persona first).

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { ExportScale, ExportSize } = require('/exportconfig.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node) { console.log('Select a node.');
} else {
  const config = node.exportableInterface?.exportConfig;
  if (!config) {
    console.log('This node has no export config yet.');
    console.log('Set one up once via the UI (Export persona), then re-
run.');
  } else {
    console.log('Existing formats:', config.formatCount);
    for (const format of config.formats) {
      console.log('  format sizes:', format.sizeCount);
      for (const size of format.sizes) {
        const s = size.exportScale;
        console.log('    multiplier:', s.hasMultiplier ? s.multiplier :
`$${s.width}x${s.height}`);
      }
    }

    if (config.formatCount > 0) {
      const twoX =
ExportSize.createWithExportScale(ExportScale.createWithMultiplier(2));
      config.formats[0].appendSize(twoX);

doc.executeCommand(DocumentCommand.createSetExportConfig(doc.selection,
config));
      console.log('Appended 2x size — format now has',
config.formats[0].sizeCount, 'size(s).');
    }
  }
}
}

```


Lesson 10 — Text Threading

When a story is too long for a single text frame, the overflow text needs somewhere to go. In Affinity, you can **thread** text frames together so that overflow flows automatically from one frame into the next — like columns in a magazine or pages in a book. This lesson explains how to query, create, and break those threads programmatically.

Text threading is the foundation for multi-page layouts. A novel, a catalogue, or any document where text flows across pages relies on threaded frames. Without threading, overflow text simply vanishes — it still exists in the story, but there is nowhere on screen to see it. With threading, you control exactly where text goes when a frame fills up.

Two things are worth knowing before you start:

- Threaded frames share a **single story**. Reading `storyInterface.text` from any frame in the chain returns the full text across all linked frames, not just the portion visible in that one frame.
- The SDK lets you **query** thread state and **break** existing threads, but creating new threads programmatically requires care — the `createLinkTextFrame` command links two existing `FrameTextNodes`, and both must already exist on the spread.

By the end of this lesson you will be able to detect whether a text frame is part of a thread, link frames together so overflow text flows between them, and unlink them when you need to restructure a layout.

14.1 What this lesson covers

- How text threading works in Affinity
- Querying thread state with `TextFrameInterface` properties

- Linking two text frames with `DocumentCommand.createLinkTextFrame`
- Unlinking frames with `DocumentCommand.createUnlinkTextFrame`
- Reading text across a linked story
- Gotchas and limitations

14.2 How threading works

A text frame in Affinity has a `textFrameInterface` that exposes several properties for querying its thread state:

```
// Snippet – assumes node is a text frame node
const tfi = node.textFrameInterface;
console.log('Is multi-frame:', tfi.isMultiFrameTextFlow);
console.log('Is front of chain:', tfi.isTextFlowFront);
console.log('Is back of chain:', tfi.isTextFlowBack);
console.log('Flow index:', tfi.textFlowIndex);
```

Here is what each property means:

Property	Meaning
<code>isMultiFrameTextFlow</code>	true if this frame is part of a thread chain (two or more linked frames)
<code>isTextFlowFront</code>	true if this frame is the first in the chain (the story starts here)
<code>isTextFlowBack</code>	true if this frame is the last in the chain (overflow ends here)
<code>textFlowIndex</code>	The position of this frame in the chain (0 = first, 1 = second, etc.)

A frame that is not linked to any other frame will have `isMultiFrameTextFlow === false` and both `isTextFlowFront` and `isTextFlowBack` will also be false. Once you link two frames, the first reports `isTextFlowFront === true` and the last reports `isTextFlowBack === true`.

The `storyInterface` also has an `isMultiFrameTextFlow` property that reports the same thing — it is simply a convenience so you do not need to reach through `textFrameInterface` every time.

Check the capability guards before changing frame behaviour: `tfi.canHideOverflow`, `tfi.canUseBaselineGrid`, `tfi.canUseTextWraps` tell you whether the frame supports the corresponding flag, and `tfi.hasScaledText` reports scaled text.

14.3 Linking frames

To thread two frames together, use `DocumentCommand.createLinkTextFrame`. It takes two arguments: the **source** frame (where text flows from) and the **destination** frame (where overflow goes):

```
// Snippet – assumes frameA and frameB are FrameTextNode references
const doc = Document.current;
const spread = doc.currentSpread;

const cmd = DocumentCommand.createLinkTextFrame(frameA, frameB);
doc.executeCommand(cmd);
```

After linking, any text that overflows `frameA` will automatically flow into `frameB`. The two frames now share a single story. If you read the story text from either frame, you get the full content across both.

You can chain more than two frames — link `frameB` to `frameC`, and text flows `A → B → C`. Each frame's `textFlowIndex` reflects its position in the chain.

```
// Snippet – assumes frameA, frameB and frameC are FrameTextNode
references
// Chain three frames
doc.executeCommand(DocumentCommand.createLinkTextFrame(frameA, frameB));
doc.executeCommand(DocumentCommand.createLinkTextFrame(frameB, frameC));
```

14.4 Unlinking frames

To break a thread, use `DocumentCommand.createUnlinkTextFrame`. It takes a single argument — the frame where you want to break the chain:

```
// Snippet – assumes frameB is part of an existing thread chain
// Break the chain after frameB
// frameA's story ends at frameA; frameB and frameC form a new chain (or
// become independent)
doc.executeCommand(DocumentCommand.createUnlinkTextFrame(frameB));
```

Unlinking splits the story. The frame you pass to `createUnlinkTextFrame` and everything after it forms a separate chain (or becomes unlinked if it is the only frame left). The frames before it retain their original story.

After unlinking, overflow from the first chain no longer reaches the second. Text that was previously visible in the second chain may disappear if the frames are too small to contain their share of the story — but the text is still in the story, just not rendered.

14.5 Reading text across linked frames

When frames are threaded, they share one story. Reading `storyInterface.text` from any frame in the chain returns the **full** text — not just the portion visible in that particular frame:

```
// Snippet – assumes frameB is part of a thread chain
const si = frameB.storyInterface;
// Returns ALL text across the entire thread, not just what's in frameB
console.log('Full story:', si.text);
console.log('Story range:', si.storyRange);
```

This is a common source of confusion. If you have a 10,000-word story flowing across five frames, reading the text from frame 3 still gives you all 10,000 words. The `storyRange` covers the entire story, not the subset visible in that frame.

To determine which portion of the story is visible in a specific frame, you need the frame's `textBegin` property and the story ranges of adjacent frames. This is an advanced pattern — most scripts work with the full story text and let the layout engine handle the visual distribution.

14.6 Putting it together

Here is a minimal example that queries the thread state of every text frame in the document:

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  for (const node of doc.layers.all) {
    if (!node.isFrameTextNode) continue;
    const tfi = node.textFrameInterface;
    if (!tfi) continue;

    const name = node.descriptionInterface?.description || '(unnamed)';
    if (tfi.isMultiFrameTextFlow) {
      const pos = tfi.textFlowIndex;
      const role = tfi.isTextFlowFront ? '[FRONT]' :
        tfi.isTextFlowBack ? '[BACK]' : '';
    }
  }
}
```

```

        console.log(`${name}" - thread position ${pos}${role}`);
    } else {
        console.log(`${name}" - standalone (not threaded)`);
    }
}
}

```

This is useful for auditing a document — finding which frames are linked, which are the front and back of a chain, and which are standalone.

14.7 Key gotchas

Overflow text is invisible but still there. When a frame is too small to display all its text and the overflow has nowhere to go (no linked frame), the extra text still exists in the story. It just has no visible home. Linking a new frame to the back of the chain makes it reappear.

storyInterface.text returns everything. Do not assume that reading text from frame B in a chain gives you only frame B's portion. It gives you the entire story. Use textBegin and adjacent frame ranges to isolate a specific frame's visible text if needed.

storyInterface.isMultiFrameTextFlow mirrors textFrameInterface.isMultiFrameTextFlow. Both properties report the same value. Use whichever is more convenient — storyInterface if you are already working with the story, textFrameInterface if you are inspecting frame properties.

Linking requires both frames to exist on the spread. You cannot link a frame that has not been added to the document yet. Create both frames first, then link them.

The SDK does not support creating pre-linked frames in a single call. There is no FrameTextNodeDefinition.createLinked(pair) factory. You must create each frame individually and then issue createLinkTextFrame as a separate command.

FrameTextNodeDefinition.createFromStoryBuilder bounds are in points, not pixels. The SDK geometry system uses points as its native unit. At 300dpi, raw numbers like width: 200 produce frames only 53pt wide — barely visible. Always convert: const pxPerPt = doc.dpi / 72; { x: N * pxPerPt, y: N * pxPerPt, width: N * pxPerPt, height: N * pxPerPt }. Without this, frames appear as tiny slivers on the spread. See examples/10a-link-and-flow.js for a working example.

Always seed the StoryBuilder with frame-text defaults. Call sb.setToFrameTextDefaultStyle(doc.dpi, doc.rasterFormat) immediately after StoryBuilder.create() and before sb.addText(). Without it, text renders at an invisibly small size (0.24pt). See examples/10a-link-and-flow.js.

14.8 Exercises

14.8.1 Exercise 10.1 — Audit threaded frames

Write a script that finds all FrameTextNodes in the document and logs their thread state: whether they are threaded, their position in the chain, and whether they are the front or back. Group the output by chain.

14.8.2 Exercise 10.2 — Link two selected frames

Write a script that takes the first two FrameTextNodes in the current selection and threads them together using createLinkTextFrame. After linking, log the thread state of both frames to confirm the link.

14.8.3 Exercise 10.3 — Break and re-thread

Write a script that finds an existing thread chain, unlinks the middle frame, and then re-links it to create a different flow order. Log the thread state before and after to show the change.

14.9 SDK modules used in this lesson

- /document — Document, DocumentCommand
- /nodes — FrameTextNode, textFrameInterface
- /storybuilder — StoryBuilder

14.10 Further examples

Example	What it shows
breakFrame.js	Break a text frame at the cursor into two frames
splitStory.js	Split a story at the cursor into two separate text frames

14.11 See also

- Lesson 07 — Text & story
- examples/

14.12 Examples**14.12.1 Link and flow**

```
'use strict';

// Example 10a — Link and flow
// Lesson 10: Text Threading
//
// Creates two text frames on the current spread, fills the first with
// enough text to overflow, links them so the overflow flows into the
// second frame, then logs the thread state of both frames.

const { Document } = require('/document.js');
const { DocumentCommand, AddChildNodesCommandBuilder } = require('/commands.js');
const { StoryBuilder } = require('/storybuilder.js');
const { FrameTextNodeDefinition } = require('/nodes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  const spread = doc.currentSpread;
  if (!spread) { console.log('No current spread.')} }
  else {
    // Build a story with enough text to overflow a small frame
    const sb = StoryBuilder.create();
    sb.setToFrameTextDefaultStyle(doc.dpi, doc.rasterFormat);
    const longText = [
      'Text threading lets overflow flow from one frame into the next.',
      'This is essential for multi-page layouts like books and',
      'catalogues.',
      'When a frame fills up, the excess text automatically appears in',
      'the',
      'linked frame downstream. You can chain multiple frames together,',
      'and each frame in the chain shares a single story. Reading the',
      'story',
      'from any frame returns the full text across the entire chain, not',
      'just',
      'the portion visible in that particular frame. This shared-story',
      'model',
      'is what makes long-form text layout possible in Affinity.',
    ]
```

```

    ''
    'To link frames, use DocumentCommand.createLinkTextFrame(src,
dest).',
    'To break a link, use DocumentCommand.createUnlinkTextFrame(node).',
    'The textFrameInterface properties isMultiFrameTextFlow,
isTextFlowFront,',
    'isTextFlowBack, and textFlowIndex let you inspect the thread
state.',
    ''
    'This example creates two frames, links them, and logs the result.'
].join(' ');
sb.addText(longText);

// Bounds are in points (like all geometry in the SDK); convert to
document
// pixels for createFromStoryBuilder. At 300dpi: pts * 300/72 =
pixels.
// Using 100pt / 250pt / 350pt as convenient round-point values.
const pxPerPt = doc.dpi / 72;

// Create the first (source) frame — small so text overflows
const defA = FrameTextNodeDefinition.createFromStoryBuilder(
  { x: 100 * pxPerPt, y: 50 * pxPerPt, width: 250 * pxPerPt, height:
350 * pxPerPt },
  sb
);
const addA = AddChildNodesCommandBuilder.create();
addA.addNode(defA);
addA.setInsertionTarget(spread);
const cmdA = addA.createCommand(true);
doc.executeCommand(cmdA);

// Find the frame we just created (last node added)
const frameA = cmdA.newNodes[cmdA.newNodes.length - 1];
console.log('Created frame A:',
frameA.descriptionInterface?.description || '(unnamed)');

// Create the second (destination) frame — off to the right
const defB = FrameTextNodeDefinition.createFromStoryBuilder(
  { x: 400 * pxPerPt, y: 50 * pxPerPt, width: 250 * pxPerPt, height:
350 * pxPerPt },
  sb // reuse the same StoryBuilder so both frames have the long text
);
const addB = AddChildNodesCommandBuilder.create();
addB.addNode(defB);
addB.setInsertionTarget(spread);
const cmdB = addB.createCommand(true);
doc.executeCommand(cmdB);

const frameB = cmdB.newNodes[cmdB.newNodes.length - 1];
console.log('Created frame B:',
frameB.descriptionInterface?.description || '(unnamed)');

// Link frame A -> frame B
const linkCmd = DocumentCommand.createLinkTextFrame(frameA, frameB);
doc.executeCommand(linkCmd);
console.log('Linked frame A -> frame B');

// Query and log thread state
const tfiA = frameA.textFrameInterface;
const tfiB = frameB.textFrameInterface;

console.log('');
console.log('--- Thread state after linking ---');
console.log('Frame A:');
console.log(' isMultiFrameTextFlow:', tfiA.isMultiFrameTextFlow);
console.log(' isTextFlowFront:', tfiA.isTextFlowFront);
console.log(' isTextFlowBack:', tfiA.isTextFlowBack);

```

```
    console.log('  textFlowIndex:', tfiA.textFlowIndex);

    console.log('Frame B:');
    console.log('  isMultiFrameTextFlow:', tfiB.isMultiFrameTextFlow);
    console.log('  isTextFlowFront:', tfiB.isTextFlowFront);
    console.log('  isTextFlowBack:', tfiB.isTextFlowBack);
    console.log('  textFlowIndex:', tfiB.textFlowIndex);

    // Show that reading from either frame gives the full story
    const siA = frameA.storyInterface;
    const siB = frameB.storyInterface;
    console.log('');
    console.log('Story from frame A:', siA.text.substring(0, 80) + '...');
    console.log('Story from frame B:', siB.text.substring(0, 80) + '...');
    console.log('Same story?', siA.text === siB.text ? 'YES' : 'NO');
  }
}
```


Phase 4 — Advanced

PART

IV

Phase 4 — Advanced

15

Phase 3 gave you the tools to build scripts that interact with users, process text, and produce output. Phase 4 goes deeper — into the capabilities of the SDK that require more careful handling, reward more careful thinking, and unlock the most powerful automation possibilities the platform offers.

This phase covers seven areas that sit at the edge of what scripts can do: AI-powered image operations, layer effects and adjustments, production-grade async patterns, a survey of the SDK modules, direct pixel manipulation, programmatic font and typography control, and connecting to Affinity from external programs via MCP. None of these are prerequisites for writing useful scripts — you could spend months working productively with only Phases 1 through 3. But each lesson here opens a category of work that simply is not possible without it.

The AI commands lesson is particularly significant. Affinity’s generative capabilities — image generation, inpainting, background removal, depth mapping, colourisation — are available directly from JavaScript without any external API keys, quotas, or network dependencies. Scripts can drive them programmatically, chain them with export pipelines, and present them through dialogs in ways the standard UI does not support. **Note:** AI commands require a Canva account with an active Pro subscription — they are not available on free accounts.

Pixel processing is the other lesson that unlocks a qualitatively different class of scripts. Once you have direct read/write access to individual pixels, any image processing algorithm you can express mathematically becomes implementable: custom colour grading, histogram analysis, procedural texture generation, per-channel adjustments, and effects that depend on spatial relationships between pixels.

The pro patterns lesson ties the phase together — it covers the async techniques, error handling, and external communication that production scripts need when they move from personal experiments to tools used by a team.

15.1 Lessons

Lesson	Topic	Key skill you'll gain
11 — AI commands	<code>generateImage</code> , <code>removeBackground</code> , <code>selectSubject</code>	Driving Affinity's AI features from scripts
12 — Layer effects	Shadows, glows, adjustments, filters	Applying and batching visual effects across many nodes
13 — Pro patterns	Async pipelines, networking, timers, error handling	Writing scripts that are robust enough for production use
14 — Special modules	Pixel access, fonts, brushes, buffers, metadata	Using the full breadth of the SDK's module library
15 — Pixel processing	<code>PixelReaderWriterRGBA8</code> , <code>Bitmap</code> , <code>NodeRenderingEngine</code>	Direct per-pixel read and write on any raster layer
16 — Fonts & typography	<code>FontFamily</code> , <code>PANOSE</code> , font distance, typography automation	Inspecting, classifying, and applying fonts programmatically
17 — Raster adjustments	<code>BrightnessContrast</code> , <code>Levels</code> , <code>Curves</code> , <code>HSL</code> , all 19 types	Creating and modifying non-destructive adjustment layers
18 — Align & distribute	<code>GroupTransformData</code> , <code>GroupTransformType</code> , <code>GroupTransformAnchor</code>	Aligning and distributing nodes on any axis programmatically

15.2 What to expect

By the end of Phase 4 you should be able to write a script that:

- Generates an image from a text prompt and exports it in one pipeline
- Applies a consistent set of layer effects across every node in a document
- Processes every pixel of a raster layer to implement a custom image filter
- Enumerates all installed fonts, classifies them by type, and finds the closest substitute for a missing typeface
- Handles save failures on unsaved documents, network errors, and export failures gracefully
- Chains multiple async operations — execute, export, save — without blocking the UI
- Connects to Affinity from an external program via MCP and executes scripts remotely

You will also encounter the SDK's hardest-won lessons in this phase: that `LineType.Dash` is required before dashes render, that `mustSaveAs` must be checked before any save operation, and that pixel alpha is always `px.alpha` never `px.a`. These are not intuitive, they do not produce helpful error messages, and they

are documented here precisely because they cost time to discover independently.

Lesson 11 — AI Commands

Affinity exposes its AI capabilities directly to scripts. This lesson covers the seven AI operations — image generation, generative inpainting, background removal, subject selection, depth map generation, colourisation, and image tracing — and shows how to combine them with the rest of the SDK.

These commands run natively inside Affinity, so they respect the document model, integrate with the undo stack, and need no external API keys. Every AI command shares a setup requirement that's easy to miss: it must run on the correct spread, with the right selection, and be **awaited**. Scripts that call them synchronously or without an active spread fail silently. The sequencing — set spread, set selection, then await the command — is covered here so you don't have to discover it through failed runs.

The commands also differ in what they operate on:

- `generateImage` creates a new raster node from a prompt (no selection needed).
- `generativeEditImage` fills a masked area of an existing raster layer.
- `removeBackground` and `selectSubject` operate on the selected raster node.
- `detectDepth` and `colourise` produce a new layer from an existing one.
- `imageTrace` converts a raster to a vector `PolyCurve`.

By the end you'll be able to call all seven correctly, chain them with export pipelines, and drive them from dialog-driven workflows.

16.1 What this lesson covers

- `generateImage(prompt)` — AI image generation
- `generativeEditImage(prompt)` — inpainting / generative fill

Note
AI features require a paid Canva subscription. These APIs only work when the user is signed into a Canva account with an active subscription. Scripts that call AI commands silently fail or return errors without one.

- `removeBackground()` — background removal
- `selectSubject()` — smart subject selection
- `detectDepth()` — depth map generation
- `colourise()` — grayscale to colour
- `imageTrace()` — raster to vector
- Running AI commands correctly (spread, selection, async)

16.2 Why use the SDK AI commands?

① **Note**
Prefer these over external AI APIs. The SDK AI commands are native to Affinity, respect the edit model, integrate with the undo stack, and don't require network credentials.

16.3 Generate an image from a prompt

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No doc.'); }
else {
  // Set the spread first – generateImage places onto the CURRENT spread
  doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  doc.generateImage('A misty mountain landscape at dawn, digital art style');
  console.log('Image generation started.');
```

① **Note**
Set currentSpread explicitly before calling generateImage. The image is placed on whichever spread is current at the time the command runs.

16.4 Generative edit (inpainting)

Select the region you want to edit (use a raster selection or select a layer), then:

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const doc = Document.current;
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

// Requires a raster selection or selected image node
doc.generativeEditImage('Replace with a clear blue sky');
```

16.5 Remove background

`createRemoveBackground()` strips the background from a selected image node, leaving the subject on a transparent layer.

```
'use strict';
const { DocumentCommand } = require('/commands.js');
const { Document } = require('/document.js');

const doc = Document.current;
const node = doc.selection.nodes.filter(n => n.isImageNode || n.isRasterNode).first;

if (!node) {
  console.log('Select an image node first.');
```

16.6 Select subject

AI-powered selection of the main subject in the current image:

```
'use strict';
const { Document, DocumentCommand } = require('/document.js');
```



```

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));
doc.executeCommand(DocumentCommand.createSelectSubject());
console.log('Subject selection applied.')}
// The raster selection is now set – you can use it for further
operations
}

```

16.7 Detect depth

Generates a depth map from the selected image:

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
doc.detectDepth();
console.log('Depth detection started.')}
}

```

16.8 Colourise

Converts a grayscale image to colour:

```

'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
doc.colourise();
console.log('Colourisation started.')}
}

```

16.9 Image trace (raster to vector)

Converts the selected raster/image node into vector curves:

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else if (doc.selection.nodes.isEmpty) { console.log('Select a raster or
image node first.')} }
else {
const edgeThreshold = 0.5; // 0.0–1.0 – how sharp edges need
to be detected
const curveFittingTolerance = 2.0; // lower = tighter fit, more nodes

// doc.imageTrace is the convenience form
doc.imageTrace(edgeThreshold, curveFittingTolerance, doc.selection);
console.log('Image trace complete.')}
}

```

16.10 Batch image generation

Generate one image per prompt, each on its own artboard:

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

const PROMPTS = [
'A serene forest path in autumn',
'A futuristic cityscape at night',
'An abstract geometric pattern in blue and gold',

```

```

];

const doc = Document.current;
if (!doc) { console.log('No doc.');
```

16.11 Using AI commands with doc.promises (async)

For long-running AI operations, use `doc.promises` to avoid blocking the script (e.g. when chaining AI generation with export or document operations).

```

'use strict';
const { DocumentCommand } = require('/commands.js');
async function generateAndExport(prompt, outputPath) {
  const { DocumentPromises, FileExportOptions, FileExportArea } =
    require('/document.js');
  const docP = await DocumentPromises.getCurrent();
  if (!docP) return;

  const doc = docP.document;
  await
docP.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

  doc.generateImage(prompt);

  // Export after generation
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForCurrentSpread();
  const records = await docP.export(outputPath, opts, area, null);
  console.log(records.all[0]?.isSuccess ? 'Done.' : 'Failed.');
```

16.12 Dialog-driven image generator

Combine Dialog (Lesson 08) with AI commands for a practical tool:

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');

const doc = Document.current;
if (!doc) { console.log('No doc.');
```

```

doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));
    doc.generateImage(prompt);
    console.log('Generating:', prompt);
  }
}
}

```

16.13 Key gotchas

Always set currentSpread before AI commands. `generateImage`, `generativeEditImage`, `removeBackground`, and `detectDepth` all operate on the current spread. Forgetting this means the command runs on the wrong page.

AI commands are async under the hood. They trigger processing but return before it finishes. Use `executeCommandAsync` or `doc.promises.executeCommand` if you need to chain operations after completion.

`selectSubject` and `removeBackground` require an image node to be selected. They fail silently if the selection is wrong.

16.13.1 DocumentCommand factory equivalents

The AI commands shown here use `doc` convenience methods (`doc.generateImage()`, `doc.removeBackground()`, etc.). They also exist as `DocumentCommand` factories — `createRemoveBackground(sel)`, `createSelectSubject(sel)` — which let you combine them with `CompoundCommandBuilder` and pass explicit selections instead of relying on `doc.selection`.

16.14 Exercises

16.14.1 Exercise 11.1 — Single image generator

Build a dialog with a text box for a prompt. Generate an image on the current spread when the user confirms.

16.14.2 Exercise 11.2 — Background removal batch

Find all image nodes in the selection. Remove the background from each one.

16.14.3 Exercise 11.3 — Multi-prompt generator

Accept a multi-line prompt box where each line is a separate prompt. Generate one image per line.

16.14.4 Exercise 11.4 — Trace and style

Select a raster image. Run `imageTrace()` to vectorise it, then apply a solid colour fill to the result.

16.15 SDK modules used in this lesson

- `/commands` — `DocumentCommand`
- `/document` — `Document`, `DocumentPromises`
- `/dialog` — `Dialog`, `DialogResult`

16.16 Further examples

Example	What it shows
<code>boldItalics.js</code>	<code>DocumentCommand</code> for text attribute changes (bold/italic), <code>Dialog</code> usage

16.17 See also

- Lesson 09 — Export & Files
- Lesson 12 — Layer Effects
- `examples/`

16.18 Examples**16.18.1 Dialog-driven image generator**

```

'use strict';

// Example 10a — Dialog-driven image generator
// Lesson 10: AI Commands
//
// Shows a prompt dialog and generates an image on the current spread.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const dlg = Dialog.create('AI Image Generator');
  const col = dlg.addColumn();

  const grp1 = col.addGroup('Prompt');
  const promptBox = grp1.addTextBox('Describe the image you want', '');
  promptBox.isMultiline = true;
  promptBox.rowSpan = 5;

  const grp2 = col.addGroup('Options');
  const onNewSpread = grp2.addSwitch('Place on new spread', false);

  const result = dlg.runModal();

  if (result == DialogResult.Ok) {
    const prompt = promptBox.text.trim();

    if (!prompt) {
      console.log('Enter a prompt and try again.')}
    } else {
      // Always set current spread before AI commands
      doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

      doc.generateImage(prompt);
      console.log('Image generation started for prompt:');
      console.log(' ', prompt);
    }
  } else {
    console.log('Cancelled.')}
  }
}

```

Lesson 12 — Layer Effects & Adjustments

17

Layer effects — shadows, glows, outlines, bevels, blurs — are among the most visually impactful properties you can set from a script. This lesson covers the full effect system: applying and configuring individual effects, batching them across many nodes, applying adjustment layers, and using destructive filters on raster content.

Scripting effects is valuable wherever you need consistent visual treatment across layers: matching shadows on every icon in a set, a bloom on highlight layers, or clearing effects before re-exporting.

The effect system has three categories, each applied differently:

- **Live effects** (shadows, glows, outlines, bevels) — applied via individual `DocumentCommand` calls; they persist as non-destructive properties on the node.
- **Adjustment layers** (Levels, Curves, HSL, Brightness/Contrast) — created via `*AdjustmentRasterNodeDefinition` factories and added as live child nodes (see Lesson 16).
- **Destructive filters** (Blur, Noise, Vignette, Bloom) — applied directly to raster pixel data; they can only be removed with undo.

One pattern matters for live effects: the `enableIfDisabled` flag. Per-property effect commands (e.g. `createSetOuterShadowLayerEffectColour`) modify individual properties of an existing indexed effect — they do **not** replace the whole effects list. When `true`, `enableIfDisabled` adds the effect at that index if it doesn't exist; when `false`, it silently does nothing if the effect isn't already present. For reliable updates without duplicates, the safer pattern is: read existing effects from `node.layerEffectsInterface.effects`, clone the effect

at the target index, mutate its properties, then apply back via the full effect setter command.

By the end you'll be able to apply live effects to any node, set adjustment-layer parameters, apply destructive filters, and clear or replace effects in bulk.

17.1 What this lesson covers

- Outer shadow, outer glow, inner shadow, inner glow
- Colour overlay, gradient overlay, bevel and emboss, phong bevel
- Outline (stroke effect) and Gaussian blur layer effect
- Adjustment layer parameters (set on existing adjustment nodes)
- Destructive filters on raster nodes (`create*Filter` factory methods)
- The `enableIfDisabled` flag and index parameter
- Removing all effects at once
- Drawing scale and margins
- Raster brush properties

17.2 Effect categories

The SDK splits effects into three categories:

Category	Examples	How they're applied
Layer effects	Outer/Inner Shadow, Outer/Inner Glow, Outline, Bevel & Emboss, Colour Overlay, Gaussian Blur	Individual <code>doc.executeCommand</code> calls per property using <code>Selection.create(doc, node)</code>
Adjustment layers	Levels, Curves, HSL, Vibrance	Created via <code>*AdjustmentRasterNodeDefinition</code> factories and/or modified via parameter commands
Destructive filters	Blur, Noise, Vignette, Bloom	Raster nodes only — use <code>create*Filter</code> factory methods

17.3 Outer shadow

Apply shadow effects using `OuterShadowLayerEffect.create()` to build a complete effect object, then apply it with `createSetOuterShadowLayerEffect`. This is more reliable than individual property commands which have inconsistent argument orders.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');
const { OuterShadowLayerEffect } = require('/layereffects.js');

const doc = Document.current;
const node = doc.selection.nodes.first;

if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);
  const effect = OuterShadowLayerEffect.create();

  effect.colour = RGBA8(0, 0, 0, 180);
  effect.offset = 6; // pixels
  effect.angle = 315 * Math.PI / 180; // 315 degrees =
```

```

bottom-right (radians)
  effect.radius = 8; // blur radius in
pixels
  effect.opacity = 0.7; // 0.0-1.0
  effect.enabled = true;

doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffect(sel,
effect, 0));

console.log('Shadow applied.');
```

17.4 The 'enableIfDisabled' parameter

Most per-property effect commands accept `enableIfDisabled` as a final parameter. When `true`, the effect is automatically added at that index if it doesn't already exist. When `false`, the command silently does nothing if the effect isn't present.

```

// Snippet – for per-property commands only (not
createSetOuterShadowLayerEffect)
// If there's no outer shadow, this creates one AND sets its radius
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 10, true);

// If there's no outer shadow, this does nothing
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 10, false);
```

17.5 The index parameter

Effects that can be duplicated (`OuterShadow`, `InnerShadow`, `Outline`, `ColourOverlay`, `GradientOverlay`) use an index to address each instance. The first effect is index 0.

```

// Snippet – per-property commands with correct argument orders
// Radius: (sel, index, value, enable) – index before value
DocumentCommand.createSetOuterShadowLayerEffectRadius(sel, 0, 8, true);

// Opacity: (sel, value, index, enable) – value before index
(inconsistent!)
DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel, 0.7, 0, true);

// Duplicate the first shadow, then address index 1
compound.addCommand(DocumentCommand.createDuplicateOuterShadowLayerEffect(sel,
0));
compound.addCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
1, 20, false));
```

17.6 Outline (stroke effect)

`Outline` adds a stroke as a layer effect (separate from the line style stroke on vector nodes). By default the outline uses a solid colour, but you can also use a `FillDescriptor` (gradient, bitmap fill, etc.) via `createSetOutlineLayerEffect Fill` for gradient outlines.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { RGBA8 } = require('/colours.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);
```

① Note

Note: `createSetOuterShadowLayerEffectOffsetAngle` and the `.angle` property use **radians**, not degrees. Convert with `degrees * Math.PI / 180`. `OuterShadowLayerEffect.create()` returns `enabled = false` and `opacity = 0` by default – set these explicitly or the shadow will be invisible.

① Note

The full `createSetOuterShadowLayerEffect(sel, effect, index)` setter shown in the example above does not use `enableIfDisabled` – it creates or replaces the effect at the given index directly. Prefer this approach over per-property commands for reliability.

① Note

The per-property commands have inconsistent argument orders: `radius` uses (index, value) while `opacity` uses (value, index). The full effect setter avoids this problem entirely.

```

doc.executeCommand(DocumentCommand.createSetOutlineLayerEffectColour(sel,
RGBA8(255, 0, 0, 255), 0, true)); // bright red

doc.executeCommand(DocumentCommand.createSetOutlineLayerEffectRadius(sel,
0, 20, true)); // 20px outline
}

```

For a gradient outline, use `createSetOutlineLayerEffectFill` with a `FillDescriptor` instead of a solid colour:

```

// Snippet – assumes sel, doc are defined
const { Gradient } = require('/colours.js');
const { GradientFill, FillDescriptor } = require('/fills.js');
const { RGBA8 } = require('/colours.js');

const gradient = Gradient.create([
  { colour: RGBA8(255, 0, 0, 255), position: 0 },
  { colour: RGBA8(0, 0, 255, 255), position: 1 },
]);
const gradFill = GradientFill.create(gradient, 0); // 0 = linear
const fill = FillDescriptor.create(gradFill, true);

doc.executeCommand(DocumentCommand.createSetOutlineLayerEffectFill(sel,
fill, 0, true));

```

17.7 Inner glow

Applies a glow effect inside the edges of the layer — uses colour, radius, opacity, and intensity.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { RGBA8 } = require('/colours.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectColour(sel,
  RGBA8(255, 200, 0, 255), true)); // bright yellow

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectRadius(sel,
  50, true)); // 50px radius

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectOpacity(sel,
  1.0, true)); // full opacity

  doc.executeCommand(DocumentCommand.createSetInnerGlowLayerEffectIntensity(sel,
  1.0, true)); // full intensity
}

```

17.8 Gaussian blur (layer effect)

A live blur effect applied to the entire layer — non-destructive and adjustable after application.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (!node) { console.log('Select something first.')}
else {
  const sel = Selection.create(doc, node);

```



```

doc.executeCommand(DocumentCommand.createSetGaussianBlurLayerEffectRadius(sel,
30, true)); // 30px – clearly visible

doc.executeCommand(DocumentCommand.createSetGaussianBlurLayerEffectOpacity(sel,
1.0, true));
}

```

17.9 Outer glow

A glow effect radiating outward from the layer edges — set colour, radius, opacity, and intensity.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectColour(sel,
RGBA8(255, 230, 100, 255), true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectRadius(sel,
25, true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectOpacity(sel,
0.9, true));

  doc.executeCommand(DocumentCommand.createSetOuterGlowLayerEffectIntensity(sel,
0.8, true));
  console.log('Outer glow applied.');
```

17.10 Inner shadow

Inner shadow uses an index (like outer shadow) and adds an intensity parameter.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectColour(sel,
RGBA8(0, 0, 0, 200), 0, true));

  doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectOffsetAngle(sel,
8, 315, 0, true));

  doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectRadius(sel,
0, 15, true));

  doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectOpacity(sel,
0, 0.7, true));

  doc.executeCommand(DocumentCommand.createSetInnerShadowLayerEffectIntensity(sel,
0, 0.5, true));
  console.log('Inner shadow applied.');
```

17.11 Colour overlay

Colour overlay tints the entire layer with a solid colour at a given blend mode.

```

'use strict';
const { Document } = require('/document.js');
```

```

const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetColourOverlayLayerEffectColour(sel,
  RGBA8(255, 0, 100, 255), 0, true));

  doc.executeCommand(DocumentCommand.createSetColourOverlayLayerEffectOpacity(sel,
  0, 0.5, true));
  console.log('Colour overlay applied.');
```

17.12 Bevel and emboss

Bevel and emboss adds a 3D lighting effect. Unlike indexed effects, there is only one bevel per layer.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
if (node) {
  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectRadius(sel,
  10, true));

  doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectDepth(sel,
  2.0, true));

  doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectOpacity(sel,
  0.8, true));

  doc.executeCommand(DocumentCommand.createSetBevelEmbossLayerEffectDirection(sel,
  135, 45, true)); // azimuth, elevation
  console.log('Bevel and emboss applied.');
```

17.12.1 Additional effect commands

- `createSetAllLayerEffectsScaleWithObject(sel, scale)` — toggle whether all effects on the node scale when the node is transformed
- `createSetOuterShadowLayerEffectKnocksOut(sel, knocksOut, index, enableIfDisabled)` — toggle shadow knock-out per instance
- `createSetBevelEmbossLayerEffectAzimuth(sel, azimuth, enableIfDisabled)` — set bevel light angle independently
- `createSetBevelEmbossLayerEffectElevation(sel, elevation, enableIfDisabled)` — set bevel light height independently

17.12.2 Effect instance management

Every stackable effect (OuterShadow, InnerShadow, OuterGlow, InnerGlow, BevelEmboss, PhongBevel, ColourOverlay, GradientOverlay, Outline, GaussianBlur) shares the same management pattern — duplicate to stack a second instance, remove one instance, move instances to reorder, and toggle blend mode / enabled / scale-with-object per effect:

```

// Snippet — assumes sel is a Selection
// Duplicate (stack a second instance)
doc.executeCommand(DocumentCommand.createDuplicateOuterShadowLayerEffect(sel));
// Remove one instance (by index)
doc.executeCommand(DocumentCommand.createRemoveInnerGlowLayerEffect(sel,
0));
```

```
// Reorder stacked instances
doc.executeCommand(DocumentCommand.createMoveColourOverlayLayerEffect(sel,
0, 1));
// Per-effect blend mode, enabled toggle, scale-with-object
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectBlendMode(sel,
'Multiply', 0, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectEnabled(sel,
true, 0, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectScaleWithObject(sel,
true, 0, true));
```

The createDuplicate*, createRemove* (beyond createRemoveAllLayerEffects), createMove*, and createSet*LayerEffectBlendMode/Enabled/ScaleWithObject families exist for all ten effects with identical signatures — substitute the effect name.

17.13 Remove all effects

createRemoveAllLayerEffects clears every live effect from the selected node at once.

```
// Snippet — assumes doc and sel are already defined
doc.executeCommand(DocumentCommand.createRemoveAllLayerEffects(sel));
console.log('All effects removed.');
```

17.14 Gradient overlay

Gradient overlay applies a gradient fill on top of the layer. Like ColourOverlay and Outline it is indexed — you can stack multiple gradient overlays.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { GradientFill } = require('/fills.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const sel = Selection.create(doc, node);

  // Enable a gradient overlay at index 0 (creates it if not present)
  const gf = GradientFill.createDefault(); // black-to-white linear gradient
  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectFill(sel,
  gf, true));

  // Adjust blend mode and opacity
  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectBlendMode(sel,
  doc.blendMode, 0, true));
  doc.executeCommand(DocumentCommand.createSetGradientOverlayLayerEffectOpacity(sel,
  0.6, 0, true));

  console.log('Gradient overlay applied.');
```

Remove a specific gradient overlay with createRemoveGradientOverlayLayerEffect(sel, index).

17.14.1 GradientOverlayLayerEffect.calculateTransform

A static helper for computing the fill transform for gradient overlay effects:

```
// Snippet
const { GradientOverlayLayerEffect } = require('/layereffects.js');
const xf = GradientOverlayLayerEffect.calculateTransform(1, 1, 0, 0);
// scaleX, scaleY, transformX, transformY → returns a Transform
```

OutlineLayerEffect has the same static helper: OutlineLayerEffect.calculateTransform(scaleX, scaleY, transformX, transformY).

17.15 Phong bevel

Phong bevel is a lighting-based 3D bevel effect distinct from the flat Bevel & Emboss. It adds ambient, diffuse, and specular light parameters for a more realistic surface simulation.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const sel = Selection.create(doc, node);

  // Radius (bevel depth in pixels), opacity
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectRadius(sel,
    8, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectOpacity(sel,
    0.9, true));

  // Lighting parameters
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectAmbient(sel,
    0.2, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectDiffuse(sel,
    0.8, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectShininess(sel,
    40, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectSpecular(sel,
    0.5, true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectDepth(sel,
    5.0, true));

  // Ambient and specular colours
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectAmbientColour(sel,
    RGBA8(255, 255, 255, 255), true));
  doc.executeCommand(DocumentCommand.createSetPhongBevelLayerEffectSpecularColour(sel,
    RGBA8(255, 255, 220, 255), true));

  console.log('Phong bevel applied.');
```

Remove with createRemovePhongBevelLayerEffect(sel).

Adjustment layer parameter commands modify an *existing* adjustment layer. Adjustment layers can also be created programmatically via their *AdjustmentRasterNodeDefinition factory classes — see **Lesson 17 — Raster Adjustments** for the full workflow. The snippet below shows how to modify parameters on an existing node:

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { Nodes } = require('/nodes.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;
```

```

if (!node) {
  console.log('Select a Brightness/Contrast adjustment layer first.');
```

```

} else if (!(node instanceof
Nodes.BrightnessContrastAdjustmentRasterNode)) {
  console.log('Selected node is not a Brightness/Contrast adjustment
layer.');
```

```

} else {
  const sel = Selection.create(doc, node);
doc.executeCommand(DocumentCommand.createSetBrightnessContrastAdjustmentParameters(sel,
{
  brightness: 0.5,    // -1.0 to 1.0
  contrast:   0.4,    // -1.0 to 1.0
  isLinear:   false
}));
  console.log('Parameters set.');
```

```

}

```

17.16 Destructive filters on raster nodes

Destructive filters modify pixel data directly and cannot be undone without the undo stack. They only work on raster nodes — applying them to a shape or group is a silent no-op.

There are two sets of filter commands:

- `createGaussianBlurFilter(sel, params)` — creates and applies the filter. Use this.
- `createSetGaussianBlurFilterParameters(sel, params)` — sets parameters on an *already existing* filter. Same pattern as adjustment layers — silent no-op if no filter exists yet.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, RasterSelectionLogicalOperation } = require('/
commands.js');
const { HalftoneDotType, HalftoneScreenType } = require('affinity:dom');
```

```

const doc = Document.current;
if (!doc) { console.log('No document open.');
```

```

} else {
  // createGaussianBlurFilter — creates and applies in one step
doc.executeCommand(DocumentCommand.createGaussianBlurFilter(doc.selection,
{ radius: 20 }));

doc.executeCommand(DocumentCommand.createVignetteFilter(doc.selection, {
  radius:    0.6,
  intensity: 0.8
}));

doc.executeCommand(DocumentCommand.createAddNoiseFilter(doc.selection, {
  intensity: 0.3,
  monochromatic: false
}));

// Halftone filter
doc.executeCommand(DocumentCommand.createHalftoneFilter(doc.selection, {
  dotType:    HalftoneDotType.Dot,
  screenType: HalftoneScreenType.Frequency,
  frequency:  20,
  angle:      45,
  dotSize:    0.5
}));

console.log('Destructive filters applied.');
```

```

}

```

① **Note**
Adjustment parameter commands require an adjustment node. Calling a parameter command on a shape, group, or `doc.selection` directly is a silent no-op. Always build the selection with `Selection.create(doc, node)` where `node` is the adjustment layer.

① **Note**
Use `create*Filter` not `createSet*FilterParameters` to apply a new filter. The `createSet*` variants modify an existing filter and are silent no-ops if none exists yet.

17.16.1 More live-filter families

The same add-then-edit pattern covers every filter family — `createXFilter(sel, params)` to apply, `createSetXFilterParameters(sel, params)` to edit an existing one:

Family	Effect
BilateralBlur	Edge-preserving blur
BoxBlur	Uniform box blur
MedianBlur	Median noise reduction
MaximumBlur / MinimumBlur	Max/min (dilate/erode-style) blurs
MotionBlur	Directional motion blur
LensBlur	Aperture blur
FieldBlur	Spatially-varying blur
DepthOfField	Depth-based focus blur
DiffuseGlow / Diffuse	Soft glow diffusion / grain diffusion
Bloom	Highlight bloom
Clarity	Local-contrast clarity
HighPass / UnsharpMask	Sharpening filters
Denoise	Noise reduction
DustAndScratch	Dust/scratch removal
Defringe	Chromatic-fringe removal
Voronoi	Voronoi cell pattern
Pixelate	Mosaic pixelation
Ripple / Twirl / Spherical / Pinch-Punch / RadialBlur	Distortions
ShadowsHighlights	Filter (not adjustment) version of shadows/highlights

17.17 Drawing scale & margins

Read page geometry and drawing scale properties from the current spread and document preset.

```
'use strict';
const { Document } = require('/document.js');
const { DrawingScale } = require('/drawingscale.js');

const doc = Document.current;
const spread = doc.currentSpread;

// Margins - spread.marginsInterface returns undefined
// Use physicalRootPropertiesInterface for page geometry instead
const prpi = spread.physicalRootPropertiesInterface;
console.log('Page count:', prpi?.pageCount);
const box = prpi?.pageBoxInterface?.getPageBoundingBox(0);
console.log('Page box:', box);

// Drawing scale on the document preset
// IMPORTANT: pass a raw integer - NOT UnitType.* (those return undefined
// and cause INVALID_ARGS)
// 15 = Metres, 8 = Millimetres, 9 = Centimetres, 11 = Inches, 5 = Pixels
```

```
const defaults = DrawingScale.getDefaults(15); // 15 = Metres
console.log(`${defaults.length} standard metre scales available`);
for (const s of defaults.slice(0, 5)) {
  console.log(' ', s.toString(false, true, false));
}

// Create a custom scale
const myScale = DrawingScale.createFromIntegers(1, 200, true);
console.log('My scale:', myScale.toString(false, true, false));
```

17.18 Raster brush properties

The `/rasterbrush` module exposes raster brush dynamics. Access is via a brush handle from the application context (when a brush is active in the UI):

```
'use strict';
const { RasterBrush, BrushDynamic, BrushDynamicControllerType } =
require('/rasterbrush.js');

// Once you have a brush handle (from app context):
// const brush = ...;

// All brush dynamic properties have .value, .variance,
// and .controllerType
// brush.size.value           - base diameter
// brush.flow.value           - flow rate (0-1)
// brush.opacity              - brush opacity (0-1)
// brush.spacing              - spacing between dabs
// brush.hardness.value        - edge hardness
// brush.angle.value          - brush angle
// brush.scatterX.value        - X scatter
// brush.scatterY.value        - Y scatter
// brush.hueShift.value        - hue shift (HSL)
// brush.satShift.value        - saturation shift
// brush.lumShift.value        - luminance shift
// brush.wetEdges              - wet edges on/off
// brush.blendMode             - blend mode enum
```

17.19 Key gotchas

create*Filter not createSet*FilterParameters to apply a new filter. `createSetGaussianBlurFilterParameters` modifies an existing filter — silent no-op if none exists. Use `createGaussianBlurFilter` to create and apply in one step.

Adjustment layers are created via static factories, not new. Use `*Adjustment RasterNodeDefinition.create(params)` or `.createDefault()` — instantiating nodes directly with `new` throws `Invalid handle`. See Lesson 17 for the full workflow.

Effect commands do not work inside `CompoundCommandBuilder`. Wrapping effect commands in a compound command is a silent no-op — the effect appears in the panel but doesn't render. Execute each effect command individually with `doc.executeCommand`.

Use `Selection.create(doc, node)` not `doc.selection`. Effect commands require a per-node selection built with `Selection.create`. Passing `doc.selection` directly silently does nothing.

The argument order is `(sel, opacity, index, enableIfDisabled)` — not `(sel, index, opacity, enableIfDisabled)`. Passing the index value (0) as opacity makes the shadow fully transparent with no error or warning.

The index parameter. For effects that support multiple instances (shadows, outlines), always pass the index explicitly. Forget it and you'll address the wrong instance.

Note

`RasterBrush.createDefault()` is not implemented. The only way to obtain a `RasterBrush` instance is from a brush handle when the C++ layer provides one from tool state. There is no direct SDK API to get the currently active brush.

Check `adjustment_ranges` and `filter_ranges` docs in the SDK for valid parameter ranges before applying numeric values — out-of-range values may be clamped or rejected silently.

`removeAllLayerEffects` vs removing one. Use `createRemoveOuterShadowLayerEffect(sel, index)` to remove a specific effect; `createRemoveAllLayerEffects(sel)` removes everything.

① **Note**
`enableIfDisabled: true` for new effects. Without it, setting a property on a non-existent effect does nothing silently.

17.20 Exercises

17.20.1 Exercise 12.1 — Shadow batch

Apply a consistent drop shadow (6px offset, 45° angle, 8px blur, 70% opacity) to every selected node using a compound command.

17.20.2 Exercise 12.2 — Glow ring

Apply an outer glow and an inner glow to selected nodes. Make the outer glow amber and the inner glow white.

17.20.3 Exercise 12.3 — Effect cleaner

Write a script that removes all layer effects from every node in the document.

17.20.4 Exercise 12.4 — Adjustment stack

Select an image node. Apply a Brightness/Contrast adjustment and a Vibrance adjustment in sequence.

17.21 SDK modules used in this lesson

- `/commands` — `DocumentCommand`, `CompoundCommandBuilder`
- `/layereffects` — layer effect parameter types
- `/colours` — RGBA8
- `/drawingscale` — `DrawingScale`
- `/rasterbrush` — `RasterBrush`
- `adjustment_ranges` — valid parameter ranges for adjustments
- `filter_ranges` — valid parameter ranges for filters

17.22 See also

- Lesson 11 — AI Commands
- Lesson 13 — Pro Patterns
- `examples/`

17.23 Examples

17.23.1 DIAGNOSTIC — Shadow command tester

```
'use strict';

// DIAGNOSTIC — Shadow command tester
// Lesson 11: Layer Effects
//
// Tests createSetOuterShadowLayerEffect with a properly constructed
// OuterShadowLayerEffect object — then shows per-property commands
// with verified argument orders as alternatives.
//
// Select a single object and run. Check the canvas after each log line.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');
const { OuterShadowLayerEffect } = require('/layereffects.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const nodes = doc.selection.nodes;
```



```

if (nodes.isEmpty) { console.log('Select one object and run again.')} }
else {
  const node = nodes.first;
  const sel = Selection.create(doc, node);

  console.log('Running shadow diagnostic...');

  // Full effect setter approach – most reliable
  const effect = OuterShadowLayerEffect.create();
  effect.colour = RGBA8(255, 0, 0, 255); // bright red
  effect.offset = 30; // 30px distance
  effect.angle = 315 * Math.PI / 180; // 315 degrees = bottom-
right (in radians)
  effect.radius = 20; // 20px blur
  effect.opacity = 1.0; // 100%
  effect.enabled = true;

  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffect(sel,
effect, 0));
  console.log('1. Full effect setter applied – shadow should be clearly
visible bottom-right');

  // Per-property commands with verified argument orders (for reference)
  // NOTE: these have inconsistent argument orders – use the full setter
above instead

  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
RGBA8(0, 255, 0, 255), 0, true));
  console.log('2. Colour changed to green via per-property command');

  // Opacity: (sel, value, index, enable) – value BEFORE index
  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel,
1.0, 0, true));
  console.log('3. Opacity set to 100%% via per-property command – (sel,
value, index, enable)');

  // Radius: (sel, index, value, enable) – index BEFORE value
  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
0, 20, true));
  console.log('4. Radius set to 20px via per-property command – (sel,
index, value, enable)');

  // Offset+Angle: (sel, offset, angle, index, enable) – angle in
RADIANS
  doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel,
30, 315 * Math.PI / 180, 0, true));
  console.log('5. Offset+angle set via per-property command – angle in
RADIANS (315deg = 5.498rad)');
}
}

```

17.23.2 Shadow batch

```

'use strict';

// Example 11a – Shadow batch
// Lesson 11: Layer Effects
//
// Applies a consistent drop shadow to every selected node.
//
// Uses createSetOuterShadowLayerEffect with a properly constructed
// OuterShadowLayerEffect object – more reliable than individual
// property commands which have inconsistent argument orders.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { RGBA8 } = require('/colours.js');

```

```

const { OuterShadowLayerEffect } = require('/layereffects.js');

const SHADOW_COLOUR = RGBA8(0, 0, 0, 180);
const SHADOW_OFFSET = 6; // pixels
const SHADOW_ANGLE = 315; // degrees (315 = bottom-right)
const SHADOW_RADIUS = 8; // blur radius in pixels
const SHADOW_OPACITY = 0.7; // 0.0-1.0

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const nodes = doc.selection.nodes;
  if (nodes.isEmpty) {
    console.log('Select some objects and run again.')}
  else {
    const angleRad = SHADOW_ANGLE * Math.PI / 180;

    for (const node of nodes) {
      const sel = Selection.create(doc, node);
      const effect = OuterShadowLayerEffect.create();

      effect.colour = SHADOW_COLOUR;
      effect.offset = SHADOW_OFFSET;
      effect.angle = angleRad;
      effect.radius = SHADOW_RADIUS;
      effect.opacity = SHADOW_OPACITY;
      effect.enabled = true;

      doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffect(sel,
        effect, 0));
    }

    console.log(`Drop shadow applied to ${nodes.length} node(s).`);
  }
}

```

Lesson 13 — Pro Patterns: Async, Hints & Network

18

Once your scripts work, the next step is making them work reliably in production — handling errors gracefully, chaining async operations safely, talking to external services, and saving scripts for reuse. This lesson covers `doc.promises` for async workflows, HTTP networking, timers, and the community hint system.

These patterns matter when scripts move from personal experiments to tools other people use. A script that crashes on an unsaved document, ignores a failed export, or blocks the UI during a long operation is frustrating. The techniques here — promise chaining, `mustSaveAs` checks, error handling, and the hint pool — are what separate a finished tool from a fragile prototype.

Two ideas are worth holding onto:

- **Async has a specific shape.** `doc.promises` wraps operations like `executeCommand`, `export`, `save`, and `saveAs` in Promises you can await. The synchronous `doc.executeCommand/doc.export` do **not** return promises and can't be awaited. Mixing both in one async function is a common source of ordering bugs — pick one style and use it throughout.
- **Always check `mustSaveAs` before saving.** New, never-saved documents have no file path; `doc.mustSaveAs` is true and `doc.promises.save()` fails on them. Branch on `mustSaveAs` to decide between `saveAs()` and `save()`.

The hint system — `search_sdk_hints` and `add_sdk_hint` — is a community knowledge base in the SDK tooling. When the docs don't cover a behaviour, searching the hint pool often surfaces a working pattern. When you discover something new, contributing a hint makes it available to everyone. Checking

the hint pool before debugging from scratch is one of the highest-leverage habits in SDK development.

18.1 What this lesson covers

- The `doc.promises` API — full Promise-based workflow
- Chaining async operations safely
- Error handling in async scripts
- `network.js` — HTTP from scripts
- `timers.js` — timeouts and intervals
- `search_sdk_hints` — community knowledge base
- `add_sdk_hint` — contributing solutions back
- `save_script_to_library` — saving production scripts
- Building robust, production-grade scripts

18.2 The `doc.promises` API

Every document operation has a Promise-based equivalent under `doc.promises`. This is the preferred pattern for scripts that do multiple sequential operations.

```
'use strict';
const { Document, DocumentPromises } = require('/document.js');
const { app } = require('/application.js');

async function main() {
  // Get current document (Promise-based)
  const docP = await DocumentPromises.getCurrent();
  if (!docP) {
    console.log('No document open.');
    return;
  }

  const doc = docP.document;
  console.log('Document:', doc.title);

  // Save — new/unsaved documents need saveAs first
  try {
    if (doc.mustSaveAs) {
      // Document has never been saved — save to Desktop first
      const desktop = app.userDesktopPath;
      await docP.saveAs(`${desktop}/${doc.title || 'untitled'}.af`);
      console.log('Saved to Desktop.');
    } else {
      await docP.save();
      console.log('Saved.');
    }
  } catch (err) {
    console.log('Save failed:', err);
  }
}

main();
```

`doc.mustSaveAs` is true for unsaved documents. Calling `docP.save()` on a document that has never been saved throws `SAVE_TO_TEMPORARY_ARCHIVE_ERROR`. Always check `doc.mustSaveAs` and use `docP.saveAs(path)` with a Desktop path for new documents.

18.3 Processing and exporting the current document

```
'use strict';
const { DocumentPromises, FileExportOptions, FileExportArea } = require('/document.js');
const { app } = require('/application.js');

async function main() {
```

```

const docP = await DocumentPromises.getCurrent();
if (!docP) { console.log('No document open.');
```

`return;`

```

}

const doc = docP.document;
const desk = app.userDesktopPath;

// Export as PNG to the Desktop
const opts = FileExportOptions.createWithPresetName('PNG');
const area = FileExportArea.createForWholeDocument();
const outPath = `${desk}/${doc.title}.png`;

try {
  const records = await docP.export(outPath, opts, area, null);
  const rec = records.all[0];
  console.log(rec?.isSuccess ? 'Exported: ' + outPath : 'Export
failed.');
```

`catch`

```

  (err) {
    console.log('Export error:', err);
  }
}

main();
```

18.4 Full async pipeline

Load → generate AI image → export → save → close, all chained safely:

```

'use strict';
const { DocumentPromises, FileExportOptions, FileExportArea } = require('./
document.js');
const { DocumentCommand } = require('/commands.js');
const { app } = require('/application.js');

async function generateAndExport(prompt) {
  const desktop = app.userDesktopPath;
  const docP = await DocumentPromises.getCurrent();
  if (!docP) throw new Error('No document open.');
```

`const doc = docP.document;`

```


  // Set spread before AI command
  await docP.executeCommand(
    DocumentCommand.createSetCurrentSpread(doc.currentSpread)
  );

  // Generate image (async – resolves when generation starts)
  doc.generateImage(prompt);

  // Export
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForCurrentSpread();
  const path = `${desktop}/generated.png`;
  const records = await docP.export(path, opts, area, null);

  // Save – check mustSaveAs for unsaved documents
  if (doc.mustSaveAs) {
    await docP.saveAs(`${desktop}/generated.af`);
  } else {
    await docP.save();
  }

  return records.all[0]?.path ?? null;
}

async function main() {
  try {
    const path = await generateAndExport('Abstract blue waves');
    console.log('Done:', path);
  } catch (err) {
```

```

        console.log('Pipeline failed:', err.message ?? err);
    }
}

main();

```

18.5 Network requests

- ① **Note**
Network access requires permission. Enable it in Affinity Settings → Scripts.

`HttpRequest.create()` throws `PERMISSION_DENIED` if not enabled.

The response body is in `result.response.content` (not `.body`).

```

'use strict';
const { HttpRequest, RequestMethod } = require('/network.js');

// GET
try {
    const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos/1', RequestMethod.Get);
    req.setTimeoutInSec(10);
    const result = req.do();
    if (result.response.statusCode === 200) {
        const data = JSON.parse(result.response.content);
        console.log('Title:', data.title);
    }
} catch(err) {
    console.log('Network error:', err.message);
    // If PERMISSION_DENIED: enable network access in Affinity Settings
}

// POST with JSON body
try {
    const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos', RequestMethod.Post);
    req.setHeaderValue('Content-Type', 'application/json');
    const result = req.do();
    console.log('Response status:', result.response.statusCode);
    console.log('Response body: ', result.response.content);
} catch(err) {
    console.log('Network error:', err.message);
}

```

18.6 Remote data → shapes pattern

Fetch data from an API and create one shape per item:

```

'use strict';
const { Document } = require('/document.js');
const { CompoundCommandBuilder } = require('/commands.js');
const { HttpRequest, RequestMethod } = require('/network.js');
const { RGBA8 } = require('/colours.js');

const doc = Document.current;
if (!doc) { console.log('No doc.');
}
else {
    let items = [];
    try {
        const req = HttpRequest.create('https://jsonplaceholder.typicode.com/todos?_limit=5', RequestMethod.Get);
        req.setTimeoutInSec(10);
        const result = req.do();
        if (result.response.statusCode === 200) {
            items = JSON.parse(result.response.content);
        }
    } catch(e) {
        console.log('Network error:', e.message);
        // If PERMISSION_DENIED: enable network access in Affinity Settings
    }

    console.log(`Fetched ${items.length} items.`);

    items.forEach((item, i) => {
        const colour = item.completed ? RGBA8(80, 200, 120, 255) : RGBA8(220,

```

```
80, 60, 255);
// ... build and add shape nodes here
// See Lesson 06 for AddChildNodesCommandBuilder usage
console.log(`${i + 1}. ${item.title} - ${item.completed ? 'done' :
'pending'}`);
});
}
```

18.7 Timers

```
'use strict';
const { setTimeout, setInterval } = require('/timers.js');

// setTimeout(delay, callback) - delay in milliseconds, callback receives
errorCode
setTimeout(2000, (err) => {
  if (err) { console.log('Timer error:', err); return; }
  console.log('Ran after 2 seconds.');
```

```
});

// setInterval(delay, callback) - repeats until timer.cancel() is called
let count = 0;
const timer = setInterval(1000, (err) => {
  if (err) return; // err is set when cancelled
  count++;
  console.log('Tick', count);
  if (count >= 3) timer.cancel();
});
```

18.8 search_sdk_hints — community knowledge base

Before experimenting on a hard problem, always search hints first:

```
// Via the MCP tool (when using Claude with Affinity):
search_sdk_hints("How to replace text in a frame without appending")
search_sdk_hints("TextSelection end - 1")
search_sdk_hints("setCurrentSpread clears selection")
```

This queries a global pool of solved problems from other sessions. It often returns exactly the gotcha you're about to spend an hour debugging.

18.9 add_sdk_hint — contributing back

When you solve a novel problem through experimentation, add a hint:

```
// Via the MCP tool:
add_sdk_hint("To replace all text in a TextNode, create a TextSelection
with
end = storyRange.end - 1. Using storyRange.end causes text to append
rather
than replace.")
```

Your hint will be visible to all future sessions.

18.10 save_script_to_library

When a script is working reliably and you want to keep it, use the `Affinity_save_script_to_library` MCP tool to persist it to the script library. The library stores scripts in `scripts.propcol` — a proprietary binary file. They are **not** accessible as `.js` files from within scripts at runtime.

18.11 Production script checklist

Before shipping a script for others to use:

- [] Guard against no document open (if (!doc) return)
- [] Guard against empty selection (if (sel.nodes.isEmpty) return)
- [] Use compound commands for multi-step changes (one undo step)
- [] Capture `doc.selection` BEFORE any `setCurrentSpread` call

Note
Timers only run while the script engine is active. Do not use them for deferred saves or long idle waits — the engine may not be running.

Note
There is no Save to Library button in the Scripts panel UI. Library saving is available via the MCP server.

- [] Wrap file operations in try/catch
- [] Log success/failure clearly with `console.log`
- [] Use `app.userDesktopPath` for all file paths
- [] Test with a document that has multiple spreads
- [] Test with nothing selected (should fail gracefully)
- [] Save to library when stable

18.12 Key gotchas

Always call `main()` at the bottom. Async functions are not auto-invoked.

`setTimeout(delay, callback)` — delay comes first. This is the opposite of Node/browser where the callback is first. Passing a callback as the first argument produces `TypeError: expected parameter callback to be Function` from `waitAsync`. The callback receives an `errorCode` which is set on cancellation.

`timer.cancel()` not `clearInterval(timer)`. The SDK timer returns an object with a `.cancel()` method. There is no global `clearInterval`.

`generate_image` is fire-and-forget. It starts the generation but returns immediately. Chain with `executeCommandAsync` callback if you need to act on the result.

① **Note**
`search_sdk_hints` before experimenting. Many subtle bugs (selection cleared by `setCurrentSpread`, end-1 rule, `Dialog.Ok` casing) are already documented. Search first.

18.13 Exercises

18.13.1 Exercise 13.1 — Async pipeline

Write a fully async script using `doc.promises` that: loads the current document, exports it as PDF, then saves it — with proper error handling at each step.

18.13.2 Exercise 13.2 — Remote shapes

Fetch a JSON array from `https://jsonplaceholder.typicode.com/todos?_limit=8`. For each item, log its title and completion status. (Bonus: create a coloured shape for each.)

18.13.3 Exercise 13.3 — Production tool

Take your favourite script from any earlier lesson and harden it with the production checklist above. Add a dialog for configuration and save it to the library.

18.13.4 Exercise 13.4 — Hint contribution

Solve a problem you encountered during the tutorial that isn't covered in the preamble hints. Formulate it as a clear, concise hint and note it down for contributing via `add_sdk_hint`.

18.14 SDK modules used in this lesson

- `/document` — `DocumentPromises`
- `/network` — `HttpRequest`, `RequestMethod`, `HttpStatusCode`
- `/timers` — `setTimeout`, `setInterval`, `setImmediate`
- `/commands` — `DocumentCommand`
- `/fs` — `File`, `Buffer`, `Directory`, `FileSystemPromises`

18.15 Further examples

Example	What it shows
<code>logToFile.js</code>	<code>/logging (LogFile)</code> , <code>/fs</code> file I/O, set Timeout timing patterns

18.16 See also

- Lesson 12 — Layer Effects
- Tutorial README
- examples/

18.17 Examples**18.17.1 Full async pipeline**

```
'use strict';

// Example 12a — Full async pipeline
// Lesson 12: Pro Patterns
//
// Loads the current document, exports as PNG, then saves.
// Demonstrates the doc.promises pattern with error handling.

const { Document, DocumentPromises, FileExportArea, FileExportOptions } =
require('/document.js');
const { app } = require('/application.js');

async function main() {
  const desktop = app.userDesktopPath;

  // Step 1 — get current document
  let docP;
  try {
    docP = await DocumentPromises.getCurrent();
  } catch (err) {
    console.log('Could not get document:', err);
    return;
  }

  if (!docP) {
    console.log('No document is open.');
    return;
  }

  const doc = docP.document;
  const title = doc.title;
  console.log('Document:', title);

  // Step 2 — export as PNG
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const outPath = `${desktop}/${title}.png`;

  let records;
  try {
    records = await docP.export(outPath, opts, area, null);
  } catch (err) {
    console.log('Export failed:', err);
    return;
  }

  const rec = records.all[0];
  if (!rec?.isSuccess) {
    console.log('Export error:', rec?.errorMessage ?? 'unknown');
    return;
  }
  console.log('Exported to:', rec.path);

  // Step 3 — save the document
  // New/unsaved documents need saveAs() first — save() throws
  // SAVE_TO_TEMPORARY_ARCHIVE_ERROR
  try {
    if (docP.document.mustSaveAs) {
      await docP.saveAs(`${desktop}/${title}.af`);
    }
  }
}
```

```
    } else {  
      await docP.save();  
    }  
    console.log('Document saved.');
```

```
} catch (err) {  
  console.log('Save failed:', err);  
}
```

```
console.log('Pipeline complete.');
```

```
}
```

```
// Always call the async entry point at the bottom  
main();
```

Lesson 14 — Special Modules: Pixel Access, Brushes & More

This lesson surveys the SDK modules that extend the core API with powerful but less commonly discovered capabilities: direct pixel access, brush inspection, binary buffers, and document metadata. Each module is introduced with its key types, access patterns, and a practical example.

These modules unlock whole categories of scripts the core API alone can't support — pixel-level image processing, binary data handling, and document metadata inspection.

Each module is self-contained and solves a different problem, so this lesson works as a survey rather than a single narrative. Each section covers what the module contains, what problem it solves, the correct access pattern, and a minimal example. `/pixelaccessor` gets the full treatment in Lesson 14; font enumeration is covered in full in Lesson 15.

One module deserves special attention: `/pixelaccessor`. Its access pattern isn't obvious from the name — you don't instantiate a reader directly, you obtain one with the free functions `createPixelReader(bitmap) / createPixelReaderWriter(bitmap)`, or with the typed `PixelReaderWriterRGBA8.create(bitmap)`. Knowing that up front avoids the natural but wrong assumption that you're looking for a class constructor.

Use this lesson as a map. When a future script needs something you're not sure the SDK provides, this is the first place to look.

19.1 What this lesson covers

This lesson documents the SDK's supporting modules that don't fit in the earlier lessons — direct pixel read/write, per-node metadata, document/drawing-scale properties, brushes, image resources, picture frames, timers, unit conversion, hatch patterns, system and environment modules, and the additional node types. Font enumeration is covered in Lesson 15.

- `/pixelaccessor` — read and write individual pixels
- `/taginterface` — per-node custom metadata
- `/documentproperties`, `/drawingscale`, `/marginsinterface` — document properties
- `/rasterbrush`, `/pathbrush` — brush property access
- `/imageresourceinterface` — placed-image file metadata
- `/pictureframeinterface` — picture-frame state
- `/rasterselection` — pixel-selection state
- `/timers` — Timer objects
- `/units` — `UnitValueConverter` DPI conversions
- `/hatch` — building hatch patterns
- `/os`, `/environment`, `/logging`, `/configuration`, `/inspect` — system modules
- `DocumentCommand` method reference, `AddChildNodesCommandBuilder`
- Additional node types — `TableTextNode`, `PatternRasterNode`, `EnclosureRasterNode`, `MeasurementNode`, `DevelopNode`

19.2 Modules at a glance

Module	What it adds
<code>/pixelaccessor</code>	Read and write individual pixels in any colour format
<code>/fonts</code>	Font enumeration — covered in full in Lesson 15
<code>/rasterbrush</code>	Inspect and modify raster brush properties
<code>/buffer</code>	Binary buffer creation and manipulation
<code>/rasterobject</code>	Raster object wrapper (used by brushes)
<code>/documentproperties</code>	Document metadata — DPI, size, colour space
<code>/drawingscale</code>	Drawing scale (map scale) access
<code>/taginterface</code>	Per-node custom key-value tag storage
<code>/marginsinterface</code>	Margin information on spread nodes
<code>/pageboxinterface</code>	Page box (bleed, trim, slug) access
<code>/artboardproperties</code>	Artboard metadata
<code>/imageresourceinterface</code>	Placed-image source file metadata
<code>/pictureframeinterface</code>	Picture-frame state on vector nodes
<code>/rasterselection</code>	Read-only pixel-selection state

Module	What it adds
/timers	Timer objects behind setTimeout/setInterval
/units	UnitValueConverter DPI-based unit conversion
/hatch	HatchPattern / HatchLine construction
/os	Host OS info — platform, arch, version, line endings
/environment	Runtime permissions, filesystem roots, SDK version
/logging	File-based logging via LogFile
/configuration	Parse JSON config into a typed tree
/inspect	Node.js-style inspect for deep object introspection
/pathbrush	Vector path brush property access

19.3 `/pixelaccessor` — Direct pixel read/write

Read and write individual pixels across all of Affinity's colour formats. The accessor class must match the bitmap's format exactly — mismatches throw `INVALID_ARGS`.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
if (!node) { console.log('Select a raster or image node.')} }
else {
  const ri = node.rasterInterface;

  // Always check format before creating an accessor — mismatch throws
  INVALID_ARGS
  if (String(ri.format) !== 'RGBA8') {
    console.log('Format is', ri.format, '- this example requires RGBA8. ');
    console.log('Open an RGBA8 image (File > New or a standard JPEG/PNG)
and run again. ');
  } else {
    const bitmap = ri.createCompatibleBitmap(true); // true = copy
    contents
    const rw = PixelReaderWriterRGBA8.create(bitmap);

    // Read a pixel — returns { r, g, b, alpha } all 0-255
    // Note: the alpha key is 'alpha', not 'a'
    const px = rw.readPixel(10, 10);
    console.log('Pixel at (10,10):', px.r, px.g, px.b, px.alpha);

    // Write a pixel
    rw.writePixel(10, 10, { r: 255, g: 0, b: 0, alpha: 255 });

    // Push changes back — see 14a-pixel-inverter.js for a full loop
    example
    doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
    bitmap));
    console.log('Pixel written.');
```

```

    }
}

```

19.3.1 All supported pixel formats

Class	Format	Bits/channel	Notes
PixelReader RGBA8 / Pixel ReaderWriter RGBA8	RGBA	8-bit	Standard web/ screen
PixelReader RGBA16 / Pixel ReaderWriter RGBA16	RGBA	16-bit	High precision
PixelReader IA8 / Pixel ReaderWriter IA8	Intensity+Alpha	8-bit	Greyscale
PixelReader IA16 / Pixel ReaderWriter IA16	Intensity+Alpha	16-bit	Greyscale HD
PixelReader CMYKA8 / Pixel ReaderWriter CMYKA8	CMYK+Alpha	8-bit	Print
PixelReader LABA16 / Pixel ReaderWriter LABA16	L_a_b*+Alpha	16-bit	Perceptual
PixelReaderM8 / PixelReader WriterM8	Mask	8-bit	Single channel
PixelReader M16 / Pixel ReaderWriter M16	Mask	16-bit	Single channel HD
PixelReaderRGB Auf / Pixel ReaderWriter RGBAuf	RGBA	Float	HDR
PixelReaderMf / PixelReader WriterMf	Mask	Float	HDR single channel

① **Note**
PixelReaderWriter
RGBA8.create(bitmap)
throws **INVALID_ARGS** if the
format isn't **RGBA8**. Always
check `node.raster`
`Interface.format` before cre-
ating an accessor. Use
`String(ri.format)` to com-
pare — the format value is
an enum object, not a plain
string.

Open the image as a document (File > Open) rather than placing it on a page. A placed image has `isImageNode: true` and `isRasterNode: undefined` — `createReplaceBitmap` silently does nothing on it. If you need to process a placed image in-place, rasterise it first via `DocumentCommand.createRasteriseObjects(doc.selection, false, false)`.

19.4 `/taginterface` — Per-node custom metadata

TagInterface lets you write arbitrary string key-value pairs to any node via DocumentCommand. These persist inside the .af file. You can also read them back at runtime through node.tagInterface (hasKey, getValueForKey, hasPredefinedKey, getValueForPredefinedKey). See Lesson 03 — Per-node tag metadata for the full read/write pattern.

```
'use strict';
const { PredefinedTagKey } = require('/taginterface.js');
const { DocumentCommand } = require('/commands.js');
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else if (doc.selection.nodes.isEmpty) { console.log('Select a node first.')}
else {
  const sel = doc.selection;
  const node = sel.nodes.first;

  // Write a custom tag (persists in the file)
  doc.executeCommand(
    DocumentCommand.createSetTagValueForKey(sel, 'my-script-data', 'hello world')
  );

  // Write a predefined tag
  doc.executeCommand(
    DocumentCommand.createSetTagValueForPredefinedKey(
      sel, PredefinedTagKey.ScriptLabel, 'my-label')
  );

  // Read a tag back
  const tagIface = node.tagInterface;
  console.log('Has my-script-data:', tagIface.hasKey('my-script-data'));
  console.log('Value:', tagIface.getValueForKey('my-script-data'));
}
```

19.5 `/documentproperties` — Document metadata

```
'use strict';
const { Document } = require('/document.js');

// Document properties are accessed directly on the doc object
const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  console.log('DPI: ', doc.dpi);
  console.log('Width (px): ', doc.widthPixels);
  console.log('Height (px): ', doc.heightPixels);
  console.log('Format: ', doc.format);
  console.log('Colour profile:', doc.colourProfile);
}
```

19.6 `/drawingscale` — Drawing/map scale

For technical drawing and mapping workflows:

```
'use strict';
const { DrawingScale } = require('/drawingscale.js');

// Create a 1:100 scale
const scale = DrawingScale.createFromIntegers(1, 100, true);
console.log(scale.toString(false, true, false)); // "1 : 100"
console.log('Left:', scale.leftValue);
console.log('Right:', scale.rightValue);

// Get default scales for a unit
```

Note
doc.documentProperties does not exist as a property. Document meta-data is read directly from doc.dpi, doc.widthPixels, doc.heightPixels, doc.format, etc. The Document Properties export from /commands is an internal type used by createSetDocumentProperties — it is not user-constructable.

```
// IMPORTANT: pass a raw integer — NOT UnitType.* (those return undefined
// and cause INVALID_ARGS)
// 8 = Millimetres, 9 = Centimetres, 15 = Metres, 11 = Inches, 5 = Pixels
const defaults = DrawingScale.getDefaults(8); // 8 = Millimetres
for (const s of defaults) {
  console.log(s.toString(true, true, false));
}

// Identity scale (1:1)
const identity = DrawingScale.identity;
```

19.7 `/marginsinterface` — Spread margins

Page geometry can be accessed via `spread.physicalRootPropertiesInterface`:

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const spread = doc.currentSpread;

  // Available page geometry
  const prpi = spread.physicalRootPropertiesInterface;
  const props = spread.physicalRootInterface?.physicalRootProperties;

  console.log('Page count:', prpi?.pageCount);

  // Page bounding box (in document pixels)
  const pbi = prpi?.pageBoxInterface;
  if (pbi) {
    const box = pbi.getPageBoundingBox(0); // index 0 = first page
    console.log('Page box:', box);
  }
}
```

19.8 `/rasterbrush` — Brush property access

`RasterBrush` objects are returned from other API calls (for example, from a node's brush state). The class is fully functional once you have a handle:

```
'use strict';
const { RasterBrush, BrushDynamic, BrushDynamicControllerType,
  RasterBrushTextureMode } = require('/rasterbrush.js');

// Once you have a brush handle, inspect its properties:
// console.log('Size value:', brush.size.value);
// console.log('Opacity:', brush.opacity);
// console.log('Spacing:', brush.spacing);
// console.log('Blend mode:', brush.blendMode);
```

19.9 `/os` — Host operating system info

Read the host platform, architecture, OS version, and line-ending convention. Static properties only — no instance needed.

```
'use strict';
const { OS } = require('/os.js');

console.log('Platform:', OS.platform); // "Darwin", "Windows", "Linux"
console.log('Arch: ', OS.arch); // "arm64", "x64"
console.log('Machine: ', OS.machine);
console.log('Release: ', OS.release);
console.log('OS name: ', OS.osName, OS.osVersion);
console.log('EOL: ', JSON.stringify(OS.Eol)); // "\n" or "\r\n"
```

① **Note**
Current limitation: `spread.marginsInterface` returns undefined — the `MarginsInterface` class is defined with `hasMargins` and `useMargins` properties, but `MarginsInterfaceApi` has no exported methods, making it inaccessible at runtime.

19.10 `/environment` — Runtime permissions & config

Environment reports what the script is allowed to do (filesystem, network, AI) and which folders it can touch, plus the internal SDK version.

```
'use strict';
const { Environment, EnvironmentPermission } = require('/environment.js');

console.log('Permissions:', Environment.permissions); // { filesystem,
network, genAI }
console.log('Can use network:',
Environment.hasPermission(EnvironmentPermission.Network));
console.log('FS roots:', Environment.fileSystemRoots); // ["/Users/you/
Desktop"]
console.log('SDK ABI:', Environment.sdkVersionStr); // internal SDK ABI
string, not the app version
console.log('V8 version:', Environment.v8VersionStr);

// Run a task after the current synchronous turn
Environment.postTask(() => console.log('queued'));
// Environment.quit(); // exit the app
```

19.11 `/logging` — File-based logging

LogFile mirrors the console output to a file at a given verbosity level. Create it, start(), write logs via the normal console.* methods, then flush() and stop().

```
'use strict';
const { LogFile, LogLevel } = require('/logging.js');
const { app } = require('/application.js');

const log = LogFile.create(app.userDesktopPath + '/script.log',
LogLevel.Info, true);
log.start();
console.info('This is captured');
console.debug('This is excluded (below Info level)');
log.flush();
log.logLevel = LogLevel.Error; // raise the threshold mid-run
log.stop();
```

LogLevel values: All, Trace, Debug, Info, Warn, Error, None.

19.12 `/configuration` — Typed JSON config

ConfigurationItem.createFromJsonString(json, allowComments) parses JSON into a typed tree you can read with getProperty, getItem, and typed accessors.

```
'use strict';
const { ConfigurationItem } = require('/configuration.js');

const cfg = ConfigurationItem.createFromJsonString(
  '{ "width": 1200, "title": "Poster", "tags": ["a", "b"] }'
);
console.log(cfg.value); // { width: 1200, title:
"Poster", ... }
console.log(cfg.getPropertyInt('width')); // 1200
console.log(cfg.getPropertyString('title')); // "Poster"
console.log(cfg.getItem('tags')?.value); // ["a", "b"]
console.log(cfg.isObject, cfg.isArray);
```

19.13 `/inspect` — Node.js-style object inspector

/inspect provides a Node.js-compatible util.inspect for the Affinity runtime. It produces readable string representations of any value — including HandleObject instances whose internal state lives behind prototype getters. Getters that throw render as <error: ...> rather than failing the whole inspect.

Note
Environment.sdkVersionStr
is the internal SDK ABI ver-
sion, not the app version.
 Use app.shortVersion from /
 application for the Affinity
 build number.

```
'use strict';
const { inspect } = require('/inspect.js');

// Basic usage
console.log(inspect(doc));
console.log(inspect(doc.selection.nodes.first, { depth: 3 }));

// Format string (printf-style: %s %d %i %f %j %o %O)
const { format } = require('/inspect.js');
console.log(format('Node: %s, Type: %s', name, type));

// Automatically format all console output
const { patchConsole } = require('/inspect.js');
patchConsole({ depth: 4, getters: true });
console.log(doc); // now formatted by inspect
// ... later, when done:
const { unpatchConsole } = require('/inspect.js');
unpatchConsole();
```

Key options: depth (default 2, null = unlimited), showHidden (non-enumerable), getters (true/'get'/'set'), sorted (lexicographic sort), enums ('brief' shows RGBA8, 'full' shows RasterFormat.RGBA8, 'raw' shows the enum object).

19.14 `/pathbrush` — Vector path brush access

PathBrush exposes a vector brush's width, size/opacity variance, and corner behaviour. Like RasterBrush, there is no createDefault() — obtain a brush from an existing node's brush state.

```
'use strict';
const { PathBrush, CornerStrategy, PathBrushDynamicControllerType } =
require('/pathbrush.js');

// Obtain a PathBrush from a node's brush state:
// const brush = node.pathBrushInterface?.pathBrush;
// console.log(brush.brushWidth, brush.sizeVariance,
// brush.opacityVariance);
// console.log(brush.cornerStrategy); // CornerStrategy.Pull/Fold/Overlap
```

① **Note**
PathBrush.createDefault()
throws — same as RasterBrush. Brushes are read from node state, not constructed from scratch. CornerStrategy = Pull/Fold/Overlap; PathBrushDynamicControllerType = None/Pressure/Velosity/VelosityInverse.

19.15 `/imageresourceinterface` — Placed-image metadata

node.imageResourceInterface exposes everything about the source file behind an image node: its path, byte size, file type, original DPI, ICC profile, placement mode, and thumbnails. Available on ImageNode and EmbeddedDocumentNode.

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node) { console.log('Select a node.')}
else {
  const iri = node.imageResourceInterface;
  if (!iri) { console.log('Not a placed image.')}
  else {
    console.log('Path: ', iri.imageFilePath ?? '(embedded)');
    console.log('Bytes: ', iri.imageFileSize);
    console.log('Type: ', iri.fileName);
    console.log('Original DPI: ', iri.originalDPI);
    console.log('Placement: ', iri.imagePlacement);
    console.log('Modified: ', iri.modifiedTime);
  }
}
```

`saveOriginalFile(filename)` writes the original source file back out to Desktop, and `getSmallThumbnail()` / `getLargeThumbnail()` return preview bitmaps — handy for building asset-catalogue scripts.

19.16 `/pictureframeinterface` — Picture-frame state

Vector nodes can act as picture frames — rectangles that clip content placed inside them. `node.pictureFrameInterface` reads that state; `DocumentCommand.createPopulatePictureFrame(contentNode, selection)` places a node into the frame programmatically.

```
// Snippet — assumes node is a vector node
const pfi = node.pictureFrameInterface;
if (pfi) {
  console.log('Frame enabled: ', pfi.enabled);
  console.log('Has contents: ', pfi.hasFrameContents);
  if (pfi.hasFrameContents) {
    console.log('Contents: ',
pfi.frameContents?.descriptionInterface?.description);
    console.log('Anchor: ', pfi.anchor);
  }
}
```

`calculateAnchor(node, hint)` and `calculateConstraints(node, hint)` reproduce the fitting logic the UI uses when content is placed, letting you predict how an image will sit inside the frame before committing.

19.17 `/rasterselection` — Pixel-selection state

`doc.rasterSelection` returns a read-only snapshot of the current pixel-selection state. Use it to avoid running selection-mutating commands pointlessly — for example, skipping `rasterInvertSelection` when nothing is selected.

```
// Snippet — assumes doc is defined
const rs = doc.rasterSelection;
console.log('Has pixel selection:', rs.isCurrentPixelSelection);
console.log('Is select-all/none: ', rs.isSelectAllOrNone);
console.log('Is select-none: ', rs.isSelectNone);
```

The state-changing commands themselves (`createRasterSelectAll`, `createSetRasterSelectionFromPolygon`, ...) are covered in Lesson 14.

19.18 `/timers` — Timer objects

Beyond the `setTimeout/setInterval` shims (covered in Lesson 12), the `/timers` module exposes the underlying `Timer` class for high-resolution scheduling.

```
'use strict';
const { Timer } = require('/timers.js');

const t = Timer.create();
t.expiryFromNow = 2.5; // seconds from now
t.waitForAsync((err) => {
  if (!err) console.log('2.5s elapsed:', Timer.now);
});
```

`Timer.now` / `Timer.nowBigInt` read the monotonic clock, `Timer.cancelAll()` clears every pending timer, and `setTimeout(delay, callback)` returns a `Timer` you can `cancel()` later. Remember the shim's argument order — delay first, callback second.

19.19 `/units` — UnitValue and conversion factors

`doc.unitValueConverter` converts between unit types using the document DPI — essential whenever the UI shows points but the SDK works in document pixels.

```

'use strict';
const { Document } = require('/document.js');
const { UnitType } = require('/units.js');

const doc = Document.current;
if (doc) {
  const converter = doc.unitValueConverter;
  console.log('DPI:', converter.dpi, '| view DPI:', converter.viewDpi);

  const pxPerPt = converter.getConversionFactor(UnitType.Point,
UnitType.Pixel);
  console.log('1 pt =', pxPerPt, 'px');

  const mmPerInch = converter.getConversionFactor(UnitType.Inch,
UnitType.Millimetre);
  console.log('1 in =', mmPerInch, 'mm');
}

```

UnitValue itself (re-exported from /units) carries prototype helpers: getValueAsPixels(converter), getValueAsUnitType(converter, unitType), getValueAsDegrees(), getValueAsRadians(), and isFinite.

19.20 `/hatch` — Building hatch patterns

Lesson 05 showed how to apply a hatch fill; the /hatch module is where the patterns themselves come from. A HatchPattern is a list of HatchLines, each with an origin, rotation, step distance, and optional dash pattern — and patterns are fully constructible, unlike brushes.

```

'use strict';
const { HatchPattern, HatchLine } = require('/hatch.js');

const pattern = HatchPattern.createEmpty();
const line = HatchLine.create(
  { x: 0, y: 0 }, // origin
  Math.PI / 4, // rotation (radians)
  8, // step between lines
  [4, 2] // dash pattern (4 on, 2 off) — omit for solid
);
pattern.appendHatchLine(line);
console.log('Lines in pattern:', pattern.hatchLineCount);

```

Pass the finished pattern to HatchFill.create(pattern, units, penColour, brushColour, lineWeight) from Lesson 05. HatchPattern.createDefault() and .all (enumerating the built-in library patterns) are also available.

19.21 `DocumentCommand` — full method reference

commands.js exposes 43 static factory methods across the following groups:

19.21.1 Macro recording/playback

```

// Snippet
DocumentCommand.createStartRecordingMacro()
DocumentCommand.createStopRecordingMacro()
DocumentCommand.createReplayMacro()
DocumentCommand.createClearMacro()
DocumentCommand.createImportMacro(path)
DocumentCommand.createExportMacro(path)

```

19.21.2 Raster selection commands

```

// Snippet
DocumentCommand.createRasterSelectAll()
DocumentCommand.createRasterDeselect()
DocumentCommand.createRasterInvertSelection()
DocumentCommand.createRasterReselect()
DocumentCommand.createSetRasterSelectionFromPolygon(polygon, operation,
antialias, feather) // replaces createRasterSelectPolygon

```

19.21.3 Opacity control (separate from visibility)

```
// Snippet
DocumentCommand.createSetOpacity(selection, opacity)           // layer
opacity
DocumentCommand.createSetBrushFillOpacity(selection, opacity, options) //
fill opacity
DocumentCommand.createSetLineFillOpacity(selection, opacity, options) //
stroke opacity
```

19.21.4 Fill anchor control

```
// Snippet
DocumentCommand.createSetBrushFillIsAnchoredToSpread(selection, bool,
options)
DocumentCommand.createSetPenFillIsAnchoredToSpread(selection, bool,
options)
DocumentCommand.createSetTransparencyFillIsAnchoredToSpread(selection,
bool, options)
```

19.21.5 Tag metadata

```
// Snippet
DocumentCommand.createSetTagValueForKey(selection, key, value)
DocumentCommand.createSetTagValueForPredefinedKey(selection,
predefinedKey, value)
```

19.21.6 Blend options

```
// Snippet
DocumentCommand.createSetBlendGamma(selection, gamma)
DocumentCommand.createSetBlendRanges(selection, blendOptions)
DocumentCommand.createSetAntialiasingMode(selection, antialiasingMode)

// Read the current values via node.blendModeInterface.blendOptions:
// .gamma, and spline-controlled channel ranges
// (getChannelSourceLayerRanges / getChannelUnderlyingCompositionRanges)
```

19.21.7 Document-level

```
// Snippet
DocumentCommand.createSetDocumentProperties(documentProperties)
DocumentCommand.createConvertDocumentFormat(format)
DocumentCommand.createGroupTransform(selection, xDataOrNull, yDataOrNull)
DocumentCommand.createSetWindingMode(selection, windingOrder)
DocumentCommand.createSetExportConfig(selection, exportConfig)
DocumentCommand.createMoveNodes(selection, target, moveType,
childListType)
DocumentCommand.createHideSelection(selection)
DocumentCommand.createFlatten()
DocumentCommand.createMergeVisible()
DocumentCommand.createSplitCurve(selection, parametricDistance)
DocumentCommand.createInsertGlyph(selection, glyph)
```

19.21.8 Snapshots

```
// Snippet
DocumentCommand.createAddDocumentSnapshot(description)
DocumentCommand.createDeleteDocumentSnapshot(snapshot)
DocumentCommand.createRestoreDocumentSnapshot(snapshot)
DocumentCommand.createSetCurrentSnapshot(snapshot)
DocumentCommand.createSetCurrentSnapshotFromHistoryItem(historyItem)
```

19.21.9 History navigation

```
// Snippet
DocumentCommand.createSetHistoryIndex(index)
DocumentCommand.createCycleAlternateFutures(index)
```

19.22 `createSetEditable` — argument order

Selection is the first argument:

```
// Snippet
// CORRECT: selection first, editable flag second
```

```
DocumentCommand.createSetEditable(selection, true) // unlock
DocumentCommand.createSetEditable(selection, false) // lock
```

19.23 `AddChildNodesCommandBuilder` — insertion options

Setters for controlling insertion behaviour:

```
'use strict';
const { AddChildNodesCommandBuilder, InsertionMode } = require('./
commands.js');
const builder = AddChildNodesCommandBuilder.create();
builder.autoName = true; // auto-assign layer name
builder.forcePixelAlignment = true; // snap to pixel grid
builder.clearCurrentSelection = false; // don't deselect before adding
builder.appendCurrentSelection = true; // add to existing selection
builder.setInsertionTarget(targetNode); // insert relative to a
node
builder.setInsertionTargetSelection(targetSel); // insert relative to a
selection
builder.setInsertionMode(InsertionMode.After); // Before / After /
Into
```

`addContainerNode` adds a container node, and `createCommandAndReset` allows reusing the builder.

19.24 SDK native binding modules

Two native C++ binding modules underpin the pixel and brush APIs:

Module	Contains
<code>affinity:raster</code>	All <code>PixelReader*Api</code> and <code>PixelReaderWriter*Api</code> classes
<code>affinity:brushes</code>	<code>RasterBrushApi</code> , <code>BrushDynamicApi</code> , brush enums

19.25 New node types

Three new concrete node classes are available:

19.25.1 TableTextNode

A grid-based text node with cells separated by `HardBreakType.TableCell` glyphs. See Lesson 07 for the full pattern.

```
// Snippet — assumes doc is defined
const { TableTextNodeDefinition } = require('./nodes.js');
const def = TableTextNodeDefinition.create({ x: 0, y: 0, width: 300,
height: 200 }, { width: 3, height: 3 });
doc.addNode(def);
```

19.25.2 PatternRasterNode

A raster node that tiles a repeating pattern. Useful for procedural fill effects.

```
// Snippet — assumes doc is defined
const { PatternRasterNodeDefinition } = require('./nodes.js');
const def = PatternRasterNodeDefinition.createDefault();
```

19.25.3 EnclosureRasterNode

A raster node that wraps raster content in an enclosure. Less commonly used directly — typically created through other SDK operations.

```
// Snippet — assumes doc is defined
const { EnclosureRasterNodeDefinition } = require('./nodes.js');
const def = EnclosureRasterNodeDefinition.createDefault();
```

All three use standard node patterns: `doc.addNode(def)`, `AddChildNodesCommandBuilder` for insertion targeting, and access to `rasterInterface` / `storyInterface` where applicable.

19.25.4 MeasurementNode

A measurement annotation — a dimension line between two points. Create it with `MeasurementNodeDefinition.create(start, end, factor, unitType)` and add it via `builder.addMeasurementNode()`.

```
'use strict';
const { MeasurementNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { UnitType } = require('/units.js');
const { Document } = require('/document.js');

const doc = Document.current;
if (doc) {
  const def = MeasurementNodeDefinition.create(
    { x: 100, y: 100 }, // start point
    { x: 400, y: 100 }, // end point
    1.0, // scale factor
    UnitType.Pixel // measurement unit
  );
  const builder = AddChildNodesCommandBuilder.create();
  builder.addMeasurementNode(def);
  doc.executeCommand(builder.createCommand(true));
  console.log('Measurement added.');
```

Companion `DocumentCommand` factories configure measurement display globally: `createSetMeasurementUnits(unitType)`, `createSetMeasurementPrecision(useDocumentPrecision, decimalPlaces)`, `createSetMeasurementAnnotationOffset(node, offset)`, and `createSetMeasurementShowEndpointMarkers(show)`. Note there is no `isMeasurementNode` type guard.

19.25.5 DevelopNode

Represents a RAW image in the Develop persona. Read its state via `isDevelopNode`, `.parameters` (a `DevelopParameters`), and `.sourceIsRaw` push changes back with `createSetDevelopParameters(sel, developParameters)`. See Lesson 16 for the full develop workflow.

19.26 Exercises

19.26.1 Exercise 14.1 — Pixel inverter

Select a raster node. Read every pixel and write back its inverted colour (255-r, 255-g, 255-b). Use `PixelReaderWriterRGBA8`.

19.26.2 Exercise 14.2 — Tag all selected nodes

Build a dialog asking for a key and a value. Apply that tag to every selected node using `createSetTagValueForKey`.

19.26.3 Exercise 14.3 — Macro recorder UI

Build a dialog with buttons to start recording, stop recording, replay, and clear a macro. Wire each to the corresponding `DocumentCommand`.

19.26.4 Exercise 14.4 — Raster selection from polygon

Create a rectangular raster selection programmatically using `createSetRasterSelectionFromPolygon` (replaces `createRasterSelectPolygon`), then invert it with `createRasterInvertSelection`.

19.26.5 Exercise 14.5 — Image asset catalogue

Walk every `ImageNode` in the document and log its layer name, source file path, original DPI, and file size using `imageResourceInterface`. Flag any image below 150 DPI with a [LOW-RES] marker.

19.27 SDK modules used in this lesson

- /pixelaccessor — All *PixelReader* and *PixelReaderWriter* classes
- /taginterface — *TagInterface*, *PredefinedTagKey*
- /rasterbrush — *RasterBrush*, *BrushDynamic*
- /imageresourceinterface — *ImageResourceInterface*
- /pictureframeinterface — *PictureFrameInterface*
- /rasterselection — *RasterSelection*
- /timers — *Timer*
- /units — *UnitType*, *UnitValue*, *UnitValueConverter*
- /hatch — *HatchPattern*, *HatchLine*
- /drawingscale — *DrawingScale*
- /marginsinterface — *MarginsInterface*
- /documentproperties — *DocumentProperties*
- /commands — updated *DocumentCommand*, *AddChildNodesCommandBuilder*
- /nodes — *TableTextNodeDefinition*, *PatternRasterNodeDefinition*, *EnclosureRasterNodeDefinition*, *MeasurementNodeDefinition*, *DevelopNode*
- /os — *OS*
- /environment — *Environment*, *EnvironmentPermission*
- /logging — *LogFile*, *LogLevel*
- /configuration — *ConfigurationItem*, *ConfigurationValueType*
- /inspect — *inspect*, *format*, *patchConsole*, *unpatchConsole*
- /pathbrush — *PathBrush*, *CornerStrategy*, *PathBrushDynamicController*
Type

19.28 See also

- Lesson 03 — Node Model
- Lesson 04 — Command Pattern
- Lesson 15 — Pixel Processing
- examples/

19.29 Examples**19.29.1 Pixel inverter**

```
'use strict';

// Example 13a — Pixel inverter
// Lesson 13: New modules
//
// Inverts every pixel in the selected raster node.
// Demonstrates the PixelReaderWriterRGBA8 API.
//
// Open the image as a document (File > Open), not as a placed image on a
// page.
// Select the layer and run.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.')}
  else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    if (String(ri.format) !== 'RGBA8') {
```



```

    console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.');
```

```

    } else {
    const bitmap = ri.createCompatibleBitmap(true); // copy contents
    const rw      = PixelReaderWriterRGBA8.create(bitmap);

    console.log(`Inverting ${w}x${h} pixels...`);

    for (let y = 0; y < h; y++) {
      for (let x = 0; x < w; x++) {
        const px = rw.readPixel(x, y);
        rw.writePixel(x, y, {
          r: 255 - px.r,
          g: 255 - px.g,
          b: 255 - px.b,
          alpha: px.alpha
        });
      }
    }

    doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
    console.log('Done - pixels inverted.');
```

```

    }
  }
}

```

19.29.2 Resource inspector

```

'use strict';

// Example 13b - Resource inspector
// Lesson 13: Special Modules
//
// Reports image resource metadata, picture-frame state, blend options,
// and DPI-aware unit conversions for the selected node.

const { Document } = require('/document.js');
const { UnitType } = require('/units.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node) { console.log('Select a node.');
```

```

} else {
  const iri = node.imageResourceInterface;
  if (iri) {
    console.log('Image file:   ', iri.imageFilePath ?? '(embedded)');
    console.log('File size:   ', iri.imageFileSize, 'bytes');
    console.log('Original DPI: ', iri.originalDPI);
    console.log('File type:   ', iri.fileName);
  } else {
    console.log('Not a placed image (no imageResourceInterface).');
  }

  const pfi = node.pictureFrameInterface;
  if (pfi) {
    console.log('Picture frame enabled:', pfi.enabled);
    console.log('Has frame contents:   ', pfi.hasFrameContents);
    if (pfi.hasFrameContents) {
      console.log('Contents:',
pfi.frameContents?.descriptionInterface?.description ?? '(unnamed)');
    }
  }

  const opts = node.blendModeInterface?.blendOptions;
  if (opts) {
    console.log('Blend gamma:', opts.gamma);
  }
}

```

```
const pxPerPt =  
doc.unitValueConverter.getConversionFactor(UnitType.Point,  
UnitType.Pixel);  
console.log('1 pt =', pxPerPt, 'px at', doc.dpi, 'dpi');  
}
```

Lesson 15 — Pixel Processing

Pixel processing gives you direct access to the raw colour data of any raster layer. This lesson covers the full pipeline: getting a bitmap from a raster node, reading and writing individual pixels, creating bitmaps from scratch, loading images from disk, and rendering any node to a bitmap. It also covers the colour formats and the `Buffer` class for raw binary data.

Pixel-level access unlocks algorithms the high-level SDK can't express — custom colour grading, per-pixel analysis, procedural textures, histograms, and spatial effects. If you can describe it mathematically, you can implement it here.

Two things are easy to get wrong, so keep them in mind:

- **Pixel access starts with a Bitmap, not the node.** Get a bitmap via the node's `bitmap` property, `Bitmap.loadFromFile()`, or `NodeRenderingEngine`. Reading pixel data directly from a node doesn't work.
- **Pixels expose alpha as `px.alpha`, not `px.a`.** Code that reads `px.a` returns undefined silently, producing subtle bugs. The `RGBA8` constructor uses `r`, `g`, `b`, `alpha` too.

Performance matters in pixel loops — iterating millions of pixels naively can make the app feel unresponsive. Work on smaller bitmaps, use typed access formats, and structure loops to minimise repeated lookups.

By the end you'll have a working mental model of the pixel pipeline — node → bitmap → reader/writer → back — plus working examples of inversion, grayscale, procedural generation, and histograms to use as templates.

20.1 What this lesson covers

- What a pixel is and when to use pixel processing vs built-in filters
- The complete pixel pipeline: bitmap → pixel accessor → write back
- All 10 colour formats and how to choose the right one
- Reading and writing individual pixels
- Creating bitmaps from scratch and loading from file
- Rendering any node to a bitmap with `NodeRenderingEngine`
- The `Buffer` class for raw binary data
- Performance considerations for pixel loops
- Real-world patterns: histogram analysis, posterisation, procedural generation

20.2 What a pixel is in this context

When you open an image in Affinity, what you see on screen is a rendered representation. The underlying data is a rectangular grid of numbers — each cell in the grid is a pixel, and each pixel is a small set of channel values that describe its colour.

For an RGBA8 image — the most common format — each pixel has four channels: red, green, blue, and alpha (transparency). Each channel is an 8-bit integer: a whole number between 0 and 255. A pure red pixel is { `r: 255, g: 0, b: 0, alpha: 255` }. A 50% transparent mid-grey is { `r: 128, g: 128, b: 128, alpha: 128` }. That's all a pixel is — a small structure of numbers in a large grid.

Pixel processing means reading those numbers, computing new ones, and writing them back. Any operation you can express as a mathematical transformation on those channel values is something you can implement as a pixel script.

20.3 Why pixel processing rather than filters or adjustments

Affinity's built-in filters and adjustment layers cover a wide range of common operations — levels, curves, HSL, blur, noise — and they run fast because they execute in C++ on optimised code paths. For those operations, you should always prefer the built-in tools.

Pixel processing becomes the right choice when you need something the built-in tools cannot express:

- **Custom algorithms** — a specific colour grading formula, a specialised sharpening kernel, a per-pixel mathematical transform that isn't in the filter list.
- **Analysis without editing** — reading pixel data to compute statistics, detect clipping, measure average colour, or build a histogram. The built-in tools don't expose this data to scripts.
- **Conditional processing** — applying different transforms to different regions of the image based on pixel values. For example: desaturate only pixels where red dominates, or boost contrast only in shadow regions.
- **Procedural generation** — creating images from scratch using mathematical functions, noise algorithms, or data from outside the document.
- **Format-specific operations** — working directly in CMYK or LAB colour space without a conversion round-trip.

20.4 Why pixel processing is slow — and what to do about it

JavaScript is not a fast language for numerical loops. A 4000×3000 image has 12 million pixels. Visiting each one, calling `readPixel`, performing arithmetic, and calling `writePixel` takes measurably longer in JavaScript than the same operation in C++ — potentially several seconds for a large image.

This doesn't mean pixel processing is unusable, but it does mean you should think before writing a loop:

- **Do you need every pixel?** Analysis tasks often work well on a sampled grid — visiting every 4th pixel in each direction gives you 1/16th the work with results that are still statistically representative.
- **Do you need to write back?** Read-only analysis (histograms, colour statistics) doesn't need `createReplaceBitmap`. Skip the write-back entirely.
- **Can you work at a smaller size?** If the script's effect is not detail-sensitive, export a scaled-down version, process it, and re-import.
- **Is there a built-in alternative?** A Curves adjustment layer that approximates your formula will always be faster than a per-pixel script. Use pixel processing for what the SDK genuinely can't do otherwise.

20.5 Why pixel processing matters

Scripts can read raster metadata (width, height, format) and write pixels through high-level operations (filters, adjustments, AI). Pixel processing drops below all of that and gives you direct access to the numbers. You can implement any image processing algorithm you want, entirely in JavaScript, with results pushed back as undoable commands.

20.6 The pipeline

```
node.rasterInterface
  ↓
.createCompatibleBitmap(true) ← copy current pixels into a writable
bitmap
  ↓
PixelReaderWriterRGBA8.create(bitmap) ← create a typed accessor
  ↓
rw.readPixel(x, y) / rw.writePixel(x, y, pixel)
  ↓
DocumentCommand.createReplaceBitmap(selection, bitmap) ← push back
(undoable)
```

Every step is explicit. The bitmap is a detached copy — you never modify live document pixels directly. The final `createReplaceBitmap` command is what actually changes the document, and it sits in the undo stack like any other operation.

20.7 Choosing the right format

Before creating a pixel accessor, check the node's format:

```
'use strict';
const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node first.')}
  else {
    const ri = node.rasterInterface;
    const format = ri.format; // RasterFormat enum value
    console.log('Format:', format);
    console.log('Width: ', ri.width, 'Height:', ri.height);
  }
}
```

Then match the accessor class to the format:

RasterFormat value	Reader class	ReaderWriter class	Pixel object shape
RGBA8	PixelReaderRGBA8	PixelReaderWriterRGBA8	{ r, g, b, alpha } 0–255
RGBA16	PixelReaderRGBA16	PixelReaderWriterRGBA16	{ r, g, b, alpha } 0–65535
IA8	PixelReaderIA8	PixelReaderWriterIA8	{ i, alpha } 0–255
IA16	PixelReaderIA16	PixelReaderWriterIA16	{ i, alpha } 0–65535
CMYKA8	PixelReaderCMYKA8	PixelReaderWriterCMYKA8	{ c, m, y, k, alpha } 0–255
LABA16	PixelReaderLABA16	PixelReaderWriterLABA16	{ l, a, b, alpha }
M8	PixelReaderM8	PixelReaderWriterM8	{ m } 0–255
M16	PixelReaderM16	PixelReaderWriterM16	{ m } 0–65535
RGBAuf	PixelReaderRGBAuf	PixelReaderWriterRGBAuf	{ r, g, b, alpha } float
Mf	PixelReaderMf	PixelReaderWriterMf	{ m } float

Note
Mismatching format and accessor throws `INVALID_ARGS`.
 Always check `ri.format` before choosing your accessor class. The format is an enum object — use `String(ri.format)` to compare: `String(ri.format) === 'RGBA8'`.

20.8 Core pixel read/write pattern

The pipeline always follows the same five steps: get the raster interface, copy pixels into a writable bitmap, create a typed accessor, process pixels, push back with `createReplaceBitmap`.

```
// Snippet
const ri = node.rasterInterface;
const bitmap = ri.createCompatibleBitmap(true); // true = copy current pixels
const rw = PixelReaderWriterRGBA8.create(bitmap);

for (let y = 0; y < ri.height; y++) {
  for (let x = 0; x < ri.width; x++) {
    const px = rw.readPixel(x, y); // { r, g, b, alpha } — 'alpha' not 'a'
    rw.writePixel(x, y, { /* modified values */ });
  }
}
```

```
doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
```

See `14a-pixel-inverter.js` for a complete runnable example using this pattern.

20.9 Read-only analysis (no write-back needed)

When you only need to inspect pixels — colour sampling, histogram, dominant colour — use the read-only `PixelReaderRGBA8` (no `Writer` in the name). It doesn't need a writable bitmap.

```
'use strict';
const { Document } = require('/document.js');
```

```

const { PixelReaderRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) { console.log('Select a raster or image node.')} }
  else {
    const ri = node.rasterInterface;
    const bitmap = ri.createCompatibleBitmap(true);
    const reader = PixelReaderRGBA8.create(bitmap);

    // Sample a 5x5 grid across the image
    const w = ri.width, h = ri.height;
    let totalR = 0, totalG = 0, totalB = 0, count = 0;

    for (let gy = 0; gy < 5; gy++) {
      for (let gx = 0; gx < 5; gx++) {
        const x = Math.floor(gx * w / 5);
        const y = Math.floor(gy * h / 5);
        const px = reader.readPixel(x, y);
        totalR += px.r; totalG += px.g; totalB += px.b;
        count++;
      }
    }

    console.log('Average colour:',
      Math.round(totalR/count),
      Math.round(totalG/count),
      Math.round(totalB/count)
    );
  }
}

```

20.10 Creating a bitmap from scratch

You don't need an existing node to start. The Bitmap class lets you create one from dimensions and format, or load one from a file:

```

'use strict';
const { Document } = require('/document.js');
const { Bitmap, RasterFormat } = require('/rasterobject.js');
const { RasterNodeDefinition } = require('/nodes.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')} }
else {
  // Create a new 200x200 RGBA8 bitmap
  const bitmap = Bitmap.create(200, 200, RasterFormat.RGBA8);
  const rw = PixelReaderWriterRGBA8.create(bitmap);

  // Draw a gradient
  for (let y = 0; y < 200; y++) {
    for (let x = 0; x < 200; x++) {
      rw.writePixel(x, y, {
        r: Math.round(x / 200 * 255),
        g: Math.round(y / 200 * 255),
        b: 128,
        alpha: 255
      });
    }
  }

  // Add it to the document as a new raster node
  const nodeDef = RasterNodeDefinition.create(RasterFormat.RGBA8);
  nodeDef.bitmap = bitmap;
}

```

```

const builder = AddChildNodesCommandBuilder.create();
builder.addRasterNode(nodeDef);
doc.executeCommand(builder.createCommand());
console.log('Gradient raster node added.');
```

20.11 Loading a bitmap from a file

`Bitmap.loadFromFile` reads any image from the Desktop into a `Bitmap` object. The file must exist — it throws a generic `ERROR` if the path is wrong or the file is missing.

```

'use strict';
const { Document, FileExportOptions, FileExportArea } = require('/document.js');
const { Bitmap, RasterFormat } = require('/rasterobject.js');
const { app } = require('/application.js');

const doc = Document.current;
const desktop = app.userDesktopPath;

if (!doc) { console.log('No document open.')}
else {
  // Export the current document as PNG first, then load it back as a
  // Bitmap
  const outPath = `${desktop}/sdk_bitmap_test.png`;
  const opts = FileExportOptions.createWithPresetName('PNG');
  const area = FileExportArea.createForWholeDocument();
  const records = doc.export(outPath, opts, area, null);

  if (!records.all[0]?.isSuccess) {
    console.log('Export failed — cannot load.');
```

```

  } else {
    // Now load it back
    const bitmap = Bitmap.loadFromFile(outPath, RasterFormat.RGBA8);
    console.log('Loaded:', bitmap.width, 'x', bitmap.height);

    // bitmap.format returns a plain integer (0 = RGBA8, 1 = RGBA16, etc.)
    // Use RasterFormat.keys/values to look up the name
    const fmtKeys = RasterFormat.keys.toString().split(',');
    const fmtVals = RasterFormat.values.toString().split(',').map(Number);
    const fmtIdx = fmtVals.indexOf(bitmap.format);
    const fmtName = fmtIdx !== -1 ? fmtKeys[fmtIdx] :
String(bitmap.format);
    console.log('Format:', fmtName); // e.g. 'RGBA8'
  }
}
```

ⓘ **Note**
Desktop + granted folders restriction applies.
`Bitmap.loadFromFile` can only read files from the Desktop or folders you explicitly grant access to.

Any other path throws a generic `ERROR`. Always use `app.userDesktopPath` as your base path.

20.12 Rendering a node to a bitmap with `NodeRenderingEngine`

This lets you render any node — including vector, text, or group nodes — to pixels. Useful for generating thumbnails, doing pixel analysis on non-raster nodes, or compositing.

```

'use strict';
const { Document } = require('/document.js');
const { NodeRenderingEngine, RasterFormat } = require('/rasterobject.js');
const { PixelReaderRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.first;
  if (!node) { console.log('Select any node first.')}
  else {
    const engine = NodeRenderingEngine.createDefault(node,
RasterFormat.RGBA8);
```



```

// engine has width/height from the node's bounding box
const bitmap = engine.createCompatibleBitmap(true);
const reader = PixelReaderRGBA8.create(bitmap);

// Sample the centre pixel
const cx = Math.floor(engine.width / 2);
const cy = Math.floor(engine.height / 2);
console.log('Centre pixel:', reader.readPixel(cx, cy));
}
}

```

20.13 The Buffer class

Buffer is a raw byte container — not directly for pixel data (use Bitmap or Pixel Buffer for that), but useful for binary I/O, encoding text, or building binary structures.

```

'use strict';
const { Buffer } = require('/buffer.js');

// Create and fill
const buf = Buffer.create(4);
const arr = buf.array; // Uint8Array view
arr[0] = 0xFF; arr[1] = 0x00; arr[2] = 0x80; arr[3] = 0xFF;

// UTF-8 encode a string
const utf8buf = Buffer.utf8('Hello, Affinity!');
console.log('Byte length:', utf8buf.size);
console.log('As string: ', utf8buf.toString());

// Concatenate
const combined = buf.concat(utf8buf);
console.log('Combined size:', combined.size);

// Slice (copy)
const sliced = buf.slice(0, 2);

// ArrayBuffer for interop with JS typed arrays
const ab = buf.arrayBuffer; // standard ArrayBuffer

```

Buffer also offers compareSome(), toString(enc, start, end) overloads, and the array / items views alongside arrayBuffer. For rendering control, NodeRenderingEngine.create(node, format, options) accepts a NodeRenderingEngineOptions object (create() it first): down/up resampler types, antialiasing, gradient dithering, clipping, and isolated-rendering flags — createDefault() is just the presets version.

20.13.1 `PixelBuffer` — pixel data without a GPU bitmap

PixelBuffer is a CPU-side pixel array, useful when you want the raw bytes without the overhead of a full Bitmap:

```

'use strict';
const { RGBA8 } = require('/colours.js');
const { PixelBuffer, RasterFormat } = require('/rasterobject.js');

const pb = PixelBuffer.create(100, 100, RasterFormat.RGBA8);
const ab = pb.buffer; // raw ArrayBuffer — 100×100×4 bytes
const u8 = new Uint8Array(ab);

// Fill with solid red
for (let i = 0; i < u8.length; i += 4) {
  u8[i] = 255; // R
  u8[i+1] = 0; // G
  u8[i+2] = 0; // B
  u8[i+3] = 255; // A
}

```

20.14 Performance considerations

JavaScript pixel loops are slow. For large images (> 2000×2000px) a full-image nested for-loop will take several seconds. Strategies to manage this:

- Use `PixelReaderWriterRGBA8` not a generic accessor — the typed classes have direct C++ backing.
- Work on a sampled grid rather than every pixel when analysis (not editing) is the goal.
- Use `doc.executeCommandAsync` to avoid blocking the UI during heavy processing.
- Prefer the SDK's built-in filters (adjustments, layer effects) for common operations — they run in C++ at full speed. Drop to pixel level only when you need something the SDK doesn't offer natively.

20.15 Key gotchas

alpha not a — on both read and write. `readPixel` returns `{ r, g, b, alpha }` — the key is alpha, not a. Using `px.a` reads undefined. On `writePixel`, passing `{ r, g, b, a: someValue }` silently ignores the value and writes 255 (fully opaque) instead — no error is thrown. Always use alpha as the key name on both sides.

createReplaceBitmap requires a raster document, not a placed image. Open the image as a document (File > Open) rather than placing it on a page. A placed image has `isImageNode: true` — `createReplaceBitmap` silently does nothing on it. Rasterise first via `DocumentCommand.createRasteriseObjects(doc.selection, false, false)` if you need to process a placed image in-place.

Format mismatch gives corrupt data silently. Always check `ri.format` first. If the node is CMYK and you use `PixelReaderWriterRGBA8`, you'll read wrong values and write corrupted channels with no error message.

createCompatibleBitmap(false) gives an uninitialized bitmap. Pass `true` to copy existing pixel content. Pass `false` only when you're going to fill every pixel yourself (e.g. procedural generation).

PixelBuffer vs Bitmap — a `PixelBuffer` is a straight byte array for raw bytes; a `Bitmap` has GPU backing for rendering and is what you pass to pixel accessors (`PixelReaderWriterRGBA8.create(bitmap)`).

Desktop + granted folders for file loading. `Bitmap.loadFromFile` follows the same Desktop and granted-folder restriction as `fs` — other paths are not accessible.

20.16 Raster pixel selection

The SDK exposes commands for working with the pixel-level selection (the “marching ants” selection in Affinity Photo). These are distinct from the node selection used elsewhere.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand, RasterSelectionLogicalOperation } = require('/commands.js');

const doc = Document.current;
if (doc) {
  // Check current pixel selection state
  const rsel = doc.rasterSelection;
  console.log('Has pixel selection:', !rsel.isSelectNone);
  console.log('Is select-all/none:', rsel.isSelectAllOrNone);
  console.log('Is current pixel sel:', rsel.isCurrentPixelSelection);
}
```

```

// Select all pixels
doc.rasterSelectAll();

// Invert the pixel selection
doc.rasterInvertSelection();

// Deselect all
doc.rasterDeselect();

// Reselect last selection
doc.rasterReselect();

// Select a polygon region (array of {x, y} points)
const polygon = [
  { x: 100, y: 100 },
  { x: 300, y: 100 },
  { x: 300, y: 300 },
  { x: 100, y: 300 },
];
doc.setRasterSelectionFromPolygon(
  polygon,
  RasterSelectionLogicalOperation.New, // New / Add / Subtract /
Intersect
  true, // antialias
  0 // feather radius (pixels)
);

// Raster selection refinement
doc.growShrinkRasterSelection(10, false); // expand by 10px
(circular=false = square)
doc.featherRasterSelection(5); // soften edge by 5px
doc.smoothRasterSelection(3); // smooth boundary by 3px
doc.outlineRasterSelection(8, alignment, false); // outline at radius
8px

// Select based on a node's alpha channel (use intensity=false for
binary)
doc.setRasterSelectionFromObject(node, false,
RasterSelectionLogicalOperation.New);
}

```

`doc.rasterSelection` returns a `RasterSelection` object with read-only state — `isCurrentPixelSelection`, `isSelectAllOrNone`, and `isSelectNone`. Check these before mutating: for example, the auto-fix section below guards with `!doc.rasterSelection.isSelectNone` so one-click corrections never run against an empty selection. The modification commands (`setRasterSelectionFromPolygon`, `setRasterSelectionFromObject`, `rasterSelectAll`, `rasterDeselect`, etc.) are all on `doc` directly. `RasterSelectionLogicalOperation` controls how a new selection combines with the existing one: New (0), Add (1), Subtract (2), Intersect (3) — there is no Replace value.

Magic wand (flood select): `createRasterFloodSelect(x, y, tolerance, antialias, feather)` performs a flood-fill raster selection starting at pixel (x, y), similar to the Magic Wand tool in the UI. Use positive tolerance for similar-colour selection, lower values for tighter matching.

20.17 Raster auto-fixes and fill

One-click tonal corrections and paint-style fills operate on the current raster selection. These are `doc` convenience methods (with `DocumentCommand` equivalents).

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');

```

Note
Both `setRasterSelectionFromPolygon` and `rasterSelectPolygon` work — prefer `setRasterSelectionFromPolygon`.

```

const doc = Document.current;
if (doc && !doc.rasterSelection.isSelectedNone) {
  // One-click tonal corrections (operate on the raster selection)
  doc.rasterAutoLevels();           // stretch to full tonal range
  doc.rasterAutoContrast();         // normalize contrast
  doc.rasterAutoColours();          // fix colour cast
  doc.rasterAutoWhiteBalance();     // neutralise white balance

  // Fill the raster selection (mode = flood-fill style)
  // DocumentCommand.createRasterFill(sel, mode, colour, opacity,
  blendMode);
  doc.rasterFill(); // fill with the current foreground colour

  // Magic-wand fill starting at a point, with tolerance + contiguity
  // DocumentCommand.createRasterFloodFill(sel, point, tolerance,
  isContiguous,
  //                                     antialias, samplingSource,
  blendMode, colour);
}

```

`createRasterFill(sel, mode, colour, opacity, blendMode)` paints the whole selection; `createRasterFloodFill(sel, point, tolerance, isContiguous, antialias, samplingSource, blendMode, colour)` paints outward from a point, stopping at colour boundaries like the Flood Fill tool.

20.18 Exercises

20.18.1 Exercise 15.1 — Colour inverter

See `14a-pixel-inverter.js` in lesson 14 for a working implementation.

20.18.2 Exercise 15.2 — Average colour sampler

Select any node (raster, image, or vector). Sample a 10×10 grid of pixels from a rendered bitmap and compute the average R, G, B. Display the result in a `console.log`.

20.18.3 Exercise 15.3 — Greyscale converter

See `15b-greyscale-converter.js` for a working implementation using the luminance formula.

20.18.4 Exercise 15.4 — Procedural gradient node

See `15c-procedural-gradient.js` for a working implementation.

20.18.5 Exercise 15.5 — Histogram analysis

See `15e-histogram-analysis.js` for a full implementation with per-channel statistics, clipping detection, and tonal distribution.

20.18.6 Exercise 15.6 — Posterise

See `15f-posterise.js` for a working implementation with a configurable level count via dialog.

20.18.7 Exercise 15.7 — Node thumbnail

Render the selected (non-raster) node to a bitmap using `NodeRenderingEngine`, then save it as a PNG to the Desktop using `Bitmap` and file export.

20.19 SDK modules used in this lesson

- `/pixelaccessor` — All `PixelReader*` and `PixelReaderWriter*` classes
- `/commands` — `DocumentCommand`, `AddChildNodesCommandBuilder`, `RasterSelectionLogicalOperation`
- `/document` — `Document`
- `/rasterobject` — `Bitmap`, `PixelBuffer`, `NodeRenderingEngine`, `RasterFormat`

- /rasterselection — RasterSelection (read-only pixel-selection state)
- /buffer — Buffer
- /nodes — RasterNodeDefinition

20.20 Further examples

Example	What it shows
bitmapWriter.js	/rasterobject (Bitmap, Pixel Buffer, NodeRenderingEngine), /buffer byte manipulation, pixel writing to a bitmap

20.21 See also

- Lesson 03 — Node Model — how to find raster nodes
- Lesson 04 — Command Pattern — how commands integrate with undo
- Lesson 12 — Layer Effects — fast built-in image operations
- examples/

20.22 Examples

20.22.1 Colour inverter

```
'use strict';

// Example 14a — Colour inverter
// Lesson 14: Pixel Processing
//
// Inverts all pixel colours in the selected raster/image node.
// Demonstrates the full bitmap pipeline: read → process → write back.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
  } else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;

    console.log(`Node: ${node.descriptionInterface?.description ??
'<strong>(unnamed)</strong>'}`);
    console.log(`Size: ${w}x${h} Format: ${ri.format}`);

    if (String(ri.format) !== 'RGBA8') {
      console.log('This script requires RGBA8 format. Open an RGBA8 image
and run again.');
    } else {
      // Copy current pixels into a detached, writable bitmap
      const bitmap = ri.createCompatibleBitmap(true);
      const rw = PixelReaderWriterRGBA8.create(bitmap);

      let count = 0;
      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = rw.readPixel(x, y);
          rw.writePixel(x, y, {
            r: 255 - px.r,
            g: 255 - px.g,
```

```

        b: 255 - px.b,
        alpha: px.alpha // preserve alpha channel
    });
    count++;
  }
}

// Push back as a single undoable command
doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
console.log(`Done — inverted ${count} pixels.`);
}
}
}

```

20.22.2 Greyscale converter

```

'use strict';

// Example 14b — Greyscale converter
// Lesson 14: Pixel Processing
//
// Converts a selected RGBA8 raster node to greyscale using the
// photometric luminance formula:  $L = 0.299R + 0.587G + 0.114B$ 

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.')}
  else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;

    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '— this script requires RGBA8.
Open an RGBA8 image and run again.')}
    else {
      const bitmap = ri.createCompatibleBitmap(true);
      const rw = PixelReaderWriterRGBA8.create(bitmap);

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = rw.readPixel(x, y);
          // Photometric luminance
          const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 * px.b);
          rw.writePixel(x, y, { r: L, g: L, b: L, alpha: px.alpha });
        }
      }

      doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,
bitmap));
      console.log(`Converted ${w}*${h} pixels to greyscale.`);
    }
  }
}

```

20.22.3 Procedural gradient node

```

'use strict';
const { RGBA8 } = require('/colours.js');

// Example 14c — Procedural gradient node

```

```
// Lesson 14: Pixel Processing
//
// Creates a new 512x512 raster node from scratch containing a radial
// gradient.
// White at the centre, black at the edges.
// Demonstrates creating bitmaps without an existing node.

const { Document } = require('/document.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const { Bitmap, RasterFormat } = require('/rasterobject.js');
const { RasterNodeDefinition } = require('/nodes.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

const W = 512;
const H = 512;

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // 1. Create a blank bitmap
  const bitmap = Bitmap.create(W, H, RasterFormat.RGBA8);
  const rw = PixelReaderWriterRGBA8.create(bitmap);

  const cx = W / 2;
  const cy = H / 2;
  const maxDist = Math.sqrt(cx * cx + cy * cy);

  // 2. Fill with radial gradient
  for (let y = 0; y < H; y++) {
    for (let x = 0; x < W; x++) {
      const dx = x - cx;
      const dy = y - cy;
      const dist = Math.sqrt(dx * dx + dy * dy);
      const t = Math.min(dist / maxDist, 1.0); // 0 = centre, 1 =
edge
      const v = Math.round((1 - t) * 255); // bright centre,
dark edge

      rw.writePixel(x, y, { r: v, g: v, b: v, alpha: 255 });
    }
  }

  // 3. Wrap in a raster node definition
  const nodeDef = RasterNodeDefinition.create(RasterFormat.RGBA8);
  nodeDef.bitmap = bitmap;

  // 4. Add to document
  const builder = AddChildNodesCommandBuilder.create();
  builder.addRasterNode(nodeDef);
  const cmd = builder.createCommand(true);
  doc.executeCommand(cmd);

  const newNode = cmd.newNodes[0];
  if (newNode) {
    newNode.userDescription = 'Radial Gradient';
  }

  console.log(`Radial gradient (${W}x${H}) added to document.`);
}

```

20.22.4 Colour histogram

```
'use strict';

// Example 14d – Colour histogram
// Lesson 14: Pixel Processing
//
// Reads every pixel and builds a luminance histogram.
// Reports the 5 most common brightness values and average colour.

```

```

const { Document } = require('/document.js');
const { PixelReaderRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.')}
  else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.')}
    else {
      const bitmap = ri.createCompatibleBitmap(true);
      const reader = PixelReaderRGBA8.create(bitmap); // read-only, no
write needed

      const histogram = new Array(256).fill(0);
      let totalR = 0, totalG = 0, totalB = 0, opaqueCount = 0;

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = reader.readPixel(x, y);
          if (px.alpha > 0) {
            const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 *
px.b);
            histogram[L]++;
            totalR += px.r; totalG += px.g; totalB += px.b;
            opaqueCount++;
          }
        }
      }

      // Find top 5 most common luminance values
      const sorted = histogram
        .map((count, value) => ({ value, count }))
        .sort((a, b) => b.count - a.count)
        .slice(0, 5);

      console.log('=== Histogram:
${node.descriptionInterface?.description ?? '(unnamed)'} ===');
      console.log(`Size: ${w}x${h} Opaque pixels: ${opaqueCount}`);
      console.log('');
      console.log('Top 5 luminance values:');
      for (const { value, count } of sorted) {
        const bar = '█'.repeat(Math.round(count / opaqueCount * 40));
        console.log(` ${String(value).padStart(3)}: ${bar} (${count})`);
      }

      if (opaqueCount > 0) {
        console.log('');
        console.log('Average colour:',
          Math.round(totalR / opaqueCount),
          Math.round(totalG / opaqueCount),
          Math.round(totalB / opaqueCount)
        );
      }
    }
  }
}

```


20.22.5 Histogram analysis

```

'use strict';

// Example 14e – Histogram analysis
// Lesson 14: Pixel Processing
//
// Reads every pixel in the selected raster document and produces a
detailed
// tonal and colour report: per-channel statistics, clipping detection,
// shadow/midtone/highlight distribution, and a text-based histogram bar
chart.
//
// This is a read-only analysis – no pixels are modified and no write-back
is needed.
//
// Open the image as a document (File > Open), select the layer, and run.

const { Document } = require('/document.js');
const { PixelReaderRGBA8 } = require('/pixelaccessor.js');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
  if (!node) {
    console.log('Select a raster or image node and run again.');
  } else {
    const ri = node.rasterInterface;
    const w = ri.width;
    const h = ri.height;
    const name = node.descriptionInterface?.description ?? '(unnamed)';

    // Read-only accessor – no write-back needed for analysis
    if (String(ri.format) !== 'RGBA8') {
      console.log('Format is', ri.format, '– this script requires RGBA8.
Open an RGBA8 image and run again.');
    } else {
      const bitmap = ri.createCompatibleBitmap(true);
      const reader = PixelReaderRGBA8.create(bitmap);

      // Per-channel histograms (256 buckets each)
      const histR = new Array(256).fill(0);
      const histG = new Array(256).fill(0);
      const histB = new Array(256).fill(0);
      const histL = new Array(256).fill(0); // luminance

      // Per-channel min/max
      let minR = 255, maxR = 0;
      let minG = 255, maxG = 0;
      let minB = 255, maxB = 0;

      // Totals for averages
      let totalR = 0, totalG = 0, totalB = 0;

      // Tonal zone counts
      let shadows = 0, midtones = 0, highlights = 0;

      // Clipping counts (pixels at 0 or 255)
      let clippedShadow = 0, clippedHighlight = 0;

      let opaqueCount = 0;

      console.log(`Analysing ${w}x${h} pixels...`);

      for (let y = 0; y < h; y++) {
        for (let x = 0; x < w; x++) {
          const px = reader.readPixel(x, y);

```

```

        // Skip fully transparent pixels — they don't contribute to
visible tones
        if (px.alpha === 0) continue;
        opaqueCount++;

        // Accumulate channel histograms
        histR[px.r]++;
        histG[px.g]++;
        histB[px.b]++;

        // Luminance (perceptual weighting)
        const L = Math.round(0.299 * px.r + 0.587 * px.g + 0.114 * px.b);
        histL[L]++;

        // Per-channel min/max
        if (px.r < minR) minR = px.r;
        if (px.r > maxR) maxR = px.r;
        if (px.g < minG) minG = px.g;
        if (px.g > maxG) maxG = px.g;
        if (px.b < minB) minB = px.b;
        if (px.b > maxB) maxB = px.b;

        // Totals
        totalR += px.r;
        totalG += px.g;
        totalB += px.b;

        // Tonal zones: shadows 0-84, midtones 85-169, highlights 170-255
        if (L <= 84) shadows++;
        else if (L <= 169) midtones++;
        else highlights++;

        // Clipping
        if (L === 0) clippedShadow++;
        if (L === 255) clippedHighlight++;
    }
}

if (opaqueCount === 0) {
    console.log('No opaque pixels found.');
```

```

} else {
    const avgR = Math.round(totalR / opaqueCount);
    const avgG = Math.round(totalG / opaqueCount);
    const avgB = Math.round(totalB / opaqueCount);
    const avgL = Math.round(0.299 * avgR + 0.587 * avgG + 0.114 * avgB);

    console.log('');
    console.log(`=== ${name} — ${w}x${h} (${opaqueCount} opaque pixels)
===`);

    console.log('');
    console.log('— Channel ranges —');
    console.log(`  Red:  min ${String(minR).padStart(3)} max
${String(maxR).padStart(3)}  avg ${String(avgR).padStart(3)}`);
    console.log(`  Green: min ${String(minG).padStart(3)} max
${String(maxG).padStart(3)}  avg ${String(avgG).padStart(3)}`);
    console.log(`  Blue:  min ${String(minB).padStart(3)} max
${String(maxB).padStart(3)}  avg ${String(avgB).padStart(3)}`);
    console.log(`  Luma:  avg ${String(avgL).padStart(3)}`);

    console.log('');
    console.log('— Tonal distribution —');
    const pct = (n) => ((n / opaqueCount) * 100).toFixed(1).padStart(5)
+ '%';
    console.log(`  Shadows   (0-84):  ${pct(shadows)}  (${shadows}
px)`);
    console.log(`  Midtones   (85-169): ${pct(midtones)}  (${midtones}
```

```

px)`);
    console.log(` Highlights(170-255): ${pct(highlights)}
    (${highlights} px)`);

    console.log('');
    console.log(`— Clipping —`);
    const clipPctS = ((clippedShadow / opaqueCount) *
100).toFixed(2);
    const clipPctH = ((clippedHighlight / opaqueCount) *
100).toFixed(2);
    console.log(` Shadow clip (L=0): ${clipPctS}% (${clippedShadow}
px)`);
    console.log(` Highlight clip (L=255): ${clipPctH}%
    (${clippedHighlight} px)`);
    if (parseFloat(clipPctS) > 1) console.log(' ⚠ Shadow clipping
exceeds 1%');
    if (parseFloat(clipPctH) > 1) console.log(' ⚠ Highlight clipping
exceeds 1%');

    // Luminance histogram bar chart – 32 buckets of 8 values each
    console.log('');
    console.log(`— Luminance histogram (0=dark → 255=bright) —`);
    const bucketCount = 32;
    const buckets = new Array(bucketCount).fill(0);
    for (let i = 0; i < 256; i++) {
        buckets[Math.floor(i / (256 / bucketCount))] += histL[i];
    }
    const maxBucket = Math.max(...buckets);
    const barWidth = 30;
    for (let i = 0; i < bucketCount; i++) {
        const low = Math.round(i * 256 / bucketCount);
        const filled = Math.round((buckets[i] / maxBucket) * barWidth);
        const bar = '█'.repeat(filled) + '░'.repeat(barWidth - filled);
        console.log(` ${String(low).padStart(3)} ${bar}`);
    }
}
}
}
}

```

20.22.6 Posterise

```

'use strict';

// Example 14f – Posterise
// Lesson 14: Pixel Processing
//
// Reduces each colour channel to N discrete levels, creating a flat,
// graphic
// poster-like effect. The level count is configurable via a dialog (2-
// 16).
//
// Posterisation works by quantising each channel value to the nearest
// step
// in a set of N evenly-spaced levels across 0-255. With 2 levels each
// channel
// is either 0 or 255. With 4 levels the steps are 0, 85, 170, 255.
//
// This effect cannot be expressed as a simple adjustment layer – it
// requires
// per-pixel arithmetic, which is why it belongs in a pixel script.
//
// Open the image as a document (File > Open), select the layer, and run.

const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');

```

```
// — Dialog


---


// addComboBox is used for level selection — the SDK dialog has no slider
control.

const dlg    = Dialog.create('Posterise');
const col    = dlg.addColumn();
const grp    = col.addGroup('Settings');
const LEVELS = ['2', '3', '4', '5', '6', '8'];
const combo  = grp.addComboBox('Levels', LEVELS, 2); // default index 2 =
4 levels

const result = dlg.runModal();
if (result !== DialogResult.Ok) {
  console.log('Cancelled.');
```

```
  } else {
    const levels = parseInt(LEVELS[combo.selectedIndex], 10); // read
selectedIndex, not .value

    // — Pixel processing
    

---


    const doc = Document.current;
    if (!doc) { console.log('No document open.');
```

```
    } else {
      const node = doc.selection.nodes.filter(n => n.isRasterNode ||
n.isImageNode).first;
      if (!node) {
        console.log('Select a raster or image node and run again.');
```

```
      } else {
        const ri    = node.rasterInterface;
        const w      = ri.width;
        const h      = ri.height;
        if (String(ri.format) !== 'RGBA8') {
          console.log('Format is', ri.format, '- this script requires RGBA8.
Open an RGBA8 image and run again.');
```

```
        } else {
          const bitmap = ri.createCompatibleBitmap(true);
          const rw      = PixelReaderWriterRGBA8.create(bitmap);

          console.log(`Posterising ${w}x${h} to ${levels} levels...`);

          // Pre-compute a lookup table: for each input value 0-255, what is
the
          // nearest quantised output? This avoids repeating the arithmetic
per pixel.
          //
          // For N levels, the step size is 255 / (N-1).
          // The quantised value is: round(input / step) * step, clamped to 0-
255.
          const lut = new Uint8Array(256);
          const step = 255 / (levels - 1);
          for (let i = 0; i < 256; i++) {
            lut[i] = Math.min(255, Math.round(Math.round(i / step) * step));
          }

          for (let y = 0; y < h; y++) {
            for (let x = 0; x < w; x++) {
              const px = rw.readPixel(x, y);
              rw.writePixel(x, y, {
                r:    lut[px.r],
                g:    lut[px.g],
                b:    lut[px.b],
                alpha: px.alpha // preserve transparency
              });
            }
          }
        }
      }
    }
  }
}
```

```
doc.executeCommand(DocumentCommand.createReplaceBitmap(doc.selection,  
bitmap));  
    console.log(`Done – posterised to ${levels} levels.`);  
  }  
}  
}
```


Lesson 16 — Fonts & Typography

Typography automation means knowing what fonts are available, how to classify them, and how to apply them to text. This lesson covers the font inspection API: enumerating families, querying weight, width, and serif classification, using the PANOSE system for font matching, and combining font data with the text/story API to build practical typography tools.

Font tasks are common in professional workflows: checking a document uses only approved fonts, substituting a missing typeface with the closest alternative, generating a usage report, or applying consistent styling across a document.

The font API has two distinct layers. The `/fonts` module is the **enumeration** layer — `FontFamily.all`, metadata, classification, and PANOSE descriptors — for auditing, reporting, and font-matching. The `/glyphatts` and `/paragraphatts` modules are the **application** layer — setting specific fonts on text ranges via `StoryDelta`. Confusing the two (e.g. applying formatting through the enumeration API) produces no-ops or cryptic errors.

PANOSE is a ten-digit numeric classification describing a typeface's visual characteristics (serif style, weight, proportion, contrast, and more). It lets you find the closest available substitute for a missing font by similarity rather than name matching — usually a more visually appropriate result.

By the end you'll be able to enumerate installed fonts, classify them, find the closest substitute for any named typeface, and apply fonts to text ranges via the story API.

21.1 What this lesson covers

- Enumerating all installed fonts and font families

- Font classification: serif, sans-serif, monospaced, condensed
- The PANOSE system for font matching
- Querying font properties: weight, width, bold/italic availability
- Font collections
- Combining font data with the text/story API for practical scripts
- Finding the best available font for a given style need
- Document font usage reporting

21.2 The fonts module

The `/fonts` module adds a complete view of the system font library. Before this, you could apply a named style via `StoryDelta`, but you couldn't ask "is this font installed?" or "what serif fonts do I have?". Now you have the full picture.

```
'use strict';
const { Font, FontFamily, FontCollection, FontWeight, FontWidth } =
  require('/fonts.js');
```

21.3 Three levels of font data

```
FontCollection ← named groups (e.g. "Microsoft Office Fonts", "Adobe
Fonts")
  └─ FontFamily ← a named family (e.g. "Helvetica Neue")
    └─ Font ← an individual variant (e.g. "Helvetica Neue Bold
Italic")
```

You can enter the hierarchy at any level.

21.4 Enumerating font families

```
'use strict';
const { FontFamily } = require('/fonts.js');
const { EnumerationResult } = require('affinity:common');

// Array of all families (convenient but allocates a lot)
const families = FontFamily.all;
console.log(`${families.length} font families installed`);

// Callback-based (memory-efficient for large font libraries)
FontFamily.enumerate(family => {
  console.log(family.name, '-', family.fontCount, 'font(s)');
  return EnumerationResult.Continue;
});
```

21.5 Inspecting fonts within a family

```
'use strict';
const { FontFamily } = require('/fonts.js');
const family = FontFamily.all.find(f => f.name === 'Arial');
if (!family) {
  console.log('Arial not installed.');
```

```
  } else {
    console.log(`${family.name}: ${family.fontCount} variant(s)`);
    console.log(`Has variable fonts: ${family.hasVariations}`);

    for (let i = 0; i < family.fontCount; i++) {
      const font = family.getFont(i);
      console.log(`  [${i}] ${font.traitsName}`);
      console.log(`    Bold: ${font.isBold} Italic: ${font.isItalic}`);
      console.log(`    Weight: ${font.weight} Width: ${font.width}`);
      console.log(`    PostScript: ${font.postscriptName}`);
    }

    // Or iterate directly
    for (const font of family.fonts) {
      console.log(' ', font.postscriptName);
    }
  }
}
```


21.6 Getting a Font object

The correct way to get a Font object is via FontFamily.

```
'use strict';
const { FontFamily } = require('/fonts.js');

// Build a name-keyed map of all installed families
const familyMap = new Map();
for (const fam of FontFamily.all) {
  familyMap.set(fam.name.toLowerCase(), fam);
}

// Get the regular weight font from a named family
function getFontByFamily(name, preferBold = false) {
  const fam = familyMap.get(name.toLowerCase());
  if (!fam) return null;
  let best = null, bestDist = Infinity;
  const targetWeight = preferBold ? 0.75 : 0;
  for (const f of fam.fonts) {
    if (!f.isValid) continue;
    const d = Math.abs(f.normalizedWeight - targetWeight);
    if (d < bestDist) { bestDist = d; best = f; }
  }
  return best;
}

const georgia = getFontByFamily('Georgia');
if (georgia) {
  console.log('Valid: ', georgia.isValid);
  console.log('Serif: ', georgia.isSerif);
  console.log('PS name: ', georgia.postscriptName);
}

// Platform defaults (these always work with no arguments)
const { Font } = require('/fonts.js');
const def = Font.createDefault();
const mono = Font.createDefaultMonospaced();
const sym = Font.createDefaultSymbol();
```

21.7 Font classification via PANOSE

PANOSE is a 10-byte classification system for fonts. The SDK exposes the key classification queries directly on the Font object:

```
'use strict';
const { Font } = require('/fonts.js');

for (const font of Font.all.slice(0, 50)) {
  const family = font.familyName;
  if (font.isSerif) console.log('Serif: ', family, font.traitsName);
  if (font.isSansSerif) console.log('Sans: ', family, font.traitsName);
  if (font.isMonospaced) console.log('Mono: ', family, font.traitsName);
}
```

For fine-grained classification, access the Panose object directly:

```
'use strict';
const { Font, FontFamily, PanoseType } = require('/fonts.js');

// Get a font via FontFamily — this is always safe
const tnr = FontFamily.all.find(f => f.name === 'Times New Roman');
const font = tnr ? tnr.getFont(0) : Font.createDefault();

const panose = font.panose;

console.log('Is serif: ', panose.isSerif);
console.log('Is sans-serif:', panose.isSansSerif);
console.log('Is monospaced:', panose.isMonospaced);
console.log('Is empty: ', panose.isEmpty);
console.log('PANOSE string:', panose.toString());
```

```
// Type-specific classification
console.log('Is serif type:', panose.is(PanoseType.Serif));
```

21.8 Font distance (finding the closest match)

Both Font and Panose expose a `getDistance(other)` method. Smaller distance = closer match. Useful for finding a substitute when a font isn't installed:

```
'use strict';
const { Font, FontFamily } = require('/fonts.js');

// Try to find 'Futura' via FontFamily — no risk of INVALID_ARGS
const futuraFamily = FontFamily.all.find(f => f.name === 'Futura');

if (!futuraFamily) {
  console.log('Futura not installed — finding closest match...');

  // Use createEmpty() as a baseline for distance comparison
  // Or better: use the default font as the reference point
  const target = Font.createDefault();

  let bestFont = null;
  let bestDist = Infinity;

  for (const font of Font.all) {
    const dist = target.getDistance(font);
    if (dist < bestDist) {
      bestDist = dist;
      bestFont = font;
    }
  }

  if (bestFont) {
    console.log('Best match:', bestFont.familyName, bestFont.traitsName);
    console.log('Distance: ', bestDist);
  }
} else {
  console.log('Futura is installed:', futuraFamily.fontCount,
    'variant(s)');
}
```

To find a substitute for a specific missing font, the most reliable approach is to compare PANOSE data: get the PANOSE of any installed font with a similar style, then measure `font.panose.getDistance(candidate.panose)` across all installed fonts.

21.9 Font collections

Collections are named groups of fonts (Adobe Fonts, Microsoft Office Fonts, etc.):

```
'use strict';
const { FontCollection } = require('/fonts.js');

for (const collection of FontCollection.all) {
  console.log('Collection:', collection.displayName);
}
```

21.10 Practical: check which fonts a document uses

The Document object exposes `enumerateFontNames` — fonts referenced in text nodes:

```
'use strict';
const { Document } = require('/document.js');
const { Font, FontWeight, FontWidth } = require('/fonts.js');

const doc = Document.current;
if (!doc) { console.log('No document.');
```

```

else {
  const fontNames = doc.getFontNames(); // returns [{ name,
isInstalled }] pairs via enumeration

  // Or use the raw enumeration
  const missing = [];
  doc.enumerateFontNames((name, isInstalled) => {
    if (!isInstalled) missing.push(name);
    return 0; // continue
  });

  console.log('Fonts used in document:');
  doc.enumerateFontNames((name, isInstalled) => {
    console.log(` ${isInstalled ? 'v' : 'x'} ${name}`);
    return 0;
  });

  if (missing.length > 0) {
    console.log(`\n⚠️ ${missing.length} font(s) missing:`);
    for (const name of missing) console.log(' -', name);
  }
}

```

21.11 Practical: font picker dialog

Combine the fonts module with the Dialog API to build a font chooser:

```

'use strict';
const { Document } = require('/document.js');
const { Font, FontFamily } = require('/fonts.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { DocumentCommand } = require('/commands.js');
const { StoryDelta } = require('/storydelta.js');
const { GlyphAttStringType } = require('/glyphatts.js');
const { Selection, TextSelection } = require('/selections.js');

const doc = Document.current;
if (!doc) { console.log('No document.'); }
else {
  const families = FontFamily.all;
  const names = families.map(f => f.name);

  const dlg = Dialog.create('Choose Font');
  const col = dlg.addColumn();
  const grp = col.addGroup('Font family');
  const combo = grp.addComboBox('Family', names, 0);

  const result = dlg.runModal();
  if (result !== DialogResult.Ok) { console.log('Cancelled.'); }
  else {
    const chosenFamily = families[combo.selectedIndex];
    const firstFont = chosenFamily.fontCount > 0 ?
chosenFamily.getFont(0) : null;
    if (!firstFont) { console.log('No fonts in family.'); }
    else {
      // Apply to selected text via StoryDelta
      const textNode = doc.selection.nodes.filter(n =>
n.isTextNode).first;
      if (!textNode) {
        console.log('Select a text node too.');
      } else {
        doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

        const si = textNode.storyInterface;
        const range = si.storyRange;
        const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
        const sel = Selection.create(doc, textNode);
        sel.addSubSelectionForNode(textNode, textSel);

```

```

        const delta =
StoryDelta.createGlyphString(GlyphAttStringType.FontPostscriptName,
firstFont.postscriptName);
        doc.executeCommand(DocumentCommand.createFormatText(sel, delta));
        console.log(`Applied font: ${firstFont.postscriptName}`);
    }
}
}
}

```

21.12 Font properties reference

Property	Type	Description
font.isValid	bool	Font exists and is usable
font.isBold	bool	Bold weight
font.isItalic	bool	Italic style
font.isVariableBold	bool	Has variable bold axis
font.isVariableItalic	bool	Has variable italic axis
font.isBoldAvailable	bool	Bold variant exists in family
font.isItalicAvailable	bool	Italic variant exists in family
font.isCondensed	bool	Condensed width class
font.isExpanded	bool	Expanded width class
font.isSerif	bool	Serif (via PANOSE)
font.isSansSerif	bool	Sans-serif (via PANOSE)
font.isMonospaced	bool	Monospaced (via PANOSE)
font.weight	number	Numeric weight (400 = Regular, 700 = Bold)
font.normalizedWeight	number	Normalised weight 0–1
font.width	C++ enum	Width class — read from existing font; cannot be constructed directly
font.familyName	string	Family name
font.wwsFamily	string	WWS family name (weight, width, slant grouping)
font.ribbiFamily	string	RIBBI family name (regular, italic, bold, bold-italic)

Property	Type	Description
<code>font.traitsName</code>	string	Traits string (Bold, Italic, etc.)
<code>font.postscriptName</code>	string	PostScript name (unique identifier)
<code>font.postscriptPrefix</code>	string	PS name prefix
<code>font.variableBold</code>	number	Variable bold axis value
<code>font.variableItalic</code>	number	Variable italic axis value
<code>font.panose</code>	Panose	Full PANOSE classification

21.13 FontFamily properties reference

Property	Type	Description
<code>family.name</code>	string	Family name
<code>family.fontCount</code>	number	Number of fonts in family
<code>family.hasVariations</code>	bool	Contains variable fonts
<code>family.hasFixed</code>	bool	Contains fixed-pitch fonts
<code>family.fonts</code>	Font[]	All fonts in family
<code>family.getFont(i)</code>	Font	Font at index

21.14 Key gotchas

font.postscriptName may be empty on some fonts. Even when `isValid` is true, some properties like `postscriptName` may be empty. Always prefer getting fonts via `FontFamily.all` and `family.getFont(i)` for reliable property access.

panose.is(PanoseType.Serif) and panose.isSerif can disagree. The convenience property uses a broader classification; `panose.is()` applies a stricter type test. Prefer `font.isSerif` for general classification checks.

Font.all enumerates every individual font — not families. A family with 8 variants appears as 8 entries. For family-level work, use `FontFamily.all`.

PANOSE is not always populated. Some fonts have an empty PANOSE record (`panose.isEmpty == true`). In this case `isSerif`, `isSansSerif`, `isMonospaced` will all return false, not because the font isn't serif, but because the data isn't there.

font.postscriptName is the stable identifier. Use this when applying fonts via `StoryDelta` — it's more reliable than the family name, which can have platform-specific variations.

doc.enumerateFontNames callback convention. The callback receives (`name`, `isInstalled`) and must return 0 to continue (not `Enumeration`

Result.Continue as in other SDK callbacks). This is an inconsistency in the API.

21.15 Exercises

21.15.1 Exercise 16.1 — Font census with classification

List all installed font families. For each, classify it as Serif, Sans-serif, Monospaced, or Unknown. Print a summary count at the end.

21.15.2 Exercise 16.2 — Missing font reporter

Open a document with some text. Use doc.enumerateFontNames to find any fonts that are referenced but not installed. Log their names.

21.15.3 Exercise 16.3 — Font distance finder

Ask the user for a font name via a Dialog TextBox. If not installed, find and report the three closest installed alternatives using font.getDistance().

21.15.4 Exercise 16.4 — Font picker tool

Build a Dialog with a ComboBox of all installed family names. When confirmed, apply the first font in the chosen family to the current text selection using StoryDelta and GlyphAttStringType.FontPostscriptName.

21.15.5 Exercise 16.5 — Variable font detector

Find and list all font families that contain at least one variable font (family.hasVariations === true). For each, list the family name and how many fonts it has.

21.16 SDK modules used in this lesson

- /fonts — Font, FontFamily, FontCollection, FontWeight, FontWidth, Panose, PanoseType
- /document — Document (for enumerateFontNames)
- /dialog — Dialog, DialogResult
- /storydelta — StoryDelta
- /glyphatts — GlyphAttStringType
- /selections — Selection, TextSelection
- /commands — DocumentCommand

21.17 See also

- Lesson 07 — Text & Story
- Lesson 08 — Dialog UI
- examples/

21.18 Examples

21.18.1 Font census

```
'use strict';

// Example 15a — Font census
// Lesson 15: Fonts & Typography
//
// Lists all installed font families with classification and a summary.

const { Font, FontFamily } = require('/fonts.js');

const families = FontFamily.all;
let serif = 0, sans = 0, mono = 0, variable = 0, unknown = 0;

console.log(`=== Font Census: ${families.length} families ===\n`);

for (const family of families) {
  const first = family.fontCount > 0 ? family.getFont(0) : null;
  let tag = '?';
```

```

if (first && !first.panose.isEmpty) {
  if (first.isMonospaced) { tag = 'MONO'; mono++; }
  else if (first.isSerif) { tag = 'SERF'; serif++; }
  else if (first.isSansSerif) { tag = 'SANS'; sans++; }
  else { tag = '?'; unknown++; }
} else {
  unknown++;
}

const varFlag = family.hasVariations ? ' [variable]' : '';
if (family.hasVariations) variable++;

console.log(`[${tag}] ${family.name} (${family.fontCount})${varFlag}`);
}

console.log(`\n== Summary ==`);
console.log(`Serif:      ${serif}`);
console.log(`Sans-serif:    ${sans}`);
console.log(`Monospaced:     ${mono}`);
console.log(`Unknown/empty:   ${unknown}`);
console.log(`Variable:       ${variable}`);
console.log(`Total:         ${families.length}`);

```

21.18.2 Missing font reporter

```

'use strict';

// Example 15b – Missing font reporter
// Lesson 15: Fonts & Typography
//
// Reports all fonts used in the document that are not installed.
// Uses doc.enumerateFontNames() – callback returns 0 to continue
// (different from the EnumerationResult pattern used elsewhere in the
// SDK).

const { Document } = require('/document.js');
const { FontFamily } = require('/fonts.js');

const doc = Document.current;
if (!doc) { console.log('No document open.'); }
else {
  const installed = [];
  const missing = [];

  doc.enumerateFontNames((name, isInstalled) => {
    if (isInstalled) installed.push(name);
    else missing.push(name);
    return 0; // 0 = continue (not EnumerationResult.Continue)
  });

  const total = installed.length + missing.length;
  console.log(`Document: ${doc.title}`);
  console.log(`Fonts referenced: ${total}`);
  console.log(`Installed:      ${installed.length}`);
  console.log(`Missing:       ${missing.length}`);

  if (installed.length > 0) {
    console.log(`\n✓ Installed:');
    for (const name of installed) console.log(`  ${name}`);
  }

  if (missing.length > 0) {
    // Build a lookup of installed family names for substitute suggestions
    const installedFamilies = [];
    for (const fam of FontFamily.all) {
      installedFamilies.push(fam.name.toLowerCase());
    }
  }
}

```

```

console.log('\nX Missing (not installed):');
for (const name of missing) {
  console.log(`  ${name}`);

  // Suggest the closest name match among installed families
  const target = name.toLowerCase();
  let bestName = null, bestScore = -1;
  for (const famName of installedFamilies) {
    // Simple shared-prefix length as a rough similarity score
    let score = 0;
    const minLen = Math.min(target.length, famName.length);
    while (score < minLen && target[score] === famName[score]) score++;

    if (score > bestScore) { bestScore = score; bestName = famName; }
  }

  if (bestName && bestScore > 0) {
    console.log(`    → Closest installed name: ${bestName}`);
  }
} else {
  console.log('\nAll fonts are installed ✓');
}
}

```


Lesson 17 — Raster Adjustments

Raster adjustments are non-destructive layers that sit above a pixel layer and modify its appearance without altering its pixel data. Every adjustment type has two things: a **definition class** (used to create a new node) and a **command factory** (used to modify an existing node's parameters).

22.1 What this lesson covers

- Creating adjustment layers programmatically via `*AdjustmentRasterNode` Definition factories
- Reading existing adjustment parameters from a selected node
- Setting parameters via `DocumentCommand.createSet*AdjustmentParameters`
- Every adjustment type the SDK exposes, with parameter ranges
- Inserting adjustments above a specific raster node
- RAW Develop nodes
- Destructive Halftone filter

22.2 Creating an adjustment layer

The pattern is the same for every adjustment type:

1. Create a definition via `XxxAdjustmentRasterNodeDefinition.create(params)` or `.createDefault()`
2. Add it via `AddChildNodesCommandBuilder.addXxxAdjustmentRasterNode(def)`
3. Set `InsertionMode.OutsideAtTop` and target the raster node to place it inside that layer

```
'use strict';  
const { Document } = require('/document.js');
```

```

const { BrightnessContrastAdjustmentRasterNodeDefinition } = require('./
nodes.js');
const { AddChildNodesCommandBuilder, InsertionMode } = require('./
commands.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!node || !node.isRasterNode) {
  console.log('Select a raster node first.');
} else {
  const def = BrightnessContrastAdjustmentRasterNodeDefinition.create({
    brightness: 0.3,
    contrast: 0.2,
    isLinear: false
  });

  const builder = AddChildNodesCommandBuilder.create();
  builder.addBrightnessContrastAdjustmentRasterNode(def);
  builder.setInsertionTarget(node);
  builder.setInsertionMode(InsertionMode.InsideAtTop);

  doc.executeCommand(builder.createCommand(true));
  console.log('Brightness/Contrast adjustment added.');
}

```

The adjustment appears in the Layers panel as a child of the raster node, identical to one added manually via the menu.

22.3 Reading parameters from an existing adjustment node

To read the current parameters of an adjustment layer, check its type and access `.parameters`:

```

'use strict';
const { Document } = require('/document.js');
const Nodes = require('/nodes.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node instanceof Nodes.BrightnessContrastAdjustmentRasterNode) {
  const p = node.parameters;
  console.log('Brightness:', p.brightness);
  console.log('Contrast: ', p.contrast);
  console.log('Linear:    ', p.isLinear);
}

if (node instanceof Nodes.HSLShiftAdjustmentRasterNode) {
  const p = node.parameters;
  const m = p.masterParameters;
  console.log('Hue shift: ', m.hueShift);           // radians
  console.log('Sat shift: ', m.saturationShift);
  console.log('Lum shift: ', m.luminosityShift);
}

```

22.4 Brightness / Contrast

```

// Snippet
const def = BrightnessContrastAdjustmentRasterNodeDefinition.create({
  brightness: 0.0, // -1.0 to 1.0
  contrast: 0.0, // -1.0 to 1.0
  isLinear: false
});

```

To modify an existing node:

```

// Snippet
doc.executeCommand(DocumentCommand.createSetBrightnessContrastAdjustmentParameters(sel,
{
  brightness: 0.3,

```

```

    contrast: 0.2,
    isLinear: false
  }));

```

22.5 Levels

LevelsAdjustmentParameters holds a masterParameters object plus per-channel parameter objects (red, green, blue, and two colour-space-specific channels). All values are normalized 0.0–1.0; gamma range is 0.0–2.0.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const {
  LevelsAdjustmentRasterNodeDefinition,
  LevelsAdjustmentChannelParameters
} = require('/nodes.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  // Create with a default definition, then adjust master parameters
  const def = LevelsAdjustmentRasterNodeDefinition.createDefault(doc);
  const params = def.parameters;

  params.masterParameters = {
    blackLevel: 0.05, // lift shadows (0.0–1.0)
    whiteLevel: 0.95, // clip highlights (0.0–1.0)
    gamma: 1.0, // midpoint gamma (0.0–2.0; 1.0 = linear)
    outputBlackLevel: 0.0,
    outputWhiteLevel: 1.0
  };

  def.parameters = params; // write back after mutation

  // Apply to selection
  const sel = Selection.create(doc, node);
  doc.executeCommand(DocumentCommand.createSetLevelsAdjustmentParameters(sel,
    params));
}

```

22.6 Curves

CurvesAdjustmentParameters holds a master spline and optional per-channel splines. Import Spline from /geometry.

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const { CurvesAdjustmentRasterNodeDefinition, CurvesAdjustmentParameters } =
  require('/nodes.js');
const { Spline } = require('/geometry.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const params = CurvesAdjustmentParameters.create();

  // Build an S-curve: slightly boost contrast
  const sCurve = Spline.create();
  sCurve.insertPointXY(0.0, 0.0); // shadow anchor
  sCurve.insertPointXY(0.25, 0.15); // shadow lift
  sCurve.insertPointXY(0.75, 0.85); // highlight pull-down
  sCurve.insertPointXY(1.0, 1.0); // highlight anchor
  params.masterSpline = sCurve;
}

```

```

const def = CurvesAdjustmentRasterNodeDefinition.createDefault(doc);
def.parameters = params;

const sel = Selection.create(doc, node);

doc.executeCommand(DocumentCommand.createSetCurvesAdjustmentParameters(sel,
params));
}

```

Per-channel splines use `params.setChannelSpline(channel, spline)` where channel indices are 0 = Red, 1 = Green, 2 = Blue.

22.7 HSL Shift

`HSLShiftAdjustmentParameters.create()` returns a parameters object. Call `.setChannelParameters(channel, params)` where channel 0 = master. hueShift is in radians (full circle = $2\pi \approx 6.28$).

```

'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { Selection } = require('/selections.js');
const {
  HSLShiftAdjustmentRasterNodeDefinition,
  HSLShiftAdjustmentParameters
} = require('/nodes.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (node) {
  const hslParams = HSLShiftAdjustmentParameters.create();

  // Shift master channel
  hslParams.setChannelParameters(0, {
    hueShift: 0.1, // radians
    saturationShift: 0.2, // -1.0 to 1.0
    luminosityShift: 0.0 // -1.0 to 1.0
  });

  const def = HSLShiftAdjustmentRasterNodeDefinition.create(hslParams);

  const sel = Selection.create(doc, node);

  doc.executeCommand(DocumentCommand.createSetHSLShiftAdjustmentParameters(sel,
hslParams));
}

```

22.8 White Balance

```

// Snippet
doc.executeCommand(DocumentCommand.createSetWhiteBalanceAdjustmentParameters(sel,
{
  whiteBalance: 0.3, // -1.0 (cool) to 1.0 (warm)
  tint: 0.0 // -1.0 to 1.0
}));

```

22.9 Vibrance

```

// Snippet
doc.executeCommand(DocumentCommand.createSetVibranceAdjustmentParameters(sel,
{
  vibrance: 0.4, // -1.0 to 1.0
  saturation: 0.1 // -1.0 to 1.0
}));

```

22.10 Exposure

```

// Snippet
doc.executeCommand(DocumentCommand.createSetExposureAdjustmentParameters(sel,
{

```

```
    exposure: 1.5    // -20.0 to 20.0 (EV stops)
  }));
```

22.11 Colour Balance

Applies to shadow, midtone, and highlight tonal ranges independently. Range index: 0 = Highlights, 1 = Midtones, 2 = Shadows (use `TonalRangeType` enum if available, or the integer directly).

```
// Snippet
doc.executeCommand(DocumentCommand.createSetColourBalanceAdjustmentParameters(sel,
{
  values: [
    { cyanRed: 0.1, magentaGreen: -0.1, yellowBlue: 0.0 }, // highlights
    { cyanRed: 0.0, magentaGreen: 0.1, yellowBlue: 0.0 }, // midtones
    { cyanRed: 0.0, magentaGreen: 0.0, yellowBlue: 0.1 }  // shadows
  ],
  preserveLuminosity: true
}));
```

22.12 Posterise

```
// Snippet
doc.executeCommand(DocumentCommand.createSetPosteriseAdjustmentParameters(sel,
{
  levels: 4    // 2 to 256 integer
}));
```

22.13 Threshold

```
// Snippet
doc.executeCommand(DocumentCommand.createSetThresholdAdjustmentParameters(sel,
{
  threshold: 0.5    // 0.0 to 1.0
}));
```

22.14 Invert

`InvertAdjustmentRasterNodeDefinition` has no parameters — it just inverts all channels.

```
// Snippet
const { InvertAdjustmentRasterNodeDefinition } = require('/nodes.js');
const def = InvertAdjustmentRasterNodeDefinition.create();
```

22.15 Recolour

Maps all colours to a single hue/saturation (like a tinted duotone).

```
// Snippet
doc.executeCommand(DocumentCommand.createSetRecolourAdjustmentParameters(sel,
{
  hue: 1.0,    // 0.0 to 2π radians
  saturation: 0.8, // 0.0 to 1.0
  lightness: 0.0 // -1.0 to 1.0
}));
```

22.16 Split Toning

Applies separate hue/saturation to highlights and shadows.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetSplitToningAdjustmentParameters(sel,
{
  highlightsHue: 0.1,    // 0.0 to 2π
  highlightsSaturation: 0.3,
  shadowsHue: 3.5,
  shadowsSaturation: 0.2,
  balance: 0.5    // 0.0 (shadows) to 1.0 (highlights)
}));
```

22.17 Black & White

Converts to greyscale with per-channel luminance weights.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetBlackAndWhiteAdjustmentParameters(sel,
{
  red:    0.4,    // -2.0 to 3.0
  green:  0.6,
  blue:   0.2,
  cyan:   0.5,
  magenta: 0.3,
  yellow: 0.5
}));
```

22.18 Tone Compression and Tone Stretch

Used in HDR / tone-mapping workflows.

```
// Snippet
const { ToneCompressionMethod } = require('/nodes.js');
doc.executeCommand(DocumentCommand.createSetToneCompressionAdjustmentParameters(sel,
{
  method: ToneCompressionMethod.Reinhard, // or check available enum
  values
  exposure: 1.0, // 0.0 to 2.0
  gamma:    1.0, // 0.0 to 2.0
  colour:   1.0 // 0.0 to 1.0
}));
```

22.19 Normals (normal map rotation)

Used on normal map textures.

```
// Snippet
doc.executeCommand(DocumentCommand.createSetNormalsAdjustmentParameters(sel,
{
  rotation: 0.0, // -π to π radians
  scale:    1.0, // 0.0 to 2.0
  flipX:    false,
  flipY:    false
}));
```

22.20 RAW Develop

A `DevelopNode` represents a RAW image opened in the Develop persona. Unlike the adjustment nodes above, it is not created programmatically — it appears when the user opens a RAW file. Read its `DevelopParameters` via `.parameters` and `.sourceIsRaw`, then push changes back with `createSetDevelopParameters`.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const Nodes = require('/nodes.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const developNode = doc.layers.all.find(n => n.isDevelopNode);
  if (!developNode) {
    console.log('No RAW develop node found — open a RAW file first.')}
  else {
    const params = developNode.parameters;
    console.log('Source is RAW:', developNode.sourceIsRaw);

    // Tweak the develop parameters, then write back:
    // doc.executeCommand(DocumentCommand.createSetDevelopParameters(
    //   doc.selection, params));
    console.log('Develop parameters read.')}
  }
}
```

Supporting enums (from /nodes): DevelopDetailRefinementMethod (UnsharpMask/HighPass/MultiBand), DevelopToneCurveMethod (Standard/Compressed/Natural/HighContrast/Log), and DevelopInvertMethod (InfraredRB/InfraredRGB/Negative). There is no addDevelopNode builder method — RAW develop nodes are created by the application, not by scripts.

22.21 All adjustment types — quick reference

Adjustment	Definition class	Parameter command
Brightness/Contrast	BrightnessContrastAdjustmentRasterNodeDefinition	createSetBrightnessContrastAdjustmentParameters
Levels	LevelsAdjustmentRasterNodeDefinition	createSetLevelsAdjustmentParameters
Curves	CurvesAdjustmentRasterNodeDefinition	createSetCurvesAdjustmentParameters
HSL Shift	HSLShiftAdjustmentRasterNodeDefinition	createSetHSLShiftAdjustmentParameters
White Balance	WhiteBalanceAdjustmentRasterNodeDefinition	createSetWhiteBalanceAdjustmentParameters
Vibrance	VibranceAdjustmentRasterNodeDefinition	createSetVibranceAdjustmentParameters
Exposure	ExposureAdjustmentRasterNodeDefinition	createSetExposureAdjustmentParameters
Colour Balance	ColourBalanceAdjustmentRasterNodeDefinition	createSetColourBalanceAdjustmentParameters
Shadows/Highlights	ShadowsHighlightsAdjustmentRasterNodeDefinition	createSetShadowsHighlightsAdjustmentParameters
Posterise	PosteriseAdjustmentRasterNodeDefinition	createSetPosteriseAdjustmentParameters
Threshold	ThresholdAdjustmentRasterNodeDefinition	createSetThresholdAdjustmentParameters
Invert	InvertAdjustmentRasterNodeDefinition	<i>(no parameters)</i>
Recolour	RecolourAdjustmentRasterNodeDefinition	createSetRecolourAdjustmentParameters
Black & White	BlackAndWhiteAdjustmentRasterNodeDefinition	createSetBlackAndWhiteAdjustmentParameters

Adjustment	Definition class	Parameter command
Split Toning	SplitToningAdjustmentRasterNodeDefinition	createSetSplitToningAdjustmentParameters
Selective Colour	SelectiveColourAdjustmentRasterNodeDefinition	createSetSelectiveColourAdjustmentParameters
Tone Compression	ToneCompressionAdjustmentRasterNodeDefinition	createSetToneCompressionAdjustmentParameters
Tone Stretch	ToneStretchAdjustmentRasterNodeDefinition	createSetToneStretchAdjustmentParameters
Normals	NormalsAdjustmentRasterNodeDefinition	createSetNormalsAdjustmentParameters

22.22 HalftoneFilter

HalftoneFilter is a raster filter that applies a halftone screen effect. Two approaches:

22.22.1 Direct apply (via `/nodes`)

Applied via createSetHalftoneFilterParameters on a selection — no CompoundCommandBuilder.

```
'use strict';
const { Document } = require('/document.js');
const { DocumentCommand } = require('/commands.js');
const { HalftoneFilterParameters, HalftoneScreenType, HalftoneDotType } =
  require('/nodes.js');

const doc = Document.current;
const sel = doc.selection;

if (!sel.nodes.isEmpty) {
  const params = new HalftoneFilterParameters();
  params.screenType = HalftoneScreenType.Colour; // Colour, Grey, Line,
  Dot
  params.dotType = HalftoneDotType.Round; // Round, Square,
  Diamond, Line
  params.cellSize = 32; // cell size in pixels
  params.screenAngle = Math.PI / 6; // angle in radians
  params.contrast = 75; // contrast percentage
  params.greyComponentReplacement = 25; // GCR percentage
  params.underColourRemoval = 10; // UCR percentage
  doc.executeCommand(DocumentCommand.createSetHalftoneFilterParameters(sel,
    params));
  console.log('Halftone filter applied.');
```

① **Note**
HalftoneFilter is not a CompoundCommandBuilder operation. Execute it directly via executeCommand. Do not wrap it in a compound command — it silently fails.

HalftoneScreenType values: Colour, Grey, Line, Dot. HalftoneDotType values: Round, Square, Diamond, Line, Ellipse, Character, Custom.

22.22.2 Live filter node (via `/nodes`)

HalftoneFilterRasterNode is a live filter node with full HalftoneFilterParameters:

```
'use strict';
const { Document } = require('/document.js');
```



```

const { HalftoneFilterRasterNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder, InsertionMode } = require('/
commands.js');

const doc = Document.current;
const node = doc?.selection.nodes.filter(n => n.isRasterNode).first;

if (!node) { console.log('Select a raster node.')}
else {
  const def = HalftoneFilterRasterNodeDefinition.createDefault();
  const builder = AddChildNodesCommandBuilder.create();
  builder.addHalftoneFilterRasterNode(def);
  builder.setInsertionTarget(node);
  builder.setInsertionMode(InsertionMode.InsideAtTop);
  doc.executeCommand(builder.createCommand(true));

  // Modify parameters later
  const { HalftoneScreenType, HalftoneDotType } = require('/nodes.js');
  doc.executeCommand(DocumentCommand.createSetHalftoneFilterRasterNodeParameters(
    sel, // the halftone node
    HalftoneScreenType.Screen, // Screen, Dot, Line
    HalftoneDotType.Sine, // Sine, Round, Diamond, Line,
    Square, Ellipse
    0.5, // cell size
    45, // screen angle (degrees)
    0.1, // contrast
    0, // grey-component replacement
    0 // under-colour removal
  ));
  console.log('Live halftone filter node added.');
```

22.23 Key gotchas

createDefault(doc) requires the document argument for Levels and Curves. Both `LevelsAdjustmentRasterNodeDefinition.createDefault(doc)` and `CurvesAdjustmentRasterNodeDefinition.createDefault(doc)` require a document handle. Calling them with no argument throws.

Write back after mutating a parameters object. For Levels: read `def.parameters`, mutate the result, then set `def.parameters = params`. The object is not a live reference.

setChannelParameters for HSL takes an integer channel index, not an enum. Channel 0 = master, 1–6 = individual colour ranges. Using the wrong index is a silent no-op.

Hue values are in radians. Both `RecolourAdjustmentParameters.hue` and `HSLShiftAdjustmentChannelParameters.hueShift` are in radians. Full circle = 2π (≈ 6.283). To convert degrees: `degrees * Math.PI / 180`.

InsertionMode.InsideAtTop places the adjustment above the raster node's content. For a simple document where the raster node has no children, any insertion mode works. If the raster node already has children (e.g. masks), use `InsideAtTop` to ensure the adjustment is the topmost child.

22.24 Exercises

22.24.1 Exercise 17.1 – Batch brightness boost

Select multiple raster nodes. Add a Brightness/Contrast adjustment (brightness: 0.2, contrast: 0.1) inside each. Use `CompoundCommandBuilder` to make the whole operation a single undo step.

Note
Adjustment layers only affect raster pixel data. Adding a Curves layer above a vector shape has no visual effect – the shape must be rasterised first.

22.24.2 Exercise 17.2 — Levels dialog

Show a dialog with five `UnitValueEditor` controls (black, white, gamma, output black, output white). Apply the chosen values as a Levels adjustment on the selected raster node. Use live preview while the dialog is open.

22.24.3 Exercise 17.3 — HSL shift batch

Walk every raster node in the document. On each, add an HSL adjustment that boosts saturation by 0.15 and shifts hue by +0.1 radians. Log a count of adjusted layers.

22.24.4 Exercise 17.4 — Invert stack reporter

Walk every node in the document tree. Find any `InvertAdjustmentRasterNode` and log its parent node's name. Count the total.

22.25 SDK modules used in this lesson

- /nodes — All `*AdjustmentRasterNodeDefinition` and `*AdjustmentRasterNode` classes, `CurvesAdjustmentParameters`, `HSLShiftAdjustmentParameters`, `LevelsAdjustmentChannelParameters`, `ToneCompressionMethod`, `ToneStretchMethod`
- /commands — `DocumentCommand`, `AddChildNodesCommandBuilder`, `InsertionMode`
- /document — `Document`
- /selections — `Selection`
- /geometry — `Spline` (for Curves)
- /layereffects — `HalftoneFilter`

22.26 See also

- Lesson 12 — Layer Effects
- Lesson 15 — Pixel Processing
- examples/

22.27 Examples**22.27.1 Brightness batch adjuster**

```
'use strict';

// Example 16a — Brightness batch adjuster
// Lesson 16: Raster Adjustments
//
// Adds a Brightness/Contrast adjustment layer inside every selected
// raster node.
// All additions are wrapped in a CompoundCommand for single-step undo.
//
// Usage: select one or more raster/image nodes, then run.

const { Document } = require('/document.js');
const { BrightnessContrastAdjustmentRasterNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder, CompoundCommandBuilder, InsertionMode } = require('/commands.js');

const BRIGHTNESS = 0.2; // -1.0 to 1.0
const CONTRAST = 0.1; // -1.0 to 1.0

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const nodes = [...doc.selection.nodes].filter(n => n.isRasterNode || n.isImageNode);
  if (nodes.length === 0) {
    console.log('Select at least one raster or image node.')}
  else {
    const compound = CompoundCommandBuilder.create();
```

```

    for (const node of nodes) {
        const def =
BrightnessContrastAdjustmentRasterNodeDefinition.create({
            brightness: BRIGHTNESS,
            contrast:    CONTRAST,
            islinear:    false
        });

        const builder = AddChildNodesCommandBuilder.create();
        builder.addBrightnessContrastAdjustmentRasterNode(def);
        builder.setInsertionTarget(node);
        builder.setInsertionMode(InsertionMode.InsideAtTop);

        compound.addCommand(builder.createCommand(false));
    }

    doc.executeCommand(compound.createCommand());
    console.log('Brightness/Contrast adjustment added to ${nodes.length}
node(s).');
}
}

```

22.27.2 Levels dialog with live preview

```

'use strict';

// Example 16b – Levels dialog with live preview
// Lesson 16: Raster Adjustments
//
// Shows a dialog with controls for the five Levels master parameters.
// Applies a live preview while the dialog is open and commits on OK.
//
// Usage: select a raster or image node, then run.

const { Document } = require('/document.js');
const { DocumentCommand, AddChildNodesCommandBuilder, InsertionMode } =
require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');
const { LevelsAdjustmentRasterNodeDefinition } = require('/nodes.js');
const { Selection } = require('/selections.js');
const { UnitType } = require('/units.js');

const doc = Document.current;
const node = doc?.selection.nodes.first;

if (!doc || !node || (!node.isRasterNode && !node.isImageNode)) {
    console.log('Select a raster or image node first.');
} else {

    const dlg = Dialog.create('Levels');
    const col = dlg.addColumn();
    const grp = col.addGroup('Master channel');

    const blackEdit = grp.addUnitValueEditor('Black',
UnitType.Percentage, UnitType.Percentage, 0, 0, 100);
    const whiteEdit = grp.addUnitValueEditor('White',
UnitType.Percentage, UnitType.Percentage, 100, 0, 100);
    const gammaEdit = grp.addUnitValueEditor('Gamma',
UnitType.Percentage, UnitType.Percentage, 100, 1, 200);
    const outBEdit = grp.addUnitValueEditor('Output black',
UnitType.Percentage, UnitType.Percentage, 0, 0, 100);
    const outWEdit = grp.addUnitValueEditor('Output white',
UnitType.Percentage, UnitType.Percentage, 100, 0, 100);

    function buildParams() {
        return {
            blackLevel:    blackEdit.value / 100,
            whiteLevel:    whiteEdit.value / 100,

```

```
        gamma:          gammaEdit.value / 100,
        outputBlackLevel: outBEdit.value / 100,
        outputWhiteLevel: outWEdit.value / 100
    };
}

function applyPreview(preview) {
    const def = LevelsAdjustmentRasterNodeDefinition.createDefault(doc);
    const params = def.parameters;
    Object.assign(params.masterParameters, buildParams());
    def.parameters = params;

    const builder = AddChildNodesCommandBuilder.create();
    builder.addLevelsAdjustmentRasterNode(def);
    builder.setInsertionTarget(node);
    builder.setInsertionMode(InsertionMode.InsideAtTop);

    doc.executeCommand(builder.createCommand(false), preview);
}

dlg.onControlValueChangedHandler = () => applyPreview(true);
applyPreview(true);

while (dlg.runModal() == DialogResult.Ok) {
    applyPreview(false);
    break;
}
doc.clearPreviews();
console.log('Done.');
```

Lesson 18 — Align & Distribute

Aligning and distributing nodes is one of the most common layout operations — centring a title on a page, distributing icons evenly across a toolbar, or snapping objects to a grid. This lesson covers the `GroupTransform` system: a single entry point that handles every alignment and distribution variant through structured data objects rather than dozens of separate command factories.

Automating layout is where scripting delivers some of its most visible results. Instead of manually positioning each node with `createTransform` and calculating offsets yourself, the `applyGroupTransform` method handles the math — all you specify is what operation to perform and what reference frame to use. The system separates X and Y axes so you can align left in one dimension while distributing in the other, or leave an axis untouched.

By the end of this lesson you will be able to align nodes to edges and centres, distribute them evenly with consistent spacing, and anchor operations to the spread, page, or specific selection boundaries — all in a single command call.

23.1 What this lesson covers

- `doc.applyGroupTransform(xData, yData, selection)` — the single entry point for all align and distribute operations
- `GroupTransformData` — the data object that specifies what to do on one axis
- `GroupTransformType` — the operation (align to edge, centre, distribute, space)
- `GroupTransformAnchor` — the reference frame (spread, page, first selected, last selected, etc.)

- Practical patterns: align left, centre on page, distribute horizontally, space evenly

23.2 How it works

Every align and distribute operation in Affinity is a `GroupTransform` command. You describe what to do on the X axis and what to do on the Y axis separately using `GroupTransformData` objects, then pass both to `doc.applyGroupTransform`. Passing `null` for an axis means “do nothing on this axis”.

```
'use strict';
const { Document } = require('/document.js');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // Align selected nodes to their left edges, using the spread as
  // reference
  const xData = new GroupTransformData();
  xData.type = GroupTransformType.Min; // align to minimum
  // (left) edge
  xData.anchor = GroupTransformAnchor.FirstSelected; // relative to the
  // leftmost selected node

  doc.applyGroupTransform(xData, null, doc.selection);
}
```

23.3 GroupTransformType — what to do

Value	Constant	Meaning
0	None	No operation on this axis
1	Min	Align all to the minimum edge (left / top)
2	Mid	Align all centres
3	Max	Align all to the maximum edge (right / bottom)
4	DistributeMin	Distribute by minimum edges (equal gaps between leading edges)
5	DistributeMid	Distribute by centres (equal gaps between centres)
6	DistributeMax	Distribute by maximum edges (equal gaps between trailing edges)
7	Space	Space evenly (equal gaps between objects, not edges)

23.4 GroupTransformAnchor — reference frame

Value	Constant	Meaning
0	LogicalEdge	The logical bounding edge of the selection
1	Spread	The spread (canvas) boundaries
2	SpreadMargins	The spread's margin guides
3	FirstSelected	The first (bottom in panel) selected node's bounds
4	LastSelected	The last (top in panel) selected node's bounds
5	Page	The current page bounds
6	PageMargins	The current page margin guides

23.5 Common operations**23.5.1 Align left edges (to leftmost selected node)**

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Min;
xData.anchor = GroupTransformAnchor.FirstSelected;
doc.applyGroupTransform(xData, null, doc.selection);
```

23.5.2 Centre horizontally on the spread

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Mid;
xData.anchor = GroupTransformAnchor.Spread;
doc.applyGroupTransform(xData, null, doc.selection);
```

23.5.3 Centre both axes on the page

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Mid;
xData.anchor = GroupTransformAnchor.Page;

const yData = new GroupTransformData();
yData.type = GroupTransformType.Mid;
yData.anchor = GroupTransformAnchor.Page;

doc.applyGroupTransform(xData, yData, doc.selection);
```

23.5.4 Distribute horizontally with equal spacing between objects

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Space;
xData.anchor = GroupTransformAnchor.LogicalEdge;
doc.applyGroupTransform(xData, null, doc.selection);
```

23.5.5 Distribute centres vertically

```
// Snippet
const yData = new GroupTransformData();
yData.type = GroupTransformType.DistributeMid;
```

```
yData.anchor = GroupTransformAnchor.LogicalEdge;
doc.applyGroupTransform(null, yData, doc.selection);
```

23.5.6 Align right edges to the spread's right margin

```
// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Max;
xData.anchor = GroupTransformAnchor.SpreadMargins;
doc.applyGroupTransform(xData, null, doc.selection);
```

23.6 Full example — align & distribute dialog

```
'use strict';
const { Document } = require('/document.js');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands.js');
const { Dialog, DialogResult } = require('/dialog.js');

const doc = Document.current;
if (!doc || doc.selection.nodes.length < 2) {
  console.log('Select at least 2 nodes first.');
```

```
  } else {
    const dlg = Dialog.create('Align & Distribute');

    const col = dlg.addColumn();

    const alignGrp = col.addGroup('Align horizontal');
    const hAlignCombo = alignGrp.addComboBox('To', [
      'Left edges', 'Centres', 'Right edges'
    ], 0);

    const anchorGrp = col.addGroup('Reference');
    const anchorCombo = anchorGrp.addComboBox('Anchor', [
      'First selected', 'Last selected', 'Spread', 'Page'
    ], 0);

    const distGrp = col.addGroup('Distribute horizontal');
    const distCombo = distGrp.addComboBox('Method', [
      'None', 'Leading edges', 'Centres', 'Trailing edges', 'Space evenly'
    ], 0);

    if (dlg.runModal() !== DialogResult.Ok) { console.log('Cancelled.');
```

```
    } else {
      const anchorMap = [
        GroupTransformAnchor.FirstSelected,
        GroupTransformAnchor.LastSelected,
        GroupTransformAnchor.Spread,
        GroupTransformAnchor.Page
      ];
      const alignTypeMap = [
        GroupTransformType.Min,
        GroupTransformType.Mid,
        GroupTransformType.Max
      ];
      const distTypeMap = [
        GroupTransformType.None,
        GroupTransformType.DistributeMin,
        GroupTransformType.DistributeMid,
        GroupTransformType.DistributeMax,
        GroupTransformType.Space
      ];

      const anchor = anchorMap[anchorCombo.selectedIndex];
      const alignIdx = hAlignCombo.selectedIndex;
      const distIdx = distCombo.selectedIndex;

      // Apply align
      const xAlign = new GroupTransformData();
      xAlign.type = alignTypeMap[alignIdx];
```



```

xAlign.anchor = anchor;
doc.applyGroupTransform(xAlign, null, doc.selection);

// Apply distribute if selected
if (distIdx > 0) {
  const xDist = new GroupTransformData();
  xDist.type = distTypeMap[distIdx];
  xDist.anchor = GroupTransformAnchor.LogicalEdge;
  doc.applyGroupTransform(xDist, null, doc.selection);
}

console.log('Done.');
```

23.7 Combining align and distribute

Both axes are processed independently and can be combined in a single apply GroupTransform call. For example, snap all selected nodes to a grid: align to the left of the spread on X, distribute evenly on Y:

```

// Snippet
const xData = new GroupTransformData();
xData.type = GroupTransformType.Min;
xData.anchor = GroupTransformAnchor.Spread;

const yData = new GroupTransformData();
yData.type = GroupTransformType.Space;
yData.anchor = GroupTransformAnchor.LogicalEdge;

doc.applyGroupTransform(xData, yData, doc.selection);
```

23.8 Using DocumentCommand directly

doc.applyGroupTransform is the convenience form. The underlying command is:

```

// Snippet
const { DocumentCommand } = require('/commands.js');
const cmd = DocumentCommand.createGroupTransform(selection, xData, yData);
doc.executeCommand(cmd);
```

Use the command form when you need to batch the transform inside a CompoundCommandBuilder alongside other operations.

23.9 Key gotchas

Requires at least 2 nodes selected for distribute operations. With only one node selected, Space, DistributeMin, DistributeMid, and DistributeMax are silent no-ops. Min, Mid, and Max work on a single node (they move it to align against the anchor).

GroupTransformData uses new, not a static factory. Unlike most SDK types, GroupTransformData is constructed with new GroupTransformData(). Its type and anchor properties are then set directly.

node.baseBoxInterface.baseBox reflects LOCAL coordinates after transform. The visual positions of nodes change but baseBox values may appear unchanged because the actual translation is encoded in the node's transform Interface. Use node.getSpreadBaseBox(false) to read the post-transform spread-space bounding box if you need to verify results.

Anchor anchors that reference selected nodes (FirstSelected, LastSelected) use panel order, not selection order. First selected in panel terms is the bottommost node in the Layers panel.

Note
Operations are not previewable via the preview flag — GroupTransform commands commit immediately.

23.10 Exercises

23.10.1 Exercise 18.1 — Centre on canvas

Select any number of nodes. Centre them all horizontally and vertically on the spread in a single operation.

23.10.2 Exercise 18.2 — Distribute evenly

Select 5 or more shapes in a row. Distribute them with equal horizontal spacing between objects. The leftmost and rightmost should stay in place.

23.10.3 Exercise 18.3 — Grid snap

Write a script that aligns all selected nodes into a 3-column grid: the first three align horizontally, the next three at a fixed Y offset, and so on. Hint: iterate selected nodes in groups of 3, using FirstSelected/LastSelected anchors.

23.10.4 Exercise 18.4 — Align dialog

Build a dialog with dropdowns for horizontal align (None / Left / Centre / Right), vertical align (None / Top / Middle / Bottom), and reference (Spread / Page / Selection). Apply the chosen combination in a single command.

23.11 SDK modules used in this lesson

- /commands — DocumentCommand, GroupTransformData, GroupTransformType, GroupTransformAnchor, CompoundCommandBuilder
- /document — Document
- /dialog — Dialog, DialogResult
- /selections — Selection

23.12 See also

- Lesson 04 — Command Pattern
- Lesson 06 — Geometry
- examples/

23.13 Examples

23.13.1 Align selection

```
'use strict';

// Example 17a — Align selection
// Lesson 17: Align & Distribute
//
// Demonstrates the six most common alignment operations using
// GroupTransform.
// Runs through each in sequence with a 0 delay between them for
// illustration.
//
// Usage: select 3 or more nodes, then run.

const { Document } = require('/document.js');
const { GroupTransformData, GroupTransformType, GroupTransformAnchor } =
  require('/commands.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  const count = doc.selection.nodes.length;
  if (count < 2) {
    console.log('Select at least 2 nodes first.')}
  else {
    console.log(`Operating on ${count} nodes.`);

    function align(xType, xAnchor, yType, yAnchor, label) {
      const xData = xType !== null ? Object.assign(new
        GroupTransformData(), { type: xType, anchor: xAnchor }) : null;
```

```
    const yData = yType !== null ? Object.assign(new
GroupTransformData(), { type: yType, anchor: yAnchor }) : null;
    doc.applyGroupTransform(xData, yData, doc.selection);
    console.log(label, 'done.');
```

```
  }

  // Centre all nodes on the spread
  align(
    GroupTransformType.Mid, GroupTransformAnchor.Spread,
    GroupTransformType.Mid, GroupTransformAnchor.Spread,
    'Centre on spread'
  );

  // Now distribute them horizontally with equal spacing
  align(
    GroupTransformType.Space, GroupTransformAnchor.LogicalEdge,
    null, null,
    'Space evenly (horizontal)'
  );

  // Align tops to the topmost selected node
  align(
    null, null,
    GroupTransformType.Min, GroupTransformAnchor.FirstSelected,
    'Align tops'
  );

  console.log('All operations complete.');
```

```
  }
}
```


Phase 5 — Official Examples

PART



Phase 5 – Official Affinity SDK Examples

24

This phase is different from the others. Instead of lessons we wrote, it is a curated set of **example scripts shipped by the Affinity team inside the application** at `JSLib/examples` (within the Affinity app bundle). They are the closest thing the SDK has to official “best practice” references — small, focused scripts that each demonstrate one area of the API the way the people who built it intended it to be used.

Treat them as a companion to the lessons. Where a lesson explains a concept, these show it in a complete, runnable form. They are reproduced here verbatim, including their BSD 3-Clause license headers (© Canva Pty Ltd.), so you can trust that they reflect the real, current SDK.

Each example lives in its own folder alongside its source file. The table below links to each folder’s write-up.

24.1 How to run these examples

The official scripts are written as **modules**: each one defines a `main` function and ends with `module.exports.main = main` rather than calling `main()` itself. That is why they live in the app’s `/examples/` folder — they are meant to be loaded and invoked, not pasted-and-run as a standalone script.

To run one from the script editor:

1. Open the script in the editor (or paste its contents in).
2. Add a single line at the end: `main()` ;
3. Click **Run**.

Alternatively, if you run a script *from within the app’s own examples folder*, you can invoke another example as `require('/examples/<name>.js').main()` ;

— that is exactly what the comments in `bulgedPolyline.js`, `bulgeVersinePlayground.js`, and `tableFromJson.js` do.

① **Note**
Caveat — require('/examples/...') paths. A few scripts (`tableFromJson.js`, `bulgedPolyline.js`, `bulgeVersinePlayground.js`) call `require('/examples/...')` to load data or other examples. That absolute path only resolves when the file is sitting in the Affinity app's own JS `Lib/examples` directory. If you paste a copy from this tutorial, replace that `require` with the data you need (for `tableFromJson.js`, the dataset it loads is included here as `countries.json`) or run it directly from the app's examples folder.

① **Note**
Mixed require styles. The official examples are not entirely consistent with each other: some use `require('/document')` and some `require('/document.js')` some pull constants from `require('affinity:common')`. Our lessons always use the `.js` form (`require('/module.js')`), which is what `SDK-PRIMER.md` documents — follow that convention in your own scripts.

24.2 Examples at a glance

24.2.1 Layout & print

#	Example	Area	What it shows
01	<code>addGuides.js</code>	Layout	Adding margin guides around objects, with a live-preview dialog
02	<code>alignToPage.js</code>	Layout	Aligning the selection to the page (or page margins)
03	<code>adjustPageItems.js</code>	Layout	Per-page offset of items across a page range
04	<code>artboardGrid.js</code>	Layout	Generating a grid of artboards programmatically
20	<code>cropMarks.js</code>	Print	Crop/bleed marks for print output
21	<code>makeGrid.js</code>	Layout	Grid of shapes or text frames with spacing
22	<code>stepAndRepeat.js</code>	Layout	Duplicate selection with progressive offset, rotation and scale
23	<code>makeNumbersSequence.js</code>	Text	Number selected text objects with prefix/suffix/padding

24.2.2 Document & selection

#	Example	Area	What it shows
05	<code>setDocumentFormat.js</code>	Document	Changing the document's raster format
33	<code>countSelectedItems.js</code>	Selection	Count selected objects by

#	Example	Area	What it shows
			type, including nested children
34	selectObjects.js	Selection	Select objects by type/state through a dialog

24.2.3 Geometry & paths

#	Example	Area	What it shows
06	addPoints.js	Geometry	Subdividing curves at their midpoints
07	bulgedPolyline.js	Geometry	Building a polyline with bulge ToXY sections
08	bulgeVersinePlayground.js	Geometry	Interactive bulge/versine curve explorer
16	cornerEffects.js	Geometry	Corner effects on curves (rounded, bevel, inset, fancy)
17	divideLength.js	Geometry	Divide curve segments into equal lengths
18	pathEffects.js	Geometry	Punk, bloat, twirl and retract path effects
19	roundAnyCorner.js	Geometry	Round corners of selected curves with radius clamping

24.2.4 Text & story

#	Example	Area	What it shows
09	textOnCurvesAndShapes.js	Shapes / Text	The four curve- and shape-backed text node types
10	boldItalics.js	Text	Bolding and italicising the selected text
11	tableFromJson.js	Text / Data	Populating a TableTextNode from JSON (needs countries.json)

#	Example	Area	What it shows
26	breakFrame.js	Text	Break a text frame at the cursor position
27	opticalBackward.js	Text / Arrange	Move selection backward only past overlapping siblings
28	opticalForward.js	Text / Arrange	Move selection forward only past overlapping siblings
29	splitStory.js	Text	Split a story at the cursor into two text frames

24.2.5 Strokes & lines

#	Example	Area	What it shows
30	arrowheads.js	Stroke	Arrowheads on curve start/end with scale
31	strokesWeightDown.js	Stroke	Decrease stroke weight by 1pt across selection
32	strokesWeightUp.js	Stroke	Increase stroke weight by 1pt across selection

24.2.6 Transform & effects

#	Example	Area	What it shows
24	randomise.js	Transform	Randomise position, rotation, scale and opacity
25	swapObjects.js	Transform	Swap positions of two selected objects

24.2.7 Raster, colour & dialog

#	Example	Area	What it shows
12	bitmapWriter.js	Raster	Two ways to write pixels into a bitmap
13	meshBubble.js	Colour	Building a mesh-gradient “soap bubble” fill

#	Example	Area	What it shows
14	flexibleLayout.js	Dialog UI	Column stacks, colour pickers, and dynamic visibility
15	logToFile.js	Diagnostics	Writing structured log records to a file

24.3 Attribution & license

All scripts in this phase are © 2026 Canva Pty Ltd. and are reproduced under the **BSD 3-Clause License**. The original files ship inside the Affinity application bundle at `JSLib/examples`. They are included here unchanged (license headers intact) as a reference collection; they are not original tutorial content.

addGuides.js

25

Area: Layout

Adds margin guides around each selected object, or around the whole selection's bounding box. The dialog offers switches for each edge plus horizontal/vertical centre, an "around each object vs. entire selection" toggle, and optional absolute or "inflate" offsets.

This is the canonical example of a **live-preview dialog**: every control change calls `doc.executeCommand(cmd, true)` (the `true` flag renders a preview without committing), and `doc.clearPreviews()` restores state when nothing valid is selected. The guides themselves are created with `DocumentCommand.createAddGuide(isVertical, position)`, batched through a `CompoundCommandBuilder` when there is more than one. It also shows `node.getSpreadBaseBox()` and `unionRects` for computing bounds.

Source: `addGuides.js`

alignToPage.js

26

Area: Layout

A minimal wrapper around the alignment system. The dialog picks a horizontal and vertical alignment; the script maps those to `GroupTransformType` values (Min / Mid / Max / None) and calls `doc.applyGroupTransform(xData, yData)` — exactly the pattern covered in Lesson 17, here with the option to anchor to `GroupTransformAnchor.PageMargins` instead of the bare page.

Source: `alignToPage.js`

adjustPageItems.js

27

Area: Layout

Shifts items on a chosen page range by separate odd/even offsets. It walks `doc.spreads`, bins every non-container node onto its page via `spread.getPageIndexOfBox`, then builds one `DocumentCommand.createTransform` per page wrapped in a `Selection.create(doc, nodes, true)` and a `Transform.createTranslate`. Worth studying for the correct way to iterate pages and the `Collection.range(1, doc.pageCount).toArray()` helper from the `/collection` module.

Source: `adjustPageItems.js`

artboardGrid.js

28

Area: Layout

Builds a grid of artboards from dialog inputs (position, size, spacing, count). Each artboard is a `ShapeNodeDefinition` with `ShapeRectangle.create()` and `setBoundingRectangle`, added via `DocumentCommand.createAddArtboard`. The same live-preview pattern is used: the dialog's `onControlValueChanged` Handler re-runs the command with `preview = true` so you see the grid form as you type.

Source: `artboardGrid.js`

setDocumentFormat.js

29

Area: Document

The shortest example: a combo box of raster formats backed by `RasterFormat`, and on OK it simply assigns `doc.format = <format>`. A good illustration that many document properties are plain assignable fields, not commands.

Source: `setDocumentFormat.js`

addPoints.js

30

Area: Geometry

Subdivides every selected polycurve by inserting a new point at the midpoint of each bezier. It splits each segment with `bez.getParamAtLength(bez.length * 0.5)` and `bez.split(t)`, rebuilds the curve with `Curve Builder`, and writes it back through `DocumentCommand.createSetPolyCurve NodeCurves`. Demonstrates reading and rewriting curve geometry rather than just transforms.

Source: `addPoints.js`

bulgedPolyline.js

31

Area: Geometry

An interactive builder for a polyline where each section is defined by an endpoint and a **bulge** value, drawn with `CurveBuilder.bulgeToXY`. The dialog supports absolute or relative endpoints, computes the resulting chord, versine (sagitta), sweep angle, and radius live, and adds sections one at a time. A great reference for the bulge/versine curve model and for `PolyCurveNodeDefinition.create(...)`.

Source: `bulgedPolyline.js`

bulgeVersinePlayground.js 32

Area: Geometry

A sibling to the above that lets you flip between the **bulge** and **versine** parameterisations of the same arc and watch the computed geometry update. It exercises `bulgeToXY`/`versineToXY` and their `...RelativeXY` variants, and shows the relationship $\text{versine} = \text{bulge} * \text{chord} / 2$. The most thorough demonstration in the set of how Affinity represents arcs.

Source: `bulgeVersinePlayground.js`

textOnCurvesAndShapes.js 33

Area: Geometry / Text

Creates one of each curve/shape-backed text node type on a new A4 landscape document: `CurvePathTextNode` (text on a wavy bezier), `PolyCurveTextNode` (text inside a blob), `ShapePathTextNode` (text on a heart outline), and `ShapeTextNode` (text inside a star). It uses `StoryBuilder` with `applyGlyphDelta` / `applyParagraphDelta` to style the text and `Shape.create(ShapeType.Heart)` / `ShapeType.Star` for the containers. The single best tour of the text-on-path family.

Source: `textOnCurvesAndShapes.js`

boldItalics.js

34

Area: Text

Two lines of real work: it builds a composite `StoryDelta` from `StoryDelta.createWeight(FontWeight.Bold)` and `StoryDelta.createItalic(true)` and applies it with `doc.formatText`. The simplest possible illustration of the `StoryDelta` formatting model from Lesson 07.

Source: `boldItalics.js`

tableFromJson.js

35

Area: Text / Data

Builds a new A4 landscape document and fills a `TableTextNode` `Definition.create(box, { width, height })` from a dataset (the first 20 of the bundled `countries.json`, with a network fetch that falls back to the local copy). It writes cells via `doc.setText(text, selection)`, using `HardBreak` `Type.TableCell` to delimit them, and styles a bold 14pt header row with `Story Delta`. A dense but complete example combining networking, table nodes, and text formatting. Note its `require('/examples/countries.json')` — see the run caveat in the phase README.

Source: `tableFromJson.js`

bitmapWriter.js

36

Area: Raster

Demonstrates **two** ways to paint a bitmap: writing sparse pixels one at a time with `PixelReaderWriterRGBA8` (wrapped in `try/finally` with `w.dispose()`), and filling a contiguous block fast via a `PixelBuffer + Uint8Array` then `buffer.createCompatibleBitmap(true)`. The result is added as an `Image NodeDefinition` with `doc.addNode(nodeDef, null, undefined, preview)`. Directly relevant to Lesson 14, and it shows the speed trade-off between the two approaches.

Source: `bitmapWriter.js`

meshBubble.js

37

Area: Colour

The most advanced example. It creates a new document, drops in a circle, and fills it with a `ColourMesh / MeshFill` “soap bubble” — concentric colour bands, an S-shaped dark area, and highlight dots — computed per mesh node. It shows `ColourMesh.createFromColour`, `setNodePosition / setNodeColour / setCurveNodePosition` (tangent handles), and applying the fill via `FillDescriptor.create(MeshFill.create(mesh), ...)` and `DocumentCommand.createSetBrushFill(sel, desc)`. A working reference for the mesh-gradient fill type.

Source: `meshBubble.js`

flexibleLayout.js

38

Area: Dialog UI

A pure dialog-layout demonstration: `addColumnStack` for multi-column control arrangement, `addColourPicker` with RGB/CMYK toggle, `Horizontal Alignment` for button positioning, and — importantly — `dlg.setItemsVisibility(showIds, hideIds)` to swap control sets (the supported way to do dynamic visibility, not the no-op selector methods). Read this before building any non-trivial dialog.

Source: `flexibleLayout.js`

logToFile.js

39

Area: Diagnostics

Shows the `/logging` module: `LogFile.create(path, LogLevel, append)` followed by `start()`, `flush()`, and `stop()`. It demonstrates that the file's level filters records (`console.debug` is excluded at `Info`, `console.warn` after dropping to `Error`), and that the path must be allowed in the app's Scripting settings. Useful when an example misbehaves and `console.log` is not enough.

Source: `logToFile.js`

cornerEffects.js

40

Area: Geometry

Applies Rounded, Inverse Rounded, Bevel, Inset, or Fancy corner effects to selected curve points, with a pattern picker (all points, first/last, odd/even, and more). It decomposes each curve into anchors with incoming/outgoing handles, rebuilds the affected points with `CurveBuilder`, and writes the result back with `DocumentCommand.createSetCurves`. Previews live.

Source: `cornerEffects.js`

divideLength.js

41

Area: Geometry

Divides every segment of the selected curves into equal-length pieces while preserving handles. Each cubic is split by arc length with `getParamAtLength` and `split`, then rebuilt with `CurveBuilder` and applied via `DocumentCommand.createSetCurves`. Previews live.

Source: `divideLength.js`

pathEffects.js

42

Area: Geometry

Punk, Bloat, PunkBloat, BloatPunk, Twirl, AntiTwirl, and Retract All effects for selected curves, plus Make Rectangle and Make Ellipse replacements. Handles are pushed toward or away from the object centre, the curves are rebuilt with `CurveBuilder`, and the result is applied with `Document.Command.createSetCurves`.

Source: `pathEffects.js`

roundAnyCorner.js

43

Area: Geometry

Rounds the corners of the selected curves at a given radius. Smooth points are left alone and the radius is clamped to half the shorter adjacent segment. Curves are rebuilt with `CurveBuilder` and written back with `DocumentCommand.createSetCurves`. Previews live.

Source: `roundAnyCorner.js`

cropMarks.js

44

Area: Print

Adds crop marks and registration marks around each selected object or the whole selection. Marks are built from `Curve.createLineXY` and `Curve.createEllipse` with a `LineStyleDescriptor` in `SVG11.black`, combined with `union` `Rects`, and added as `PolyCurveNodeDefinition` nodes under a `ContainerNodeDefinition("Printers Marks")`.

Source: `cropMarks.js`

makeGrid.js

45

Area: Layout

Lays the selection out into a rows-by-columns grid with row and column gutters. Each source rectangle is mapped onto its cells with a `fitTransform`, creating rectangle, text, or picture frames via `AddChildNodesCommandBuilder`, or duplicating the source with `createTransform`. Optionally deletes the original.

Source: `makeGrid.js`

stepAndRepeat.js

46

Area: Layout

Duplicates the selection a number of times, each copy offset, rotated, and scaled a little more than the last. Copies are made with `DocumentCommand.createTransform(..., { duplicateNodes: true })` around the selection's bounds centre, batched through a `CompoundCommandBuilder`. Previews live.

Source: `stepAndRepeat.js`

makeNumbersSequence.js 47

Area: Text

Fills the selected text objects with a numbered sequence, with optional prefix, suffix, and zero padding, ordered by selection, left to right, or top to bottom. Each number is applied with `DocumentCommand.createSetText` on a `Selection` plus `TextSelection`. Previews live.

Source: `makeNumbersSequence.js`

randomise.js

48

Area: Transform

Randomises the position, rotation, scale, and opacity of each selected object within the chosen ranges. A seeded `splitmix32` PRNG drives the pattern and Reshuffle picks a new seed; transforms go through `createTransform` and opacity through `createSetOpacity`. Previews live.

Source: `randomise.js`

swapObjects.js

49

Area: Transform

Swaps the positions of exactly two selected objects, keeping their centres. Each centre comes from `getSpreadBaseBox().centre`, and the swap is two `createTransform` translate commands in a single `CompoundCommandBuilder`.

Source: `swapObjects.js`

breakFrame.js

50

Area: Text

Breaks a text flow at a frame: it unlinks everything after the flow head, relinks the kept frames, and moves the text between the resulting stories. Glyphs and attribute runs are captured with `getGlyph`, `getGlyphAttRunsFrom`, and `getParagraphAttRunsFrom`, then restored with `createInsertGlyph` and `createFormatText`.

Source: `breakFrame.js`

opticalBackward.js

51

Area: Arrange

Moves the selection backward in the z-order past only the nearest sibling whose bounds overlap it, so every step produces a visible change. Overlap is tested with `rectsIntersect` on `getSpreadVisibleBox` bounds merged by `unionRects`, and the move goes through `DocumentCommand.createMoveNodes(..., NodeMoveType.Before)`.

Source: `opticalBackward.js`

opticalForward.js

52

Area: Arrange

Moves the selection forward in the z-order past only the nearest sibling whose bounds overlap it, so every step produces a visible change. Overlap is tested with `rectsIntersect` on `getSpreadVisibleBox` bounds merged by `union` `Rects`, and the move goes through `DocumentCommand.createMoveNodes(..., NodeMoveType.After)`.

Source: `opticalForward.js`

splitStory.js

53

Area: Text

Splits a story at the cursor into two text frames. Frames after the split point are unlinked, the visible ranges are captured with their glyph and attribute runs, and the text is restored into the separate frames with `createInsertGlyph` and `createFormatText`, preserving formatting.

Source: `splitStory.js`

arrowheads.js

54

Area: Stroke

Applies arrowheads to the start and end of the selected curves' strokes with a scale factor. Each head is built with `ArrowHead.create(ArrowHeadStyle.*, { scaleX, scaleY })` and attached via `LineStyleDescriptor.cloneWithNewArrowHeads`, then applied with `createSetLineStyleDescriptor`. Previews live.

Source: arrowheads.js

strokesWeightDown.js

55

Area: Stroke

Decreases the stroke weight of every selected vector or text object by 1pt. The step is converted with `dpi / 72`, applied through a fresh `LineStyle` masked with `LineStyleMask.Weight`, and batched in a single `CompoundCommandBuilder`.

Source: `strokesWeightDown.js`

strokesWeightUp.js

56

Area: Stroke

Increases the stroke weight of every selected vector or text object by 1pt. The step is converted with `dpi / 72`, applied through a fresh `LineStyle` masked with `LineStyleMask.Weight`, and batched in a single `CompoundCommandBuilder`.

Source: `strokesWeightUp.js`

countSelectedItems.js

57

Area: Selection

Counts the selected objects and reports a breakdown by type, including nested children. Nodes are classified with the `is*` type guards (images, placed documents, picture frames, shapes, curves, text, groups, layers), and the result is shown with `alert`.

Source: `countSelectedItems.js`

selectObjects.js

58

Area: Selection

Selects objects by type through a dialog with per-type switches (rectangles, ellipses, other shapes via ShapeType, curves, text frames, other text, groups, picture frames, images, placed documents). Matching runs over the current spread or the whole document, and the result is assigned with `doc.selection = Selection.create(doc, nodes, true)`.

Source: selectObjects.js

Affinity SDK – Quick Reference Cheat Sheet

Keep this file open while coding. One page covering the patterns you'll use every session.

59.1 Boilerplate – every script starts here

```
'use strict';

const { Document } = require('/document.js');

const doc = Document.current;
if (!doc) { console.log('No document open.')}
else {
  // your code here
}
```

59.2 Imports by task

```
// Document access
const { Document, FileExportOptions, FileExportArea, DocumentPromises } =
require('/document.js');
const { app } = require('/application.js');

// Making changes
const { DocumentCommand, CompoundCommandBuilder,
AddChildNodesCommandBuilder } = require('/commands.js');

// Selections
const { Selection, TextSelection } = require('/selections.js');

// Colours and fills
const { RGBA8, Colour } = require('/colours.js');
const { FillDescriptor, SolidFill } = require('/fills.js');
```

```
// Strokes
const { LineStyle, LineStyleMask } = require('/linestyle.js');

// Geometry
const { Transform, CurveBuilder, PolyCurve } = require('/geometry.js');

// Text
const { StoryBuilder } = require('/storybuilder.js');
const { StoryDelta } = require('/storydelta.js');
const { GlyphAttStringType } = require('/glyphatts.js');

// Dialogs
const { Dialog, DialogResult, DialogSpatialAnchor, SpatialAnchor } =
require('/dialog.js');

// Units
const { UnitType } = require('/units.js');

// Collections
const { Collection } = require('/collection.js');

// Node tree
const Nodes = require('/nodes.js');

// Files
const { File } = require('/fs.js');
const { Buffer } = require('/buffer.js');
const { HttpRequest, RequestMethod } = require('/network.js');

// System / logging / config
const { OS } = require('/os.js');
const { Environment } = require('/environment.js');
const { LogFile, LogLevel } = require('/logging.js');
const { ConfigurationItem } = require('/configuration.js');
```

59.3 Programmatic selection

```
const { Selection } = require('/selections.js');
const { DocumentCommand } = require('/commands.js');

// Select a single node
const sel = Selection.create(doc, targetNode);
doc.executeCommand(DocumentCommand.createSetSelection(sel));

// Select multiple nodes
const multiSel = Selection.create(doc, node1, node2, node3);
doc.executeCommand(DocumentCommand.createSetSelection(multiSel));
```

59.4 The command loop

```
const sel = doc.selection;
const compound = CompoundCommandBuilder.create();

for (const node of sel.nodes) {
  const nodeSel = Selection.create(doc, node);
  compound.addCommand( DocumentCommand.createXxx(nodeSel, ...) );
}

doc.executeCommand(compound.createCommand());
```

59.5 Colours

```
const red = RGBA8(220, 50, 50, 255); // r, g, b, a — all 0-255
const clear = RGBA8(0, 0, 0, 0);
const fill = FillDescriptor.createSolid(red);
const noFill = FillDescriptor.createNone();
```

59.6 Apply fill / stroke / opacity

```
// Via DocumentCommand — selection first, then fill
doc.executeCommand(DocumentCommand.createSetBrushFill(sel, fill)); //
interior
```

```

doc.executeCommand(DocumentCommand.createSetPenFill(sel, fill));    //
stroke

// Via doc convenience methods – accept Colour, SolidFill, or
FillDescriptor
doc.setBrushFillDescriptor(colour, sel);
doc.setPenFillDescriptor(fill, sel);

// Opacity and blend mode
doc.setOpacity(0.5, sel);           // layer opacity 0.0–1.0
doc.setBrushFillOpacity(0.7, sel); // fill opacity
doc.setLineFillOpacity(0.5, sel);  // stroke opacity

// Line style shorthands
doc.setLineWeight(4, sel);          // pixels
doc.setLineWeightPts(2, sel);       // points
doc.setLineCap(cap, sel);
doc.setLineJoin(join, sel);

// Common doc convenience methods
doc.undo(); doc.redo();
doc.selectAll();
doc.deleteSelection(sel);
doc.showSelection(sel); doc.hideSelection(sel);
doc.lockSelection(sel); doc.unlockSelection(sel);
doc.convertToCurves(sel);
doc.rasteriseObjects(sel);
doc.flatten(); doc.mergeVisible();

```

59.7 Text replacement

```

// end - 1 excludes the hidden story terminator
doc.executeCommand(DocumentCommand.createSetCurrentSpread(doc.currentSpread));

const si    = node.storyInterface;
const range = si.storyRange;
const textSel = TextSelection.create([ { begin: range.begin, end:
range.end - 1 } ]);
const selection = Selection.create(doc, node); // NOT
Selection.createEmpty()
selection.addSubSelectionForNode(node, textSel);
doc.executeCommand(DocumentCommand.createSetText(selection, 'New text'));

```

59.8 Dialog

```

const dlg = Dialog.create('My Tool');
const col = dlg.addColumn();
const grp = col.addGroup('Options');

// Attach controls to dlg for clean access in helper functions
dlg.enable = grp.addCheckBox('Enable', false);
dlg.format = grp.addComboBox('Format', ['PNG', 'PDF'], 0);
dlg.colour = grp.addColourPicker('Colour');
dlg.size = grp.addUnitValueEditor('Size', UnitType.Pixel,
UnitType.Millimetre, 100); // store px, display mm
dlg.count = grp.addUnitValueEditor('Count', UnitType.Number,
UnitType.Number, 6, 1, 100); // unitless integer
dlg.anchor = grp.addSpatialAnchor('Anchor', SpatialAnchor.None); //
spatial anchor picker

// Reactive – per-control (no argument)
dlg.enable.onValueChangedHandler = () => { dlg.size.isEnabled =
dlg.enable.value; };

// Reactive – dialog-level (receives changed control)
dlg.onControlValueChangedHandler = (ctrl) => { /* ctrl === dlg.enable etc.
*/ };

// Simple run
const result = dlg.runModal();

```

```

if (result == DialogResult.Ok) { // note: Ok not OK (title-case)
  console.log(dlg.enable.value, dlg.format.selectedIndex,
    dlg.colour.value);
}

// Live preview loop
dlg.onControlValueChangedHandler = () => doc.executeCommand(cmd,
true); // true = preview
while (dlg.runModal() == DialogResult.Ok) {
  if (!valid) { alert('Fix this.');
```

59.9 Export

```

const desktop = app.userDesktopPath;
const opts = FileExportOptions.createWithPresetName('PNG');
const area = FileExportArea.createForWholeDocument();
const records = doc.export(`${desktop}/output.png`, opts, area, null);
console.log(records.all[0]?.isSuccess ? 'OK' : 'FAIL');
```

59.10 Layer effects (shadow example)

```

// sel must be Selection.create(doc, node) – not doc.selection
// Effect commands must be executed individually – CompoundCommandBuilder
// is a silent no-op for effects
const idx = 0;

doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectColour(sel,
  RGBA8(0,0,0,180), idx, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOffsetAngle(sel,
  6, 315, idx, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectRadius(sel,
  idx, 8, true));
doc.executeCommand(DocumentCommand.createSetOuterShadowLayerEffectOpacity(sel,
  0.7, idx, true)); // opacity first, then index
// true = enableIfDisabled – adds the effect if not already present
```

59.10.1 Outline with gradient/pattern fill

```

// Outline accepts a FillDescriptor (not just solid colour)
const { Gradient } = require('/colours.js');
const { GradientFill, FillDescriptor } = require('/fills.js');
const gradient = Gradient.create([
  { colour: RGBA8(255, 0, 0, 255), position: 0 },
  { colour: RGBA8(0, 0, 255, 255), position: 1 },
]);
const fill = FillDescriptor.create(GradientFill.create(gradient, 0),
true);
doc.executeCommand(DocumentCommand.createSetOutlineLayerEffectFill(sel,
fill, 0, true));
```

59.11 Async pipeline

```

async function main() {
  const docP = await DocumentPromises.getCurrent();
  if (!docP) { console.log('No doc.');
```

```

}
main(); // always invoke the async entry point

```

59.12 Additional APIs

59.12.1 Pixel read/write

```

const { PixelReaderWriterRGBA8 } = require('/pixelaccessor.js');
const bitmap = node.rasterInterface.createCompatibleBitmap(true);
const rw = PixelReaderWriterRGBA8.create(bitmap);
const px = rw.readPixel(x, y); // { r, g, b, alpha } - NOT
{ r, g, b, a }
rw.writePixel(x, y, { r, g, b, alpha }); // key is 'alpha' on write too
// // using 'a:' silently writes
255 - no error!
doc.executeCommand(DocumentCommand.createReplaceBitmap(sel, bitmap));

```

59.12.2 Font enumeration

```

const { Font, FontFamily } = require('/fonts.js');
const families = FontFamily.all; // all installed families
const fonts = Font.all; // all individual fonts

// Get a specific font via FontFamily
const arial = FontFamily.all.find(f => f.name === 'Arial');
const f = arial?.fontCount > 0 ? arial.getFont(0) : null;
if (f) console.log(f.isSerif, f.isMonospaced, f.postscriptName);

```

59.12.3 Custom node tags

```

const { PredefinedTagKey } = require('/taginterface.js');
// Write a tag (persists in file)
DocumentCommand.createSetTagValueForKey(sel, 'my-key', 'value');
DocumentCommand.createSetTagValueForPredefinedKey(sel,
PredefinedTagKey.ScriptLabel, 'label');
// Read tags back via tagInterface
if (node.tagInterface) {
  console.log(node.tagInterface.hasKey('my-key'));
  console.log(node.tagInterface.getValueForKey('my-key'));
}

```

59.12.4 Opacity commands

```

DocumentCommand.createSetOpacity(sel, 0.5) // layer opacity
DocumentCommand.createSetBrushFillOpacity(sel, 0.8) // fill opacity
DocumentCommand.createSetLineFillOpacity(sel, 0.6) // stroke opacity

```

59.12.5 ContainerNode (named layer group)

```

const { ContainerNodeDefinition } = require('/nodes.js');
const { AddChildNodesCommandBuilder } = require('/commands.js');
const def = ContainerNodeDefinition.create('My Group'); // named
container
const builder = AddChildNodesCommandBuilder.create();
builder.addContainerNode(def);
doc.executeCommand(builder.createCommand(true));

doc.addNode(def); // shorthand for single-node add (no builder needed)

```

59.12.6 Node convenience methods

```

node.userDescription = 'Renamed'; // set layer name directly
node.lock(); // make non-editable
node.unlock();
const copy = node.duplicate(); // returns new node (lands on top)
node.delete(); // remove node
if (node.isColouredLogicalNode) {
  console.log('Colour label:', node.layerColour); // read-only
}

```

59.12.7 Application settings

```

const { app } = require('/application.js');
app.settings.undoLimit // read-only

```

```
app.settings.loadPSDWithEditableText // read/write
app.settings.loadPSDWithEditableText = true
```

59.12.8 Document save and close

```
const { PackageResourcesPolicy } = require('/document.js');
doc.save(); // save in-place
doc.saveAs('/path/to/file.af');
doc.saveAsPackage('/path/pkg', PackageResourcesPolicy.CopyLinks);
// doc.close(); // NOT_IMPLEMENTED - always throws
```

59.12.9 Macro commands

```
DocumentCommand.createStartRecordingMacro()
DocumentCommand.createStopRecordingMacro()
DocumentCommand.createReplayMacro()
DocumentCommand.createImportMacro(desktop + '/macro.afmacro')
```

59.12.10 Raster selection commands

```
DocumentCommand.createRasterSelectAll()
DocumentCommand.createRasterDeselect()
DocumentCommand.createRasterInvertSelection()

// Renamed from rasterSelectPolygon
DocumentCommand.createSetRasterSelectionFromPolygon(polygon, operation,
antialias, feather)

// Selection from object bounds
DocumentCommand.createSetRasterSelectionFromObject(sel, operation,
antialias, feather)

// Refine existing raster selection
DocumentCommand.createGrowShrinkRasterSelection(10, false) // grow by
10px (circular kernel)
DocumentCommand.createFeatherRasterSelection(5) // soften
edge by 5px
DocumentCommand.createSmoothRasterSelection(3) // smooth
boundary by 3px

const { RasterSelectionOutlineAlignment } = require('/commands.js');
DocumentCommand.createOutlineRasterSelection(4,
RasterSelectionOutlineAlignment.Centre, false) // outline at
radius 4px

// Doc convenience methods - same signatures without create*
doc.growShrinkRasterSelection(8, true); // grow, circular kernel
doc.featherRasterSelection(5);
doc.smoothRasterSelection(3);
doc.outlineRasterSelection(4,
RasterSelectionOutlineAlignment.Inside, false);
```

59.12.11 Drawing scale

```
const { DrawingScale } = require('/drawingscale.js');
const scale = DrawingScale.createFromIntegers(1, 100, true); // 1:100
```

59.13 `createSetEditable` — argument order

```
// Selection is first argument, editable flag is second
DocumentCommand.createSetEditable(selection, false) // lock
DocumentCommand.createSetEditable(selection, true) // unlock
```

59.14 Document convenience shorthands

```
// Text operations
doc.setText('Hello', sel); // replace text in selection
doc.formatText(delta, sel); // apply formatting to selection
doc.insertGlyph(0x2665, sel); // insert ♥ glyph

// Undo/redo
doc.undo();
doc.redo();
```

```
// Stroke enhancements
doc.setDashPhase(0, sel);           // dash phase
doc.setBalancedDashes(true, sel);   // balance dashes at corners

// Dash pattern variants
doc.setDashPattern([8, 4], sel);     // regular dash
doc.setDashPattern([8, 4, 2, 4], sel); // dash-dot-dot
```

59.15 Document line style shorthands

```
// These doc.setXxx() shorthands are simpler than building commands
manually
doc.setLineWeightPts(2, sel);        // set weight in points (auto-
converts to px)
doc.setLineCap(2, sel);              // 0=Butt, 1=Square, 2=Round
doc.setLineJoin(2, sel);             // 0/1=Miter, 2=Round, 3=Bevel
doc.setLineType(LineType.Dash, sel); // set line type
doc.setDashPattern([8, 4], sel);     // set dash pattern
doc.setStrokeAlignment(StrokeAlignment.Centre, sel); // stroke alignment
```

59.16 StoryDelta convenience factories

```
const { StoryDelta } = require('/storydelta.js');
const { FontWeight } = require('/fonts.js');

// Simple bold + italic
const bold      = StoryDelta.createWeight(FontWeight.Bold);
const italic    = StoryDelta.createItalic(true);
const combined  = StoryDelta.createComposite([bold, italic]);
doc.formatText(combined); // applies to current selection – no Selection
object needed
```

59.17 Geometry utilities

```
const { unionRects, TransformBuilder, Polygon, Curve, CubicBezier, Point }
= require('/geometry.js');
const combined = unionRects(rect1, rect2); // merge two bounding boxes

// Polygon convenience factories
const tri = Polygon.createTriangle(Point.zero, new Point(100, 0), new
Point(50, 80));
const dia = Polygon.createDiamond(new Point(0, 0), new Point(100, 100));
const loz = Polygon.createLozenge(new Point(0, 50), new Point(100, 50),
20);

// Precision unit circle (for measuring/fitting)
const circle = Curve.createPrecisionUnitCircle();

// Transform properties
const tx = node.baseToSpreadTransform;
console.log(tx.xAxis, tx.yAxis, tx.origin);
const inv = tx.inverted;

// Chainable transform methods
const moved = Transform.translationMatrix(100, 0);           //
translate
const scaled = Transform.scaleMatrix(2, 2, point);           // scale
about point
const added = moved.translate(50, 0).scale(1.5, 1.5, origin); //
chainable

// around() replaces about()
const rot = Transform.rotationMatrix(45, Transform.Degrees); // 45deg
about origin
const rot2 = Transform.rotationMatrix(45, Transform.Degrees, center); //
about center

// Static multiply, decompose
const combined2 = Transform.multiply(txA, txB);               // combine
transforms
```

```

const [sc, tr, rot3] = Transform.decompose(combined2);      //
decompose

// TransformBuilder fluent builder
const builder = TransformBuilder.create();
const built = builder
  .setXAxis(1, 0)
  .setYAxis(0, 1)
  .setOrigin(100, 100)
  .build();

// CubicBezier prototype methods
const bez = new CubicBezier();
const pt = bez.evaluate(0.5);          // point at t=0.5
const { left, right } = bez.split(0.5); // split into two beziers
const tangent = bez.getTangent(0.5);   // tangent vector
const normal = bez.getNormal(0.5);     // normal vector
const curv = bez.getCurvature(0.5);   // curvature value
const closest = bez.getClosestPoint(pt); // nearest point on curve
const len = bez.length;                // total arc length

```

59.18 HalftoneFilter

```

const { HalftoneFilter } = require('/layereffects.js');

// Direct apply (no CompoundCommandBuilder — silent no-op for layer
effects)
doc.executeCommand(DocumentCommand.createSetHalftoneParameters(
  sel, HalftoneFilter.Screen, HalftoneFilter.Sine, 0.5, 45, 0.1
));

// Or use live HalftoneFilterRasterNode
const { HalftoneFilterRasterNodeDefinition, HalftoneScreenType,
HalftoneDotType } = require('/nodes.js');
const def = HalftoneFilterRasterNodeDefinition.createDefault();

```

59.19 DialogButton and DialogColumnStack

```

// Button — use addButton() on a group
const btn = grp.addButton('Click Me');
btn.alignment = HorizontalAlignment.Left;
btn.onClickHandler = () => { console.log('Clicked!'); };

// Column stack layout
const stack = grp.addColumnStack();
const leftCol = stack.addColumn();
const rightCol = stack.addColumn();

// Detect if dialog is running modal (property on Dialog instance)
if (dlg.isRunningModal) { console.log('Dialog is modal.');
```

```

// Toggle item visibility (method on Dialog instance)
dlg.setItemsVisibility([label.itemID], []); // show label
dlg.setItemsVisibility([], [label.itemID]); // hide label

// Fill/stroke editor
const { DialogFillEditor, DialogStrokeEditor } = require('/dialog.js');
const fillEditor = DialogFillEditor.create();
const strokeEditor = DialogStrokeEditor.create();

// Font picker
dlg.font = grp.addFontPicker('Font'); // DialogFontPicker
- .font, .fontFamily, .fontCollection

// Find a control by ID (returns typed DialogControl or null)
const ctrl = dlg.findControl('my-control-id');
if (ctrl) ctrl.isEnabled = false;

```


59.20 Fills – selection first

```
const desc = FillDescriptor.create(SolidFill, { r: 255, g: 0, b: 0, alpha:
255 });
DocumentCommand.createSetBrushFill(sel, desc); // selection first, then
fill
DocumentCommand.createSetPenFill(sel, desc);

// Hatch fill
const hatchCmd = DocumentCommand.createSetHatchFillAttributes(sel, /*
builder */);
```

59.21 TableTextNode

```
const { TableTextNodeDefinition } = require('/nodes.js');
const def = TableTextNodeDefinition.create({ x: 0, y: 0, width: 300,
height: 200 }, { width: 3, height: 3 });
doc.addNode(def);
```

59.21.1 TableSubSelection – read table cell/edge state

```
const { TableSubSelection } = require('/selections.js');
// After selecting a TableTextNode, check for table sub-selection
const item = doc.selection.items[0];
const sub = item.getSubSelectionOfType(SubSelectionType.Table);
if (sub) {
  const ts = TableSubSelection.fromSubSelection(sub);
  console.log(ts.anchor, ts.caret, ts.isRectangle);
  ts.enumerateCells((row, col, rect) => { /* ... */ });
}
```

59.21.2 NodeChildren.all – recursive descendants

```
// node.children.all returns ALL descendants (not just direct children)
for (const child of node.children.all) {
  if (child.isRasterNode) console.log('Found raster:',
child.userDescription);
}
```

59.22 GlyphAtts / ParagraphAtts

```
const { GlyphAtts, GlyphAttStringType, GlyphAttDoubleType } = require('/
glyphatts.js');
const { ParagraphAtts } = require('/paragraphatts.js');

const ga = GlyphAtts.create();
ga.setStringValue(GlyphAttStringType.StyleName, 'Heading');
ga.setDoubleValue(GlyphAttDoubleType.Height, 24);

const pa = ParagraphAtts.create();
pa.isPreventWidows = true;
```

59.23 Attribute runs on Story

```
for (const run of story.glyphAttRuns) {
  console.log(run.begin, run.end, run.glyphAtts);
}
// Also: story.paragraphAttRuns – yields {paragraphAtts, begin, end}
// Also: story.attRuns – yields {glyphAtts, paragraphAtts, begin, end}
```

59.24 Fluent node-definition builders

```
const def = PolyCurveNodeDefinition.createDefault()
  .setUserDescription('My Shape')
  .setTransform(xf);
```

59.25 Colour → FillDescriptor auto-convert

```
// Fill/line methods accept Colour directly
def.addBrushFillDescriptor(RGBA8(200, 80, 40, 255));
```

59.26 `node.spreadBaseBox`

```
const box = node.spreadBaseBox; // equivalent to getSpreadBaseBox(true)
```

59.27 Spread / artboard document properties

```
const { ArtboardDocumentProperties } = require('/documentproperties.js');
const props =
  ArtboardDocumentProperties.create().setUseMargin(true).setMargin(margin);
doc.setArtboardDocumentProperties(artboard, props);
doc.setArtboardSizeWithAnchor(artboard, 400, 300, SpatialAnchor.Centre);
```

59.28 Boolean operations & shape division

```
// Combine/divide overlapping vector nodes (needs 2+ selected)
doc.boolOpUnion(); // or
DocumentCommand.createBoolOpUnion(doc.selection)
doc.boolOpSubtract(); // top shape cuts the shape(s) below
doc.boolOpIntersect(); // keep only the overlap
doc.boolOpXor(); // keep everything except the overlap
doc.divideShapes(); // split into pieces along intersections
```

59.29 Text frame threading

```
// Thread text from one frame into another (no doc convenience method)
doc.executeCommand(DocumentCommand.createLinkTextFrame(srcFrame,
  destFrame));
doc.executeCommand(DocumentCommand.createUnlinkTextFrame(srcFrame));
// srcFrame.textFrameInterface.isMultiFrameTextFlow → true when threaded
```

59.30 Raster auto-fixes & fill

```
doc.rasterAutoLevels(); // one-click tonal correction on selection
doc.rasterAutoContrast();
doc.rasterAutoColours();
doc.rasterAutoWhiteBalance();
doc.rasterFill(); // fill raster selection with current
colour
```

59.31 System / logging / config

```
OS.platform; // "Darwin"
Environment.permissions; // { fileSystem, network, genAI }
Environment.fileSystemRoots; // ["/Users/you/Desktop"]

const log = LogFile.create(`${app.userDesktopPath}/run.log`,
  LogLevel.Info, true);
log.start(); log.flush(); log.stop();

const cfg = ConfigurationItem.createFromJsonString(`{"width": 1200}`);
cfg.value.width; // 1200
```

59.32 Per-node export config, glyphs, units & hatch

```
// Per-node export settings – config must already exist (set once via UI
Export persona)
const { ExportScale, ExportSize } = require('/exportconfig.js');
const cfg = node.exportableInterface.exportConfig;
cfg.formats[0].appendSize(ExportSize.createWithExportScale(ExportScale.createWithMultiplie
doc.executeCommand(DocumentCommand.createSetExportConfig(sel, cfg));

// Glyphs – typed character model (code point, not a string)
const { CharGlyph } = require('/glyphs.js');
const g = CharGlyph.create(0x2665);
// story.glyphs / story.charGlyphs / story.hardBreakGlyphs – pre-filtered
collections

// DPI-based unit conversion
const pxPerPt = doc.unitValueConverter.getConversionFactor(UnitType.Point,
UnitType.Pixel);

// Hatch pattern construction
const { HatchPattern, HatchLine } = require('/hatch.js');
const pat = HatchPattern.createEmpty();
pat.appendHatchLine(HatchLine.create({ x: 0, y: 0 }, Math.PI / 4, 8, [4,
2]));
```

59.33 Critical rules

Rule	Why
<code>require('/module.js')</code> not <code>./module</code>	SDK module registry, not file system
<code>doc.title</code> not <code>doc.name</code>	<code>.name</code> is undefined
<code>RGBA8()</code> not <code>Colour.fromRGBA()</code>	<code>.fromRGBA</code> doesn't exist
<code>doc.executeCommand(cmd, true)</code> for preview, <code>doc.clearPreviews()</code> after loop	Without <code>clearPreviews()</code> , cancelled previews stay on canvas permanently
Previews don't stack — rebuild in one command each update	SDK replaces previous preview; layering commands doesn't accumulate
<code>result == DialogResult.Ok</code>	Use title-case <code>Ok</code> not <code>OK</code> loose equality <code>==</code> works correctly
<code>range.end - 1</code> for text selections	Excludes the hidden story terminator
<code>Selection.create(doc, node)</code> for text	<code>Selection.createEmpty()</code> causes silent failure
Read <code>doc.selection</code> BEFORE set <code>CurrentSpread</code>	Spread change clears selection
Compound commands for multi-step changes	One undo step, not N separate entries
Desktop + granted folders for file access	Other paths → <code>PERMISSION_DENIED</code>
No <code>toLocaleString()</code>	Not supported — use plain number or <code>String(n)</code>
Check <code>String(ri.format) === 'RGBA8'</code> before <code>PixelReaderWriter RGBA8.create()</code>	Format mismatch throws <code>INVALID_ARGS</code> — format is an enum object, use <code>String()</code> to compare
<code>bitmap.format</code> returns a plain integer not a name	Use <code>RasterFormat.keys/values</code> to look up — see lesson 15 loading example
<code>res.response.statusCode == 200</code> not <code>res.statusCode</code>	<code>statusCode</code> is on <code>res.response</code> , not <code>res</code> directly; body is <code>res.response.content</code>
<code>create*Filter</code> not <code>createSet*FilterParameters</code> to apply a new filter	<code>createSet*</code> is a silent no-op if no filter exists yet
No <code>CompoundCommandBuilder</code> for effect commands	Silent no-op — execute each effect command individually
<code>setTimeout(delay, callback)</code> not <code>setTimeout(callback, delay)</code>	SDK order is opposite to Node/browser
<code>main()</code> call at bottom of async scripts	Async functions don't auto-run
<code>Transform.around()</code> — use <code>around()</code> , not <code>about()</code>	<code>about()</code> does not exist

Rule	Why
<code>createSetRasterSelectionFromPolygon()</code> for polygon selections	Prefer this over <code>rasterSelectPolygon()</code>
<code>require('/vectorbrush')</code> fails — that module does not exist	Use <code>/pathbrush</code> for vector brushes, <code>/rasterbrush</code> for pixel brushes
<code>ExportConfig</code> has no constructor	Obtain via <code>node.exportableInterface.exportConfig</code> — set once via the UI Export persona
<code>createSetEditable(sel, bool)</code> — selection first	Selection is the first argument
<code>createSetBrushFill(sel, fill)</code> — selection first	Selection is the first argument, then the fill
<code>dialog.setIsEnabledBySelector()</code> / <code>setIsVisibleBySelector()</code> are silent no-ops	Use <code>setItemsVisibility()</code> instead
<code>TableTextNodeDefinition.create(rect, {width, height})</code> not <code>create(rect, width, height)</code>	Grid dimensions are an object
<code>formatText</code> across cell separators only partially applies	Format each cell individually in a loop
Dialog values are read-only after build phase	Must set values before <code>runModal()</code>
<code>setCurrentSpread</code> clears selection	Read <code>doc.selection</code> first
<code>node.spreadBaseBox</code> equivalent to <code>getSpreadBaseBox(true)</code>	Convenience property, defaults include <code>Clips</code> to <code>true</code>
<code>RasterSelectionOutlineAlignment</code> — Centre, Inside, Outside	Required for <code>createOutlineRasterSelection</code>
Fill/line methods on definitions accept <code>Colour</code> directly	A solid fill is created automatically
Node-definition setters are chainable	<code>.setDescription()</code> , <code>.setTransform()</code> , etc. return this
<code>DialogControl.setEnabled()</code> works correctly	Updates <code>isEnabled</code> properly
Handlers fire on user interaction only	Programmatic value changes do not trigger <code>onValueChangedHandler</code>
Geometry structs have proper wrapper types	<code>Rectangle</code> , <code>Point</code> , <code>Vector</code> , etc. are documented types, not anonymous objects

Affinity SDK — C++ Exposure Map

What this document is: A complete map of how the Affinity C++ engine is exposed to JavaScript, derived by analysing every SDK source file. This answers: what can you touch, what is hidden, and what signals clearly indicate a boundary between JS and C++.

60.1 How the binding works

The SDK is a three-layer sandwich:

Your JavaScript script

↓

JS wrapper `classes` (the `.js` files you `require()`)

↓

Native C++ `bindings` (`affinity:dom`, `affinity:commands`, `affinity:common`, `affinity:story`, `affinity:ui`)

↓

Affinity C++ `engine` (NOT accessible — no escape hatch)

The native bindings are loaded with `require('affinity:xxx')` — a special scheme that is **not** the file system. These resolve to compiled C++ extension objects baked into the Affinity binary.

You can never call into a C++ binding that the JS SDK doesn't already expose. There is no `ffi`, no `nativeModule`, no way to call arbitrary C++ from JS. If something isn't in the binding table below, it does not exist for your scripts.

60.2 The four native C++ binding modules

Module	What it provides
<code>affinity:dom</code>	Document object model — all nodes, spreads, adjustments, filters
<code>affinity:commands</code>	Command construction — <code>DocumentCommand</code> , <code>CompoundCommandBuilder</code> , etc.
<code>affinity:common</code>	Shared types — <code>EnumerationResult</code> , <code>ErrorCode</code>
<code>affinity:story</code>	Text story model — <code>StoryIoFormat</code>
<code>affinity:ui</code>	Dialog system — all <code>Dialog*</code> APIs, <code>UiApi</code> for alerts/prompts
<code>affinity:application</code>	App-level — <code>ApplicationApi</code> , <code>ApplicationSettingsApi</code>

Everything in the JS SDK is a wrapper around calls into one of these six modules.

60.3 The architecture signal: `\*Api` classes

Every C++ method is accessed through a class ending in `Api`. These are the exact boundary points between JS and C++. The JS wrapper classes receive a native handle from these `Api` calls and wrap it in a typed JS object.

```
// This is what happens inside the SDK — not in your scripts
class ShapeNode extends VectorNode {
  get shapeInterface() {
    // ShapeNodeApi is C++ — returns a native handle
    return new
ShapeInterface(ShapeNodeApi.getShapeInterface(this.handle));
    // ^^^^^^^^^^^^^ C++ boundary
  }
}
```

The handle is an opaque integer pointer into the C++ heap. You never construct handles — you only receive them from `Api` calls and pass them back.

60.4 What IS exposed to JavaScript**60.4.1 Node types (`affinity:dom` → `nodes.js`)**

All 29 concrete node classes are accessible:

Node class	Type guard	Key interfaces
<code>SpreadNode</code>	<code>isSpreadNode</code>	<code>physicalRootInterface</code> , artboards, layers
<code>DocumentNode</code>	<code>isDocumentNode</code>	<code>pageCount</code> , <code>spread-Count</code>
<code>GroupNode</code>	<code>isGroupNode</code>	<code>children</code> , <code>visibility</code>
<code>ContainerNode</code>	<code>isContainerNode</code>	<code>children</code> , <code>fill</code> , <code>line</code>
<code>ShapeNode</code>	<code>isShapeNode</code>	<code>shapeInterface</code> , <code>fill</code> , <code>line</code> , <code>artboard</code>
<code>PolyCurveNode</code>	<code>isPolyCurveNode</code>	<code>curvesInterface</code> , <code>art-board</code>

Node class	Type guard	Key interfaces
ImageNode	isImageNode	rasterInterface, imageResourceInterface
RasterNode	isRasterNode	rasterInterface
ArtTextNode	isArtTextNode	storyInterface, fill, line
FrameTextNode	isFrameTextNode	storyInterface, fill, line
TableTextNode	isTableTextNode	storyInterface (grid-based cells)
CurvePathTextNode	isCurvePathTextNode	storyInterface
PolyCurveTextNode	isPolyCurveTextNode	storyInterface
ShapeTextNode	isShapeTextNode	storyInterface, shapeInterface
ShapePathTextNode	isShapePathTextNode	storyInterface, shapeInterface
EmbeddedDocumentNode	isEmbeddedDocumentNode	spreads, artboards, layers
VectorNode	isVectorNode	fill, line, curves, pictureFrame
LogicalNode	isLogicalNode	fill, line, transparency
ColouredLogicalNode	isColouredLogicalNode	layerColour
PhysicalNode	isPhysicalNode	(base class)
PatternRasterNode	isPatternRasterNode	rasterInterface (tiled image)
EnclosureRasterNode	isEnclosureRasterNode	rasterInterface (wrapped raster)
MeasurementNode	<i>(created via MeasurementNodeDefinition.create(start, end, factor, unit Type))</i>	measurement annotation
DevelopNode	isDevelopNode	.parameters, .source IsRaw, .imageSize (RAW develop)

60.4.2 Adjustment layer nodes (all exposed)

Node class	Parameters type
BlackAndWhiteAdjustmentRasterNode	BlackAndWhiteAdjustmentParameters
BrightnessContrastAdjustmentRasterNode	BrightnessContrastAdjustmentParameters
ColourBalanceAdjustmentRasterNode	ColourBalanceAdjustmentParameters
CurvesAdjustmentRasterNode	CurvesAdjustmentParameters (handle-based)

Node class	Parameters type
ExposureAdjustmentRasterNode	ExposureAdjustmentParameters
HSLShiftAdjustmentRasterNode	HSLShiftAdjustmentParameters (handle-based)
InvertAdjustmentRasterNode	<i>(no parameters)</i>
LevelsAdjustmentRasterNode	LevelsAdjustmentParameters + colourSpace
NormalsAdjustmentRasterNode	NormalsAdjustmentParameters
PosteriseAdjustmentRasterNode	PosteriseAdjustmentParameters
RecolourAdjustmentRasterNode	RecolourAdjustmentParameters
SelectiveColourAdjustment RasterNode	SelectiveColourAdjustmentPara- meters
ShadowsHighlightsAdjustment RasterNode	ShadowsHighlightsAdjustmentPa- rameters
SplitToningAdjustmentRaster Node	SplitToningAdjustmentParame- ters
ThresholdAdjustmentRasterNode	ThresholdAdjustmentParameters
ToneCompressionAdjustment RasterNode	ToneCompressionAdjustmentPara- meters
ToneStretchAdjustmentRaster Node	ToneStretchAdjustmentParame- ters
VibranceAdjustmentRasterNode	VibranceAdjustmentParameters
WhiteBalanceAdjustmentRaster Node	WhiteBalanceAdjustmentParame- ters

60.4.3 Filter layer nodes (all exposed)

Node class
AddNoiseFilterRasterNode
BilateralBlurFilterRasterNode
BloomFilterRasterNode
BoxBlurFilterRasterNode
ClarityFilterRasterNode
DefringeFilterRasterNode
DenoiseFilterRasterNode
DepthOfFieldFilterRasterNode
DiffuseFilterRasterNode
DiffuseGlowFilterRasterNode
DustAndScratchFilterRasterNode
FieldBlurFilterRasterNode
GaussianBlurFilterRasterNode
HighPassFilterRasterNode
LensBlurFilterRasterNode

Node class
MaximumBlurFilterRasterNode
MedianBlurFilterRasterNode
MinimumBlurFilterRasterNode
MotionBlurFilterRasterNode
PinchPunchFilterRasterNode
PixelateFilterRasterNode
RadialBlurFilterRasterNode
RippleFilterRasterNode
ShadowsHighlightsFilterRasterNode
SphericalFilterRasterNode
TwirlFilterRasterNode
UnsharpMaskFilterRasterNode
VignetteFilterRasterNode
VoronoiFilterRasterNode
HalftoneFilterRasterNode

60.4.4 Node interfaces (exposed as lazy-loaded properties)

Every node exposes some of these interfaces — which ones depend on the node type:

Interface	Accessed via	Key properties/methods
BaseBoxInterface	<code>node.baseBoxInterface</code>	<code>.baseBox</code> , <code>.constrainingBaseBox</code>
BlendModeInterface	<code>node.blendModeInterface</code>	<code>.blendMode</code> , <code>.blendOptions</code> , <code>.antialiasingMode</code>
BrushFillInterface	<code>node.brushFillInterface</code>	<code>.currentDescriptor</code> , <code>.isNoFill</code>
CompoundOperationInterface	<code>node.compoundOperationInterface</code>	<code>.compoundOperation</code>
CurvesInterface	<code>node.curvesInterface</code>	<code>.polyCurve</code>
DescriptionInterface	<code>node.descriptionInterface</code>	<code>.description</code> , <code>.userDescription</code> , <code>.tag</code> <code>Colour</code>
EditabilityInterface	<code>node.editabilityInterface</code>	<code>.isEditable</code> , <code>.isLocalEditable</code> , <code>.isMasterEditable</code>
ExportableInterface	<code>node.exportableInterface</code>	<code>.exportConfig</code>

Interface	Accessed via	Key properties/methods
ImageResourceInterface	node.imageResourceInterface	.imageFilePath, .imageFileSize, .fileType
LayerEffectsInterface	node.layerEffectsInterface	.effects
LineStyleInterface	node.lineStyleInterface	.lineStyle, .penFillDescriptor, .lineWeight
PhysicalRootInterface	node.physicalRootInterface	.physicalRootProperties
PictureFrameInterface	node.pictureFrameInterface	.enabled, .description
RasterInterface	node.rasterInterface	.width, .height, .format, .pixelSize
ShapeInterface	node.shapeInterface	.shape, .type, .boundingBox
StoryInterface	node.storyInterface	.story, .storyRange, .getText()
TransformInterface	node.transformInterface	.transform, .baseBox
TransparencyInterface	node.transparencyInterface	.fillDescriptor
VisibilityInterface	node.visibilityInterface	.globalOpacity, .fillOpacity, .isVisibleInExport
ArtboardInterface	node.artboardInterface	.enabled, .name, .baseBox, .origin
TagInterface	node.tagInterface	.tagColour
MarginsInterface	node.marginsInterface	margins data
TextFrameInterface	node.textFrameInterface	.ignoreBaselineGrid, .ignoreTextWraps, .isMultiFrameTextFlow

60.4.5 C++ Api classes from affinity:dom — full method inventory

From nodes.js alone, 142 C++ Api binding classes are imported. Here are the key ones with their exposed method counts:

Api class	Methods exposed to JS	Notes
NodeApi	32	Core tree navigation + box queries
NodeCastApi	74	Type dispatch — one handler per node class

Api class	Methods exposed to JS	Notes
NodeDefinitionApi	2	setTransform, set UserDescription
EmbeddedDocument NodeApi	26	Placed document properties
VectorNodeDefini- tionApi	17	Multi-fill/stroke slot management
DepthOfFieldFilter ParametersApi	12	Depth of field parame- ters
SpreadNodeApi	11	Spread extents, page indices, artboards
ImageNodeApi	10	Image properties, stock info
CurvesAdjustmentPa- rametersApi	9	Curves spline manage- ment
HSLShiftAdjustment ParametersApi	9	HSL channel parame- ters
FieldBlurFilterPa- rametersApi	7	Field blur point man- agement
TextNodeApi	6	Text-specific inter- faces
VectorNodeApi	7	Vector-specific inter- faces
LogicalNodeApi	5	Fill/line/transparency for logical nodes
CurvesAdjustment RasterNodeDefini- tionApi	6	Curves + colour space
LevelsAdjustment RasterNodeDefini- tionApi	6	Levels + colour space

Most filter/adjustment Api classes have exactly 4 methods: create, createDefault, getParameters, setParameters.

60.5 What is NOT exposed to JavaScript

This is the critical part. The following categories of C++ functionality have **no binding** visible in the SDK — there is no JS wrapper for them:

60.5.1 1. Pixel-level raster manipulation

/pixelaccessor adds PixelReaderRGBA8, PixelReaderWriterRGBA8, and equivalents for all 10 colour formats (RGBA8, RGBA16, IA8, IA16, CMYKA8, LABA16, M8, M16, RGBAuf, Mf). Both read and write pixel access is supported across all formats. Use node.rasterInterface.createCompatible Bitmap(true) to get a writable bitmap, then PixelReaderWriterRGBA8.create(bitmap) to read/write pixels, then DocumentCommand.createReplace Bitmap(sel, bitmap) to push changes back.

60.5.2 2. Master pages / master spreads

The `EmbeddedDocumentNode` can enumerate spreads including masters via `enumerateSpreads(includeMasters)`, but there is no `MasterSpreadApi` or `MasterPageApi`. You can detect and navigate to master content but cannot programmatically create, link, or manage master pages from JS.

60.5.3 3. Styles (character, paragraph, object)

The story system exposes `GlyphAttributedStringType.StyleName` for applying named character styles by name. But there is no `StyleSheetApi`, `ParagraphStyleApi`, or `CharacterStyleApi`. You can **apply** a style by name if you already know it, but you cannot enumerate all styles, create new styles, modify existing styles, or read a node's applied style name from the C++ side.

60.5.4 4. Tables — partially available

`TableTextNode` exposes a grid-based text node accessible via `/nodes`. It has a `storyInterface` where cells are separated by `HardBreakType.TableCell` glyphs. There is no dedicated `TableApi`, `TableCellApi`, or `TableRowApi` — you navigate cells through the story text model. Complex table operations (row/column insert/delete, merge, split) are still not exposed.

60.5.5 5. Find/Replace

No `FindReplaceApi`. You can implement your own text search by reading `storyRanges`, but the application's built-in find/replace engine is not accessible.

60.5.6 6. Symbols / Assets panel

No `SymbolApi` or `AssetApi`. Affinity's Symbols feature (reusable components) has no JS representation.

60.5.7 7. Live paint / pixel persona tools

No `PaintBrushApi`, `LivePaintApi`, or `InpainterApi` beyond the high-level `DocumentCommand.createGenerativeEditImage()`. The pixel-level painting tools have no programmatic API.

60.5.8 8. Constraints / layout rules

No `ConstraintApi`. The auto-layout and constraint system used in Affinity for responsive artboards has no JS binding.

60.5.9 9. Linked resources / package management

The SDK has `PackageResourcesPolicy` in exports but no `LinkedResourceApi` or `PackageApi` for managing linked file relationships or repackaging documents.

60.5.10 10. Colour swatches / palettes

No `SwatchApi`, `PaletteApi`, or `ColourGroupApi`. Document colour palettes and global swatches are not accessible.

60.5.11 11. Grids and snapping

No `GridApi`, `SnapApi`. The grid and snapping system is UI-side only.

60.5.12 12. Print / preflight

No `PrintApi` or `PreflightApi`. Printing and PDF preflight checking are not exposed.

60.5.13 13. History beyond undo/redo

`DocumentHistory` exposes position, size, `canUndo`, `canRedo`, and enumeration of history items. But items are read-only — you cannot inject arbitrary history entries or branch history. The `setPosition` setter exists but only navigates linearly.

60.5.14 14. Plugin metadata / custom properties

/taginterface provides TagInterface — a per-node key-value store that persists in the file. Use node.tagInterface.hasKey(key), .getValueForKey(key), and DocumentCommand.createSetTagValueForKey(sel, key, value) to read and write arbitrary string metadata on any node. Predefined TagKey enumerates Affinity’s built-in tag keys.

60.6 Boundary signals — how to detect what's missing

When working with the SDK, these patterns indicate you’ve hit a boundary:

Pattern 1: No *Api class exists If you want to do something and can’t find an *Api class for it in the native module imports, the feature is not bound. Example: searching for StyleSheetApi — doesn’t exist.

Pattern 2: Interface exists but is read-only Some interfaces expose getters but no corresponding DocumentCommand.createSet* factory. Example: you can read node.visibilityInterface.fillOpacity but the only way to set it is through DocumentCommand.createSetVisibility().

Pattern 3: NOT_ALLOWED error code If a script command returns NOT_ALLOWED, it means the user has restricted AI, filesystem, or network access in Affinity settings — not that the API doesn’t exist.

Pattern 4: Class defined but not exported Some classes are defined in the SDK source but not in module.exports. Example: NodeFactory is defined but not exported — it’s used internally by createTypedNode().

60.7 The complete native method call inventory (nodes.js only)

The following 95 unique C++ method calls are made from nodes.js alone. This is the exact set of C++ operations that the node system relies on:

60.7.1 Navigation

getFirstChild, getLastChild, getNextSibling, getPreviousSibling, getParent, moveToFirstChild, moveToLastChild, moveToNextSibling, moveToPreviousSibling, moveToParent, isSameNode

60.7.2 Bounding boxes

getSpreadBaseBox, getExactSpreadBaseBox, getExactSpreadVisibleBox, getSpreadVisibleBox, getLocalVisibleBox, getLineBox, getTransformedLineBox, getContentExtentsBox, getContentExtentsBoxOfChildren

60.7.3 Transforms

getBaseToSpreadTransform, getLocalToSpreadTransform, getSpreadToBaseTransform

60.7.4 Interface accessors

getBaseBoxInterface, getBlendModeInterface, getBrushFillInterface, getCompoundOperationInterface, getCurvesInterface, getDescriptionInterface, getEditabilityInterface, getExportableInterface, getImageResourceInterface, getLayerEffectsInterface, getLineStyleInterface, getPhysicalRootInterface, getPhysicalRootPropertiesInterface, getPictureFrameInterface, getRasterInterface, getShapeInterface, getStoryInterface, getTransformInterface, getTransparencyInterface, getVisibilityInterface, getArtboardInterface

60.7.5 Document/spread

getDocument, getPageCount, getSpreadCount, getSpreadExtents, getFirstPageIndex, getLastPageIndex, isFirstPage, getPageIndexOfPoint, getPageIndexOfBox, enumerateArtboards, getArtboard, getArtboardCount

60.7.6 Image/raster

getBitmapBrushFillDescriptor, getExtendType, getUpsamplerType, getStockURL, getStockUserProfileURL, getStockAuthor, getLastRendered, isKOnly, getExtendEmpty, getLayerColour

60.7.7 Embedded documents

getSelectedSpreadId, getSelectedArtboardId, embeddedDocumentHasSpreads, embeddedDocumentHasArtboards, embeddedDocumentHasPageBoundingBoxes, embeddedDocumentHasLayers, getSelectedArtboardOffset, getPageBoundingBoxType, getPDFPassthrough, canSetPDFPassthrough, getRasterDPI, getOriginalHostDPI, isAffinityFile, canMakeLinked, canEditEmbeddedImage, getLoadDocumentOptions, shouldAllowSelectDocument, isArtboardSelected, isArtboardDoc, needsPassword, enumerateSpreads, enumeratePageBoundingBoxes, enumerateLayerVisibilities

60.7.8 Logical nodes

hasNonMaskChildren, hasNonMaskOrAdjustmentChildren

60.7.9 Physical nodes

canTransformWhileProtectingChildList, canBeExpressedAsVectorClip

60.8 Runtime capability detection

There is no `SDK_VERSION` constant or capability query system. You cannot ask the SDK at runtime “does this method exist?” — you have to try it and catch errors, or check the source. The `module.exports` list in each wrapper file is the authoritative list of what’s available, and `app.shortVersion` from `/application` reports the running Affinity build.

60.9 SDK modules

Module	Native binding	Key capability
<code>/pixelaccessor</code>	<code>affinity:raster</code>	Read+write pixels in all 10 colour formats
<code>/fonts</code>	<code>affinity:fonts</code>	Enumerate all fonts, families, collections
<code>/exportconfig</code>	<code>affinity:dom</code>	Per-node export settings (<code>ExportConfig/ExportFormat/ExportScale</code>)
<code>/glyphs</code>	<code>affinity:story</code>	Typed glyph classes, <code>CharGlyph.create()</code> , glyph enums
<code>/hatch</code>	<code>affinity:dom</code>	<code>HatchPattern/HatchLine</code> construction
<code>/rasterbrush</code>	<code>affinity:brushes</code>	Raster brush properties (size, flow, opacity, etc.)
<code>/buffer</code>	(internal)	Binary buffer manipulation
<code>/rasterobject</code>	(internal)	Raster object handle wrapper

Module	Native binding	Key capability
/documentproperties	affinity:dom	Document metadata (DPI, size, colour space)
/drawingscale	affinity:dom	Map/drawing scale (1:100 etc.)
/taginterface	affinity:dom	Custom key-value tags per node
/marginsinterface	affinity:dom	Spread margin information
/pageboxinterface	affinity:dom	Bleed, trim, slug boxes
/artboardproperties	affinity:dom	Artboard metadata
/os	affinity:os	OS platform/arch/os-Name/osVersion/Eol
/environment	affinity:application	Environment permissions, fileSystem-Roots, sdkVersion
/logging	affinity:common	LogFile file-based logging
/configuration	affinity:common	ConfigurationItem JSON config parsing
/pathbrush	affinity:brushes	PathBrush vector brush access

60.9.1 New native C++ binding modules (affinity:xxx)

Module	Provides
affinity:raster	All PixelReader/Writer Api classes for 10 colour formats
affinity:fonts	FontApi, FontFamilyApi, FontCollectionApi, PanoseApi, font enums
affinity:brushes	RasterBrushApi, BrushDynamicApi, brush enums

60.9.2 DocumentCommand factories

Key additions grouped by purpose:

Macro system: createStartRecordingMacro, createStopRecordingMacro, createReplayMacro, createClearMacro, createImportMacro(path), createExportMacro(path)

Raster selections: createRasterSelectAll, createRasterDeselect, createRasterInvertSelection, createRasterReselect, createSetRasterSelectionFromPolygon(polygon, op, antialias, feather) (replaces createRasterSelectPolygon), createSetRasterSelectionFromObject(sel, op, antialias, feather), createGrowShrinkRasterSelection(radius, circular), createFeatherRasterSelection(radius), createSmoothRasterSelection(radius), createOutlineRasterSelection(radius, alignment, circular), RasterSelectionOutlineAlignment enum

Opacity control: `createSetOpacity(sel, opacity), createSetBrushFillOpacity(sel, opacity, opts), createSetLineFillOpacity(sel, opacity, opts)`

Fill anchoring: `createSetBrushFillIsAnchoredToSpread, createSetPenFillIsAnchoredToSpread, createSetTransparencyFillIsAnchoredToSpread`

Tag metadata: `createSetTagValueForKey(sel, key, value), createSetTagValueForPredefinedKey(sel, predefinedKey, value)`

Blend options: `createSetBlendGamma(sel, gamma), createSetBlendRanges(sel, blendOptions), createSetAntialiasingMode(sel, mode)`

Snapshots: `createAddDocumentSnapshot, createDeleteDocumentSnapshot, createRestoreDocumentSnapshot, createSetCurrentSnapshot, createSetCurrentSnapshotFromHistoryItem`

Document: `createSetDocumentProperties, createConvertDocumentFormat(format), createGroupTransform, createSetWindingMode, createSetExportConfig, createMoveNodes, createFlatten, createMergeVisible, createSetSelection(sel)`

Spread: `createSetSpreadSizeWithAnchor, createSetSpreadDocumentProperties`

History: `createSetHistoryIndex(index), createCycleAlternateFutures(index)`

Text frame: `createSetTextFrameIgnoreBaselineGrid(sel, ignore), createSetTextFrameIgnoreTextWraps(sel, ignore)`

Layer effects — outline fill: `createSetOutlineLayerEffectFill(sel, fill, index, enableIfDisabled), createSetOutlineLayerEffectFillTransform(sel, transform, index, enableIfDisabled), createSetOutlineLayerEffectFillType(sel, type, index, enableIfDisabled), createSetGradientOverlayLayerEffectFill(sel, fill, index, enableIfDisabled), createSetGradientOverlayLayerEffectFillTransform(sel, transform, index, enableIfDisabled)`

AI: `createRemoveBackground(sel), createSelectSubject(sel)` **Artboard:** `createAddArtboard(spread, bounds)` **Curve editing:** `createAddCurve(sel), createAddCurveNode(sel, point, type), createSmoothCurves(sel), createDeleteCurveNodes(sel), createScissorCut(sel), createKnifeCut(sel)`

Document: `createSetDocumentUnits(sel, units), createSetPageDocumentProperties(sel, props)` **Guides:** `createAddHorizontalGuide(pos), createAddVerticalGuide(pos), createMoveHorizontalGuide(sel, pos), createMoveVerticalGuide(sel, pos), createRemoveHorizontalGuide(sel), createRemoveVerticalGuide(sel)`

Embedded documents: `createSetEmbeddedDocumentAllLayersVisibility(sel, visible), createSetEmbeddedDocumentLayerVisibility(sel, layerIndex, visible), createSetEmbeddedDocumentPDFPassthrough(sel, passthrough), createSetEmbeddedDocumentPageBoundingBoxType(sel, type), createSetEmbeddedDocumentSelectedArtboardID(sel, id), createSetEmbeddedDocumentSelectedSpreadID(sel, id)` **Layer effects:** `createSetAllLayerEffectsScaleWithObject(sel, scale), createSetOuterShadowLayerEffectKnocksOut(sel, knocksOut, index, enableIfDisabled), createSetBevelEmbossLayerEffectAzimuth(sel, azimuth, enableIfDisabled), createSetBevelEmbossLayerEffectElevation(sel, elevation, enableIfDisabled)` **Raster selection:** `createRasterFloodSelect(x, y, tolerance, antialias, feather) (magic wand), createRasterSelectPolygon(polygon, operation, antialias, feather) (avail-`

able alongside `createSetRasterSelectionFromPolygon`) **Other:** `createHideSelection`, `createSplitCurve`, `createInsertGlyph`

60.9.3 Additional factories

Boolean operations: `createBoolOpUnion(sel)`, `createBoolOpSubtract(sel)`, `createBoolOpIntersect(sel)`, `createBoolOpXor(sel)`, `createDivideShapes(sel)`

Text frame threading: `createLinkTextFrame(srcNode, destNode)`, `createUnlinkTextFrame(node)`

Merge: `createMergeDown()`, `createMergeSelected()`

Picture frame: `createPopulatePictureFrame(contentNode, selection)`

Raster auto-fixes: `createRasterAutoLevels(sel)`, `createRasterAutoContrast(sel)`, `createRasterAutoColours(sel)`, `createRasterAutoWhiteBalance(sel)`

Raster fill: `createRasterFill(sel, mode, colour, opacity, blendMode)`, `createRasterFloodFill(sel, point, tolerance, isContiguous, antialias, samplingSource, blendMode, colour)`

RAW Develop: `createSetDevelopParameters(sel, developParameters)`

Measurement: `createSetMeasurementUnits(unitType)`, `createSetMeasurementPrecision(useDocumentPrecision, decimalPlaces)`, `createSetMeasurementAnnotationOffset(node, offset)`, `createSetMeasurementShowEndpointMarkers(show)`

60.9.4 Enum member reference

Values below marked ★ were read at runtime (`Object.entries` dump in Affinity 3.3.0); the rest are member names from the official docs at <https://sdk.affinity.studio/> (which publishes no values — never infer values from documentation order).

Runtime-verified (name = value) ★:

Enum	Members
GradientFillType ★	Linear=0, Elliptical=1, Radial=2, Conical=3
RasterSelectionLogicalOperation ★	New=0, Add=1, Subtract=2, Intersect=3 (no Replace — use New)
NodeMoveType ★	Before=0, After=1, Inside=2
NodeChildType ★	Main=0, Enclosure=1, MasterSpread=2, WrapShape=3, StoryPin=4
ShapeCornerType ★	Round=0, Straight=1, RoundInverse=2, Cutout=3, None=4
ContentType ★	Main=0, Frame=1, Cell=2
VisibilityMode ★	Main=0, Export=1
ShapeArrowEndStyle ★	None=0, Arrow=1, TailTriangular=2, TailSquare=3, Circle=4, HalfCircleInner=5, HalfCircleOuter=6, Diamond=7, Box=8, Barb=9,

Enum	Members
	Chevron=10 (no Triangle – use Arrow)
QRPayloadType ★	Text=0, URL=1, Phone=2, SMS=3, Email=4, WhatsApp=5, FaceTime=6, Location=7, Wifi=8, VEvent=9, VCard=10, DataMerge=11
FillType ★	Unknown=0, None=1, Solid=2, Gradient=3, Bitmap=4, Mesh=5, Hatch=6, Diffusion=7

Member names from official docs (values not published):

Enum	Members
SubSelectionType	CurveEdge, CurveNode, Text, Table, ExportFormat, Fill, LineFill, Transparency, FillMesh, LineFillMesh, TransparencyMesh, GeometryModifier, FillDiffusion
DialogItemType	Unknown, Separator, Switch, ComboBox, ButtonSet, RadioGroup, StaticText, UnitValueEditor, CheckBox, SpatialAnchor, ColourPicker, TextBox, FontPicker, FillEditor, StrokeEditor, Group, Column, ColumnStack, Button
SpatialAnchor	None, TopLeft, CentreLeft, BottomLeft, TopCentre, Centre, BottomCentre, TopRight, CentreRight, BottomRight, FocalPoint
PackageResourcesPolicy	None, IncludeImages, IncludeTexts, IncludeCSS, IncludeUnrestrictedFonts, IncludeRestrictedFonts
PageBoundingBoxType	MediaBox, CropBox, TrimBox, ArtBox, BleedBox, MinimumContent, MinimumVisibleContent, MaximumContent, MaximumVisibleContent
RasterFillMode	Colour, Foreground, Background, Inpainting, History
SamplingSource	CurrentLayer, AllLayers
RasterFloodFillSamplingSource	CurrentLayer, CurrentLayerAndBelow, LayersBeneath
RasterSelectionOutlineAlignment	Outside, Centre, Inside
TonalRangeType	Shadows, Midtones, Highlights

Enum	Members
ToneCompressionMethod	Basic, Natural, Bright, Contrast, Filmic, Punchy, PBRNeutral, Log
ToneStretchMethod	Basic, Arcsinh, Logarithmic, Structural, ColourPreserving
DevelopDetailRefinementMethod / DevelopToneCurveMethod / DevelopInvertMethod	UnsharpMask, HighPass, Multi-Band / Standard, Compressed, Natural, HighContrast, Log / InfraredRB, InfraredRGB, Negative
DocumentLoadMode	Application, Spreads, Artboards
HalftoneDotType / HalftoneScreenType	Cosine, Round, Line / Mono, Colour, Line, Circular
AntialiasingMode	Inherit, ForceOn, ForceOff
CompoundOperation	Union, Intersect, Subtract, Xor
ExportScalePreset	Standard, Mult2x, Mult3x, Width100, Height50, Square64, Square64Mult2x, Width32Height32
ExportScaleSizeType	None, Width, Height, Square, WidthHeight
FileType	JPEG, PNG, TIFF, WEBP, HDPhoto, RAW, GIF, JPEG2000, BMP, TGA, OpenEXR, HDR, Unknown, Affinity, HEIF, FITS, Plugin
ShapeMajorAxis / ShapeSpiralStyle / Shape*Param / ShapeCornerIndex	See /shapes module; corner indices TopLeft/TopRight/BottomLeft/BottomRight
CapsType, SuperSubType, LeadingOverrideType, OpticalAlignmentType, TocRoleType, TypographicLineType	Glyph typography enums (see /glyphatts)
ParagraphAlignXType, ParagraphLeadingType, ParagraphStartAtHardBreakType, ParagraphUseSpaceBeforeMode, ParagraphPDFExportTagType, ParagraphLineBreakModeType	Paragraph behaviour enums (see /paragraphatts)
StoryIoFormat, TextDefaultType (Art, Frame, Unknown, Measurement), WordPartType	Story/text enums
TableEdgeSelector (None ... All, 14) / TableAxis	Table enums
WifiEncryptionType, DocumentFieldType, FieldDataType, FillerTextType, NoteType, NotePosition	Field/QR enums

Enum	Members
UnitCategory, UserUnitType, UnitType	Units enums
AddNoiseType (Gaussian, Uniform), BloomMethod (Contrast, Bright), DepthOfFieldMode (Elliptical, Tilt-Shift), SelectiveColour (9 channels), ShadowsHighlightsVersion (Default, V16)	Filter/adjustment enums
LineCommandDefaultsMode, StrokeFillType, WindingOrder, BevelEmbossType, ImagePlacement, ColourSpaceType, RasterIntent	Named in module indexes; members per official docs

Documented officially but NOT script-reachable in 3.3.0 (present in the web API index, absent from JSLib — native layer only, do not use from scripts): QRPayloadDataMerge (QR payload kind 11 exists in the QRPayloadType enum but has no JSLib wrapper; VEvent (9) likewise), ImportOptions (CAD import options), DocumentView (empty handle).

60.9.5 Geometry classes (from `/geometry`)

Class	Key factories/methods
Polygon	create(), createLine(), createTriangle(p1,p2,p3), createDiamond(rect), createLozenge(p1,p2,radius),createRectangle(rect)
Curve	create(), createLine(), createRectangle(), createEllipse(), createDiamond(), createLozenge(), createPrecisionUnitCircle()
CubicBezier	.evaluate(t), .split(t), .getTangent(t), .getNormal(t), .getCurvature(t), .getClosestPoint(pt), .length, .getParamAtLength(len), .clone(), .reversed(), .transformed(xf)
CurveBuilder	create(), .begin(), .lineTo(), .close(), .addBezier(), .addArc(), .addEllipse(), .versineTo(), .bulgeTo(), .createCurve()

60.9.6 Selections (from `/selections`)

Class	Key factories/methods
TableSubSelection	fromSubSelection(sub) — .anchor, .caret, .isRectangle, .cells, .enumerateCells(), .enumerateEdges()

Class	Key factories/methods
TextSelection	create(ranges) — .ranges, .caret, .anchor, .is Empty
FillSubSelection	fromSubSelection(sub) — .index
LineFillSubSelection	fromSubSelection(sub) — .index

60.9.7 Story attribute runs

Iterator	Yields
story.glyphAttRuns	{glyphAtts, begin, end}
story.paragraphAttRuns	{paragraphAtts, begin, end}
story.attRuns	{glyphAtts, paragraphAtts, be- gin, end}

60.9.8 Dialog controls (from `/dialog`)

Class	Key properties/methods
Dialog	create(title), .addColumn(), .run Modal(), .findControl(id), .is RunningModal, .setItemsVisibil- ity(showIDs, hideIDs)
DialogFontPicker	.font (Font), .fontFamily (Font- Family), .fontCollection (FontCol- lection), .isFullWidth
DialogFillEditor	.fill, .isStrokeFill
DialogStrokeEditor	.stroke (LineStyleDescriptor)
DialogSpatialAnchor	.value (SpatialAnchor enum)
DialogButton	.alignment, .onClickHandler, .is FullWidth
DialogUnitValueEditor	.value, .units, .noMinValue, .no MaxValue, .precision
DialogColourPicker	.value (Colour), .allowPick None, .allowNoise

60.9.9 `createSetEditable` — argument order

```
// Selection is first argument, editable flag is second
DocumentCommand.createSetEditable(selection, true) // unlock
DocumentCommand.createSetEditable(selection, false) // lock
```

60.9.10 Layer effect families

The following effect families are available in addition to OuterShadow, Out-
line, InnerGlow, and GaussianBlur:

Effect	Key commands
OuterGlow	createSetOuterGlowLayerEffect* — colour, radius, opacity, intensity, blend mode

Effect	Key commands
InnerShadow	<code>createSetInnerShadowLayerEffect*</code> — colour, offsetAngle, radius, opacity, intensity (indexed)
ColourOverlay	<code>createSetColourOverlayLayerEffect*</code> — colour, opacity, blend mode (indexed)
GradientOverlay	<code>createSetGradientOverlayLayerEffect*</code> — fill, fillTransform, blend mode (indexed)
BevelEmboss	<code>createSetBevelEmbossLayerEffect*</code> — radius, depth, direction, soften, type, shadow/highlight colours
PhongBevel	<code>createSetPhongBevelLayerEffect*</code> — ambient, diffuse, specular, shininess, <code>createSetPhongBevelLayerEffectLights(count)</code>

All layer effects must be executed individually — `CompoundCommandBuilder` is a silent no-op for effect commands.

60.9.11 Fill types

Class	Factory	Description
<code>HatchFill</code>	<code>HatchFill.create(pattern, units, penColour, brushColour, lineWeight)</code>	Repeating line pattern
<code>BitmapFill</code>	<code>BitmapFill.create(bitmap, extendType, resamplerType, ignoreAlpha)</code>	Tiled raster image
<code>GradientFill</code>	<code>GradientFill.create(gradient, gradientType)</code>	Linear/radial/conical gradient
<code>MeshFill</code>	<code>MeshFill.create(mesh) + ColourMesh.createFromColour(colour, nodes)</code>	Mesh gradient grid (<code>ColourMesh</code> with <code>setNodePosition/setNodeColour</code>)
<code>DiffusionFill</code>	<code>DiffusionFill.create(curveSet) + DiffusionCurveSet.create()</code>	Diffusion curve fill (<code>DiffusionCurveKind</code> : <code>Bezier</code> / <code>Ellipse</code> / <code>Arc</code> / <code>Line</code>)

`FillDescriptor.createSolid()` accepts a `Colour` directly, as do `createSetBrushFill` / `createSetPenFill`. The `FillType` enum includes `Mesh` and `Diffusion`.

60.9.12 Document convenience methods

Document exposes direct convenience methods for most common operations, eliminating the need to call `DocumentCommand.create*` explicitly. These include: `doc.undo()`, `doc.redo()`, `doc.selectAll()`, `doc.deleteSelection()`, `doc.showSelection()`, `doc.hideSelection()`, `doc.lockSelection()`, `doc.unlockSelection()`, `doc.setOpacity()`, `doc.setBrushFillOpacity()`, `doc.setLineFillOpacity()`, `doc.setBrushFillDescriptor()`, `doc.setPenFillDescriptor()`, `doc.setLineWeight()`, `doc.setLineWeightPts()`, `doc.setLineCap()`, `doc.setLineJoin()`, `doc.convertToCurves()`, `doc.rasteriseObjects()`, `doc.flatten()`, `doc.mergeVisible()`, `doc.addGuide()`, `doc.moveGuide()`, `doc.removeGuide()`, `doc.addRectangularArtboard()`, `doc.imageTrace()`.

AI convenience methods: `doc.generateImage(prompt)`, `doc.detectDepth()`, `doc.colourise()`, `doc.generativeEditImage(prompt)`, `doc.removeBackground()`, `doc.selectSubject()`.

60.9.13 Document classes

Class	Purpose
<code>DocumentHistory</code>	Access undo history — position, size, canUndo, canRedo, <code>getItem(index)</code> , <code>items</code>
<code>DocumentHistoryItem</code>	Individual history entry — description, thumbnail, command
<code>NewDocumentOptions</code>	Full control over new document creation — dimensions, DPI, margins, colour profile
<code>DocumentPreset</code>	Enumerate built-in presets via <code>DocumentPreset.all</code>
<code>LoadDocumentOptions</code>	Control DPI, password, colour space when loading
<code>FileExportArea</code>	<code>createForSpreads(str)</code> , <code>createForSelection(sel)</code> in addition to existing factory methods

60.10 Summary counts

Category	Count
Native C++ binding modules	9
C++ Api classes in <code>nodes.js</code>	142
JS wrapper modules	75
DocumentCommand factories	351
Node types accessible	31 concrete + 10 base
Adjustment layer types	19
Filter layer types	30 (+ <code>HalftoneFilterRasterNode</code>)
Node interfaces	21+
Major feature categories exposed	pixel access, custom tags, tables, fills, AI, develop

Category	Count
Major features still NOT exposed	several categories

Release 3.3 — analysed from live Affinity SDK via MCP.

60.11 Official SDK example scripts

The SDK ships 34 official example scripts that are instructive as canonical patterns from the Affinity team. They are mirrored 1:1 in `phase-5-official-examples/` — see its [README(./phase-5-official-examples/README.md)] for the full table:

File	What it demonstrates
<code>examples/boldItalics.js</code>	<code>StoryDelta.createWeight()</code> , <code>StoryDelta.createItalic()</code> , <code>StoryDelta.createComposite()</code>
<code>examples/addGuides.js</code>	<code>DocumentCommand.createAddGuide()</code> , <code>unionRects()</code> , <code>UnitType.Pixel/Millimetre</code>
<code>examples/setDocumentFormat.js</code>	<code>doc.format = RasterFormat.xxx</code> , dialog combo pattern
<code>examples/addPoints.js</code>	Curve node manipulation
<code>examples/alignToPage.js</code>	Alignment via transform
<code>examples/adjustPageItems.js</code>	Batch node adjustment

60.11.1 Additional confirmed APIs

StoryDelta factory methods:

- `StoryDelta.createWeight(FontWeight.Bold)` — applies font weight
- `StoryDelta.createItalic(bool)` — applies italic
- `StoryDelta.createComposite([delta1, delta2])` — combines multiple deltas

Geometry utility:

- `unionRects(rect1, rect2)` from `/geometry` — combines two Rectangle bounding boxes

LoadDocumentOptions (exported from `/document`):

- Properties: `dpi`, `hostDpi`, `password`, `colourSpace`, `format`, `loadMode`

Confirmed patterns:

- `alert(string)` is a global — shorthand for a simple message dialog
- `node.getSpreadBaseBox()` returns a Rectangle with `.x`, `.y`, `.width`, `.height`, `.centre`
- `UnitType.Millimetre` (singular) — all unit names are singular (`Pixel`, `Millimetre`, `Point`)
- `result == DialogResult.Ok` — `loose == equality` is idiomatic; `.equals()` also works